# iZ-Library

1.2.1

Generated by Doxygen 1.16.1

1 iZprime User Manual 1
1.1 What this manual covers . . . 1
1.2 Related project docs . . . 1

2 Benchmarks 3
2.1 1. Run Benchmarks . . . 3
2.2 2. Result Files Used In This Snapshot . . . 3
2.3 3. Sieve Benchmark Table and Plot . . . 4
2.4 4. Prime Generation Benchmark Table . . . 4
2.5 5. Plot Artifacts . . . 4
2.5.1 Prime generation plots . . . 4
2.6 6. SiZ_count Benchmark Table . . . 4
2.7 7. Plot From Terminal (Interactive Prompt) . . . 5

3 Language Bindings Strategy (FFI) 7
3.1 Why API-layer bindings (not CLI) . . . 7
3.2 FFI API surface (v1) . . . 7
3.3 Ownership model . . . 8
3.4 Expression support . . . 8
3.5 Recommended binding rollout . . . 8
3.6 Notes for wrapper authors . . . 8

4 izprime CLI 9
4.1 Build . . . 9
4.2 Numeric input syntax . . . 9
4.3 Commands . . . 9
4.4 stream_primes . . . 9
4.5 count_primes . . . 10
4.6 next_prime . . . 10
4.7 prev_prime . . . 10
4.8 is_prime . . . 10
4.9 gcd . . . 11
4.10 lcm . . . 11
4.11 Existing maintenance commands . . . 11

5 Contributing to iZprime 13
5.1 Scope . . . 13
5.2 Development Setup . . . 13
5.3 Code Style . . . 14
5.4 Validation Requirements . . . 14
5.5 Benchmarking Expectations . . . 14
5.6 Pull Request Checklist . . . 14
5.7 Algorithm Contributions . . . 15
5.8 Review Standard . . . 15

6 Makefile documentation (iZprime) 17
6.1 Quick start . . . 17
6.2 Dependencies . . . 17
6.3 Targets . . . 18
6.3.1 Build / run . . . 18
6.3.2 Tests . . . 18
6.3.3 Benchmarks . . . 19
6.3.4 Examples . . . 19
6.4 Configuration variables . . . 19
6.4.1 Local configuration file . . . 19
6.4.2 Dependency discovery . . . 19
6.4.3 Build toggles . . . 20
6.4.4 Test/benchmark options . . . 20
6.5 Common recipes . . . 20
6.6 Notes on outputs . . . 21

7 Native Package Plan 23
7.1 Targets . . . 23
7.2 Metadata Locations . . . 23
7.3 Package Split (Linux) . . . 23
7.4 Release Flow . . . 24
7.5 CI Gates . . . 24

8 Releasing iZprime 25
8.1 Versioning . . . 25
8.2 Pre-release Gate . . . 25
8.3 Changelog and Release Notes . . . 26
8.4 Tag and Publish . . . 26
8.5 Post-release . . . 26

9 Tests 27
9.1 1. Test Targets . . . 27
9.2 2. What Each Target Runs . . . 27
9.3 3. Current Snapshot Results . . . 27
9.4 4. Show Results In Terminal . . . 28
9.5 5. Notes . . . 28

10 Topic Index 29
10.1 Topics . . . 29

11 Data Structure Index 31
11.1 Data Structures . . . 31

12 File Index 33
12.1 File List . . . 33

13 Topic Documentation 35
13.1 Bitmap Module . . . 35
13.1.1 Detailed Description . . . 36
13.1.2 Function Documentation . . . 37
13.1.2.1 bitmap_clear_all() . . . 37
13.1.2.2 bitmap_clear_bit() . . . 37
13.1.2.3 bitmap_clear_steps() . . . 38
13.1.2.4 bitmap_clear_steps_simd() . . . 39
13.1.2.5 bitmap_clone() . . . 40
13.1.2.6 bitmap_compute_hash() . . . 41
13.1.2.7 bitmap_flip_bit() . . . 41
13.1.2.8 bitmap_fread() . . . 42
13.1.2.9 bitmap_free() . . . 43
13.1.2.10 bitmap_fwrite() . . . 43
13.1.2.11 bitmap_get_bit() . . . 44
13.1.2.12 bitmap_init() . . . 44
13.1.2.13 bitmap_set_all() . . . 46
13.1.2.14 bitmap_set_bit() . . . 46
13.1.2.15 bitmap_validate_hash() . . . 47
13.1.2.16 TEST_BITMAP() . . . 47
13.2 Integer Arrays . . . 47
13.2.1 Detailed Description . . . 50
13.2.2 Function Documentation . . . 50
13.2.2.1 TEST_GENERIC_INT_ARRAYS() . . . 50
13.3 iZ Public API . . . 50
13.3.1 Detailed Description . . . 52
13.3.2 Typedef Documentation . . . 52
13.3.2.1 INPUT_SIEVE_RANGE . . . 52
13.3.3 Function Documentation . . . 52
13.3.3.1 iZ_next_prime() . . . 52
13.3.3.2 SiZ() . . . 53
13.3.3.3 SiZ_count() . . . 54
13.3.3.4 SiZ_stream() . . . 55
13.3.3.5 SiZm() . . . 56
13.3.3.6 SiZm_vy() . . . 57
13.3.3.7 SoA() . . . 58
13.3.3.8 SoE() . . . 59
13.3.3.9 SoEu() . . . 59
13.3.3.10 SoS() . . . 60
13.3.3.11 SSoE() . . . 61
13.3.3.12 SSoS() . . . 62
13.3.3.13 vx_random_prime() . . . 63
13.3.3.14 vy_random_prime() . . . 64

13.4 Toolkit (iZ/iZm) . . . 65
13.4.1 Detailed Description . . . 67
13.4.2 Macro Definition Documentation . . . 67
13.4.2.1 MR_ROUNDS . . . 67
13.4.2.2 VX2 . . . 67
13.4.2.3 VX3 . . . 67
13.4.2.4 VX4 . . . 68
13.4.2.5 VX5 . . . 68
13.4.2.6 VX6 . . . 68
13.4.2.7 VX7 . . . 68
13.4.2.8 VX8 . . . 68
13.4.3 Function Documentation . . . 69
13.4.3.1 compute_l2_vx() . . . 69
13.4.3.2 compute_max_vx() . . . 69
13.4.3.3 compute_vx_k() . . . 70
13.4.3.4 get_root_primes() . . . 70
13.4.3.5 iZ() . . . 71
13.4.3.6 iZ_mpz() . . . 71
13.4.3.7 iZm_clone() . . . 72
13.4.3.8 iZm_construct_vx_base() . . . 73
13.4.3.9 iZm_free() . . . 73
13.4.3.10 iZm_init() . . . 74
13.4.3.11 iZm_solve_for_x0() . . . 74
13.4.3.12 iZm_solve_for_x0_mpz() . . . 75
13.4.3.13 iZm_solve_for_y0() . . . 77
13.4.3.14 process_iZ_bitmaps() . . . 78
13.4.3.15 range_info_free() . . . 78
13.4.3.16 range_info_init() . . . 79
13.4.3.17 test_primality() . . . 79
13.4.3.18 vx_collect_p_gaps() . . . 80
13.4.3.19 vx_free() . . . 80
13.4.3.20 vx_full_sieve() . . . 81
13.4.3.21 vx_init() . . . 81
13.4.3.22 vx_search_prime() . . . 82
13.4.3.23 vx_stream() . . . 83
13.4.3.24 vy_search_prime() . . . 84
13.5 FFI API . . . 85
13.5.1 Detailed Description . . . 86
13.5.2 Enumeration Type Documentation . . . 86
13.5.2.1 IZP_FFI_STATUS . . . 86
13.5.2.2 IZP_SIEVE_KIND . . . 86
13.5.3 Function Documentation . . . 86
13.5.3.1 izp_ffi_clear_error() . . . 86

13.5.3.2 izp_ffi_count_range() . . . 87
13.5.3.3 izp_ffi_free_string() . . . 88
13.5.3.4 izp_ffi_free_u64_buffer() . . . 88
13.5.3.5 izp_ffi_last_error() . . . 88
13.5.3.6 izp_ffi_next_prime() . . . 89
13.5.3.7 izp_ffi_random_prime_vx() . . . 90
13.5.3.8 izp_ffi_random_prime_vy() . . . 90
13.5.3.9 izp_ffi_sieve_u64() . . . 91
13.5.3.10 izp_ffi_status_message() . . . 92
13.5.3.11 izp_ffi_stream_range() . . . 92
13.5.3.12 izp_ffi_version() . . . 93
13.6 Logging . . . 93
13.6.1 Detailed Description . . . 94
13.6.2 Macro Definition Documentation . . . 94
13.6.2.1 LOG_DIR . . . 94
13.6.2.2 LOG_FILE . . . 94
13.6.2.3 LOG_MAX_SIZE . . . 95
13.6.2.4 LOGGER_FORMAT_PRINTF . . . 95
13.6.3 Enumeration Type Documentation . . . 95
13.6.3.1 LogLevel . . . 95
13.6.4 Function Documentation . . . 96
13.6.4.1 log_console() . . . 96
13.6.4.2 log_debug() . . . 96
13.6.4.3 log_error() . . . 96
13.6.4.4 log_fatal() . . . 97
13.6.4.5 log_info() . . . 97
13.6.4.6 log_init() . . . 97
13.6.4.7 log_level_to_string() . . . 98
13.6.4.8 log_message() . . . 98
13.6.4.9 log_message_extended() . . . 98
13.6.4.10 log_set_log_level() . . . 99
13.6.4.11 log_warn() . . . 99
13.7 Platform Abstraction . . . 99
13.7.1 Detailed Description . . . 100
13.7.2 Function Documentation . . . 100
13.7.2.1 iz_platform_cpu_cores_count() . . . 100
13.7.2.2 iz_platform_create_dir() . . . 100
13.7.2.3 iz_platform_fill_random() . . . 101
13.7.2.4 iz_platform_l2_cache_size_bits() . . . 101
13.7.2.5 iz_platform_localtime() . . . 101
13.7.2.6 iz_platform_monotonic_seconds() . . . 102
13.8 Tests and Benchmarks . . . 102
13.8.1 Detailed Description . . . 103

13.8.2 Typedef Documentation . . . 103
13.8.2.1 SIEVE_FN . . . 103
13.9 Utilities . . . 103
13.9.1 Detailed Description . . . 104
13.9.2 Macro Definition Documentation . . . 104
13.9.2.1 DIR_output . . . 104
13.9.2.2 MAX . . . 105
13.9.2.3 MAX_CORES . . . 105
13.9.2.4 MIN . . . 105
13.9.3 Function Documentation . . . 106
13.9.3.1 create_dir() . . . 106
13.9.3.2 gcd() . . . 106
13.9.3.3 gcd_mpz() . . . 107
13.9.3.4 get_cpu_cores_count() . . . 107
13.9.3.5 get_cpu_L2_cache_size_bits() . . . 108
13.9.3.6 gmp_seed_randstate() . . . 108
13.9.3.7 is_numeric_str() . . . 109
13.9.3.8 lcm() . . . 109
13.9.3.9 lcm_mpz() . . . 110
13.9.3.10 modular_inverse() . . . 110
13.9.3.11 parse_inclusive_range_mpz() . . . 111
13.9.3.12 parse_numeric_expr_mpz() . . . 112
13.9.3.13 parse_numeric_expr_u64() . . . 113
13.10 Printer, implemented in toolkit/print_utils.c . . . 113
13.10.1 Detailed Description . . . 114
13.10.2 Function Documentation . . . 114
13.10.2.1 print_centered_text() . . . 114
13.10.2.2 print_line() . . . 115
13.10.2.3 print_sha256_hash() . . . 115
13.10.2.4 print_test_fn_header() . . . 115
13.10.2.5 print_test_module_header() . . . 116
13.10.2.6 print_test_module_result() . . . 116
13.10.2.7 print_test_summary() . . . 117
13.10.2.8 print_test_table_header() . . . 118
13.11 Stopwatch . . . 118
13.11.1 Detailed Description . . . 119
13.11.2 Function Documentation . . . 119
13.11.2.1 sw_elapsed_now_seconds() . . . 119
13.11.2.2 sw_elapsed_seconds() . . . 119
13.11.2.3 sw_start() . . . 119
13.11.2.4 sw_stop() . . . 120

14 Data Structure Documentation 121

14.1 BITMAP Struct Reference . . . 121
14.1.1 Detailed Description . . . 121
14.1.2 Field Documentation . . . 121
14.1.2.1 byte_size . . . 121
14.1.2.2 data . . . 122
14.1.2.3 sha256 . . . 122
14.1.2.4 size . . . 122
14.2 CLI_BINARY_EXPR_OPTIONS Struct Reference . . . 122
14.2.1 Detailed Description . . . 122
14.2.2 Field Documentation . . . 123
14.2.2.1 a_expr . . . 123
14.2.2.2 b_expr . . . 123
14.3 CLI_COMMAND Struct Reference . . . 123
14.3.1 Detailed Description . . . 123
14.3.2 Field Documentation . . . 123
14.3.2.1 handler . . . 123
14.3.2.2 name . . . 123
14.4 CLI_RANGE Struct Reference . . . 124
14.4.1 Detailed Description . . . 124
14.4.2 Field Documentation . . . 124
14.4.2.1 lower . . . 124
14.4.2.2 range_size . . . 124
14.4.2.3 upper . . . 124
14.5 CLI_SIEVE_MODEL Struct Reference . . . 124
14.5.1 Detailed Description . . . 124
14.5.2 Field Documentation . . . 125
14.5.2.1 fn . . . 125
14.5.2.2 name . . . 125
14.6 INPUT_SIEVE_RANGE Struct Reference . . . 125
14.6.1 Detailed Description . . . 125
14.6.2 Field Documentation . . . 126
14.6.2.1 filepath . . . 126
14.6.2.2 mr_rounds . . . 126
14.6.2.3 range . . . 126
14.6.2.4 start . . . 126
14.6.2.5 stream_gaps . . . 126
14.7 IZM Struct Reference . . . 127
14.7.1 Detailed Description . . . 127
14.7.2 Field Documentation . . . 127
14.7.2.1 base_x5 . . . 127
14.7.2.2 base_x7 . . . 127
14.7.2.3 k_vx . . . 127
14.7.2.4 root_primes . . . 128

14.7.2.5 vx . . . 128
14.8 IZM_RANGE_INFO Struct Reference . . . 128
14.8.1 Detailed Description . . . 128
14.8.2 Field Documentation . . . 129
14.8.2.1 vx . . . 129
14.8.2.2 Xe . . . 129
14.8.2.3 Xs . . . 129
14.8.2.4 y_range . . . 129
14.8.2.5 Ye . . . 129
14.8.2.6 Ys . . . 130
14.8.2.7 Ze . . . 130
14.8.2.8 Zs . . . 130
14.9 IZP_U64_BUFFER Struct Reference . . . 130
14.9.1 Detailed Description . . . 131
14.9.2 Field Documentation . . . 131
14.9.2.1 data . . . 131
14.9.2.2 len . . . 131
14.10 SIEVE_LIMIT Struct Reference . . . 131
14.10.1 Detailed Description . . . 131
14.10.2 Field Documentation . . . 132
14.10.2.1 base . . . 132
14.10.2.2 exp . . . 132
14.11 SIEVE_MODEL Struct Reference . . . 132
14.11.1 Detailed Description . . . 132
14.11.2 Field Documentation . . . 132
14.11.2.1 function . . . 132
14.11.2.2 name . . . 133
14.12 STOPWATCH Struct Reference . . . 133
14.12.1 Detailed Description . . . 133
14.12.2 Field Documentation . . . 133
14.12.2.1 elapsed_sec . . . 133
14.12.2.2 end_time . . . 133
14.12.2.3 running . . . 134
14.12.2.4 start_time . . . 134
14.13 STREAM_CMD_OPTIONS Struct Reference . . . 134
14.13.1 Detailed Description . . . 134
14.13.2 Field Documentation . . . 134
14.13.2.1 has_range . . . 134
14.13.2.2 mr_rounds . . . 135
14.13.2.3 print_gaps . . . 135
14.13.2.4 print_to_console . . . 135
14.13.2.5 range . . . 135
14.13.2.6 stream_path . . . 135

14.14 UI16_ARRAY Struct Reference . . . 135
14.14.1 Detailed Description . . . 136
14.14.2 Field Documentation . . . 136
14.14.2.1 array . . . 136
14.14.2.2 capacity . . . 136
14.14.2.3 count . . . 136
14.14.2.4 ordered . . . 136
14.14.2.5 sha256 . . . 136
14.15 UI32_ARRAY Struct Reference . . . 137
14.15.1 Detailed Description . . . 137
14.15.2 Field Documentation . . . 137
14.15.2.1 array . . . 137
14.15.2.2 capacity . . . 137
14.15.2.3 count . . . 137
14.15.2.4 ordered . . . 138
14.15.2.5 sha256 . . . 138
14.16 UI64_ARRAY Struct Reference . . . 138
14.16.1 Detailed Description . . . 138
14.16.2 Field Documentation . . . 138
14.16.2.1 array . . . 138
14.16.2.2 capacity . . . 139
14.16.2.3 count . . . 139
14.16.2.4 ordered . . . 139
14.16.2.5 sha256 . . . 139
14.17 VX_SEG Struct Reference . . . 139
14.17.1 Detailed Description . . . 140
14.17.2 Field Documentation . . . 140
14.17.2.1 bit_ops . . . 140
14.17.2.2 end_x . . . 140
14.17.2.3 is_large_limit . . . 140
14.17.2.4 mr_rounds . . . 141
14.17.2.5 p_count . . . 141
14.17.2.6 p_gaps . . . 141
14.17.2.7 p_test_ops . . . 141
14.17.2.8 root_limit . . . 141
14.17.2.9 start_x . . . 142
14.17.2.10 vx . . . 142
14.17.2.11 x5 . . . 142
14.17.2.12 x7 . . . 142
14.17.2.13 y . . . 142
14.17.2.14 yvx . . . 142

15 File Documentation 143

15.1 bitmap.h File Reference . . . 143
15.1.1 Detailed Description . . . 144
15.2 bitmap.h . . . 144
15.3 cli.h File Reference . . . 145
15.3.1 Detailed Description . . . 145
15.3.2 Function Documentation . . . 145
15.3.2.1 cli_run() . . . 145
15.4 cli.h . . . 146
15.5 int_arrays.h File Reference . . . 146
15.5.1 Detailed Description . . . 148
15.6 int_arrays.h . . . 148
15.7 iZ_api.h File Reference . . . 149
15.7.1 Detailed Description . . . 151
15.8 iZ_api.h . . . 151
15.9 iZ_toolkit.h File Reference . . . 152
15.9.1 Detailed Description . . . 154
15.10 iZ_toolkit.h . . . 154
15.11 izprime_ffi.h File Reference . . . 155
15.11.1 Detailed Description . . . 156
15.11.2 Macro Definition Documentation . . . 157
15.11.2.1 IZP_FFI_API . . . 157
15.12 izprime_ffi.h . . . 157
15.13 logger.h File Reference . . . 158
15.13.1 Detailed Description . . . 159
15.14 logger.h . . . 160
15.15 platform.h File Reference . . . 161
15.15.1 Detailed Description . . . 161
15.15.2 Macro Definition Documentation . . . 161
15.15.2.1 IZ_PLATFORM_HAS_FORK . . . 161
15.15.2.2 IZ_PLATFORM_POSIX . . . 162
15.15.2.3 IZ_PLATFORM_WINDOWS . . . 162
15.16 platform.h . . . 162
15.17 test_api.h File Reference . . . 162
15.17.1 Detailed Description . . . 163
15.18 test_api.h . . . 164
15.19 utils.h File Reference . . . 164
15.19.1 Detailed Description . . . 166
15.20 utils.h . . . 166
15.21 cli_app.c File Reference . . . 167
15.21.1 Detailed Description . . . 169
15.21.2 Macro Definition Documentation . . . 169
15.21.2.1 CLI_STRING_CAP . . . 169
15.21.2.2 IZ_VERSION . . . 169

15.21.3 Typedef Documentation . . . 170
15.21.3.1 CLI_HANDLER . . . 170
15.21.3.2 MPZ_BINARY_OP . . . 170
15.21.3.3 SIEVE_FN . . . 170
15.21.4 Enumeration Type Documentation . . . 170
15.21.4.1 STREAM_PARSE_RESULT . . . 170
15.21.5 Function Documentation . . . 170
15.21.5.1 cli_run() . . . 170
15.21.5.2 dup_cli_string() . . . 171
15.21.5.3 parse_binary_mpz_args() . . . 171
15.21.5.4 parse_cli_range() . . . 171
15.21.5.5 parse_command() . . . 171
15.21.5.6 parse_cores_value() . . . 171
15.21.5.7 parse_expr_int() . . . 171
15.21.5.8 parse_expr_u64() . . . 172
15.21.5.9 parse_stream_output_option() . . . 172
15.21.5.10 parse_stream_primes_args() . . . 172
15.21.5.11 parse_stream_primes_option() . . . 172
15.21.5.12 parse_stream_range_option() . . . 172
15.21.5.13 parse_stream_rounds_option() . . . 173
15.21.5.14 print_benchmark_help() . . . 173
15.21.5.15 print_count_help() . . . 173
15.21.5.16 print_gcd_help() . . . 173
15.21.5.17 print_general_help() . . . 173
15.21.5.18 print_is_prime_help() . . . 173
15.21.5.19 print_lcm_help() . . . 173
15.21.5.20 print_next_prime_help() . . . 174
15.21.5.21 print_prev_prime_help() . . . 174
15.21.5.22 print_stream_help() . . . 174
15.21.5.23 print_test_help() . . . 174
15.21.5.24 read_cli_option_value() . . . 174
15.21.5.25 resolve_stream_path() . . . 174
15.21.5.26 run_benchmark_cmd() . . . 175
15.21.5.27 run_binary_mpz_cmd() . . . 175
15.21.5.28 run_count_primes_cmd() . . . 175
15.21.5.29 run_directional_prime_cmd() . . . 175
15.21.5.30 run_doctor_cmd() . . . 175
15.21.5.31 run_gcd_cmd() . . . 175
15.21.5.32 run_is_prime_cmd() . . . 176
15.21.5.33 run_lcm_cmd() . . . 176
15.21.5.34 run_next_prime_cmd() . . . 176
15.21.5.35 run_prev_prime_cmd() . . . 176
15.21.5.36 run_stream_primes_cmd() . . . 176

15.21.5.37 run_test_cmd() . . . 176
15.21.5.38 stream_read_option_value() . . . 177
15.21.5.39 validate_stream_primes_options() . . . 177
15.21.6 Variable Documentation . . . 177
15.21.6.1 k_commands . . . 177
15.21.6.2 k_sieve_models . . . 177
15.21.6.3 k_sieve_models_count . . . 178
15.22 cli_app.c . . . 178
15.23 izprime_ffi.c File Reference . . . 191
15.23.1 Detailed Description . . . 192
15.23.2 Macro Definition Documentation . . . 193
15.23.2.1 IZP_FFI_BUILD . . . 193
15.23.2.2 IZP_FFI_ERROR_CAP . . . 193
15.23.2.3 IZP_MAX_SIEVE_LIMIT . . . 193
15.23.3 Function Documentation . . . 193
15.23.3.1 izp_ffi_copy_mpz_to_string() . . . 193
15.23.3.2 izp_ffi_count_small_range() . . . 193
15.23.3.3 izp_ffi_export_primes() . . . 193
15.23.3.4 izp_ffi_set_error() . . . 194
15.23.3.5 izp_ffi_strdup_heap() . . . 194
15.23.3.6 izp_ffi_validate_expr_nonnegative() . . . 194
15.23.4 Variable Documentation . . . 194
15.23.4.1 g_izp_ffi_last_error . . . 194
15.24 izprime_ffi.c . . . 194
15.25 iZ_apps.c File Reference . . . 200
15.25.1 Detailed Description . . . 201
15.25.2 Macro Definition Documentation . . . 201
15.25.2.1 PRIME_STR_CAP_PADDING . . . 201
15.25.3 Function Documentation . . . 201
15.25.3.1 __attribute__() . . . 201
15.25.4 Variable Documentation . . . 201
15.25.4.1 iZmX . . . 201
15.26 iZ_apps.c . . . 202
15.27 playground.c File Reference . . . 213
15.27.1 Detailed Description . . . 213
15.28 playground.c . . . 213
15.29 prime_sieve.c File Reference . . . 213
15.29.1 Detailed Description . . . 214
15.29.2 Macro Definition Documentation . . . 215
15.29.2.1 ASSERT_LIMIT . . . 215
15.29.2.2 N_LIMIT . . . 215
15.29.2.3 Pi . . . 215
15.29.3 Function Documentation . . . 215

15.29.3.1 process_N_bitmap() . . . 215
15.29.3.2 push_small_primes() . . . 216
15.29.4 Variable Documentation . . . 216
15.29.4.1 base_primes . . . 216
15.29.4.2 base_primes_count . . . 216
15.30 prime_sieve.c . . . 216
15.31 bitmap.c File Reference . . . 224
15.31.1 Detailed Description . . . 225
15.31.1.1 Implementation Notes . . . 225
15.31.1.2 Performance Characteristics . . . 226
15.32 bitmap.c . . . 226
15.33 int_arrays.c File Reference . . . 230
15.33.1 Detailed Description . . . 231
15.33.1.1 Implementation Notes . . . 231
15.33.1.2 Performance Characteristics . . . 231
15.34 int_arrays.c . . . 232
15.35 iZ_toolkit.c File Reference . . . 232
15.35.1 Detailed Description . . . 234
15.35.2 Function Documentation . . . 234
15.35.2.1 compute_k_vx() . . . 234
15.35.2.2 gmp_random_below() . . . 235
15.35.2.3 vx_det_sieve() . . . 235
15.35.2.4 vx_prob_sieve() . . . 235
15.35.2.5 vx_set_base_values() . . . 235
15.35.3 Variable Documentation . . . 236
15.35.3.1 s_primes . . . 236
15.35.3.2 s_primes_count . . . 236
15.36 iZ_toolkit.c . . . 237
15.37 logger.c File Reference . . . 248
15.37.1 Detailed Description . . . 248
15.38 logger.c . . . 248
15.39 platform.c File Reference . . . 251
15.39.1 Detailed Description . . . 251
15.40 platform.c . . . 252
15.41 print_utils.c File Reference . . . 253
15.41.1 Detailed Description . . . 254
15.41.2 Macro Definition Documentation . . . 254
15.41.2.1 IZ_DIAG_IGNORE_FORMAT_NONLITERAL . . . 254
15.41.2.2 IZ_DIAG_POP . . . 254
15.41.2.3 IZ_DIAG_PUSH . . . 254
15.42 print_utils.c . . . 255
15.43 stopwatch.c File Reference . . . 256
15.43.1 Detailed Description . . . 256

15.44 stopwatch.c . . . . . 257
15.45 utils.c File Reference . . . . . 257
15.45.1 Detailed Description . . . . . 258
15.46 utils.c . . . . . 258

Index 267

# Chapter 1

# iZprime User Manual

iZprime is a C library for prime sieving, range counting/streaming, and random-prime generation.

## 1.1 What this manual covers

- Public API (include/iZ_api.h)
- FFI API for language bindings (include/izprime_ffi.h)
- Core toolkit types and internals (include/iZ_toolkit.h)
- Utility modules and shared data structures

## 1.2 Related project docs

- Algorithm pseudocode: docs/pseudocode.pdf
- CLI usage: docs/cli.md
- Language bindings/FFI: docs/bindings.md
- Testing guide: docs/tests.md
- Benchmark guide and results: docs/benchmarks.md
- Build system reference: docs/Makefile.md

# Chapter 2

# Benchmarks

This document captures benchmark methodology, current result snapshots, and plotting usage for the iZprime project.

## 2.1 1. Run Benchmarks

From repository root:

```
make benchmark-p_sieve
make benchmark-p_gen
make benchmark-SiZ_count
```

Save results into docs/test_results/:

```
make benchmark-p_sieve save-results
make benchmark-p_gen save-results
make benchmark-SiZ_count save-results
```

Save results and generate plot artifacts automatically:

```
make benchmark-p_sieve plot
make benchmark-p_gen plot
```

Notes:

- plot implies save-results.
- Dash-style flags also work with GNU make passthrough:

```
make -- benchmark-p_sieve --save-results --plot
make -- benchmark-p_gen --save-results --plot
make -- benchmark-SiZ_count --save-results
```

Note: plotting is currently wired for p_sieve and p_gen result formats.

## 2.2 2. Result Files Used In This Snapshot

Sieve benchmark snapshot:

- docs/test_results/psieve_10093442.txt

Prime generation benchmark snapshots:

- docs/test_results/p_gen_12094122.txt (1024-bit)
- docs/test_results/p_gen_12094126.txt (2048-bit)
- docs/test_results/p_gen_12094224.txt (4096-bit)

SiZ_count benchmark snapshot:

- docs/test_results/SiZ_count_19205134.txt

## 2.3 3. Sieve Benchmark Table and Plot

Times are in microseconds (us) from docs/test_results/psieve_10093442.txt.

| Algorithm | 10^4 | 10^5 | 10^6 | 10^7 | 10^8 | 10^9 | 10^10 |
|---|---|---|---|---|---|---|---|
| SoE | 14 | 130 | 1136 | 28874 | 136817 | 2510303 | 29944939 |
| SSoE | 21 | 230 | 1826 | 16351 | 148737 | 1393887 | 13312626 |
| SoEu | 18 | 198 | 1567 | 14153 | 139020 | 1793877 | 18707288 |
| SoS | 14 | 159 | 1395 | 14393 | 142700 | 2414564 | 28599960 |
| SoA | 24 | 197 | 1645 | 14909 | 145648 | 3600075 | 46331079 |
| SiZ | 9 | 119 | 912 | 8924 | 87100 | 1436860 | 18074650 |
| SiZm | 12 | 139 | 931 | 8825 | 73872 | 709836 | 6917661 |
| SiZm_vy | 25 | 178 | 1061 | 7997 | 68305 | 684938 | 6385593 |

## 2.4 4. Prime Generation Benchmark Table

Average times are in seconds for each bit size from the corresponding docs/test_results/p_gen_∗.txt files.

| Algorithm | 1024 | 2048 | 4096 |
|---|---|---|---|
| vy_random_prime | 0.021182 | 0.063515 | 3.181408 |
| vx_random_prime | 0.008107 | 0.082872 | 1.220519 |
| iZ_random_next_prime | 0.023231 | 0.150618 | 0.578747 |
| gmp_random_next_prime | 0.009421 | 0.071797 | 1.666787 |
| BN_generate_prime_ex | 0.026731 | 0.422709 | 4.959384 |

## 2.5 5. Plot Artifacts

Current plot files:

- docs/test_results/p_gen_12094122_avg.svg
- docs/test_results/p_gen_12094126_avg.svg
- docs/test_results/p_gen_12094224_avg.svg

### 2.5.1 Prime generation plots

## 2.6 6. SiZ_count Benchmark Table

Times are in seconds from docs/test_results/SiZ_count_19205134.txt.

Window size: 10^9 integers for each starting point.

| Start Point | Prime Count | Time (s) |
|---|---|---|
| 10^10 | 43336106 | 0.157103 |

| Start Point | Prime Count | Time (s) |
|---:|---:|---:|
| 10^20 | 21710426 | 45.271972 |
| 10^30 | 14473890 | 50.446129 |
| 10^40 | 10860250 | 75.721432 |
| 10^50 | 8684378 | 83.009872 |
| 10^60 | 7237481 | 107.691231 |
| 10^70 | 6205976 | 119.342140 |
| 10^80 | 5428021 | 167.147788 |
| 10^90 | 4825182 | 178.556225 |
| 10^100 | 4342289 | 218.545428 |

## 2.7 7. Plot From Terminal (Interactive Prompt)

You can generate/view plots directly from terminal. If no filepath is passed, scripts prompt for one:

```
python3 py_tools/plot_results.py
python3 py_tools/plot_p_sieve_results.py
python3 py_tools/plot_p_gen_results.py
```

Or pass a file explicitly:

```
python3 py_tools/plot_results.py output/psieve_10093442.txt --save
python3 py_tools/plot_results.py output/p_gen_12094224.txt --avg --save
```

# Chapter 3

# Language Bindings Strategy (FFI)

This project exposes language bindings through a stable C-ABI layer in:

- include/izprime_ffi.h
- src/ffi/izprime_ffi.c

Concrete wrappers live under bindings/:

- bindings/python (ctypes)
- bindings/rust (extern "C" safe wrapper crate)
- bindings/go (cgo)
- bindings/node (ffi-napi)

This avoids binding directly to CLI output, which is slower and brittle for long-term integrations.

## 3.1 Why API-layer bindings (not CLI)

- Typed arguments/results instead of parsing terminal text.
- Lower overhead (no process spawn for each operation).
- Deterministic memory ownership rules for foreign runtimes.
- Stable function signatures that can be versioned independent of CLI UX.

## 3.2 FFI API surface (v1)

- izp_ffi_sieve_u64: run a selected sieve model up to n <= 10^12, return all primes as a uint64_t buffer.
- izp_ffi_count_range: count primes in [start, start + range - 1].
- izp_ffi_stream_range: stream primes (or gaps) in a range to a file.
- izp_ffi_next_prime: next/previous prime from a base expression.
- izp_ffi_random_prime_vx, izp_ffi_random_prime_vy: random prime generation wrappers.
- izp_ffi_version, izp_ffi_last_error, izp_ffi_status_message: runtime/version/error helpers.

## 3.3 Ownership model

Returned buffers/strings are heap-owned by the library and must be released via:

- izp_ffi_free_u64_buffer
- izp_ffi_free_string

Do not free returned pointers directly from language runtimes.

## 3.4 Expression support

start and base expression inputs use the same parser as CLI/API (+ - * / ^ e with parentheses and grouped decimals).

## 3.5 Recommended binding rollout

1. Python (ctypes or cffi) for research/data workflows.
2. Rust (bindgen + safe wrapper crate).
3. Node.js (N-API wrapper over C FFI).
4. Go (cgo package with thin idiomatic wrappers).

## 3.6 Notes for wrapper authors

- Treat all FFI return values as status codes (IZP_FFI_STATUS).
- On non-IZP_FFI_OK, call izp_ffi_last_error() for details.
- Keep wrapper-level APIs idiomatic, but keep FFI calls one-to-one and thin.

# Chapter 4

# izprime CLI

The CLI binary is izprime and exposes task-oriented subcommands over the library API.

## 4.1 Build

```
make cli
./build/bin/izprime help
```

## 4.2 Numeric input syntax

Commands that accept numbers support:

- decimal integers: 1000000
- grouped decimals: 1,000,000
- arithmetic expressions with + - ∗ /
- powers: 10^6
- scientific shorthand: 1e6, 10e100
- parenthesized expressions: (10^6 + 5) ∗ 7

## 4.3 Commands

## 4.4 stream_primes

Streams primes in an inclusive range using SiZ_stream.

izprime stream_primes --range "[LOWER, UPPER]" [--print | --stream-to FILE] [--print-gaps] [--mr-rounds N]

Examples:

```
izprime stream_primes --range "[0, 10^5]" --print
izprime stream_primes --range "[0, 10^5]" --print-gaps
izprime stream_primes --range "[1,000,000, 1,001,000]" --stream-to output/range.txt
```

Alias: sieve.

## 4.5 count_primes

Counts primes in an inclusive range using SiZ_count.

izprime count_primes --range "[LOWER, UPPER]" [--cores N|max] [--mr-rounds N]

Examples:

izprime count_primes --range "[0, 10^9]" --cores 8
izprime count_primes --range "10^100 + 10^9, 10^100 + 10^9 + 10^9" --cores max

Alias: count.

Notes:

- --cores accepts an integer value (>= 1) or the literal max.
- --cores-number is still accepted as a backward-compatible alias.
- On platforms without fork, multi-process mode is unavailable and execution falls back to single-process mode.

## 4.6 next_prime

Finds the next prime after n using iZ_next_prime.

izprime next_prime --n VALUE

Example:

izprime next_prime --n "10^12 + 39"

## 4.7 prev_prime

Finds the previous prime before n using iZ_next_prime in reverse mode.

izprime prev_prime --n VALUE

Example:

izprime prev_prime --n "10^12 + 39"

Alias: prev.

## 4.8 is_prime

Checks primality of n using test_primality.

izprime is_prime --n VALUE [--rounds N]

Example:

izprime is_prime --n "10^61 + 1" --rounds 40

## 4.9 gcd

Computes the greatest common divisor of two arbitrary-size integers.

```
izprime gcd --a VALUE --b VALUE
izprime gcd VALUE_A VALUE_B
```

Examples:

```
izprime gcd --a "10^100 + 12" --b "10^80 + 18"
izprime gcd "10^6" "10^4"
```

Notes:

- Use either named operands (--a, --b) or two positional expressions.
- Mixed forms are rejected to keep parsing unambiguous.
- The command prints the result in base-10 to stdout.

## 4.10 lcm

Computes the least common multiple of two arbitrary-size integers.

```
izprime lcm --a VALUE --b VALUE
izprime lcm VALUE_A VALUE_B
```

Examples:

```
izprime lcm --a "48" --b "18"
izprime lcm "10^3" "6 * 10^2"
```

Notes:

- Use either named operands (--a, --b) or two positional expressions.
- Mixed forms are rejected to keep parsing unambiguous.
- The command prints the result in base-10 to stdout.

## 4.11 Existing maintenance commands

```
izprime test [--limit N]
izprime benchmark [--limit N] [--repeat N] [--algo NAME|all] [--save-results FILE]
izprime doctor
```

Use izprime help <command> for command-specific help.

# Chapter 5

# Contributing to iZprime

Thanks for contributing. This project is prepared for research-grade publication, so we hold changes to a strict quality bar.

## 5.1 Scope

Contributions are welcome for:

- correctness fixes,
- performance improvements,
- documentation quality,
- tests/benchmarks and tooling,
- new algorithms that align with the iZprime design.

## 5.2 Development Setup

Required dependencies:

- C compiler (GCC/Clang),
- make,
- GMP,
- OpenSSL.

Optional:

- pkg-config,
- Doxygen,
- Python packages in py_tools/requirements.txt for plots.

## 5.3 Code Style

- Keep modules focused and composable.
- Prefer explicit ownership and cleanup paths.
- Keep public API declarations in include/, implementations in src/.
- Document new public structs/functions with Doxygen comments.
- Avoid introducing hidden global state.

## 5.4 Validation Requirements

Before opening a pull request:

```
make clean
make test-unit
make test-integration
make -- test-all --verbose
```

If you touched performance-sensitive code, also run:

```
make benchmark-p_sieve save-results
make benchmark-p_gen save-results
make benchmark-SiZ_count save-results
```

If you changed API/docs:

```
make userManual
```

## 5.5 Benchmarking Expectations

- Include the exact command(s) used.
- Include hardware/OS/compiler context.
- Share result files from output/ when relevant.
- Prefer reporting both absolute timing and relative speedup/regression.

## 5.6 Pull Request Checklist

1. Explain the problem and the technical approach.
2. List behavioral changes and compatibility impact.
3. Link tests that validate the change.
4. Add or update benchmark evidence for performance claims.
5. Update docs (README.md, docs/∗.md, Doxygen comments) when needed.

## 5.7 Algorithm Contributions

For new sieve/generation techniques:

- map design to existing toolkit abstractions (IZM, VX_SEG, bitmaps, arrays),
- provide pseudocode-level explanation in docs/pseudocode.pdf source pipeline,
- add integration tests versus trusted baselines,
- add benchmark hooks so users can compare against existing models.

## 5.8 Review Standard

PRs are reviewed for:

- correctness and edge-case behavior,
- memory/process safety,
- API clarity and consistency,
- test coverage,
- reproducibility of benchmark claims.

# Chapter 6

# Makefile documentation (iZprime)

This document describes the modular make workflow for this repository.

## 6.1 Quick start

- Build static library + CLI and run:
  - make
- Build library only:
  - make lib
- Check dependencies:
  - make doctor
- Create release source archive:
  - make dist VERSION=1.0.0
- Run unit + integration tests:
  - make test-all
- Run sieve benchmarks:
  - make benchmark-p_sieve
- Run prime-generation benchmarks:
  - make benchmark-p_gen
- Build pseudocode PDF:
  - make pseudocode

## 6.2 Dependencies

- C toolchain: gcc or clang
- Build tool: make
- Libraries:
  - GMP (-lgmp)
  - OpenSSL 3 (-lssl -lcrypto)

The Makefile prefers pkg-config for portability; on macOS it also supports Homebrew path fallbacks.

## 6.3 Targets

### 6.3.1 Build / run

- all (default): Builds the CLI and runs it
- lib: Builds build/lib/libizprime.a and shared-library artifacts (default)
- cli: Builds build/bin/izprime (or PROGRAM override)
- run: Runs the CLI binary (builds first if needed)
- debug: Builds with debug flags (-O0 -g) and logging enabled
- release: Builds an optimized binary and disables logging
- clean: Removes build artifacts and generated output/log directories
- help: Prints a concise help summary of targets and options
- userManual: Generates docs/userManual.pdf via Doxygen + LaTeX
- pseudocode: Generates docs/pseudocode.pdf from docs/latex_src/pseudocode.tex
- install: Installs headers, static/shared libraries, CLI, and pkg-config file
- install-lib: Installs headers, static/shared libraries, and pkg-config file
- uninstall: Removes files installed by the targets above
- doctor: Verifies build dependencies with a compile/link smoke test
- dist: Builds source tarball and SHA256 checksum under dist/

### 6.3.2 Tests

The primary test runner is built at build/test/test_runner. It links against the static library target (build/lib/libizprime.a).

- test-all: Runs unit + integration tests
- test-unit: Runs unit tests only
- test-integration: Runs integration tests only

Notes:

- Test selection defaults to --unit --integration when no flags are provided.
- Test runner arguments can be extended via TEST_ARGS.

### 6.3.3 Benchmarks

Benchmarks are intentionally separate from tests, so they can be invoked explicitly and kept out of typical CI workflows.

- benchmark-p_sieve: Runs prime sieve model benchmarks
  - Invokes: ./build/test/test_runner --benchmark-p-sieve ...
- benchmark-p_gen: Runs random prime generation benchmarks
  - Invokes: ./build/test/test_runner --benchmark-p-gen ...
- benchmark-SiZ_count: Runs large-window SiZ_count benchmark
  - Invokes: ./build/test/test_runner --benchmark-siz-count ...

### 6.3.4 Examples

- examples: Builds all example programs into build/examples/
- run-example: Runs one example
  - Required variables:
    * EX=<example_name> (name of the example source without extension)
    * ARGS="..." (optional arguments passed to the example)

Example:

- make run-example EX=sieve_primes ARGS="SiZm 10000000 10"

## 6.4 Configuration variables

### 6.4.1 Local configuration file

- config.mk (optional, local)
  - If present, it overrides defaults in Makefile.
  - Start from:
    * cp config.mk.example config.mk

### 6.4.2 Dependency discovery

- USE_PKG_CONFIG (default: 1)
  - When 1, attempts to use pkg-config to discover GMP and OpenSSL.
  - When 0, skips pkg-config and uses the Homebrew/generic fallbacks.
- PKG_CONFIG (default: pkg-config)
  - Override if you need a specific pkg-config binary.

### 6.4.3 Build toggles

- LOGGING (default: 1)
  - When 1, compiles with -DENABLE_LOGGING.
  - The release target forces LOGGING=0.
- PROGRAM (default: izprime)
  - Output name for the CLI binary under build/bin/.
- CLI_ENTRY (default: src/main.c)
  - Source file used as CLI entrypoint (main()).
- VERSION (default: 1.0.0)
  - Release/version metadata for the CLI and shared library.
- SOVERSION (default: first component of VERSION)
  - Shared-library ABI major version.
- BUILD_SHARED (default: 1)
  - Build shared library artifacts in addition to static library.

### 6.4.4 Test/benchmark options

- VERBOSE (default: 0)
  - When 1, adds --verbose to the test runner.
- SAVE_RESULTS (default: 0)
  - When 1, adds --save-results to benchmark runs (where supported).
- TEST_OUTPUT (default: empty)
  - When set, redirects test runner output to the given file.
- TEST_ARGS (default: empty)
  - Extra arguments appended to the test runner invocation.

## 6.5 Common recipes

- Run integration tests with verbose output:
  - make test-integration VERBOSE=1
- Save benchmark results:
  - make benchmark-p_sieve SAVE_RESULTS=1
  - make benchmark-p_gen SAVE_RESULTS=1
  - make benchmark-SiZ_count SAVE_RESULTS=1
- Redirect test output to a file:
  - make test-all TEST_OUTPUT=output/test_all.txt

- Pass extra args to the test runner:
  - make test-all TEST_ARGS="--help"
- Install under a local prefix:
  - make install PREFIX=$HOME/.local
- Build static-only artifacts:
  - make lib BUILD_SHARED=0
- Create a release tarball:
  - make dist VERSION=1.0.0

## 6.6 Notes on outputs

- Build outputs live under build/.
- Benchmark/test logs can be redirected via TEST_OUTPUT.
- Some benchmarks may be long-running; use SAVE_RESULTS=1 to persist results when supported by that benchmark.

# Chapter 7

# Native Package Plan

This document tracks distro-native packaging targets for iZprime.

## 7.1 Targets

- macOS: Homebrew tap formula
- Debian/Ubuntu: .deb packages
- Fedora/RHEL: .rpm package
- Windows: MSYS2 build + portable ZIP + winget manifest

## 7.2 Metadata Locations

- Homebrew template: packaging/homebrew/izprime.rb.in
- Debian metadata: packaging/debian/
- RPM spec: packaging/rpm/izprime.spec
- Windows packaging templates: packaging/windows/

## 7.3 Package Split (Linux)

- izprime: CLI binary
- libizprime1: runtime shared library
- libizprime-dev: headers, static lib, pkg-config file

## 7.4 Release Flow

1. Tag release and publish source artifact + checksum.
2. Build platform-native packages from the release tarball.
3. Run post-install smoke checks:
   - izprime doctor
   - one CLI functional command (for example izprime count_primes --range "[0, 100000]")
4. Publish packages to distribution channel (tap/PPA/COPR/winget).

## 7.5 CI Gates

- Linux/macOS/Windows build and test matrix in .github/workflows/ci.yml.
- Homebrew tap auto-update on release tags in .github/workflows/update-homebrew-formula.yml.
- Minimum required checks per platform:
  - make doctor
  - make cli
  - make test-unit
  - make test-integration

# Chapter 8

# Releasing iZprime

This document defines the release workflow for v1.x and later.

## 8.1 Versioning

- Use SemVer tags: vMAJOR.MINOR.PATCH (example: v1.0.0).
- VERSION in Makefile should match the release tag.
- SOVERSION defaults to the major version and should change only on ABI breaks.

## 8.2 Pre-release Gate

Run this checklist from a clean working tree:

```
make clean
make test-unit
make test-integration
make benchmark-p_sieve save-results
make benchmark-p_gen save-results
make benchmark-SiZ_count save-results
make userManual
make dist VERSION=1.0.0
```

Confirm:

- generated docs/userManual.pdf opens correctly,
- benchmark outputs are saved in output/,
- dist/ contains tarball + checksum,
- release tarball does not include transient artifacts (build/, logs/, output/, .git/).

Windows readiness (recommended for each release):

- run CI builds on windows-latest (MSYS2/MinGW toolchain),
- run at least make doctor, make test-unit, and make test-integration,
- confirm fallback behavior for multi-process APIs on non-fork platforms.

Native package metadata is maintained under packaging/:

- packaging/debian/ (Debian/Ubuntu),
- packaging/rpm/izprime.spec (Fedora/RHEL),
- packaging/homebrew/ (Homebrew formula template),
- packaging/windows/ (portable + winget templates).

## 8.3 Changelog and Release Notes

ChangeLog should summarize user-visible changes.

Suggested generation workflow:

```
LAST_TAG=$(git describe --tags --abbrev=0 2>/dev/null || echo "")
if [ -n "$LAST_TAG" ]; then
  git log --pretty=format:"- %s (%h)" "$LAST_TAG"..HEAD
else
  git log --pretty=format:"- %s (%h)"
fi
```

Use the output as a draft, then curate into sections:

- Added
- Changed
- Fixed
- Performance
- Documentation

Release notes in GitHub/GitLab should include:

- highlights,
- migration notes (if any),
- validation commands used,
- benchmark artifacts summary.

## 8.4 Tag and Publish

```
git tag -a v1.0.0 -m "iZprime v1.0.0"
git push origin v1.0.0
```

Attach dist/izprime-<version>.tar.gz and checksum file to the release.

## 8.5 Post-release

- bump VERSION to next development version (for example 1.0.1-dev),
- update docs if commands/artifacts changed,
- track follow-up fixes as issues/milestone items.

# Chapter 9

# Tests

This document describes test targets, execution commands, and current pass snapshots.

## 9.1 1. Test Targets

From repository root:

```
make test-all
make test-unit
make test-integration
```

Verbose output:

```
make test-all verbose
make test-unit verbose
make test-integration verbose
```

GNU make passthrough form (dash flags):

```
make -- test-all --verbose
make -- test-unit --verbose
make -- test-integration --verbose
```

## 9.2 2. What Each Target Runs

- test-unit: bitmap/int-array/iZm/vx-seg module-level tests.
- test-integration: sieve hash integrity, range APIs, and prime-generation integration checks.
- test-all: unit + integration suites through the shared test runner.

## 9.3 3. Current Snapshot Results

Snapshot command set:

```
make test-unit
make test-integration
make test-all
```

Observed exit codes and summaries:

| Target | Exit code | Summary |
|---|---|---|
| make test-unit | 0 | 7/7 module groups passed (100.0%) |

| Target | Exit code | Summary |
|---|---|---|
| make test-integration | 0 | 6/6 integration groups passed (100.0%) |
| make test-all | 0 | full test runner completed successfully |

Expected success markers in output:

- Unit suite: [SUCCESS] ALL MODULE TESTS PASSED!
- Integration suite: [SUCCESS] ALL INTEGRATION TESTS PASSED!

## 9.4 4. Show Results In Terminal

Quick pass/fail runs:

```
make test-unit
make test-integration
```

Detailed run (prints per-test information and hashes):

```
make test-integration verbose
```

Capture logs for review:

```
make test-unit > /tmp/test_unit.log 2>&1
make test-integration > /tmp/test_integration.log 2>&1
```

Tail the summaries:

```
tail -n 40 /tmp/test_unit.log
tail -n 50 /tmp/test_integration.log
```

## 9.5 5. Notes

- The integrity suite intentionally skips SiZm_vy in hash equality checks because it emits unordered prime lists.
- The test runner exits non-zero when any group fails; make surfaces that as a target failure.

# Chapter 10

# Topic Index

## 10.1 Topics

Here is a list of all topics with brief descriptions:


Bitmap Module . . . . . 35
Integer Arrays . . . . . 47
iZ Public API . . . . . 50
Toolkit (iZ/iZm) . . . . . 65
FFI API . . . . . 85
Logging . . . . . 93
Platform Abstraction . . . . . 99
Tests and Benchmarks . . . . . 102
Utilities . . . . . 103
Printer, implemented in toolkit/print_utils.c . . . . . 113
Stopwatch . . . . . 118

# Chapter 11

# Data Structure Index

## 11.1 Data Structures

Here are the data structures with brief descriptions:


BITMAP
Packed bit-array with optional SHA-256 checksum . . . . . 121
CLI_BINARY_EXPR_OPTIONS . . . . . 122
CLI_COMMAND . . . . . 123
CLI_RANGE . . . . . 124
CLI_SIEVE_MODEL . . . . . 124
INPUT_SIEVE_RANGE
Input parameters for range sieving/counting . . . . . 125
IZM
Precomputed iZm assets for repeated VX-segment sieving . . . . . 127
IZM_RANGE_INFO
Precomputed iZm coordinates for an inclusive numeric interval . . . . . 128
IZP_U64_BUFFER
Owned buffer of uint64_t values returned by FFI sieve routines . . . . . 130
SIEVE_LIMIT . . . . . 131
SIEVE_MODEL . . . . . 132
STOPWATCH
Stopwatch state for elapsed wall-clock measurements . . . . . 133
STREAM_CMD_OPTIONS . . . . . 134
UI16_ARRAY
Dynamic array for uint16_t values . . . . . 135
UI32_ARRAY
Dynamic array for uint32_t values . . . . . 137
UI64_ARRAY
Dynamic array for uint64_t values . . . . . 138
VX_SEG
Runtime state for one VX segment at a specific y . . . . . 139

# Chapter 12

# File Index

## 12.1 File List

Here is a list of all documented files with brief descriptions:


bitmap.h
Bitmap container and bit operations used by sieve implementations . . . . . 143
cli.h
Public entrypoint for the iZprime CLI application layer . . . . . 145
int_arrays.h
Dynamic arrays for uint16_t, uint32_t, and uint64_t values . . . . . 146
iZ_api.h
Public API for prime sieving, range scans, and prime generation . . . . . 149
iZ_toolkit.h
Core iZ/iZm primitives used by SiZ-family algorithms . . . . . 152
izprime_ffi.h
Stable C-ABI surface for language bindings . . . . . 155
logger.h
Header file for the logging system . . . . . 158
platform.h
Cross-platform system abstraction utilities . . . . . 161
test_api.h
Public test/benchmark entry points for the iZ library . . . . . 162
utils.h
Shared utilities and common includes for the iZprime library . . . . . 164
cli_app.c
CLI application dispatcher and command handlers . . . . . 167
izprime_ffi.c
FFI-safe wrapper layer over the native iZprime API . . . . . 191
iZ_apps.c
High-level application routines built on top of the iZ_toolkit . . . . . 200
playground.c
Scratch space for testing new ideas . . . . . 213
prime_sieve.c
Implementations of classical and SiZ-family sieve algorithms . . . . . 213
bitmap.c
Implementation of the bitmap module for efficient bit array operations . . . . . 224
int_arrays.c
Implementation of dynamic integer array module for 16-bit, 32-bit, and 64-bit unsigned integers . . . . . 230

iZ_toolkit.c
Implementation of iZ index space helpers and iZm/VX segment machinery . . . . . 232
logger.c
Logging module . . . . . 248
platform.c
Platform abstraction implementation . . . . . 251
print_utils.c
Print formatting helpers for tests, benchmarks, and CLI output . . . . . 253
stopwatch.c
Stopwatch helpers based on monotonic wall-clock time . . . . . 256
utils.c
Implementations for the shared utility layer . . . . . 257

# Chapter 13

# Topic Documentation

## 13.1 Bitmap Module

Packed bit-array primitives for sieve and toolkit modules.

### Files

- file bitmap.c

  Implementation of the bitmap module for efficient bit array operations.

### Data Structures

- struct BITMAP

  Packed bit-array with optional SHA-256 checksum.

### Functions

- int TEST_BITMAP (int verbose)

  Run bitmap module tests.

### Lifecycle

- BITMAP * bitmap_init (size_t size, int set_bits)

  Allocate a bitmap with size bits.
- void bitmap_free (BITMAP **bitmap)

  Free a bitmap and set the caller pointer to NULL.
- BITMAP * bitmap_clone (BITMAP *src)

  Deep-copy a bitmap, including data and checksum.

Single-bit Operations

- int bitmap_get_bit (BITMAP *bitmap, size_t idx)

  Read bit value at idx.
- void bitmap_set_bit (BITMAP *bitmap, size_t idx)

  Set bit at idx to 1.
- void bitmap_flip_bit (BITMAP *bitmap, size_t idx)

  Toggle bit at idx.
- void bitmap_clear_bit (BITMAP *bitmap, size_t idx)

  Clear bit at idx (set to 0).

Bulk Operations

- void bitmap_set_all (BITMAP *bitmap)

  Set all bits to 1.
- void bitmap_clear_all (BITMAP *bitmap)

  Clear all bits to 0.
- void bitmap_clear_steps (BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)

  Clear every step-th bit from start_idx up to limit.
- void bitmap_clear_steps_simd (BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)

  SIMD-accelerated variant of bitmap_clear_steps().

Integrity and I/O

- void bitmap_compute_hash (BITMAP *bitmap)

  Compute SHA-256 over bitmap data and store it in BITMAP::sha256.
- int bitmap_validate_hash (BITMAP *bitmap)

  Verify BITMAP::sha256 against current bitmap data.
- int bitmap_fwrite (BITMAP *bitmap, FILE *file)

  Write bitmap payload and checksum to a binary stream.
- BITMAP * bitmap_fread (FILE *file)

  Read bitmap payload and checksum from a binary stream.

### 13.1.1 Detailed Description

Packed bit-array primitives for sieve and toolkit modules.

### 13.1.2 Function Documentation

#### 13.1.2.1 bitmap_clear_all()

```
void bitmap_clear_all (
            BITMAP * bitmap)
```

Clear all bits to 0.

Parameters

| | |
|---|---|
| bitmap | Bitmap to modify. |

Clear all bits to 0.

Parameters

| | |
|---|---|
| bitmap | A pointer to the BITMAP structure. |

Definition at line 180 of file bitmap.c.

References BITMAP::data, and BITMAP::size.

#### 13.1.2.2 bitmap_clear_bit()

```
void bitmap_clear_bit (
            BITMAP * bitmap,
            size_t idx)
```

Clear bit at idx (set to 0).

Parameters

| | |
|---|---|
| bitmap | Bitmap to modify. |
| idx | Zero-based bit index. |

Clear bit at idx (set to 0).

Parameters

| | |
|---|---|
| bitmap | The BITMAP to modify. |
| idx | The index of the bit to clear. |

Definition at line 159 of file bitmap.c.

References BITMAP::data.

Referenced by iZm_construct_vx_base(), SoEu(), SSoS(), vx_prob_sieve(), and vx_stream().

#### 13.1.2.3 bitmap_clear_steps()

```
void bitmap_clear_steps (
            BITMAP * bitmap,
            uint64_t step,
            uint64_t start_idx,
            uint64_t limit)
```

Clear every step-th bit from start_idx up to limit.

This is the core primitive for marking composite progressions in sieve code.

Parameters

| | |
|---|---|
| bitmap | Bitmap to modify. |

| | |
|---|---|
| step | Index increment between cleared bits. |
| start_idx | First index to clear. |
| limit | Inclusive upper index bound. |

Clear every step-th bit from start_idx up to limit.

Starting from start_idx, this function clears every step-th bit up to the specified limit (inclusive). This is commonly used in prime sieve algorithms where you need to mark all multiples of a number as composite.

The function includes:

- Automatic capping of limit to bitmap size
- Integer overflow protection in the loop
- Early termination to prevent infinite loops

Parameters

| | |
|---|---|
| bitmap | Pointer to the bitmap to modify |
| step | Interval between bits to clear (must be > 0, typically a prime) |
| start_idx | Starting bit index (inclusive, typically first multiple) |
| limit | Upper bound for clearing (inclusive, auto-capped to size-1) |

Note

If limit > bitmap->size, it is automatically reduced to bitmap->size - 1

No operation if bitmap is NULL, step is 0, or start_idx > limit

Includes overflow prevention: stops if next index would overflow

```
// For Sieve of Eratosthenes: mark multiples of 3 starting from 9
bitmap_clear_steps(bitmap, 3, 9, 1000);  // Clears 9, 12, 15, 18, ...
```

Definition at line 211 of file bitmap.c.

References BITMAP::data, MIN, and BITMAP::size.

Referenced by vx_search_prime().

#### 13.1.2.4 bitmap_clear_steps_simd()

```
void bitmap_clear_steps_simd (
            BITMAP * bitmap,
            uint64_t step,
            uint64_t start_idx,
            uint64_t limit)
```

SIMD-accelerated variant of bitmap_clear_steps().

Parameters

| | |
|---|---|
| bitmap | Bitmap to modify. |

| | |
|---|---|
| step | Index increment between cleared bits. |
| start_idx | First index to clear. |
| limit | Inclusive upper index bound. |

SIMD-accelerated variant of bitmap_clear_steps().

This function uses SIMD instructions (AVX2, SSE2, or NEON) to clear bits at regular intervals in the bitmap. It processes multiple bits in parallel for improved performance on large bitmaps.

Parameters

| | |
|---|---|
| bitmap | Pointer to the bitmap to modify |
| step | Interval between bits to clear (must be > 0) |
| start_idx | Starting bit index (inclusive) |
| limit | Upper bound for clearing (inclusive) |

Definition at line 236 of file bitmap.c.

References BITMAP::data, MIN, and BITMAP::size.

Referenced by iZm_construct_vx_base(), process_iZ_bitmaps(), process_N_bitmap(), SiZm(), SiZm_vy(), SoA(), SoS(), SSoE(), SSoS(), and vx_det_sieve().

#### 13.1.2.5 bitmap_clone()

```
BITMAP * bitmap_clone (
            BITMAP * src)
```

Deep-copy a bitmap, including data and checksum.

Parameters

| | |
|---|---|
| src | Source bitmap. |

Returns

Cloned bitmap, or NULL on failure.

Deep-copy a bitmap, including data and checksum.

Allocates a new bitmap with the same size as the source and copies all data including the bit array and SHA-256 hash. The returned bitmap is completely independent of the source and must be freed separately.

Parameters

| | |
|---|---|
| src | Pointer to the source bitmap to clone |

Returns

Pointer to newly allocated cloned BITMAP, or NULL on failure

Return values

| | |
|---|---|
| NULL | if src is NULL, src->size is 0, or memory allocation fails |

Note

The clone includes a copy of the SHA-256 hash from the source
Caller must free the returned bitmap using bitmap_free()
Changes to the clone do not affect the source and vice versa

Definition at line 387 of file bitmap.c.

References bitmap_init(), BITMAP::byte_size, BITMAP::data, BITMAP::sha256, and BITMAP::size.

Referenced by iZm_clone(), SiZm(), and vx_init().

#### 13.1.2.6 bitmap_compute_hash()

```
void bitmap_compute_hash (
            BITMAP * bitmap)
```

Compute SHA-256 over bitmap data and store it in BITMAP::sha256.

Parameters

| | |
|---|---|
| bitmap | Bitmap to hash. |

Compute SHA-256 over bitmap data and store it in BITMAP::sha256.

Parameters

| | |
|---|---|
| bitmap | The BITMAP for which the SHA-256 hash is generated. |

Definition at line 403 of file bitmap.c.

References BITMAP::byte_size, BITMAP::data, and BITMAP::sha256.

Referenced by bitmap_fwrite().

#### 13.1.2.7 bitmap_flip_bit()

```
void bitmap_flip_bit (
            BITMAP * bitmap,
            size_t idx)
```

Toggle bit at idx.

Parameters

| | |
|---|---|
| bitmap | Bitmap to modify. |

| | |
|---|---|
| idx | Zero-based bit index. |

Toggle bit at idx.

Parameters

| | |
|---|---|
| bitmap | The BITMAP to modify. |
| idx | The index of the bit to flip. |

Definition at line 148 of file bitmap.c.

References BITMAP::data.

Referenced by SoA().

#### 13.1.2.8 bitmap_fread()

```
BITMAP * bitmap_fread (
            FILE * file)
```

Read bitmap payload and checksum from a binary stream.

Parameters

| | |
|---|---|
| file | Readable stream. |

Returns

Newly allocated bitmap, or NULL on parse/verification failure.

Read bitmap payload and checksum from a binary stream.

Parameters

| | |
|---|---|
| file | The file pointer to read from. |

Returns

BITMAP* A pointer to the newly created BITMAP, or NULL on failure.

Definition at line 497 of file bitmap.c.

References bitmap_free(), bitmap_init(), bitmap_validate_hash(), BITMAP::byte_size, BITMAP::data, log_error(), and BITMAP::sha256.

#### 13.1.2.9 bitmap_free()

```
void bitmap_free (
            BITMAP ** bitmap)
```

Free a bitmap and set the caller pointer to NULL.

Parameters

| | |
|---|---|
| bitmap | Address of a BITMAP pointer. |

Free a bitmap and set the caller pointer to NULL.

This function safely deallocates the bitmap's data array and the structure itself, then sets the pointer to NULL to prevent use-after-free bugs and double-free errors. Safe to call with NULL pointers.

Parameters

| | |
|---|---|
| bitmap | Double pointer to the BITMAP to free |

Postcondition

*bitmap is set to NULL after successful deallocation

Safe to call multiple times on the same pointer

Note

No operation performed if bitmap == NULL or *bitmap == NULL

This is the only correct way to free a BITMAP; never use free() directly

Definition at line 103 of file bitmap.c.

Referenced by bitmap_fread(), get_root_primes(), iZm_free(), SiZ(), SiZm(), SiZm_vy(), SoA(), SoE(), SoEu(), SoS(), SSoE(), SSoS(), vx_free(), and vx_search_prime().

#### 13.1.2.10 bitmap_fwrite()

```
int bitmap_fwrite (
            BITMAP * bitmap,
            FILE * file)
```

Write bitmap payload and checksum to a binary stream.

Parameters

| | |
|---|---|
| bitmap | Bitmap to serialize. |
| file | Writable stream. |

Returns

1 on success, otherwise 0.

Write bitmap payload and checksum to a binary stream.

Parameters

| | |
|---|---|
| bitmap | The BITMAP to write. |

| | |
|---|---|
| file | The file pointer to write to. |

Returns

int 1 on success, 0 on failure.

Definition at line 445 of file bitmap.c.

References bitmap_compute_hash(), BITMAP::byte_size, BITMAP::data, log_error(), BITMAP::sha256, and BITMAP::size.

#### 13.1.2.11 bitmap_get_bit()

```
int bitmap_get_bit (
            BITMAP * bitmap,
            size_t idx)
```

Read bit value at idx.

Parameters

| | |
|---|---|
| bitmap | Bitmap to inspect. |
| idx | Zero-based bit index. |

Returns

1 if set, otherwise 0.

Read bit value at idx.

Parameters

| | |
|---|---|
| bitmap | The BITMAP to read from. |
| idx | The index of the bit to read. |

Returns

int 1 if the bit is set, 0 if unset, -1 on error.

Definition at line 137 of file bitmap.c.

References BITMAP::data.

Referenced by iZ_next_prime(), process_iZ_bitmaps(), process_N_bitmap(), SiZm(), SiZm_vy(), SoA(), SoEu(), SoS(), SSoE(), SSoS(), vx_collect_p_gaps(), vx_det_sieve(), vx_prob_sieve(), vx_search_prime(), and vx_stream().

#### 13.1.2.12 bitmap_init()

```
BITMAP * bitmap_init (
            size_t size,
            int set_bits)
```

Allocate a bitmap with size bits.

Parameters

| | |
|---|---|
| size | Number of bits to allocate (must be > 0). |

| | |
|---|---|
| set_bits | Non-zero initializes all bits to 1, otherwise to 0. |

Returns

Newly allocated bitmap, or NULL on allocation failure.

Allocate a bitmap with size bits.

Allocates memory for both the BITMAP structure and its underlying data array. The data array is allocated using calloc(), ensuring all bits start at 0. The SHA-256 hash field is initialized to all zeros.

Parameters

| | |
|---|---|
| size | The number of bits in the bitmap (must be > 0) |
| set_bits | If non-zero, initializes all bits to 1; otherwise, all bits are 0. |

Returns

Pointer to newly allocated BITMAP on success, NULL on failure

Return values

| | |
|---|---|
| NULL | if size is 0 |
| NULL | if memory allocation fails |

Note

Memory allocated: sizeof(BITMAP) + size/8 bytes

On failure, any partially allocated memory is freed before returning

Warning

Caller must free returned bitmap using bitmap_free(), not free()

Definition at line 57 of file bitmap.c.

References BITMAP::byte_size, BITMAP::data, log_error(), BITMAP::sha256, and BITMAP::size.

Referenced by bitmap_clone(), bitmap_fread(), get_root_primes(), iZm_init(), SiZ(), SiZm(), SiZm_vy(), SoA(), SoE(), SoEu(), SoS(), SSoE(), SSoS(), and vx_search_prime().

#### 13.1.2.13 bitmap_set_all()

```
void bitmap_set_all (
            BITMAP * bitmap)
```

Set all bits to 1.

Parameters

| | |
|---|---|
| bitmap | Bitmap to modify. |

Set all bits to 1.

Parameters

| | |
|---|---|
| bitmap | The BITMAP to modify. |

Definition at line 170 of file bitmap.c.

References BITMAP::data, and BITMAP::size.

Referenced by iZm_construct_vx_base(), SiZm_vy(), and SSoE().

#### 13.1.2.14 bitmap_set_bit()

```
void bitmap_set_bit (
            BITMAP * bitmap,
            size_t idx)
```

Set bit at idx to 1.

Parameters

| | |
|---|---|
| bitmap | Bitmap to modify. |
| idx | Zero-based bit index. |

Set bit at idx to 1.

Parameters

| | |
|---|---|
| bitmap | The BITMAP to modify. |
| idx | The index of the bit to set. |

Definition at line 125 of file bitmap.c.

References BITMAP::data.

#### 13.1.2.15 bitmap_validate_hash()

```
int bitmap_validate_hash (
            BITMAP * bitmap)
```

Verify BITMAP::sha256 against current bitmap data.

Parameters

| | |
|---|---|
| bitmap | Bitmap to verify. |

Returns

1 if checksum matches, otherwise 0.

Verify BITMAP::sha256 against current bitmap data.

Parameters

| | |
|---|---|
| bitmap | The BITMAP whose hash is validated. |

Returns

int 1 if the hash matches, 0 otherwise.

Definition at line 417 of file bitmap.c.

References BITMAP::byte_size, BITMAP::data, log_error(), and BITMAP::sha256.

Referenced by bitmap_fread().

#### 13.1.2.16 TEST_BITMAP()

```
int TEST_BITMAP (
            int verbose)
```

Run bitmap module tests.

Parameters

| | |
|---|---|
| verbose | Non-zero enables detailed logging. |

Returns

1 when all tests pass, otherwise 0.

## 13.2 Integer Arrays

Dynamic integer containers used by sieve and toolkit layers.

Files

- file int_arrays.c

  Implementation of dynamic integer array module for 16-bit, 32-bit, and 64-bit unsigned integers.

Data Structures

- struct UI16_ARRAY

  Dynamic array for uint16_t values.
- struct UI32_ARRAY

  Dynamic array for uint32_t values.
- struct UI64_ARRAY

  Dynamic array for uint64_t values.

Functions

- int TEST_GENERIC_INT_ARRAYS (int verbose)

  Run generic dispatch tests for C11 _Generic helper macros.

UI16 API

- UI16_ARRAY * ui16_init (int capacity)

  Allocate a UI16 array with an initial capacity.
- void ui16_free (UI16_ARRAY **array)

  Free a UI16 array and null the caller pointer.
- void ui16_resize_to (UI16_ARRAY *array, int new_capacity)

  Resize UI16 storage to new_capacity (must be >= count).
- void ui16_resize_to_fit (UI16_ARRAY *array)

  Shrink UI16 storage so capacity equals count.
- void ui16_push (UI16_ARRAY *array, uint16_t element)

  Append a uint16 value, growing storage if needed.
- void ui16_sort (UI16_ARRAY *array)

  Sort values in ascending order.
- void ui16_pop (UI16_ARRAY *array)

  Remove the last element if the array is non-empty.
- void ui16_compute_hash (UI16_ARRAY *array)

  Compute SHA-256 checksum over active payload.
- int ui16_verify_hash (UI16_ARRAY *array)

  Verify the stored checksum against current payload.
- int ui16_fwrite (UI16_ARRAY *array, FILE *file)

  Serialize count, payload, and checksum to a binary stream.
- UI16_ARRAY * ui16_fread (FILE *file)

  Deserialize a UI16 array from a binary stream.
- int TEST_UI16_ARRAY (int verbose)

  Execute UI16 test suite.

UI32 API

- UI32_ARRAY * ui32_init (int capacity)

  Allocate a UI32 array with an initial capacity.
- void ui32_free (UI32_ARRAY **array)

  Free a UI32 array and null the caller pointer.
- void ui32_resize_to (UI32_ARRAY *array, int new_capacity)

  Resize UI32 storage to new_capacity (must be >= count).
- void ui32_resize_to_fit (UI32_ARRAY *array)

  Shrink UI32 storage so capacity equals count.
- void ui32_push (UI32_ARRAY *array, uint32_t element)

  Append a uint32 value, growing storage if needed.
- void ui32_sort (UI32_ARRAY *array)

  Sort values in ascending order.
- void ui32_pop (UI32_ARRAY *array)

  Remove the last element if the array is non-empty.
- void ui32_compute_hash (UI32_ARRAY *array)

  Compute SHA-256 checksum over active payload.
- int ui32_verify_hash (UI32_ARRAY *array)

  Verify the stored checksum against current payload.
- int ui32_fwrite (UI32_ARRAY *array, FILE *file)

  Serialize count, payload, and checksum to a binary stream.
- UI32_ARRAY * ui32_fread (FILE *file)

  Deserialize a UI32 array from a binary stream.
- int TEST_UI32_ARRAY (int verbose)

  Execute UI32 test suite.

UI64 API

- UI64_ARRAY * ui64_init (int capacity)

  Allocate a UI64 array with an initial capacity.
- void ui64_free (UI64_ARRAY **array)

  Free a UI64 array and null the caller pointer.
- void ui64_resize_to (UI64_ARRAY *array, int new_capacity)

  Resize UI64 storage to new_capacity (must be >= count).
- void ui64_resize_to_fit (UI64_ARRAY *array)

  Shrink UI64 storage so capacity equals count.
- void ui64_push (UI64_ARRAY *array, uint64_t element)

  Append a uint64 value, growing storage if needed.
- void ui64_sort (UI64_ARRAY *array)

  Sort values in ascending order.
- void ui64_pop (UI64_ARRAY *array)

  Remove the last element if the array is non-empty.
- void ui64_compute_hash (UI64_ARRAY *array)

  Compute SHA-256 checksum over active payload.
- int ui64_verify_hash (UI64_ARRAY *array)

  Verify the stored checksum against current payload.
- int ui64_fwrite (UI64_ARRAY *array, FILE *file)

  Serialize count, payload, and checksum to a binary stream.
- UI64_ARRAY * ui64_fread (FILE *file)

  Deserialize a UI64 array from a binary stream.
- int TEST_UI64_ARRAY (int verbose)

  Execute UI64 test suite.

### 13.2.1 Detailed Description

Dynamic integer containers used by sieve and toolkit layers.

### 13.2.2 Function Documentation

#### 13.2.2.1 TEST_GENERIC_INT_ARRAYS()

int TEST_GENERIC_INT_ARRAYS (
int verbose)

Run generic dispatch tests for C11 _Generic helper macros.

Parameters

| | |
|---|---|
| verbose | Non-zero enables detailed logging. |

Returns

1 when all tests pass, otherwise 0.

## 13.3 iZ Public API

< iZ/iZm toolkit structures and helpers.

Files

- file iZ_apps.c
  High-level application routines built on top of the iZ_toolkit.
- file prime_sieve.c
  Implementations of classical and SiZ-family sieve algorithms.

Data Structures

- struct INPUT_SIEVE_RANGE
  Input parameters for range sieving/counting.

SiZ Range Variants

Count/stream primes over a numeric interval.

- typedef struct INPUT_SIEVE_RANGE INPUT_SIEVE_RANGE
  Input parameters for range sieving/counting.
- uint64_t SiZ_stream (INPUT_SIEVE_RANGE *range)
  Stream primes (or prime gaps) in a range to filepath (and return the count).
- uint64_t SiZ_count (INPUT_SIEVE_RANGE *input_range, int cores_num)
  Count primes in a range using multiple worker processes.

Classic Prime Sieve Algorithms

Baseline sieves up to a numeric limit.

All classic sieve functions take a limit n and return an ascending list of primes <= n.

- UI64_ARRAY * SoE (uint64_t n)
  Optimized Sieve of Eratosthenes.
- UI64_ARRAY * SSoE (uint64_t n)
  Segmented Sieve of Eratosthenes.
- UI64_ARRAY * SoS (uint64_t n)
  Sieve of Sundaram.
- UI64_ARRAY * SSoS (uint64_t n)
  Segmented Sieve of Sundaram.
- UI64_ARRAY * SoEu (uint64_t n)
  Euler (linear) sieve.
- UI64_ARRAY * SoA (uint64_t n)
  Sieve of Atkin.

iZ-based Sieve Algorithms

Sieve family operating in the 6x-1 / 6x+1 index space.

- UI64_ARRAY * SiZ (uint64_t n)
  Solid Sieve-iZ (wheel 6, iZ index space).
- UI64_ARRAY * SiZm (uint64_t n)
  Segmented Sieve-iZm (VX segmented, horizontal processing).
- UI64_ARRAY * SiZm_vy (uint64_t n)
  Segmented Sieve-iZm with vertical (y-major) traversal.

Prime Generators

Probabilistic prime searches using iZm/VX machinery.

- int vy_random_prime (mpz_t p, int bit_size, int cores_num)
  Search for a random prime using vertical (vy) sieving.
- int vx_random_prime (mpz_t p, int bit_size, int cores_num)
  Search for a random prime using horizontal (vx) sieving.
- int iZ_next_prime (mpz_t p, mpz_t base, int forward)
  Advance to the next (or previous) prime from a base value.

### 13.3.1 Detailed Description

< iZ/iZm toolkit structures and helpers.

< Common utilities, types, and dependencies.

High-level entry points for sieves and prime generation.

The API is organized into:

- Classic sieves: operate directly on the integer domain.
- SiZ / SiZm: operate in iZ index space (6x-1 and 6x+1), optionally segmented.
- Range variants: count/stream primes over large intervals.
- Prime generators: probabilistic searches using the iZm/VX machinery.

### 13.3.2 Typedef Documentation

#### 13.3.2.1 INPUT_SIEVE_RANGE

typedef struct INPUT_SIEVE_RANGE INPUT_SIEVE_RANGE

Input parameters for range sieving/counting.

The interval is interpreted as:

- Start $Z_s$ from the decimal string start.
- End $Z_e = Z_s + range - 1$.

Forward declaration; concrete definition lives in iZ_api.h.

### 13.3.3 Function Documentation

#### 13.3.3.1 iZ_next_prime()

```
int iZ_next_prime (
            mpz_t p,
            mpz_t base,
            int forward)
```

Advance to the next (or previous) prime from a base value.

Find the next prime number after a given base.

Parameters

| | |
|---|---|
| p | Output prime (initialized by caller). |

| | |
|---|---|
| base | Starting value. |
| forward | Non-zero to search forward, 0 to search backward. |

Returns

1 on success, 0 on failure.

Description: This function searches for the next/previous prime number after a given base using the iZ framework.

Parameters

| | |
|---|---|
| p | The mpz_t variable to store the found prime number. |
| base | The base number to start the search from. |
| forward | If true, search for the next prime; if false, search for the previous prime. |

Returns

1 if a prime is found, 0 otherwise.

Definition at line 891 of file iZ_apps.c.

References bitmap_get_bit(), iZ_mpz(), log_debug(), MR_ROUNDS, and VX6.

Referenced by izp_ffi_next_prime().

#### 13.3.3.2 SiZ()

```
UI64_ARRAY * SiZ (
            uint64_t n)
```

Solid Sieve-iZ (wheel 6, iZ index space).

Classic Sieve-iZ algorithm for prime generation up to n.

Parameters

| | |
|---|---|
| n | Upper bound (inclusive). |

Returns

Heap-allocated prime list, or NULL on allocation failure.

Precondition

n <= 10^12.

The Sieve-iZ algorithm operates in the iZ index space of numbers of the form 6x ± 1, excluding all multiples of 2 and 3. It maintains two bitmaps, x5 and x7, representing candidates of the form 6x − 1 (iZ−) and 6x + 1 (iZ+), respectively. For each index x whose bit remains set, the corresponding candidate is treated as prime and its composites are marked using the Xp identity in the iZ space.

This provides a wheel factorization of modulo 6, reducing the number of candidates by approximately a factor of 3 compared to a classical sieve over all integers. After sieving, any remaining set bits correspond to primes of the form 6x ± 1, which are mapped back to their integer values and appended to the output array alongside the primes 2 and 3.

Parameters

| | |
|---|---|
| n | Upper bound (inclusive) for prime generation (n <= 10^12). |

Returns

Pointer to a UI64_ARRAY containing all primes <= n on success, or NULL if allocation fails.

Definition at line 586 of file prime_sieve.c.

References UI64_ARRAY::array, ASSERT_LIMIT, bitmap_free(), bitmap_init(), UI64_ARRAY::count, Pi, process_iZ_bitmaps(), ui64_free(), ui64_init(), ui64_pop(), ui64_push(), and ui64_resize_to_fit().

Referenced by compute_max_vx(), iZm_init(), izp_ffi_sieve_u64(), SiZm(), and SiZm_vy().

#### 13.3.3.3 SiZ_count()

```
uint64_t SiZ_count (
            INPUT_SIEVE_RANGE * input_range,
            int cores_num)
```

Count primes in a range using multiple worker processes.

Multi-process prime counting over an arbitrary numeric range using iZ toolkit.

Parameters

| | |
|---|---|
| input_range | Range configuration (output disabled). |
| cores_num | Requested worker process count. |

Returns

Prime count in the interval, or 0 on error.

This function parallelizes prime counting over the interval [Zs, Ze] (interpreted from input_range) by partitioning the corresponding iZ index space into VX segments and distributing contiguous blocks of segments across multiple processes. Each child process returns a local prime count to the parent via a pipe.

The parent process aggregates all child counts, applies boundary corrections for endpoints that do not align exactly with 6x ± 1, and returns the total number of primes in [Zs, Ze]. Any failure to create pipes, fork children, or initialize per-process resources causes the function to log an error and return 0.

Parameters

| | |
|---|---|
| input_range | Pointer to an INPUT_SIEVE_RANGE structure describing the numeric interval and Miller–Rabin configuration. |

| | |
|---|---|
| cores_num | Requested number of worker processes. The actual number used is clamped to the available CPU cores and the number of VX segments in the range. |

Returns

The total number of primes found in [Zs, Ze] on success, or 0 on any error or child-process failure.

Definition at line 224 of file iZ_apps.c.

References UI64_ARRAY::array, UI64_ARRAY::count, get_cpu_cores_count(), iZ_mpz(), iZm_clone(), iZm_free(), log_error(), log_info(), MAX, MIN, INPUT_SIEVE_RANGE::mr_rounds, VX_SEG::p_count, INPUT_SIEVE_RANGE::range, range_info_free(), range_info_init(), SiZm(), INPUT_SIEVE_RANGE::start, ui64_free(), vx_free(), vx_full_sieve(), vx_init(), IZM_RANGE_INFO::Xe, IZM_RANGE_INFO::Xs, IZM_RANGE_INFO::y_range, IZM_RANGE_INFO::Ye, IZM_RANGE_INFO::Ys, IZM_RANGE_INFO::Ze, and IZM_RANGE_INFO::Zs.

Referenced by izp_ffi_count_range().

#### 13.3.3.4 SiZ_stream()

```
uint64_t SiZ_stream (
            INPUT_SIEVE_RANGE * input_range)
```

Stream primes (or prime gaps) in a range to filepath (and return the count).

Stream primes in an arbitrary numeric range using iZ toolkit.

Parameters

| | |
|---|---|
| range | Range configuration. |

Returns

Prime count in the interval, or 0 on error.

This function counts and streams primes in the interval [Zs, Ze] defined by the input range. It maps the numeric bounds into the iZ index space (6x ± 1), partitions that space into y segments of length vx, and streams primes in each segment using the iZ toolkit.

When an output file path is provided in input_range->filepath, the function additionally streams all primes in the requested interval to that file in ascending order, reconstructing them from per-segment prime gaps. Otherwise, it operates in counting mode only.

Parameters

| | |
|---|---|
| input_range | Pointer to an INPUT_SIEVE_RANGE structure describing the start value (as a decimal string), the range length, the optional output filepath, gap-stream mode, and Miller–Rabin configuration. |

Returns

The total number of primes found in [Zs, Ze] on success, or 0 on invalid input, allocation failure, or I/O error.

Definition at line 68 of file iZ_apps.c.

References UI64_ARRAY::array, UI64_ARRAY::count, INPUT_SIEVE_RANGE::filepath, log_error(), MAX, MIN, INPUT_SIEVE_RANGE::mr_rounds, VX_SEG::p_count, range_info_free(), range_info_init(), SiZm(), INPUT_SIEVE_RANGE::start, INPUT_SIEVE_RANGE::stream_gaps, ui64_free(), IZM_RANGE_INFO::vx, vx_free(), vx_init(), vx_stream(), IZM_RANGE_INFO::Xe, IZM_RANGE_INFO::Xs, IZM_RANGE_INFO::y_range, IZM_RANGE_INFO::Ye, IZM_RANGE_INFO::Ys, IZM_RANGE_INFO::Ze, and IZM_RANGE_INFO::Zs.

Referenced by izp_ffi_stream_range().

#### 13.3.3.5 SiZm()

```
UI64_ARRAY * SiZm (
            uint64_t n)
```

Segmented Sieve-iZm (VX segmented, horizontal processing).

Segmented Sieve-iZm algorithm for prime generation up to n.

Parameters

| | |
|---|---|
| n | Upper bound (inclusive). |

Returns

Heap-allocated prime list, or NULL on allocation failure.

Precondition

n <= 10^12.

Description: The Sieve-iZm algorithm is a segmented, cache-aware variant of the Sieve-iZ that operates in the iZ index space of numbers of the form 6x ± 1. It partitions the search interval into segments of length vx, where vx is a product of small primes, and constructs a pre-sieved base segment using those small primes once at initialization time.

For each segment, Sieve-iZm clones the base bitmaps and only marks composites of the remaining root primes that do not divide vx. This amortizes the cost of small-prime sieving across all segments and keeps the working set small and highly cache-resident. After marking, any remaining set bits correspond to primes of the form 6x ± 1 within that segment, which are then mapped back to their integer values and appended to the output array.

Compared to the basic Sieve-iZ, Sieve-iZm achieves the same O(n log log n) time complexity with effectively constant auxiliary memory (aside from the output) and significantly reduced constant factors, making it well-suited for enumerating primes up to large bounds.

Parameters

| | |
|---|---|
| n | Upper bound (inclusive) for prime generation (n <= 10^12). |

Returns

Pointer to a UI64_ARRAY containing all primes <= n on success, or NULL on allocation or initialization failure.

Definition at line 664 of file prime_sieve.c.

References UI64_ARRAY::array, ASSERT_LIMIT, bitmap_clear_steps_simd(), bitmap_clone(), bitmap_free(), bitmap_get_bit(), bitmap_init(), BITMAP::byte_size, compute_l2_vx(), UI64_ARRAY::count, BITMAP::data, iZ(), iZm_construct_vx_base(), iZm_solve_for_x0(), Pi, process_iZ_bitmaps(), SiZ(), ui64_free(), ui64_init(), ui64_pop(), ui64_push(), and ui64_resize_to_fit().

Referenced by izp_ffi_sieve_u64(), SiZ_count(), SiZ_stream(), and vx_search_prime().

#### 13.3.3.6 SiZm_vy()

```
UI64_ARRAY * SiZm_vy (
            uint64_t n)
```

Segmented Sieve-iZm with vertical (y-major) traversal.

Segmented Sieve-iZm with vertical (vy) traversal order.

Uses the same iZm 2D wheel model as SiZm, but scans vertical lanes for better cache behavior at large bounds. The generated primes are valid, but output order is not guaranteed to be ascending.

Parameters

| | |
|---|---|
| n | Upper bound (inclusive). |

Returns

Heap-allocated prime list, or NULL on allocation failure.

Precondition

n <= 10^12.

Reuses the iZm 2D wheel segmentation, but iterates by vertical lanes (y-major) instead of horizontal slices. This improves throughput on large ranges at the cost of natural output ordering; returned primes are valid but not guaranteed to be sorted.

Parameters

| | |
|---|---|
| n | Inclusive upper bound for prime generation. |

Returns

Heap-allocated list of primes <= n, or NULL on failure.

Definition at line 773 of file prime_sieve.c.

References UI64_ARRAY::array, ASSERT_LIMIT, bitmap_clear_steps_simd(), bitmap_free(), bitmap_get_bit(), bitmap_init(), bitmap_set_all(), UI64_ARRAY::count, gcd(), get_root_primes(), iZ(), iZm_solve_for_y0(), UI64_ARRAY::ordered, Pi, SiZ(), ui64_init(), ui64_push(), and ui64_resize_to_fit().

Referenced by izp_ffi_sieve_u64().

#### 13.3.3.7 SoA()

```
UI64_ARRAY * SoA (
            uint64_t n)
```

Sieve of Atkin.

Sieve of Atkin: Generates a list of prime numbers up to a given limit using the Sieve of Atkin algorithm.

Parameters

| | |
|---|---|
| n | Upper bound (inclusive). |

Returns

Heap-allocated prime list, or NULL on allocation failure.

Precondition

n <= 10^12.

Description: This function implements the Sieve of Atkin, a modern algorithm to find all prime numbers up to a specified integer n. It initializes a bitmap to mark potential primes and applies the Atkin conditions to identify primes. The function also marks odd multiples of squares of primes as non-prime, then collects the remaining unmarked numbers as primes.

Parameters

| | |
|---|---|
| n | The upper limit (inclusive) up to which prime numbers are to be found. |

Returns

- UI64_ARRAY* A pointer to the UI64_ARRAY structure containing the list of primes up to n.
- NULL if memory allocation fails.

Definition at line 465 of file prime_sieve.c.

References ASSERT_LIMIT, bitmap_clear_steps_simd(), bitmap_flip_bit(), bitmap_free(), bitmap_get_bit(), bitmap_init(), Pi, ui64_free(), ui64_init(), ui64_push(), and ui64_resize_to_fit().

Referenced by izp_ffi_sieve_u64().

#### 13.3.3.8 SoE()

```
UI64_ARRAY * SoE (
            uint64_t n)
```

Optimized Sieve of Eratosthenes.

Optimized Sieve of Eratosthenes algorithm to find all primes up to n.

Parameters

| | |
|---|---|
| n | Upper bound (inclusive). |

Returns

Heap-allocated prime list, or NULL on allocation failure.

Precondition

n <= 10^12.

Description: This function applies common speedups to the Sieve of Eratosthenes algorithm. It works the same way as the classic sieve but skips checking or marking even numbers and starts with 3, incrementing by 2, it also starts the sieve from p*p.

Parameters

| | |
|---|---|
| n | The upper limit to find primes. |

Returns

- A pointer to a UI64_ARRAY structure containing the list of prime numbers up to n,
- NULL if memory allocation fails.

Definition at line 82 of file prime_sieve.c.

References ASSERT_LIMIT, bitmap_free(), bitmap_init(), Pi, process_N_bitmap(), ui64_free(), ui64_init(), and ui64_resize_to_fit().

Referenced by izp_ffi_sieve_u64(), and SSoE().

#### 13.3.3.9 SoEu()

```
UI64_ARRAY * SoEu (
            uint64_t n)
```

Euler (linear) sieve.

Sieve of Euler: Generates a list of prime numbers up to a given limit using the Euler Sieve algorithm.

Parameters

| | |
|---|---|
| n | Upper bound (inclusive). |

Returns

Heap-allocated prime list, or NULL on allocation failure.

Precondition

n <= 10^12.

Description: This function uses the Euler Sieve algorithm to find all prime numbers up to a specified limit n. It marks each composite only once, allowing for a more efficient sieve process. It initializes a bitmap to track prime numbers and iterates through the numbers, marking distinct multiples of each prime found. The function also handles memory allocation and resizing of the primes array as needed.

Parameters

| | |
|---|---|
| n | The upper limit up to which prime numbers are to be found. |

Returns

- UI64_ARRAY* A pointer to the UI64_ARRAY structure containing the list of primes up to n.
- NULL if memory allocation fails.

Definition at line 397 of file prime_sieve.c.

References UI64_ARRAY::array, ASSERT_LIMIT, bitmap_clear_bit(), bitmap_free(), bitmap_get_bit(), bitmap_init(), UI64_ARRAY::count, Pi, ui64_free(), ui64_init(), ui64_push(), and ui64_resize_to_fit().

Referenced by izp_ffi_sieve_u64().

#### 13.3.3.10 SoS()

```
UI64_ARRAY * SoS (
            uint64_t n)
```

Sieve of Sundaram.

Sieve of Sundaram: Generates a list of prime numbers up to a given limit using the Sieve of Sundaram algorithm.

Parameters

| | |
|---|---|
| n | Upper bound (inclusive). |

Returns

Heap-allocated prime list, or NULL on allocation failure.

Precondition

n <= 10^12.

Description: This function implements the Sieve of Sundaram algorithm to find all prime numbers up to a specified limit n. The Sieve of Sundaram works by eliminating numbers of the form i + j + 2ij from a list of integers from 1 to k = (n-1)/2. If an integer x in this range is not eliminated, then 2x+1 is a prime number. The prime number 2 is handled as a special case, as the sieve only generates odd primes.

Parameters

| | |
|---|---|
| n | The upper limit (inclusive) up to which prime numbers are to be found. |

Returns

- UI64_ARRAY* A pointer to the UI64_ARRAY structure containing the list of primes up to n.
- NULL if memory allocation fails.

Definition at line 223 of file prime_sieve.c.

References UI64_ARRAY::array, ASSERT_LIMIT, bitmap_clear_steps_simd(), bitmap_free(), bitmap_get_bit(), bitmap_init(), UI64_ARRAY::count, Pi, ui64_free(), ui64_init(), ui64_pop(), ui64_push(), and ui64_resize_to_fit().

Referenced by izp_ffi_sieve_u64(), and SSoS().

#### 13.3.3.11 SSoE()

```
UI64_ARRAY * SSoE (
            uint64_t n)
```

Segmented Sieve of Eratosthenes.

Segmented Sieve of Eratosthenes algorithm to find all primes up to n.

Parameters

| | |
|---|---|
| n | Upper bound (inclusive). |

Returns

Heap-allocated prime list, or NULL on allocation failure.

Precondition

n <= 10^12.

Description: This function implements the Segmented Sieve of Eratosthenes algorithm to find all prime numbers up to a given limit n. It divides the range into segments and uses a bitmap to mark prime numbers in each segment. The function returns a pointer to a UI64_ARRAY structure containing the list of prime numbers found. The function also handles memory allocation and resizing of the primes array as needed.

Parameters

| | |
|---|---|
| n | The upper limit to find primes. |

Returns

- A pointer to the UI64_ARRAY structure containing the list of primes up to n,
- NULL if memory allocation fails.

Definition at line 131 of file prime_sieve.c.

References UI64_ARRAY::array, ASSERT_LIMIT, bitmap_clear_steps_simd(), bitmap_free(), bitmap_get_bit(), bitmap_init(), bitmap_set_all(), UI64_ARRAY::count, MAX, Pi, process_N_bitmap(), SoE(), ui64_free(), ui64_init(), ui64_push(), and ui64_resize_to_fit().

Referenced by izp_ffi_sieve_u64().

#### 13.3.3.12 SSoS()

```
UI64_ARRAY * SSoS (
            uint64_t n)
```

Segmented Sieve of Sundaram.

Segmented Sieve of Sundaram: Generates a list of prime numbers up to a given limit using the Sieve of Sundaram algorithm.

Parameters

| | |
|---|---|
| n | Upper bound (inclusive). |

Returns

Heap-allocated prime list, or NULL on allocation failure.

Precondition

n <= 10^12.

Description: This function implements the Sieve of Sundaram algorithm to find all prime numbers up to a specified limit n. The Sieve of Sundaram works by combining the Sundaram's basic wheel with another cache-friendly segmentation wheel (vx) that serves as a pre-sieved base from primes that divides 3vx. For each segment, it uses the Xp-identity to mark composites in the sieve bitmap, then collects the primes from the segment.

Parameters

| | |
|---|---|
| n | The upper limit (inclusive) up to which prime numbers are to be found. |

Returns

- UI64_ARRAY∗ A pointer to the UI64_ARRAY structure containing the list of primes up to n.
- NULL if memory allocation fails.

Definition at line 299 of file prime_sieve.c.

References UI64_ARRAY::array, ASSERT_LIMIT, bitmap_clear_bit(), bitmap_clear_steps_simd(), bitmap_free(), bitmap_get_bit(), bitmap_init(), BITMAP::byte_size, compute_l2_vx(), UI64_ARRAY::count, BITMAP::data, get_root_primes(), Pi, SoS(), ui64_free(), ui64_init(), ui64_pop(), ui64_push(), and ui64_resize_to_fit().

Referenced by izp_ffi_sieve_u64().

#### 13.3.3.13 vx_random_prime()

```
int vx_random_prime (
            mpz_t p,
            int bit_size,
            int cores_num)
```

Search for a random prime using horizontal (vx) sieving.

Generates a random prime candidate using the vx_search_prime routine.

Parameters

| | |
|---|---|
| p | Output prime (initialized by caller). |
| bit_size | Target size of p in bits. |
| cores_num | Requested worker processes. |

Returns

1 on success, 0 on failure.

Description: This function generates a random prime of a given bit size using the vx_search_prime routine. It initializes the random base and sets up the search parameters. The function also allows for parallel processing of the search using multiple child processes. The generated prime is stored in the provided mpz_t p variable.

Parameters

| | |
|---|---|
| p | The mpz_t variable to store the generated prime number. |

| | |
|---|---|
| bit_size | The target bit size of the prime. |
| cores_num | The number of cores to use for parallel processing. |

Returns

1 if a prime is found, 0 otherwise.

Definition at line 755 of file iZ_apps.c.

References log_error(), log_info(), log_warn(), MAX, VX5, VX6, and vx_search_prime().

Referenced by izp_ffi_random_prime_vx().

#### 13.3.3.14 vy_random_prime()

```
int vy_random_prime (
            mpz_t p,
            int bit_size,
            int cores_num)
```

Search for a random prime using vertical (vy) sieving.

Generates a random prime candidate using the vy_search_prime routine.

Parameters

| | |
|---|---|
| p | Output prime (initialized by caller). |
| bit_size | Target size of p in bits. |
| cores_num | Requested worker processes. |

Returns

1 on success, 0 on failure.

Description: This function generates a random prime of a given bit size using the vy_search_prime routine. It initializes the random base and sets up the search parameters. The function also allows for parallel processing of the search using multiple child processes. The generated prime is stored in the provided mpz_t p variable.

Parameters

| | |
|---|---|
| p | The mpz_t variable to store the generated prime number. |
| bit_size | The target bit size of the prime. |
| cores_num | The number of cores to use for parallel processing. |

Returns

1 if a prime is found, 0 otherwise.

Definition at line 598 of file iZ_apps.c.

References compute_max_vx(), log_error(), log_info(), log_warn(), MAX, and vy_search_prime().

Referenced by izp_ffi_random_prime_vy().

## 13.4 Toolkit (iZ/iZm)

Internal building blocks for SiZ and SiZm implementations.

### Files

- file iZ_toolkit.c

  Implementation of iZ index space helpers and iZm/VX segment machinery.

### Data Structures

- struct IZM

  Precomputed iZm assets for repeated VX-segment sieving.
- struct VX_SEG

  Runtime state for one VX segment at a specific y.
- struct IZM_RANGE_INFO

  Precomputed iZm coordinates for an inclusive numeric interval.

### Macros

- #define MR_ROUNDS 25

### Functions

- void iZm_construct_vx_base (uint64_t vx, BITMAP *base_x5, BITMAP *base_x7)

  Build pre-sieved base bitmaps for a VX segment.
- IZM_RANGE_INFO range_info_init (INPUT_SIEVE_RANGE *input_range, int vx)

  Map a decimal range input into iZm coordinates and segment bounds.
- void range_info_free (IZM_RANGE_INFO *info)

  Clear all GMP fields owned by info.

### iZ Mapping Helpers

- uint64_t iZ (uint64_t x, int i)

  Map iZ coordinates to an integer: 6*x + i.
- void iZ_mpz (mpz_t z, mpz_t x, int i)

  GMP variant of iZ().
- void process_iZ_bitmaps (UI64_ARRAY *primes, BITMAP *x5, BITMAP *x7, uint64_t x_↩limit)

  Traverse iZ bitmaps, emit surviving primes, and mark composites.
- void get_root_primes (UI64_ARRAY *primes, uint64_t limit)

  Generate primes up to limit for deterministic sieving.
- int test_primality (mpz_t n, int rounds)

  Check the primality of a number using GMP's probabilistic test.

IZM Lifecycle

- IZM * iZm_init (size_t vx)

  Allocate and initialize an IZM object for a given VX.
- IZM * iZm_clone (IZM *src)

  Deep-copy an IZM object for per-worker ownership.
- void iZm_free (IZM **iZm)

  Release an IZM object and set the caller pointer to NULL.

VX Selection Helpers

- uint64_t compute_vx_k (int k)

  Calculate VX_{ k }.
- uint64_t compute_l2_vx (uint64_t n)

  Choose VX using an L2-cache-aware heuristic.
- void compute_max_vx (mpz_t vx, int bit_size)

  Compute largest VX below 2^bit_size.

Modular Hit Solvers

- uint64_t iZm_solve_for_x0 (int m_id, uint64_t p, uint64_t vx, uint64_t y)

  Solve first x-hit of prime p for line m_id in segment y.
- uint64_t iZm_solve_for_x0_mpz (int m_id, uint64_t p, uint64_t vx, mpz_t y)

  GMP variant of iZm_solve_for_x0() for very large y.
- int64_t iZm_solve_for_y0 (int m_id, uint64_t p, uint64_t vx, uint64_t x)

  Solve first y-hit for fixed x in vertical (vy) scanning.

VX Segment Lifecycle and Execution

- VX_SEG * vx_init (IZM *iZm, int start_x, int end_x, char *y_str, int mr_rounds)

  Initialize and deterministically sieve one VX segment.
- void vx_free (VX_SEG **vx_obj)

  Free a VX segment and all owned resources.
- void vx_collect_p_gaps (VX_SEG *vx_obj)

  Extract prime-gap encoding from a fully sieved segment.
- void vx_full_sieve (VX_SEG *vx_obj, int collect_p_gaps)

  Complete segment processing (probabilistic stage and optional gaps).
- void vx_stream (VX_SEG *vx_obj, FILE *output, int stream_gaps)

  Stream segment primes to an output stream.

Random Prime Search Routines

- int vx_search_prime (mpz_t p, int m_id, int vx, int bit_size)

  Horizontal iZm/VX random-prime search.
- int vy_search_prime (mpz_t p, int m_id, mpz_t vx)

  Vertical iZm/VY random-prime search.

Toolkit Tests

- int TEST_IZM (int verbose)

  Run IZM construction and solver tests.
- int TEST_VX_SEG (int verbose)

  Run VX segment tests.

Standard VX Sizes (primorial products excluding 2,3)

- #define VX2 (5 ∗ 7ULL)
- #define VX3 (VX2 ∗ 11ULL)
- #define VX4 (VX3 ∗ 13ULL)
- #define VX5 (VX4 ∗ 17ULL)
- #define VX6 (VX5 ∗ 19ULL)
- #define VX7 (VX6 ∗ 23ULL)
- #define VX8 (VX7 ∗ 29ULL)

## 13.4.1 Detailed Description

Internal building blocks for SiZ and SiZm implementations.

## 13.4.2 Macro Definition Documentation

### 13.4.2.1 MR_ROUNDS

#define MR_ROUNDS 25

Default Miller-Rabin rounds used by toolkit search/sieve helpers.

Definition at line 79 of file iZ_toolkit.h.

Referenced by iZ_next_prime(), vx_init(), vx_search_prime(), and vy_search_prime().

### 13.4.2.2 VX2

#define VX2 (5 ∗ 7ULL)

35

Definition at line 69 of file iZ_toolkit.h.

### 13.4.2.3 VX3

#define VX3 (VX2 ∗ 11ULL)

385

Definition at line 70 of file iZ_toolkit.h.

#### 13.4.2.4 VX4

#define VX4 (VX3 ∗ 13ULL)

5005

Definition at line 71 of file iZ_toolkit.h.

#### 13.4.2.5 VX5

#define VX5 (VX4 ∗ 17ULL)

85085

Definition at line 72 of file iZ_toolkit.h.

Referenced by vx_random_prime().

#### 13.4.2.6 VX6

#define VX6 (VX5 ∗ 19ULL)

1616615

Definition at line 73 of file iZ_toolkit.h.

Referenced by iZ_next_prime(), and vx_random_prime().

#### 13.4.2.7 VX7

#define VX7 (VX6 ∗ 23ULL)

37260615

Definition at line 74 of file iZ_toolkit.h.

#### 13.4.2.8 VX8

#define VX8 (VX7 ∗ 29ULL)

1080558835

Definition at line 75 of file iZ_toolkit.h.

### 13.4.3 Function Documentation

#### 13.4.3.1 compute_l2_vx()

```
uint64_t compute_l2_vx (
                uint64_t n)
```

Choose VX using an L2-cache-aware heuristic.

Choose a cache-aware VX size based on detected L2 capacity.

Parameters

| | |
|---|---|
| n | Target numeric limit. |

Returns

Selected VX size.

Parameters

| | |
|---|---|
| n | Target numeric sieve bound. |

Returns

L2 compatible VX size.

Definition at line 322 of file iZ_toolkit.c.

References get_cpu_L2_cache_size_bits(), MIN, and s_primes.

Referenced by SiZm(), and SSoS().

#### 13.4.3.2 compute_max_vx()

```
void compute_max_vx (
                mpz_t vx,
                int bit_size)
```

Compute largest VX below 2^bit_size.

Compute the maximum vx such that vx < 2^bit_size.

Parameters

| | |
|---|---|
| vx | Output mpz_t containing VX. |
| bit_size | Bit-size ceiling. |

Parameters:

Parameters

| | |
|---|---|
| vx | mpz_t variable to store the computed maximum vx value. |

| | |
|---|---|
| bit_size | Integer representing the bit size limit for vx. |

Definition at line 347 of file iZ_toolkit.c.

References UI64_ARRAY::array, and SiZ().

Referenced by vy_random_prime().

#### 13.4.3.3 compute_vx_k()

```
uint64_t compute_vx_k (
            int k)
```

Calculate VX_{ k }.

Calculate a VX_{k} as the product of the first k primes > 3.

Parameters

| | |
|---|---|
| k | Number of multiplicative prime factors (>3). |

Returns

Selected VX size.

Parameters

| | |
|---|---|
| k | number of consecutive primes (> 3) to include in the product. |

Returns

Computed VX_k value.

Definition at line 302 of file iZ_toolkit.c.

References s_primes.

#### 13.4.3.4 get_root_primes()

```
void get_root_primes (
            UI64_ARRAY * primes,
            uint64_t limit)
```

Generate primes up to limit for deterministic sieving.

Generate root primes up to limit using iZ bitmap traversal.

Parameters

| | |
|---|---|
| primes | Output array, appended in ascending order. |

| | |
|---|---|
| limit | Numeric upper bound. |
| primes | Destination array. |
| limit | Numeric upper bound. |

Definition at line 128 of file iZ_toolkit.c.

References bitmap_free(), bitmap_init(), process_iZ_bitmaps(), ui64_free(), and ui64_push().

Referenced by SiZm_vy(), and SSoS().

#### 13.4.3.5 iZ()

```
uint64_t iZ (
            uint64_t x,
            int i)   [inline]
```

Map iZ coordinates to an integer: 6∗x + i.

Parameters

| | |
|---|---|
| x | iZ x-coordinate. |
| i | Matrix identifier, typically -1 or +1. |

Returns

Mapped integer value.

Map iZ coordinates to an integer: 6∗x + i.

Parameters:

Parameters

| | |
|---|---|
| x | (uint64_t) The value of x in 6x + i. |
| i | (int) The value of i in 6x + i. |

Returns

The computed value 6x + i as a 64-bit unsigned integer.

Definition at line 40 of file iZ_toolkit.c.

Referenced by process_iZ_bitmaps(), SiZm(), and SiZm_vy().

#### 13.4.3.6 iZ_mpz()

```
void iZ_mpz (
            mpz_t z,
            mpz_t x,
            int i)
```

GMP variant of iZ().

Parameters

| | |
|---|---|
| z | Output value. |

| | |
|---|---|
| x | Input x-coordinate. |
| i | Matrix identifier, typically -1 or +1. |

GMP variant of iZ().

Parameters:

Parameters

| | |
|---|---|
| z | (mpz_t) The result of the calculation 6x + i. |
| x | (mpz_t) The input value of x in 6x + i. |
| i | (int) The value of i in 6x + i. |

Definition at line 53 of file iZ_toolkit.c.

Referenced by iZ_next_prime(), SiZ_count(), vx_prob_sieve(), vx_search_prime(), vx_set_base_values(), vx_stream(), and vy_search_prime().

#### 13.4.3.7 iZm_clone()

```
IZM * iZm_clone (
            IZM * src)
```

Deep-copy an IZM object for per-worker ownership.

Deep-copy an IZM object for independent worker usage.

Parameters

| | |
|---|---|
| src | Source IZM instance. |

Returns

Newly allocated clone, or NULL on failure.

Parameters

| | |
|---|---|
| src | Source IZM object. |

Returns

Cloned IZM object, or NULL on failure.

Definition at line 248 of file iZ_toolkit.c.

References UI64_ARRAY::array, IZM::base_x5, IZM::base_x7, bitmap_clone(), UI64_ARRAY::capacity, UI64_ARRAY::count, iZm_free(), IZM::k_vx, log_error(), IZM::root_primes, ui64_init(), and IZM::vx.

Referenced by SiZ_count().

#### 13.4.3.8 iZm_construct_vx_base()

```
void iZm_construct_vx_base (
            uint64_t vx,
            BITMAP * base_x5,
            BITMAP * base_x7)
```

Build pre-sieved base bitmaps for a VX segment.

Constructs a pre-sieved iZm base segment of size vx.

Parameters

| | |
|---|---|
| vx | Segment width. |
| base_x5 | Output bitmap for 6x-1 candidates. |
| base_x7 | Output bitmap for 6x+1 candidates. |

Description: This function constructs a pre-sieved iZm base segment of size vx. It marks all composites of small primes that divide vx in the bitmaps base_x5 (iZ-) and base_x7 (iZ+). This pre-sieved base can then be used as a template for sieving all vx segments.

Parameters:

Parameters

| | |
|---|---|
| iZm | Pointer to the IZM structure containing the base segment bitmaps. |

Definition at line 379 of file iZ_toolkit.c.

References bitmap_clear_bit(), bitmap_clear_steps_simd(), bitmap_set_all(), s_primes, and s_primes_count.

Referenced by iZm_init(), and SiZm().

#### 13.4.3.9 iZm_free()

```
void iZm_free (
            IZM ** iZm)
```

Release an IZM object and set the caller pointer to NULL.

Free the memory allocated for an IZM structure.

Parameters

| | |
|---|---|
| iZm | Address of an IZM pointer. |

Parameters:

Parameters

| | |
|---|---|
| iZm | A pointer to the IZM structure to be freed. |

Definition at line 282 of file iZ_toolkit.c.

References bitmap_free(), and ui64_free().

Referenced by iZm_clone(), iZm_init(), and SiZ_count().

#### 13.4.3.10 iZm_init()

```
IZM * iZm_init (
            size_t vx)
```

Allocate and initialize an IZM object for a given VX.

Initialize an iZm structure for a given vx size.

Parameters

| | |
|---|---|
| vx | Segment width; must be odd, not divisible by 3, and >= 35. |

Returns

Initialized IZM object, or NULL on failure.

This function initializes an iZm structure for a specified vx size. It allocates memory for the structure, generates root primes for deterministic sieving, initializes base bitmaps, and constructs pre-sieved base segments for iZm5 and iZm7.

Parameters:

Parameters

| | |
|---|---|
| vx | The size of the segment for the iZm structure. |

Returns

A pointer to the initialized IZM structure.

Definition at line 200 of file iZ_toolkit.c.

References IZM::base_x5, IZM::base_x7, bitmap_init(), compute_k_vx(), iZm_construct_vx_base(), iZm_free(), IZM::k_vx, log_error(), IZM::root_primes, SiZ(), ui64_free(), and IZM::vx.

#### 13.4.3.11 iZm_solve_for_x0()

```
uint64_t iZm_solve_for_x0 (
            int m_id,
            uint64_t p,
            uint64_t vx,
            uint64_t y)
```

Solve first x-hit of prime p for line m_id in segment y.

Compute the first composite hit of p in the yth vx segment in iZm index space.

Parameters

| | |
|---|---|
| m_id | Line id (-1 for x5, +1 for x7). |

| | |
|---|---|
| p | Prime used for marking. |
| vx | Segment width. |
| y | Segment index. |

Returns

First x index to clear for this prime/line.

Description: This function solves for the smallest x that satisfies: (x + vx ∗ y)  x_p (mod p), where x_p is normalized based on the m_id and p to either x_p or p - x_p, then computes x and returns it.

Parameters:

Parameters

| | |
|---|---|
| m_id | int matrix identifier (-1 for iZm5, 1 for iZm7). |
| p | Unsigned 64-bit integer parameter. |
| vx | size_t parameter representing the vx value. |
| y | Unsigned 64-bit integer parameter. |

Returns

The computed x value as a 64-bit unsigned integer.

Definition at line 431 of file iZ_toolkit.c.

Referenced by SiZm(), and vx_det_sieve().

#### 13.4.3.12 iZm_solve_for_x0_mpz()

```
uint64_t iZm_solve_for_x0_mpz (
            int m_id,
            uint64_t p,
            uint64_t vx,
            mpz_t y)
```

GMP variant of iZm_solve_for_x0() for very large y.

Solve for x given m_id, p, vx, and y using GMP. Same as above but using GMP. Suitable for arbitrary y values.

Parameters

| | |
|---|---|
| m_id | Line id (-1 for x5, +1 for x7). |
| p | Prime used for marking. |
| vx | Segment width. |
| y | Segment index as mpz. |

Returns

First x index to clear for this prime/line.

Parameters:

Parameters

| | |
|---|---|
| m_id | int matrix identifier (-1 for iZm5, 1 for iZm7). |

| | |
|---|---|
| p | Unsigned 64-bit integer parameter. |
| vx | Size_t parameter representing the vx value. |
| y | mpz_t parameter representing the y value. |

Returns

The computed x value as a 64-bit unsigned integer.

Definition at line 463 of file iZ_toolkit.c.

Referenced by vx_det_sieve(), and vx_search_prime().

#### 13.4.3.13 iZm_solve_for_y0()

```
int64_t iZm_solve_for_y0 (
                int m_id,
                uint64_t p,
                uint64_t vx,
                uint64_t x)
```

Solve first y-hit for fixed x in vertical (vy) scanning.

Compute the first composite hit of p in the xth vy segment in iZm of width vx.

Parameters

| | |
|---|---|
| m_id | Line id (-1 for x5, +1 for x7). |
| p | Prime used for marking. |
| vx | Segment width. |
| x | Fixed x-coordinate. |

Returns

y index on success, -1 when no modular solution exists.

Description: This function solves for the smallest y that satisfies the equation (x + vx ∗ y)  x_p (mod p), where x_p is normalized based on the m_id and p. The function checks if p and vx are co-primes, and if not, it returns -1 indicating no solution can be found. If they are co-primes, it calculates the multiplicative inverse of vx modulo p and computes y using the formula y = (delta ∗ vx_inv) % p, where delta is the difference between x_p and x modulo p.

Parameters:

Parameters

| | |
|---|---|
| m_id | int matrix identifier (-1 for iZm5, 1 for iZm7). |
| p | Unsigned 64-bit integer parameter. |

| | |
|---|---|
| vx | Size_t parameter representing the vx value. |
| x | Unsigned 64-bit integer parameter. |

Returns

The computed y value as a 64-bit unsigned integer.

Definition at line 504 of file iZ_toolkit.c.

References gcd(), and modular_inverse().

Referenced by SiZm_vy().

#### 13.4.3.14 process_iZ_bitmaps()

```
void process_iZ_bitmaps (
            UI64_ARRAY * primes,
            BITMAP * x5,
            BITMAP * x7,
            uint64_t x_limit)
```

Traverse iZ bitmaps, emit surviving primes, and mark composites.

Consume iZ candidate bitmaps, emit primes, and mark root composites.

Parameters

| | |
|---|---|
| primes | Output prime array. |
| x5 | Bitmap for 6x-1 candidates. |
| x7 | Bitmap for 6x+1 candidates. |
| x_limit | Exclusive x upper bound. |
| primes | Destination array for discovered primes. |
| x5 | Bitmap for 6x-1 candidates. |
| x7 | Bitmap for 6x+1 candidates. |
| x_limit | Exclusive x upper bound. |

Definition at line 86 of file iZ_toolkit.c.

References bitmap_clear_steps_simd(), bitmap_get_bit(), iZ(), and ui64_push().

Referenced by get_root_primes(), SiZ(), and SiZm().

#### 13.4.3.15 range_info_free()

```
void range_info_free (
            IZM_RANGE_INFO * info)
```

Clear all GMP fields owned by info.

Definition at line 1032 of file iZ_toolkit.c.

References IZM_RANGE_INFO::Xe, IZM_RANGE_INFO::Xs, IZM_RANGE_INFO::Ye, IZM_RANGE_INFO::Ys, IZM_RANGE_INFO::Ze, and IZM_RANGE_INFO::Zs.

Referenced by SiZ_count(), and SiZ_stream().

#### 13.4.3.16 range_info_init()

```
IZM_RANGE_INFO range_info_init (
            INPUT_SIEVE_RANGE * input_range,
            int vx)
```

Map a decimal range input into iZm coordinates and segment bounds.

Parameters

| | |
|---|---|
| input_range | Range input (start, range). |
| vx | iZm segment width. |

Returns

Initialized range-info object; y_range < 0 indicates invalid input or an unsupported y-span for the current implementation.

Definition at line 986 of file iZ_toolkit.c.

References log_error(), INPUT_SIEVE_RANGE::range, INPUT_SIEVE_RANGE::start, IZM_RANGE_INFO::vx, IZM_RANGE_INFO::Xe, IZM_RANGE_INFO::Xs, IZM_RANGE_INFO::y_range, IZM_RANGE_INFO::Ye, IZM_RANGE_INFO::Ys, IZM_RANGE_INFO::Ze, and IZM_RANGE_INFO::Zs.

Referenced by SiZ_count(), and SiZ_stream().

#### 13.4.3.17 test_primality()

```
int test_primality (
            mpz_t n,
            int rounds)
```

Check the primality of a number using GMP's probabilistic test.

Test the primality of a number using GMP's probabilistic test.

Parameters

| | |
|---|---|
| n | Number to check. |
| rounds | Number of Miller-Rabin rounds. |

Returns

Non-zero if probably prime, 0 if composite.

Parameters

| | |
|---|---|
| n | Number to check. |

| | |
|---|---|
| rounds | Number of Miller-Rabin rounds. |

Returns

Non-zero if probably prime, 0 if composite.

This function serves as a single source of truth for primality testing, wrapping GMP's mpz_probab_↩prime_p function, allowing for changes to the underlying primality testing method in the future without affecting the API.

Definition at line 169 of file iZ_toolkit.c.

Referenced by vx_prob_sieve(), vx_search_prime(), vx_stream(), and vy_search_prime().

#### 13.4.3.18 vx_collect_p_gaps()

```
void vx_collect_p_gaps (
            VX_SEG * vx_obj)
```

Extract prime-gap encoding from a fully sieved segment.

Convert survivor bits into compact prime-gap encoding.

Parameters

| | |
|---|---|
| vx_obj | Segment object. |

This routine must run after deterministic/probabilistic sieving is complete. It populates VX_SEG::p_gaps for downstream streaming and gap traversal.

Parameters

| | |
|---|---|
| vx_obj | Segment object with validated prime survivors. |

Definition at line 801 of file iZ_toolkit.c.

References bitmap_get_bit(), VX_SEG::is_large_limit, VX_SEG::p_count, VX_SEG::p_gaps, VX_SEG::start_x, ui16_init(), ui16_push(), vx_full_sieve(), VX_SEG::x5, VX_SEG::x7, and VX_SEG::y.

Referenced by vx_full_sieve().

#### 13.4.3.19 vx_free()

```
void vx_free (
            VX_SEG ** vx_obj)
```

Free a VX segment and all owned resources.

Free all memory owned by a VX segment object.

Parameters

| | |
|---|---|
| vx_obj | Address of a VX_SEG pointer. |

| | |
|---|---|
| vx_obj | Address of the VX_SEG pointer to release. |

Definition at line 773 of file iZ_toolkit.c.

References bitmap_free(), and ui16_free().

Referenced by SiZ_count(), and SiZ_stream().

#### 13.4.3.20 vx_full_sieve()

```
void vx_full_sieve (
            VX_SEG * vx_obj,
            int collect_p_gaps)
```

Complete segment processing (probabilistic stage and optional gaps).

This function performs the sieve process on a given vx and y defined in the VX_SEG structure, and stores the primes gaps in the vx_obj->p_gaps array.

Parameters

| | |
|---|---|
| vx_obj | Segment object. |
| collect_p_gaps | Non-zero to populate VX_SEG::p_gaps. |

Description: This function combines deterministic sieving and probabilistic primality tests to identify prime candidates in a standard VX segment of a specific y in the iZ-Matrix. It could be used to only count primes in the vx segment, and optionally populates the vx_obj->p_gaps array with prime gaps between consecutive primes detected in the segment.

Parameters

| | |
|---|---|
| vx_obj | The VX_SEG to be processed. |
| collect_p_gaps | 0 only counts primes and doesn't store prime gaps, other non-zero int values populates vx_obj->p_gaps with detected prime gaps. |

Definition at line 861 of file iZ_toolkit.c.

References VX_SEG::is_large_limit, vx_collect_p_gaps(), and vx_prob_sieve().

Referenced by SiZ_count(), and vx_collect_p_gaps().

#### 13.4.3.21 vx_init()

```
VX_SEG * vx_init (
            IZM * iZm,
            int start_x,
            int end_x,
            char * y_str,
            int mr_rounds)
```

Initialize and deterministically sieve one VX segment.

Initialize the members of the VX_SEG structure with the given parameters and defaults.

Parameters

| | |
|---|---|
| iZm | Initialized toolkit context. |

| | |
|---|---|
| start_x | Inclusive x start index. |
| end_x | Inclusive x end index. |
| y_str | Segment index y as a decimal string. |
| mr_rounds | Miller-Rabin rounds (0 uses default). |

Returns

Initialized and partially processed segment, or NULL on failure.

Description: This function allocates memory for a VX_SEG structure and initializes its members. The count is initialized to 0, and the p_gaps array is allocated with an initial size of (vx/2).

Parameters:

Parameters

| | |
|---|---|
| y_str | A character pointer representing a numeric string. |

Returns

VX_SEG∗ A pointer to the initialized VX_SEG structure. NULL if memory allocation fails or y_str is not a numeric string.

Definition at line 727 of file iZ_toolkit.c.

References IZM::base_x5, IZM::base_x7, VX_SEG::bit_ops, bitmap_clone(), VX_SEG::end_x, log_error(), MAX, MIN, MR_ROUNDS, VX_SEG::mr_rounds, VX_SEG::p_count, VX_SEG::p_gaps, VX_SEG::p_test_ops, VX_SEG::start_x, IZM::vx, VX_SEG::vx, vx_det_sieve(), vx_set_base_values(), VX_SEG::x5, and VX_SEG::x7.

Referenced by SiZ_count(), and SiZ_stream().

### 13.4.3.22 vx_search_prime()

```
int vx_search_prime (
            mpz_t p,
            int m_id,
            int vx,
            int bit_size)
```

Horizontal iZm/VX random-prime search.

horizontal search routine for generating a random prime.

Parameters

| | |
|---|---|
| p | Output prime. |

| | |
|---|---|
| m_id | Requested line id (-1, +1, or random when other value). |
| vx | Segment width. |
| bit_size | Target bit size. |

Returns

1 on success, otherwise 0.

Description: This function searches for a prime number using the given parameters. It combines the iZ-Matrix filtering techniques with the Miller-Rabin primality test. The search is performed by finding suitable x values that do not correspond to composites of primes that divide vx. Then, it iterates over y values in the equation p = iZ(x + vx ∗ y) until a prime is found.

Parameters

| | |
|---|---|
| p | The prime number found in the search. |
| m_id | The identifier (1 or -1) for the iZ matrix. |
| vx | The horizontal vector of the iZ matrix. |
| bit_size | The target bit size of the prime. |

Returns

1 if a prime is found, 0 otherwise.

Definition at line 1060 of file iZ_toolkit.c.

References UI64_ARRAY::array, bitmap_clear_steps(), bitmap_free(), bitmap_get_bit(), bitmap_init(), UI64_ARRAY::count, gmp_seed_randstate(), iZ_mpz(), iZm_solve_for_x0_mpz(), log_error(), MAX, MR_ROUNDS, SiZm(), test_primality(), and ui64_free().

Referenced by vx_random_prime().

#### 13.4.3.23 vx_stream()

```
void vx_stream (
            VX_SEG * vx_obj,
            FILE * output,
            int stream_gaps)
```

Stream segment primes to an output stream.

Stream segment primes to an output stream in traversal order.

Parameters

| | |
|---|---|
| vx_obj | Segment object. |
| output | Writable output stream (e.g. stdout or a file). |

| | |
|---|---|
| stream_gaps | Non-zero to stream prime gaps instead of absolute primes. |
| vx_obj | Segment object. |
| output | Destination stream (stdout or file). |
| stream_gaps | If non-zero, output prime gaps instead of primes. |

Definition at line 881 of file iZ_toolkit.c.

References bitmap_clear_bit(), bitmap_get_bit(), VX_SEG::is_large_limit, iZ_mpz(), VX_SEG::mr_rounds, VX_SEG::p_count, VX_SEG::p_test_ops, VX_SEG::start_x, test_primality(), VX_SEG::x5, VX_SEG::x7, and VX_SEG::yvx.

Referenced by SiZ_stream().

#### 13.4.3.24 vy_search_prime()

```
int vy_search_prime (
            mpz_t p,
            int m_id,
            mpz_t vx)
```

Vertical iZm/VY random-prime search.

vertical search routine for generating a random prime.

Parameters

| | |
|---|---|
| p | Output prime. |
| m_id | Requested line id (-1, +1, or random when other value). |
| vx | Segment width. |

Returns

1 on success, otherwise 0.

Description: This function searches for a prime number using the given parameters. It combines the iZ-Matrix filtering techniques with the Miller-Rabin primality test. The search is performed by finding a suitable x value that does not correspond to a composite of a prime that divides vx. Then, it iterates over y value in the equation p = iZ(x + vx ∗ y, m_id) until a prime is found.

Parameters

| | |
|---|---|
| p | The prime number found in the search. |
| m_id | The identifier (1 or -1) for the iZ matrix. |
| vx | The horizontal vector of the iZ matrix. |

Returns

1 if a prime is found, 0 otherwise.

Definition at line 1157 of file iZ_toolkit.c.

References gmp_seed_randstate(), iZ_mpz(), MR_ROUNDS, and test_primality().

Referenced by vy_random_prime().

## 13.5 FFI API

ABI-stable wrapper functions for external language bindings.

### Data Structures

- struct IZP_U64_BUFFER

  Owned buffer of uint64_t values returned by FFI sieve routines.

### Enumerations

- enum IZP_FFI_STATUS {
  IZP_FFI_OK = 0 , IZP_FFI_ERR_INVALID_ARG = 1 , IZP_FFI_ERR_PARSE = 2 , IZP_FFI_ERR_ALLOC = 3 ,
  IZP_FFI_ERR_NOT_FOUND = 4 , IZP_FFI_ERR_OPERATION = 5 }

  Status/error codes returned by FFI functions.
- enum IZP_SIEVE_KIND {
  IZP_SIEVE_SOE = 0 , IZP_SIEVE_SSOE = 1 , IZP_SIEVE_SOS = 2 , IZP_SIEVE_SSOS = 3 ,
  IZP_SIEVE_SOEU = 4 , IZP_SIEVE_SOA = 5 , IZP_SIEVE_SIZ = 6 , IZP_SIEVE_SIZM = 7 ,
  IZP_SIEVE_SIZM_VY = 8 }

  Sieve model selector used by izp_ffi_sieve_u64.

### Functions

- IZP_FFI_API const char * izp_ffi_version (void)

  Return the iZprime semantic version string.
- IZP_FFI_API const char * izp_ffi_status_message (int code)

  Return a human-readable status message for code.
- IZP_FFI_API const char * izp_ffi_last_error (void)

  Return the last FFI error message from the current process.
- IZP_FFI_API void izp_ffi_clear_error (void)

  Clear the stored last-error string.
- IZP_FFI_API int izp_ffi_sieve_u64 (IZP_SIEVE_KIND kind, uint64_t limit, IZP_U64_BUFFER *out)

  Run a selected sieve model up to limit and return all primes.
- IZP_FFI_API int izp_ffi_count_range (const char *start, uint64_t range, int mr_rounds, int cores_num, uint64_t *out_count)

  Count primes in [start, start + range - 1].
- IZP_FFI_API int izp_ffi_stream_range (const char *start, uint64_t range, int mr_rounds, const char *filepath, int stream_gaps, uint64_t *out_count)

  Stream primes (or gaps) in [start, start + range - 1] to a file.
- IZP_FFI_API int izp_ffi_next_prime (const char *base_expr, int forward, char **out_prime_↩base10)

  Find the next/previous prime from base_expr.
- IZP_FFI_API int izp_ffi_random_prime_vx (int bit_size, int cores_num, char **out_prime_↩base10)

  Generate a random prime using vx-based search.
- IZP_FFI_API int izp_ffi_random_prime_vy (int bit_size, int cores_num, char **out_prime_↩base10)

  Generate a random prime using vy-based search.
- IZP_FFI_API void izp_ffi_free_u64_buffer (IZP_U64_BUFFER *buffer)

  Free a buffer returned by izp_ffi_sieve_u64.
- IZP_FFI_API void izp_ffi_free_string (char **str)

  Free a decimal string returned by FFI prime APIs.

### 13.5.1 Detailed Description

ABI-stable wrapper functions for external language bindings.

### 13.5.2 Enumeration Type Documentation

#### 13.5.2.1 IZP_FFI_STATUS

enum IZP_FFI_STATUS

Status/error codes returned by FFI functions.

Enumerator

| | |
|---|---|
| IZP_FFI_OK | Operation completed successfully. |
| IZP_FFI_ERR_INVALID_ARG | Invalid argument(s). |
| IZP_FFI_ERR_PARSE | Failed to parse numeric expression/input. |
| IZP_FFI_ERR_ALLOC | Memory allocation failure. |
| IZP_FFI_ERR_NOT_FOUND | Requested prime/candidate not found. |
| IZP_FFI_ERR_OPERATION | Underlying API operation failed. |

Definition at line 42 of file izprime_ffi.h.

#### 13.5.2.2 IZP_SIEVE_KIND

enum IZP_SIEVE_KIND

Sieve model selector used by izp_ffi_sieve_u64.

Definition at line 53 of file izprime_ffi.h.

### 13.5.3 Function Documentation

#### 13.5.3.1 izp_ffi_clear_error()

```
IZP_FFI_API void izp_ffi_clear_error (
            void )
```

Clear the stored last-error string.

Definition at line 166 of file izprime_ffi.c.

Referenced by izp_ffi_count_range(), izp_ffi_next_prime(), izp_ffi_random_prime_vx(), izp_ffi_random_prime_vy(
izp_ffi_sieve_u64(), and izp_ffi_stream_range().

#### 13.5.3.2 izp_ffi_count_range()

```
IZP_FFI_API int izp_ffi_count_range (
            const char * start,
            uint64_t range,
            int mr_rounds,
            int cores_num,
            uint64_t * out_count)
```

Count primes in [start, start + range - 1].

start accepts the same numeric expression syntax as the CLI/API parser.

Parameters

| | |
|---|---|
| start | Start bound expression. |

| | |
|---|---|
| range | Inclusive interval size. |
| mr_rounds | Miller-Rabin rounds (clamped by core API). |
| cores_num | Requested worker-process count. |
| out_count | Receives total prime count on success. |

Definition at line 229 of file izprime_ffi.c.

References izp_ffi_clear_error(), IZP_FFI_ERR_ALLOC, IZP_FFI_ERR_INVALID_ARG, IZP_FFI_ERR_PARSE, IZP_FFI_OK, MAX, and SiZ_count().

#### 13.5.3.3 izp_ffi_free_string()

```
IZP_FFI_API void izp_ffi_free_string (
            char ** str)
```

Free a decimal string returned by FFI prime APIs.

Parameters

| | |
|---|---|
| str | Address of owned string pointer. |

Definition at line 477 of file izprime_ffi.c.

#### 13.5.3.4 izp_ffi_free_u64_buffer()

```
IZP_FFI_API void izp_ffi_free_u64_buffer (
            IZP_U64_BUFFER * buffer)
```

Free a buffer returned by izp_ffi_sieve_u64.

Parameters

| | |
|---|---|
| buffer | Address of owned buffer object. |

Definition at line 467 of file izprime_ffi.c.

References IZP_U64_BUFFER::data, and IZP_U64_BUFFER::len.

#### 13.5.3.5 izp_ffi_last_error()

```
IZP_FFI_API const char * izp_ffi_last_error (
            void )
```

Return the last FFI error message from the current process.

The returned pointer is owned by the library and valid until the next FFI call that mutates the error state.

Definition at line 161 of file izprime_ffi.c.

#### 13.5.3.6 izp_ffi_next_prime()

```
IZP_FFI_API int izp_ffi_next_prime (
            const char * base_expr,
            int forward,
            char ** out_prime_base10)
```

Find the next/previous prime from base_expr.

Parameters

| | |
|---|---|
| base_expr | Base numeric expression. |

| | |
|---|---|
| forward | Non-zero for next prime, zero for previous prime. |
| out_prime_base10 | Receives heap-allocated decimal string. |

Definition at line 364 of file izprime_ffi.c.

References iZ_next_prime(), izp_ffi_clear_error(), IZP_FFI_ERR_INVALID_ARG, IZP_FFI_ERR_NOT_FOUND and IZP_FFI_ERR_PARSE.

#### 13.5.3.7 izp_ffi_random_prime_vx()

```
IZP_FFI_API int izp_ffi_random_prime_vx (
            int bit_size,
            int cores_num,
            char ** out_prime_base10)
```

Generate a random prime using vx-based search.

Parameters

| | |
|---|---|
| bit_size | Target prime bit length. |
| cores_num | Requested worker-process count. |
| out_prime_base10 | Receives heap-allocated decimal string. |

Definition at line 401 of file izprime_ffi.c.

References izp_ffi_clear_error(), IZP_FFI_ERR_INVALID_ARG, IZP_FFI_ERR_OPERATION, MAX, and vx_random_prime().

#### 13.5.3.8 izp_ffi_random_prime_vy()

```
IZP_FFI_API int izp_ffi_random_prime_vy (
            int bit_size,
            int cores_num,
            char ** out_prime_base10)
```

Generate a random prime using vy-based search.

Parameters

| | |
|---|---|
| bit_size | Target prime bit length. |
| cores_num | Requested worker-process count. |
| out_prime_base10 | Receives heap-allocated decimal string. |

Definition at line 434 of file izprime_ffi.c.

References izp_ffi_clear_error(), IZP_FFI_ERR_INVALID_ARG, IZP_FFI_ERR_OPERATION, MAX, and vy_random_prime().

#### 13.5.3.9 izp_ffi_sieve_u64()

```
IZP_FFI_API int izp_ffi_sieve_u64 (
            IZP_SIEVE_KIND kind,
            uint64_t limit,
            IZP_U64_BUFFER * out)
```

Run a selected sieve model up to limit and return all primes.

Parameters

| | |
|---|---|
| kind | Sieve model identifier. |

| | |
|---|---|
| limit | Inclusive upper bound (must satisfy implementation limits). |
| out | Buffer to populate (caller frees via izp_ffi_free_u64_buffer). |

Returns

IZP_FFI_OK on success.

Definition at line 171 of file izprime_ffi.c.

References IZP_U64_BUFFER::data, izp_ffi_clear_error(), IZP_FFI_ERR_INVALID_ARG, IZP_U64_BUFFER::len, SiZ(), SiZm(), SiZm_vy(), SoA(), SoE(), SoEu(), SoS(), SSoE(), and SSoS().

#### 13.5.3.10 izp_ffi_status_message()

```
IZP_FFI_API const char * izp_ffi_status_message (
            int code)
```

Return a human-readable status message for code.

Parameters

| | |
|---|---|
| code | One of IZP_FFI_STATUS values. |

Definition at line 140 of file izprime_ffi.c.

References IZP_FFI_ERR_ALLOC, IZP_FFI_ERR_INVALID_ARG, IZP_FFI_ERR_NOT_FOUND, IZP_FFI_ERR_OPERATION, IZP_FFI_ERR_PARSE, and IZP_FFI_OK.

#### 13.5.3.11 izp_ffi_stream_range()

```
IZP_FFI_API int izp_ffi_stream_range (
            const char * start,
            uint64_t range,
            int mr_rounds,
            const char * filepath,
            int stream_gaps,
            uint64_t * out_count)
```

Stream primes (or gaps) in [start, start + range - 1] to a file.

start accepts the same numeric expression syntax as the CLI/API parser.

Parameters

| | |
|---|---|
| start | Start bound expression. |
| range | Inclusive interval size. |
| mr_rounds | Miller-Rabin rounds (clamped by core API). |
| filepath | Destination file path. |

| | |
|---|---|
| stream_gaps | Non-zero to stream prime gaps instead of prime values. |
| out_count | Receives total prime count on success. |

Definition at line 288 of file izprime_ffi.c.

References izp_ffi_clear_error(), IZP_FFI_ERR_ALLOC, IZP_FFI_ERR_INVALID_ARG, IZP_FFI_ERR_OPERATION, IZP_FFI_ERR_PARSE, IZP_FFI_OK, and SiZ_stream().

#### 13.5.3.12 izp_ffi_version()

```
IZP_FFI_API const char * izp_ffi_version (
            void )
```

Return the iZprime semantic version string.

Definition at line 131 of file izprime_ffi.c.

## 13.6 Logging

Thread-safe runtime logging helpers.

Files

- file logger.c
  Logging module.

Macros

- #define LOGGER_FORMAT_PRINTF(fmt_idx, arg_idx)
  Compiler attribute wrapper for printf-style format checking.
- #define LOG_DIR "logs/"
  Directory where logs are stored.
- #define LOG_FILE LOG_DIR "log.txt"
  Default log file.
- #define LOG_MAX_SIZE 1024 * 1024 * 5
  Maximum log file size (5 MB).

Enumerations

- enum LogLevel {
  LOG_DEBUG , LOG_INFO , LOG_WARNING , LOG_ERROR ,
  LOG_FATAL }
  Enumeration of log levels.

Functions

- const char * log_level_to_string (LogLevel level)

  Returns a string representation of the log level.
- void log_init (const char *log_file)

  Initializes the logging system.
- void log_shutdown (void)

  Shuts down the logging system and cleans up resources.
- void log_set_log_level (LogLevel level)

  Sets the current log level. Messages below this level will not be logged.
- void log_message (LogLevel level, const char *format,...) LOGGER_FORMAT_PRINTF(2

  Logs a formatted message at the given log level.
- void void log_message_extended (LogLevel level, const char *file_name, int line_number, const char *format,...) LOGGER_FORMAT_PRINTF(4

  Logs a formatted message with extended information (file name, line number).
- void void void log_console (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs a timestamped message to the console only, without requiring a log level.
- void void void void log_debug (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs a debug message.
- void void void void void log_info (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs an info message.
- void void void void void void log_warn (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs a warning message.
- void void void void void void void log_error (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs an error message.
- void void void void void void void void log_fatal (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs a fatal message.

## 13.6.1 Detailed Description

Thread-safe runtime logging helpers.

## 13.6.2 Macro Definition Documentation

### 13.6.2.1 LOG_DIR

#define LOG_DIR "logs/"

Directory where logs are stored.

Definition at line 54 of file logger.h.

### 13.6.2.2 LOG_FILE

#define LOG_FILE LOG_DIR "log.txt"

Default log file.

Definition at line 55 of file logger.h.

#### 13.6.2.3 LOG_MAX_SIZE

#define LOG_MAX_SIZE 1024 ∗ 1024 ∗ 5

Maximum log file size (5 MB).

Definition at line 56 of file logger.h.

#### 13.6.2.4 LOGGER_FORMAT_PRINTF

```
#define LOGGER_FORMAT_PRINTF(
            fmt_idx,
            arg_idx)
```

Compiler attribute wrapper for printf-style format checking.

Definition at line 50 of file logger.h.

Referenced by log_console(), log_debug(), log_error(), log_fatal(), log_info(), log_message(), log_message_extended(), and log_warn().

### 13.6.3 Enumeration Type Documentation

#### 13.6.3.1 LogLevel

enum LogLevel

Enumeration of log levels.

This enumeration defines the different log levels that can be used in the logging system. Each level corresponds to a severity of importance.

Enumerator

| | |
|---|---|
| LOG_DEBUG | Debug level. |
| LOG_INFO | Info level. |
| LOG_WARNING | Warning level. |
| LOG_ERROR | Error level. |
| LOG_FATAL | Fatal level. |

Definition at line 65 of file logger.h.

### 13.6.4 Function Documentation

#### 13.6.4.1 log_console()

```
void void void log_console (
            const char * format,
            ...)
```

Logs a timestamped message to the console only, without requiring a log level.

This can be used for debugging or tracking specific points in the code.

Parameters

| | |
|---|---|
| format | The format string for the log message. |

References log_console(), and LOGGER_FORMAT_PRINTF.

Referenced by log_console().

#### 13.6.4.2 log_debug()

```
void void void void log_debug (
            const char * format,
            ...)
```

Logs a debug message.

Parameters

| | |
|---|---|
| format | The format string for the log message. |

References log_debug(), and LOGGER_FORMAT_PRINTF.

Referenced by iZ_next_prime(), and log_debug().

#### 13.6.4.3 log_error()

```
void void void void void void void log_error (
            const char * format,
            ...)
```

Logs an error message.

Parameters

| | |
|---|---|
| format | The format string for the log message. |

References log_error(), and LOGGER_FORMAT_PRINTF.

Referenced by bitmap_fread(), bitmap_fwrite(), bitmap_init(), bitmap_validate_hash(), create_dir(), iZm_clone(), iZm_init(), log_error(), range_info_init(), SiZ_count(), SiZ_stream(), vx_init(), vx_random_prime(), vx_search_prime(), vx_set_base_values(), and vy_random_prime().

#### 13.6.4.4 log_fatal()

```
void void void void void void void void log_fatal (
            const char * format,
            ...)
```

Logs a fatal message.

Parameters

| | |
|---|---|
| format | The format string for the log message. |

References log_fatal(), and LOGGER_FORMAT_PRINTF.

Referenced by log_fatal().

#### 13.6.4.5 log_info()

```
void void void void void log_info (
            const char * format,
            ...)
```

Logs an info message.

Parameters

| | |
|---|---|
| format | The format string for the log message. |

References log_info(), and LOGGER_FORMAT_PRINTF.

Referenced by log_info(), SiZ_count(), vx_prob_sieve(), vx_random_prime(), and vy_random_prime().

#### 13.6.4.6 log_init()

```
void log_init (
            const char * log_file)
```

Initializes the logging system.

Creates the log directory if it doesn't exist and rotates logs if necessary.

Parameters

| | |
|---|---|
| log_file | The path to the log file. |

#### 13.6.4.7 log_level_to_string()

```
const char * log_level_to_string (
            LogLevel level)
```

Returns a string representation of the log level.

Parameters

| | |
|---|---|
| level | The log level. |

Returns

A string corresponding to the log level.

#### 13.6.4.8 log_message()

```
void log_message (
            LogLevel level,
            const char * format,
            ...)
```

Logs a formatted message at the given log level.

This function logs messages to the log file in a thread-safe manner.

Parameters

| | |
|---|---|
| level | The log level for the message. |
| format | The format string for the log message. |

References LOGGER_FORMAT_PRINTF.

#### 13.6.4.9 log_message_extended()

```
void void log_message_extended (
            LogLevel level,
            const char * file_name,
            int line_number,
            const char * format,
            ...)
```

Logs a formatted message with extended information (file name, line number).

This function logs messages to the log file in a thread-safe manner.

Parameters

| | |
|---|---|
| level | The log level for the message. |

| | |
|---|---|
| file_name | The source file where the log was generated. |
| line_number | The line number in the source file. |
| format | The format string for the log message. |

References log_message_extended(), and LOGGER_FORMAT_PRINTF.

Referenced by log_message_extended().

#### 13.6.4.10 log_set_log_level()

```
void log_set_log_level (
            LogLevel level)
```

Sets the current log level. Messages below this level will not be logged.

Parameters

| | |
|---|---|
| level | The log level to set. |

Referenced by cli_run().

#### 13.6.4.11 log_warn()

```
void void void void void void log_warn (
            const char * format,
            ...)
```

Logs a warning message.

Parameters

| | |
|---|---|
| format | The format string for the log message. |

References log_warn(), and LOGGER_FORMAT_PRINTF.

Referenced by log_warn(), vx_random_prime(), and vy_random_prime().

## 13.7 Platform Abstraction

OS-adaptation helpers used by utility and API layers.

Files

- file platform.c
  Platform abstraction implementation.

Functions

- int iz_platform_create_dir (const char *dir)
  Create a directory if it does not already exist.
- int iz_platform_fill_random (void *buffer, size_t bytes)
  Fill a buffer with random bytes.
- int iz_platform_cpu_cores_count (void)
  Return number of online logical CPU cores (>= 1).
- int iz_platform_l2_cache_size_bits (void)
  Best-effort L2 cache size query in bits.
- double iz_platform_monotonic_seconds (void)
  Return a monotonic timestamp in seconds.
- int iz_platform_localtime (const time_t *timestamp, struct tm *out)
  Convert calendar time to local broken-down time (thread-safe).

## 13.7.1 Detailed Description

OS-adaptation helpers used by utility and API layers.

## 13.7.2 Function Documentation

### 13.7.2.1 iz_platform_cpu_cores_count()

```
int iz_platform_cpu_cores_count (
            void )
```

Return number of online logical CPU cores (>= 1).

Returns

Core count.

Definition at line 68 of file platform.c.

Referenced by get_cpu_cores_count().

### 13.7.2.2 iz_platform_create_dir()

```
int iz_platform_create_dir (
            const char * dir)
```

Create a directory if it does not already exist.

Parameters

| | |
|---|---|
| dir | Directory path. |

Returns

0 on success, -1 on failure.

Definition at line 27 of file platform.c.

Referenced by create_dir().

#### 13.7.2.3 iz_platform_fill_random()

```
int iz_platform_fill_random (
            void * buffer,
            size_t bytes)
```

Fill a buffer with random bytes.

Parameters

| | |
|---|---|
| buffer | Destination buffer. |
| bytes | Number of bytes to fill. |

Returns

1 on success, 0 on failure.

Definition at line 42 of file platform.c.

Referenced by gmp_seed_randstate().

#### 13.7.2.4 iz_platform_l2_cache_size_bits()

```
int iz_platform_l2_cache_size_bits (
            void )
```

Best-effort L2 cache size query in bits.

Returns

L2 size in bits, or a conservative fallback.

Definition at line 81 of file platform.c.

Referenced by get_cpu_L2_cache_size_bits().

#### 13.7.2.5 iz_platform_localtime()

```
int iz_platform_localtime (
            const time_t * timestamp,
            struct tm * out)
```

Convert calendar time to local broken-down time (thread-safe).

Parameters

| | |
|---|---|
| timestamp | Source calendar time. |
| out | Destination broken-down local time. |

Returns

1 on success, 0 on failure.

Definition at line 135 of file platform.c.

#### 13.7.2.6 iz_platform_monotonic_seconds()

```
double iz_platform_monotonic_seconds (
            void )
```

Return a monotonic timestamp in seconds.

Returns

Monotonic seconds.

Definition at line 115 of file platform.c.

Referenced by sw_elapsed_now_seconds().

## 13.8 Tests and Benchmarks

Test harness entry points.

Data Structures

- struct SIEVE_LIMIT
- struct SIEVE_MODEL

Typedefs

- typedef UI64_ARRAY *(* SIEVE_FN) (uint64_t)

Sieve model tests/benchmarks

- int TEST_UTILS (int verbose)

  Validate utility helpers (numeric/range parsers and shared utilities).
- int TEST_FFI (int verbose)

  Validate the bindings-oriented C FFI surface (izprime_ffi.h).
- int TEST_SIEVE_MODELS_INTEGRITY (int verbose)

  Run correctness checks across all registered sieve models.
- void BENCHMARK_SIEVE_MODELS (int save_results)

  Benchmark registered sieve models and optionally persist results.

Range sieve tests/benchmarks

- int TEST_SiZ_stream (int verbose)

  Validate SiZ_stream behavior and output formatting.
- int TEST_SiZ_count (int verbose)

  Validate SiZ_count correctness across worker counts.
- void BENCHMARK_SiZ_count (int save_results)

  Benchmark SiZ_count over increasing ranges starting from 10^10, 10^20, ..., 10^100 using max cores.

Prime generation tests/benchmarks

- int TEST_CLI (int verbose)

  Run CLI command-dispatch smoke tests.
- int TEST_iZ_next_prime (int verbose)

  Validate iZ_next_prime for forward/backward traversal cases.
- int TEST_vy_random_prime (int verbose)

  Validate vy_random_prime for primality and bit-length constraints.
- int TEST_vx_random_prime (int verbose)

  Validate vx_random_prime for primality and bit-length constraints.
- int BENCHMARK_P_GEN_ALGORITHMS (int bit_size, int test_rounds, int save_results)

  Benchmark random-prime generation routines over repeated trials.

### 13.8.1 Detailed Description

Test harness entry points.

### 13.8.2 Typedef Documentation

#### 13.8.2.1 SIEVE_FN

typedef UI64_ARRAY *(* SIEVE_FN) (uint64_t)

Sieve function type: takes a limit and returns a prime list.

Definition at line 20 of file test_api.h.

## 13.9 Utilities

Cross-cutting helpers used across modules.

Files

- file utils.c

  Implementations for the shared utility layer.

Macros

- #define MIN(a, b)

  Return the smaller of two expressions.
- #define MAX(a, b)

  Return the larger of two expressions.
- #define DIR_output "./output"
- #define MAX_CORES get_cpu_cores_count()

Functions

- int create_dir (const char *dir)

  Create a directory if it does not exist.
- int is_numeric_str (const char *str)

  Return non-zero if str contains only ASCII digits.
- int parse_numeric_expr_mpz (mpz_t out, const char *expr)

  Parse an integer expression into an mpz value.
- int parse_numeric_expr_u64 (const char *expr, uint64_t *out)

  Parse an integer expression into uint64_t.
- int parse_inclusive_range_mpz (const char *range_expr, mpz_t lower, mpz_t upper)

  Parse an inclusive range expression into lower/upper bounds.
- uint64_t gcd (uint64_t a, uint64_t b)

  Compute the greatest common divisor of a and b.
- uint64_t lcm (uint64_t a, uint64_t b)

  Compute the least common multiple of a and b.
- void gcd_mpz (mpz_t out, const mpz_t a, const mpz_t b)

  Compute the greatest common divisor of two arbitrary-precision integers.
- void lcm_mpz (mpz_t out, const mpz_t a, const mpz_t b)

  Compute the least common multiple of two arbitrary-precision integers.
- uint64_t modular_inverse (uint64_t a, uint64_t m)

  Compute the modular inverse of a modulo m.
- void gmp_seed_randstate (gmp_randstate_t state)

  Seed a GMP random state.
- int get_cpu_cores_count (void)

  Get the number of online CPU cores (>= 1).
- int get_cpu_L2_cache_size_bits (void)

  Return the CPU L2 cache size in bits (best effort).

## 13.9.1 Detailed Description

Cross-cutting helpers used across modules.

## 13.9.2 Macro Definition Documentation

### 13.9.2.1 DIR_output

#define DIR_output "./output"

Default directory for output artifacts produced by examples/tests.

Definition at line 48 of file utils.h.

Referenced by cli_run().

#### 13.9.2.2 MAX

```
#define MAX(
            a,
            b)
```

Value:
(((a) > (b)) ? (a) : (b))

Return the larger of two expressions.

Definition at line 44 of file utils.h.

Referenced by izp_ffi_count_range(), izp_ffi_random_prime_vx(), izp_ffi_random_prime_vy(), SiZ_count(), SiZ_stream(), SSoE(), vx_init(), vx_random_prime(), vx_search_prime(), and vy_random_prime().

#### 13.9.2.3 MAX_CORES

```
#define MAX_CORES get_cpu_cores_count()
```

Convenience macro: number of online CPU cores.

Definition at line 50 of file utils.h.

#### 13.9.2.4 MIN

```
#define MIN(
            a,
            b)
```

Value:
(((a) < (b)) ? (a) : (b))

Return the smaller of two expressions.

Definition at line 40 of file utils.h.

Referenced by bitmap_clear_steps(), bitmap_clear_steps_simd(), compute_l2_vx(), SiZ_count(), SiZ_stream(), and vx_init().

### 13.9.3 Function Documentation

#### 13.9.3.1 create_dir()

```
int create_dir (
            const char * dir)
```

Create a directory if it does not exist.

Parameters

| | |
|---|---|
| dir | Directory path. |

Returns

0 on success (including already exists), -1 on failure.

Parameters

| | |
|---|---|
| dir | The directory path. |

Returns

int 0 if the directory already exists or was successfully created, -1 otherwise.

Definition at line 509 of file utils.c.

References iz_platform_create_dir(), and log_error().

Referenced by cli_run().

#### 13.9.3.2 gcd()

```
uint64_t gcd (
            uint64_t a,
            uint64_t b)
```

Compute the greatest common divisor of a and b.

Compute the greatest common divisor of a and b.

Parameters

| | |
|---|---|
| a | The first integer. |
| b | The second integer. |

Returns

int The GCD of a and b.

Definition at line 526 of file utils.c.

Referenced by iZm_solve_for_y0(), lcm(), and SiZm_vy().

#### 13.9.3.3 gcd_mpz()

```
void gcd_mpz (
            mpz_t out,
            const mpz_t a,
            const mpz_t b)
```

Compute the greatest common divisor of two arbitrary-precision integers.

Parameters

| | |
|---|---|
| out | Output GCD value. |
| a | First input. |
| b | Second input. |

Compute the greatest common divisor of two arbitrary-precision integers.

Parameters

| | |
|---|---|
| out | Output GCD value. |
| a | The first integer. |
| b | The second integer. |

Definition at line 559 of file utils.c.

#### 13.9.3.4 get_cpu_cores_count()

```
int get_cpu_cores_count (
            void )
```

Get the number of online CPU cores (>= 1).

Get the number of online CPU cores (>= 1).

Returns

int The number of CPU cores.

Definition at line 641 of file utils.c.

References iz_platform_cpu_cores_count().

Referenced by SiZ_count().

#### 13.9.3.5 get_cpu_L2_cache_size_bits()

```
int get_cpu_L2_cache_size_bits (
            void )
```

Return the CPU L2 cache size in bits (best effort).

Returns

Cache size in bits, or a conservative default.

Return the CPU L2 cache size in bits (best effort).

Description: This function retrieves the L2 cache size using platform-specific methods:

- Linux: Reads from sysfs (/sys/devices/system/cpu/cpu0/cache/index2/size)
- macOS/BSD: Uses sysctl to query hw.l2cachesize
- Fallback: Returns 256 Kbits as a conservative default

The function attempts multiple detection methods to ensure robustness across different architectures and operating systems.

Returns

int The L2 cache size in bits, or 256 Kbits if unable to determine.

Definition at line 660 of file utils.c.

References iz_platform_l2_cache_size_bits().

Referenced by compute_l2_vx().

#### 13.9.3.6 gmp_seed_randstate()

```
void gmp_seed_randstate (
            gmp_randstate_t state)
```

Seed a GMP random state.

Uses platform entropy sources (OpenSSL-backed) with a time-based fallback.

Seed a GMP random state.

Description: This function seeds the GMP random state using platform entropy (OpenSSL-backed), with a time-based fallback.

Parameters

| | |
|---|---|
| state | The GMP random state. |

Definition at line 627 of file utils.c.

References iz_platform_fill_random().

Referenced by vx_search_prime(), and vy_search_prime().

#### 13.9.3.7 is_numeric_str()

```
int is_numeric_str (
            const char * str)
```

Return non-zero if str contains only ASCII digits.

Parameters

| | |
|---|---|
| str | Input string. |

Returns

1 if numeric, 0 otherwise.

Return non-zero if str contains only ASCII digits.

Parameters

| | |
|---|---|
| str | Pointer to the string. |

Returns

int 1 if the string is numeric, 0 otherwise.

Definition at line 387 of file utils.c.

#### 13.9.3.8 lcm()

```
uint64_t lcm (
            uint64_t a,
            uint64_t b)
```

Compute the least common multiple of a and b.

Returns

Least common multiple, or 0 when either input is 0.

Compute the least common multiple of a and b.

Parameters

| | |
|---|---|
| a | The first integer. |
| b | The second integer. |

Returns

uint64_t The LCM of a and b, or 0 when either input is 0.

Definition at line 544 of file utils.c.

References gcd().

#### 13.9.3.9 lcm_mpz()

```
void lcm_mpz (
            mpz_t out,
            const mpz_t a,
            const mpz_t b)
```

Compute the least common multiple of two arbitrary-precision integers.

Parameters

| | |
|---|---|
| out | Output LCM value. |
| a | First input. |
| b | Second input. |

Compute the least common multiple of two arbitrary-precision integers.

Parameters

| | |
|---|---|
| out | Output LCM value. |
| a | The first integer. |
| b | The second integer. |

Definition at line 571 of file utils.c.

#### 13.9.3.10 modular_inverse()

```
uint64_t modular_inverse (
            uint64_t a,
            uint64_t m)
```

Compute the modular inverse of a modulo m.

Returns

The inverse in [0, m-1] if it exists; otherwise an implementation-defined value.

Compute the modular inverse of a modulo m.

Description: This function computes the modular inverse of a modulo m using the Extended Euclidean Algorithm. The function takes two integers a and m as input and returns the modular inverse of a modulo m. If a and m are not co-prime, the function returns -1 indicating no modular inverse exists. The function also handles the case where m is 1, returning 0 as the modular inverse.

Parameters:

Parameters

| | |
|---|---|
| a | The integer for which the modular inverse is to be computed. |

| | |
|---|---|
| m | The modulus. |

Returns

The modular inverse of a modulo m, or -1 if no inverse exists.

Definition at line 593 of file utils.c.

Referenced by iZm_solve_for_y0().

#### 13.9.3.11 parse_inclusive_range_mpz()

```
int parse_inclusive_range_mpz (
            const char * range_expr,
            mpz_t lower,
            mpz_t upper)
```

Parse an inclusive range expression into lower/upper bounds.

Accepted forms:

- L,R
- [L, R]
- range[L, R]
- L..R
- L:R

Each bound accepts the same numeric formats as parse_numeric_expr_mpz.

Parameters

| | |
|---|---|
| range_expr | Range expression string. |
| lower | Output lower bound. |
| upper | Output upper bound. |

Returns

1 on success, 0 on parse failure.

Definition at line 435 of file utils.c.

#### 13.9.3.12 parse_numeric_expr_mpz()

```
int parse_numeric_expr_mpz (
            mpz_t out,
            const char * expr)
```

Parse an integer expression into an mpz value.

Supported term formats:

- plain decimal integer (1000000, 1,000,000)
- arithmetic operators (+, -, *, /)
- power/scientific operators (^, e, E)
- parenthesized sub-expressions ((10^6 + 5) * 3)

Parameters

| | |
|---|---|
| out | Output mpz value. |

| | |
|---|---|
| expr | Input expression string. |

Returns

1 on success, 0 on parse failure.

Definition at line 401 of file utils.c.

Referenced by parse_numeric_expr_u64().

#### 13.9.3.13 parse_numeric_expr_u64()

```
int parse_numeric_expr_u64 (
                const char * expr,
                uint64_t * out)
```

Parse an integer expression into uint64_t.

Parameters

| | |
|---|---|
| expr | Input expression string. |
| out | Output value. |

Returns

1 on success, 0 on parse failure or overflow.

Definition at line 414 of file utils.c.

References parse_numeric_expr_mpz().

## 13.10 Printer, implemented in toolkit/print_utils.c

Console formatting helpers used by tests and benchmark runners.

Files

- file print_utils.c
  Print formatting helpers for tests, benchmarks, and CLI output.

Functions

- void print_sha256_hash (const unsigned char *hash)

  Print a SHA-256 digest as hex to stdout.
- void print_line (int length, char fill_char)

  Print a repeated-character horizontal line.
- void print_centered_text (const char *text, int line_length, char fill_char)

  Print text centered inside a padded line.
- void print_test_table_header (void)

  Print the generic test runner header.
- void print_test_module_header (char *module_name)

  Print a test-suite header banner for a module.
- void print_test_fn_header (char *fn_name)

  Print a formatted header for a test function.
- void print_test_module_result (int result, int test_id, const char *unit_name, const char *format,...)

  Print a single test-row result.
- void print_test_summary (char *module_name, int passed, int failed, int verbose)

  Print module-level test summary.

## 13.10.1 Detailed Description

Console formatting helpers used by tests and benchmark runners.

## 13.10.2 Function Documentation

### 13.10.2.1 print_centered_text()

```
void print_centered_text (
            const char * text,
            int line_length,
            char fill_char)
```

Print text centered inside a padded line.

Print centered text within a line of specified length.

Parameters

| | |
|---|---|
| text | Text payload. |
| line_length | Target total line width. |
| fill_char | Padding character. |
| text | Text to center. |
| line_length | Total length of the line. |
| fill_char | Character to use for padding (' ' is used when zero). |

Definition at line 44 of file print_utils.c.

Referenced by print_test_fn_header().

#### 13.10.2.2 print_line()

```
void print_line (
            int length,
            char fill_char)
```

Print a repeated-character horizontal line.

Parameters

| | |
|---|---|
| length | Number of characters to print. |
| fill_char | Character to repeat ('-' is used when zero). |

Definition at line 28 of file print_utils.c.

Referenced by print_test_fn_header(), print_test_module_header(), print_test_summary(), and print_test_table_header().

#### 13.10.2.3 print_sha256_hash()

```
void print_sha256_hash (
            const unsigned char * hash)
```

Print a SHA-256 digest as hex to stdout.

Print the SHA-256 hash in hexadecimal format.

Parameters

| | |
|---|---|
| hash | Pointer to the SHA-256 hash array. |

Definition at line 73 of file print_utils.c.

#### 13.10.2.4 print_test_fn_header()

```
void print_test_fn_header (
            char * fn_name)
```

Print a formatted header for a test function.

Definition at line 163 of file print_utils.c.

References print_centered_text(), and print_line().

#### 13.10.2.5 print_test_module_header()

```
void print_test_module_header (
            char * module_name)
```

Print a test-suite header banner for a module.

Print module-level banner for test output.

Parameters

| | |
|---|---|
| module_name | Display name of the tested module. |

Definition at line 85 of file print_utils.c.

References print_line().

#### 13.10.2.6 print_test_module_result()

```
void print_test_module_result (
            int result,
            int test_id,
            const char * unit_name,
            const char * format,
            ...)
```

Print a single test-row result.

Print a single test unit result row with formatted details.

Parameters

| | |
|---|---|
| result | 1 for pass, 0 for fail. |
| test_id | Monotonic case index. |
| unit_↩ name | Short test unit label. |
| format | printf-style detail format. |
| result | 1 for pass, 0 for fail. |
| test_id | Monotonic case index. |
| unit_↩ name | Short test unit label. |
| format | printf-style detail format string. |
| ... | Arguments for the detail format string. |

Definition at line 111 of file print_utils.c.

#### 13.10.2.7 print_test_summary()

```
void print_test_summary (
            char * module_name,
            int passed,
            int failed,
            int verbose)
```

Print module-level test summary.

Print a summary of test results for a module.

Parameters

| | |
|---|---|
| module_name | Test module name. |

| | |
|---|---|
| passed | Number of passing tests. |
| failed | Number of failing tests. |
| verbose | Non-zero enables richer output hints. |
| module_name | Name of the tested module. |
| passed | Number of passing tests. |
| failed | Number of failing tests. |
| verbose | If non-zero, prints additional hints about the results. |

Definition at line 141 of file print_utils.c.

References print_line().

#### 13.10.2.8 print_test_table_header()

```
void print_test_table_header (
            void )
```

Print the generic test runner header.

Print standard test-table column headers.

Definition at line 95 of file print_utils.c.

References print_line().

## 13.11 Stopwatch

Monotonic wall-clock timing helpers for tests and benchmarks.

Files

- file stopwatch.c
  Stopwatch helpers based on monotonic wall-clock time.

Data Structures

- struct STOPWATCH
  Stopwatch state for elapsed wall-clock measurements.

Functions

- void sw_start (STOPWATCH *sw)
  Start or restart a stopwatch.
- void sw_stop (STOPWATCH *sw)
  Stop a running stopwatch, capturing the elapsed time in seconds.
- double sw_elapsed_seconds (const STOPWATCH *sw)
  Return elapsed seconds for the stopwatch.
- double sw_elapsed_now_seconds (void)
  Capture the current monotonic time in seconds.

### 13.11.1 Detailed Description

Monotonic wall-clock timing helpers for tests and benchmarks.

### 13.11.2 Function Documentation

#### 13.11.2.1 sw_elapsed_now_seconds()

```
double sw_elapsed_now_seconds (
            void )
```

Capture the current monotonic time in seconds.

Returns

Current monotonic timestamp in seconds.

Definition at line 68 of file stopwatch.c.

References iz_platform_monotonic_seconds().

#### 13.11.2.2 sw_elapsed_seconds()

```
double sw_elapsed_seconds (
            const STOPWATCH * sw)
```

Return elapsed seconds for the stopwatch.

If the stopwatch is still running, elapsed time is computed against the current monotonic timestamp.

Parameters

| | |
|---|---|
| sw | Stopwatch object. |

Returns

Elapsed seconds.

Definition at line 57 of file stopwatch.c.

References STOPWATCH::elapsed_sec, STOPWATCH::running, and STOPWATCH::start_time.

#### 13.11.2.3 sw_start()

```
void sw_start (
            STOPWATCH * sw)
```

Start or restart a stopwatch.

Parameters

| | |
|---|---|
| sw | Stopwatch object. |

Definition at line 39 of file stopwatch.c.

References STOPWATCH::end_time, STOPWATCH::running, and STOPWATCH::start_time.

#### 13.11.2.4 sw_stop()

```
void sw_stop (
            STOPWATCH * sw)
```

Stop a running stopwatch, capturing the elapsed time in seconds.

Parameters

| | |
|---|---|
| sw | Stopwatch object. |

Definition at line 47 of file stopwatch.c.

References STOPWATCH::elapsed_sec, STOPWATCH::end_time, STOPWATCH::running, and STOPWATCH::start_time.

# Chapter 14

# Data Structure Documentation

## 14.1 BITMAP Struct Reference

Packed bit-array with optional SHA-256 checksum.

#include <bitmap.h>

Data Fields

- size_t size
- size_t byte_size
- unsigned char * data
- unsigned char sha256 [SHA256_DIGEST_LENGTH]

### 14.1.1 Detailed Description

Packed bit-array with optional SHA-256 checksum.

size is measured in bits and byte_size is the backing storage in bytes. The checksum is maintained explicitly via bitmap_compute_hash() and verified via bitmap_validate_hash().

Definition at line 26 of file bitmap.h.

### 14.1.2 Field Documentation

#### 14.1.2.1 byte_size

size_t BITMAP::byte_size

Number of bytes in data.

Definition at line 29 of file bitmap.h.

Referenced by bitmap_clone(), bitmap_compute_hash(), bitmap_fread(), bitmap_fwrite(), bitmap_init(), bitmap_validate_hash(), SiZm(), and SSoS().

#### 14.1.2.2 data

unsigned char* BITMAP::data

Packed bits (LSB-first per byte).

Definition at line 30 of file bitmap.h.

Referenced by bitmap_clear_all(), bitmap_clear_bit(), bitmap_clear_steps(), bitmap_clear_steps_simd(), bitmap_clone(), bitmap_compute_hash(), bitmap_flip_bit(), bitmap_fread(), bitmap_fwrite(), bitmap_get_bit(), bitmap_init(), bitmap_set_all(), bitmap_set_bit(), bitmap_validate_hash(), SiZm(), and SSoS().

#### 14.1.2.3 sha256

unsigned char BITMAP::sha256[SHA256_DIGEST_LENGTH]

Cached SHA-256 checksum.

Definition at line 31 of file bitmap.h.

Referenced by bitmap_clone(), bitmap_compute_hash(), bitmap_fread(), bitmap_fwrite(), bitmap_init(), and bitmap_validate_hash().

#### 14.1.2.4 size

size_t BITMAP::size

Number of addressable bits.

Definition at line 28 of file bitmap.h.

Referenced by bitmap_clear_all(), bitmap_clear_steps(), bitmap_clear_steps_simd(), bitmap_clone(), bitmap_fwrite(), bitmap_init(), and bitmap_set_all().

The documentation for this struct was generated from the following file:

- bitmap.h

## 14.2 CLI_BINARY_EXPR_OPTIONS Struct Reference

Data Fields

- const char * a_expr
- const char * b_expr

### 14.2.1 Detailed Description

Definition at line 298 of file cli_app.c.

### 14.2.2 Field Documentation

#### 14.2.2.1 a_expr

const char* CLI_BINARY_EXPR_OPTIONS::a_expr

Definition at line 300 of file cli_app.c.

#### 14.2.2.2 b_expr

const char* CLI_BINARY_EXPR_OPTIONS::b_expr

Definition at line 301 of file cli_app.c.

The documentation for this struct was generated from the following file:

- cli_app.c

## 14.3 CLI_COMMAND Struct Reference

Data Fields

- const char * name
- CLI_HANDLER handler

### 14.3.1 Detailed Description

Definition at line 41 of file cli_app.c.

### 14.3.2 Field Documentation

#### 14.3.2.1 handler

CLI_HANDLER CLI_COMMAND::handler

Definition at line 44 of file cli_app.c.

#### 14.3.2.2 name

const char* CLI_COMMAND::name

Definition at line 43 of file cli_app.c.

The documentation for this struct was generated from the following file:

- cli_app.c

## 14.4 CLI_RANGE Struct Reference

Data Fields

- char lower [1024]
- char upper [1024]
- uint64_t range_size

### 14.4.1 Detailed Description

Definition at line 117 of file cli_app.c.

### 14.4.2 Field Documentation

#### 14.4.2.1 lower

char CLI_RANGE::lower[1024]

Definition at line 119 of file cli_app.c.

#### 14.4.2.2 range_size

uint64_t CLI_RANGE::range_size

Definition at line 121 of file cli_app.c.

#### 14.4.2.3 upper

char CLI_RANGE::upper[1024]

Definition at line 120 of file cli_app.c.

The documentation for this struct was generated from the following file:

- cli_app.c

## 14.5 CLI_SIEVE_MODEL Struct Reference

Data Fields

- const char * name
- SIEVE_FN fn

### 14.5.1 Detailed Description

Definition at line 23 of file cli_app.c.

### 14.5.2 Field Documentation

#### 14.5.2.1 fn

SIEVE_FN CLI_SIEVE_MODEL::fn

Definition at line 26 of file cli_app.c.

#### 14.5.2.2 name

const char* CLI_SIEVE_MODEL::name

Definition at line 25 of file cli_app.c.

The documentation for this struct was generated from the following file:

- cli_app.c

## 14.6 INPUT_SIEVE_RANGE Struct Reference

Input parameters for range sieving/counting.

#include <iZ_api.h>

Data Fields

- char * start
  Start of range as a base-10 numeric string.
- uint64_t range
  Interval size (number of integers to cover).
- int mr_rounds
  Miller–Rabin rounds for large primality checks.
- char * filepath
  Output path for streaming primes (NULL to disable output).
- int stream_gaps
  Non-zero streams prime gaps instead of absolute primes.

### 14.6.1 Detailed Description

Input parameters for range sieving/counting.

The interval is interpreted as:

- Start $Z_s$ from the decimal string start.
- End $Z_e = Z_s + range - 1$.

Definition at line 133 of file iZ_api.h.

## 14.6.2 Field Documentation

### 14.6.2.1 filepath

char* INPUT_SIEVE_RANGE::filepath

Output path for streaming primes (NULL to disable output).

Definition at line 138 of file iZ_api.h.

Referenced by SiZ_stream().

### 14.6.2.2 mr_rounds

int INPUT_SIEVE_RANGE::mr_rounds

Miller–Rabin rounds for large primality checks.

Definition at line 137 of file iZ_api.h.

Referenced by SiZ_count(), and SiZ_stream().

### 14.6.2.3 range

uint64_t INPUT_SIEVE_RANGE::range

Interval size (number of integers to cover).

Definition at line 136 of file iZ_api.h.

Referenced by range_info_init(), and SiZ_count().

### 14.6.2.4 start

char* INPUT_SIEVE_RANGE::start

Start of range as a base-10 numeric string.

Definition at line 135 of file iZ_api.h.

Referenced by range_info_init(), SiZ_count(), and SiZ_stream().

### 14.6.2.5 stream_gaps

int INPUT_SIEVE_RANGE::stream_gaps

Non-zero streams prime gaps instead of absolute primes.

Definition at line 139 of file iZ_api.h.

Referenced by SiZ_stream().

The documentation for this struct was generated from the following file:

- iZ_api.h

## 14.7 IZM Struct Reference

Precomputed iZm assets for repeated VX-segment sieving.

#include <iZ_toolkit.h>

Data Fields

- int vx
- int k_vx
- BITMAP * base_x5
- BITMAP * base_x7
- UI64_ARRAY * root_primes

### 14.7.1 Detailed Description

Precomputed iZm assets for repeated VX-segment sieving.

Definition at line 84 of file iZ_toolkit.h.

### 14.7.2 Field Documentation

#### 14.7.2.1 base_x5

BITMAP* IZM::base_x5

Pre-sieved base bitmap for 6x-1 line.

Definition at line 88 of file iZ_toolkit.h.

Referenced by iZm_clone(), iZm_init(), and vx_init().

#### 14.7.2.2 base_x7

BITMAP* IZM::base_x7

Pre-sieved base bitmap for 6x+1 line.

Definition at line 89 of file iZ_toolkit.h.

Referenced by iZm_clone(), iZm_init(), and vx_init().

#### 14.7.2.3 k_vx

int IZM::k_vx

Count of small primes dividing vx (excluding 2,3).

Definition at line 87 of file iZ_toolkit.h.

Referenced by iZm_clone(), iZm_init(), and vx_det_sieve().

#### 14.7.2.4 root_primes

UI64_ARRAY* IZM::root_primes

Root primes used for deterministic marking.

Definition at line 90 of file iZ_toolkit.h.

Referenced by iZm_clone(), iZm_init(), and vx_det_sieve().

#### 14.7.2.5 vx

int IZM::vx

Segment width in iZ x-units.

Definition at line 86 of file iZ_toolkit.h.

Referenced by compute_k_vx(), iZm_clone(), iZm_init(), and vx_init().

The documentation for this struct was generated from the following file:

- iZ_toolkit.h

## 14.8 IZM_RANGE_INFO Struct Reference

Precomputed iZm coordinates for an inclusive numeric interval.

#include <iZ_toolkit.h>

Data Fields

- int vx
- mpz_t Zs
- mpz_t Ze
- mpz_t Xs
- mpz_t Xe
- mpz_t Ys
- mpz_t Ye
- int y_range

### 14.8.1 Detailed Description

Precomputed iZm coordinates for an inclusive numeric interval.

This structure maps an input range [Zs, Ze] into the iZ index space used by segmented SiZ routines:

- X* are integer-domain indices such that Z ~= 6X +/- 1.
- Y* are segment indices (X / vx).
- y_range = Ye - Ys and is used to drive segment iteration counts.

A negative y_range marks an invalid/unusable mapping (bad input or range outside current implementation bounds).

Definition at line 255 of file iZ_toolkit.h.

## 14.8.2 Field Documentation

### 14.8.2.1 vx

int IZM_RANGE_INFO::vx

iZm segment width.

Definition at line 257 of file iZ_toolkit.h.

Referenced by range_info_init(), and SiZ_stream().

### 14.8.2.2 Xe

mpz_t IZM_RANGE_INFO::Xe

x_Ze = Ze / 6.

Definition at line 261 of file iZ_toolkit.h.

Referenced by range_info_free(), range_info_init(), SiZ_count(), and SiZ_stream().

### 14.8.2.3 Xs

mpz_t IZM_RANGE_INFO::Xs

x_Zs = Zs / 6.

Definition at line 260 of file iZ_toolkit.h.

Referenced by range_info_free(), range_info_init(), SiZ_count(), and SiZ_stream().

### 14.8.2.4 y_range

int IZM_RANGE_INFO::y_range

Number of y-steps minus one (Ye - Ys), or -1 when invalid.

Definition at line 264 of file iZ_toolkit.h.

Referenced by range_info_init(), SiZ_count(), and SiZ_stream().

### 14.8.2.5 Ye

mpz_t IZM_RANGE_INFO::Ye

y_Ze = Xe / vx.

Definition at line 263 of file iZ_toolkit.h.

Referenced by range_info_free(), range_info_init(), SiZ_count(), and SiZ_stream().

#### 14.8.2.6 Ys

mpz_t IZM_RANGE_INFO::Ys

y_Zs = Xs / vx.

Definition at line 262 of file iZ_toolkit.h.

Referenced by range_info_free(), range_info_init(), SiZ_count(), and SiZ_stream().

#### 14.8.2.7 Ze

mpz_t IZM_RANGE_INFO::Ze

Inclusive upper bound of the search interval.

Definition at line 259 of file iZ_toolkit.h.

Referenced by range_info_free(), range_info_init(), SiZ_count(), and SiZ_stream().

#### 14.8.2.8 Zs

mpz_t IZM_RANGE_INFO::Zs

Inclusive lower bound of the search interval.

Definition at line 258 of file iZ_toolkit.h.

Referenced by range_info_free(), range_info_init(), SiZ_count(), and SiZ_stream().

The documentation for this struct was generated from the following file:

- iZ_toolkit.h

## 14.9 IZP_U64_BUFFER Struct Reference

Owned buffer of uint64_t values returned by FFI sieve routines.

#include <izprime_ffi.h>

Data Fields

- uint64_t * data
- size_t len

### 14.9.1 Detailed Description

Owned buffer of uint64_t values returned by FFI sieve routines.

Caller must release with izp_ffi_free_u64_buffer.

Definition at line 71 of file izprime_ffi.h.

### 14.9.2 Field Documentation

#### 14.9.2.1 data

uint64_t* IZP_U64_BUFFER::data

Heap-allocated values.

Definition at line 73 of file izprime_ffi.h.

Referenced by izp_ffi_free_u64_buffer(), and izp_ffi_sieve_u64().

#### 14.9.2.2 len

size_t IZP_U64_BUFFER::len

Number of entries in data.

Definition at line 74 of file izprime_ffi.h.

Referenced by izp_ffi_free_u64_buffer(), and izp_ffi_sieve_u64().

The documentation for this struct was generated from the following file:

- izprime_ffi.h

## 14.10 SIEVE_LIMIT Struct Reference

#include <test_api.h>

Data Fields

- int base
- int exp

### 14.10.1 Detailed Description

Limit specification as $base^{exp}$.

Definition at line 23 of file test_api.h.

### 14.10.2 Field Documentation

#### 14.10.2.1 base

int SIEVE_LIMIT::base

Base value in base^exp limit notation.

Definition at line 25 of file test_api.h.

#### 14.10.2.2 exp

int SIEVE_LIMIT::exp

Exponent value in base^exp limit notation.

Definition at line 26 of file test_api.h.

The documentation for this struct was generated from the following file:

- test_api.h

## 14.11 SIEVE_MODEL Struct Reference

#include <test_api.h>

Data Fields

- SIEVE_FN function
- const char name [32]

### 14.11.1 Detailed Description

Pair a sieve function with a human-readable name (for reports/benchmarks).

Definition at line 30 of file test_api.h.

### 14.11.2 Field Documentation

#### 14.11.2.1 function

SIEVE_FN SIEVE_MODEL::function

Implementation entry point.

Definition at line 32 of file test_api.h.

#### 14.11.2.2 name

const char SIEVE_MODEL::name[32]

Display name used in test/benchmark output.

Definition at line 33 of file test_api.h.

The documentation for this struct was generated from the following file:

- test_api.h

## 14.12 STOPWATCH Struct Reference

Stopwatch state for elapsed wall-clock measurements.

#include <utils.h>

Data Fields

- struct timespec start_time
- struct timespec end_time
- int running
- double elapsed_sec

### 14.12.1 Detailed Description

Stopwatch state for elapsed wall-clock measurements.

Definition at line 220 of file utils.h.

### 14.12.2 Field Documentation

#### 14.12.2.1 elapsed_sec

double STOPWATCH::elapsed_sec

Elapsed seconds, captured at stop.

Definition at line 225 of file utils.h.

Referenced by sw_elapsed_seconds(), and sw_stop().

#### 14.12.2.2 end_time

struct timespec STOPWATCH::end_time

Captured stop timestamp.

Definition at line 223 of file utils.h.

Referenced by sw_start(), and sw_stop().

#### 14.12.2.3 running

int STOPWATCH::running

Non-zero when timing is active.

Definition at line 224 of file utils.h.

Referenced by sw_elapsed_seconds(), sw_start(), and sw_stop().

#### 14.12.2.4 start_time

struct timespec STOPWATCH::start_time

Captured start timestamp.

Definition at line 222 of file utils.h.

Referenced by sw_elapsed_seconds(), sw_start(), and sw_stop().

The documentation for this struct was generated from the following file:

- utils.h

## 14.13 STREAM_CMD_OPTIONS Struct Reference

Data Fields

- int has_range
- CLI_RANGE range
- int mr_rounds
- int print_to_console
- int print_gaps
- const char * stream_path

### 14.13.1 Detailed Description

Definition at line 281 of file cli_app.c.

### 14.13.2 Field Documentation

#### 14.13.2.1 has_range

int STREAM_CMD_OPTIONS::has_range

Definition at line 283 of file cli_app.c.

#### 14.13.2.2 mr_rounds

int STREAM_CMD_OPTIONS::mr_rounds

Definition at line 285 of file cli_app.c.

#### 14.13.2.3 print_gaps

int STREAM_CMD_OPTIONS::print_gaps

Definition at line 287 of file cli_app.c.

#### 14.13.2.4 print_to_console

int STREAM_CMD_OPTIONS::print_to_console

Definition at line 286 of file cli_app.c.

#### 14.13.2.5 range

CLI_RANGE STREAM_CMD_OPTIONS::range

Definition at line 284 of file cli_app.c.

#### 14.13.2.6 stream_path

const char* STREAM_CMD_OPTIONS::stream_path

Definition at line 288 of file cli_app.c.

The documentation for this struct was generated from the following file:

- cli_app.c

## 14.14 UI16_ARRAY Struct Reference

Dynamic array for uint16_t values.

#include <int_arrays.h>

Data Fields

- int capacity
- int count
- uint16_t * array
- int ordered
- unsigned char sha256 [SHA256_DIGEST_LENGTH]

### 14.14.1 Detailed Description

Dynamic array for uint16_t values.

Definition at line 19 of file int_arrays.h.

### 14.14.2 Field Documentation

#### 14.14.2.1 array

uint16_t* UI16_ARRAY::array

Contiguous element storage.

Definition at line 23 of file int_arrays.h.

#### 14.14.2.2 capacity

int UI16_ARRAY::capacity

Allocated element capacity.

Definition at line 21 of file int_arrays.h.

#### 14.14.2.3 count

int UI16_ARRAY::count

Number of valid elements.

Definition at line 22 of file int_arrays.h.

#### 14.14.2.4 ordered

int UI16_ARRAY::ordered

Flag indicating if the array is sorted.

Definition at line 24 of file int_arrays.h.

#### 14.14.2.5 sha256

unsigned char UI16_ARRAY::sha256[SHA256_DIGEST_LENGTH]

SHA-256 over used payload.

Definition at line 25 of file int_arrays.h.

The documentation for this struct was generated from the following file:

- int_arrays.h

## 14.15 UI32_ARRAY Struct Reference

Dynamic array for uint32_t values.

#include <int_arrays.h>

Data Fields

- int capacity
- int count
- uint32_t * array
- int ordered
- unsigned char sha256 [SHA256_DIGEST_LENGTH]

### 14.15.1 Detailed Description

Dynamic array for uint32_t values.

Definition at line 29 of file int_arrays.h.

### 14.15.2 Field Documentation

#### 14.15.2.1 array

uint32_t* UI32_ARRAY::array

Contiguous element storage.

Definition at line 33 of file int_arrays.h.

#### 14.15.2.2 capacity

int UI32_ARRAY::capacity

Allocated element capacity.

Definition at line 31 of file int_arrays.h.

#### 14.15.2.3 count

int UI32_ARRAY::count

Number of valid elements.

Definition at line 32 of file int_arrays.h.

#### 14.15.2.4 ordered

int UI32_ARRAY::ordered

Flag indicating if the array is sorted.

Definition at line 34 of file int_arrays.h.

#### 14.15.2.5 sha256

unsigned char UI32_ARRAY::sha256[SHA256_DIGEST_LENGTH]

SHA-256 over used payload.

Definition at line 35 of file int_arrays.h.

The documentation for this struct was generated from the following file:

- int_arrays.h

## 14.16 UI64_ARRAY Struct Reference

Dynamic array for uint64_t values.

#include <int_arrays.h>

Data Fields

- int capacity
- int count
- uint64_t * array
- int ordered
- unsigned char sha256 [SHA256_DIGEST_LENGTH]

### 14.16.1 Detailed Description

Dynamic array for uint64_t values.

Definition at line 39 of file int_arrays.h.

### 14.16.2 Field Documentation

#### 14.16.2.1 array

uint64_t* UI64_ARRAY::array

Contiguous element storage.

Definition at line 43 of file int_arrays.h.

Referenced by compute_max_vx(), iZm_clone(), SiZ(), SiZ_count(), SiZ_stream(), SiZm(), SiZm_vy(), SoEu(), SoS(), SSoE(), SSoS(), vx_det_sieve(), and vx_search_prime().

#### 14.16.2.2 capacity

int UI64_ARRAY::capacity

Allocated element capacity.

Definition at line 41 of file int_arrays.h.

Referenced by iZm_clone().

#### 14.16.2.3 count

int UI64_ARRAY::count

Number of valid elements.

Definition at line 42 of file int_arrays.h.

Referenced by iZm_clone(), SiZ(), SiZ_count(), SiZ_stream(), SiZm(), SiZm_vy(), SoEu(), SoS(), SSoE(), SSoS(), vx_det_sieve(), and vx_search_prime().

#### 14.16.2.4 ordered

int UI64_ARRAY::ordered

Flag indicating if the array is sorted.

Definition at line 44 of file int_arrays.h.

Referenced by SiZm_vy().

#### 14.16.2.5 sha256

unsigned char UI64_ARRAY::sha256[SHA256_DIGEST_LENGTH]

SHA-256 over used payload.

Definition at line 45 of file int_arrays.h.

The documentation for this struct was generated from the following file:

- int_arrays.h

## 14.17 VX_SEG Struct Reference

Runtime state for one VX segment at a specific y.

#include <iZ_toolkit.h>

Data Fields

- int vx
- mpz_t y
- mpz_t yvx
- mpz_t root_limit
- int is_large_limit
- int mr_rounds
- int start_x
- int end_x
- BITMAP * x5
- BITMAP * x7
- int p_count
- UI16_ARRAY * p_gaps
- int bit_ops
- int p_test_ops

## 14.17.1 Detailed Description

Runtime state for one VX segment at a specific y.

Definition at line 184 of file iZ_toolkit.h.

## 14.17.2 Field Documentation

### 14.17.2.1 bit_ops

int VX_SEG::bit_ops

Approximate deterministic mark operations.

Definition at line 198 of file iZ_toolkit.h.

Referenced by vx_det_sieve(), and vx_init().

### 14.17.2.2 end_x

int VX_SEG::end_x

Inclusive end x for this segment.

Definition at line 193 of file iZ_toolkit.h.

Referenced by vx_det_sieve(), vx_init(), and vx_prob_sieve().

### 14.17.2.3 is_large_limit

int VX_SEG::is_large_limit

Non-zero => requires probabilistic primality checks.

Definition at line 190 of file iZ_toolkit.h.

Referenced by vx_collect_p_gaps(), vx_det_sieve(), vx_full_sieve(), vx_prob_sieve(), vx_set_base_values(), and vx_stream().

#### 14.17.2.4 mr_rounds

int VX_SEG::mr_rounds

Miller-Rabin rounds when probabilistic checks are used.

Definition at line 191 of file iZ_toolkit.h.

Referenced by vx_init(), vx_prob_sieve(), and vx_stream().

#### 14.17.2.5 p_count

int VX_SEG::p_count

Count of primes found in this segment.

Definition at line 196 of file iZ_toolkit.h.

Referenced by SiZ_count(), SiZ_stream(), vx_collect_p_gaps(), vx_det_sieve(), vx_init(), vx_prob_sieve(), and vx_stream().

#### 14.17.2.6 p_gaps

UI16_ARRAY* VX_SEG::p_gaps

Optional prime-gap encoding for streamed output.

Definition at line 197 of file iZ_toolkit.h.

Referenced by vx_collect_p_gaps(), and vx_init().

#### 14.17.2.7 p_test_ops

int VX_SEG::p_test_ops

Probabilistic primality tests executed.

Definition at line 199 of file iZ_toolkit.h.

Referenced by vx_init(), vx_prob_sieve(), and vx_stream().

#### 14.17.2.8 root_limit

mpz_t VX_SEG::root_limit

sqrt(iZ(yvx + vx, +1)).

Definition at line 189 of file iZ_toolkit.h.

Referenced by vx_det_sieve(), and vx_set_base_values().

#### 14.17.2.9 start_x

int VX_SEG::start_x

Inclusive start x for this segment.

Definition at line 192 of file iZ_toolkit.h.

Referenced by vx_collect_p_gaps(), vx_det_sieve(), vx_init(), vx_prob_sieve(), and vx_stream().

#### 14.17.2.10 vx

int VX_SEG::vx

Segment width.

Definition at line 186 of file iZ_toolkit.h.

Referenced by vx_det_sieve(), vx_init(), and vx_set_base_values().

#### 14.17.2.11 x5

BITMAP* VX_SEG::x5

Candidate bitmap for 6x-1.

Definition at line 194 of file iZ_toolkit.h.

Referenced by vx_collect_p_gaps(), vx_det_sieve(), vx_init(), vx_prob_sieve(), and vx_stream().

#### 14.17.2.12 x7

BITMAP* VX_SEG::x7

Candidate bitmap for 6x+1.

Definition at line 195 of file iZ_toolkit.h.

Referenced by vx_collect_p_gaps(), vx_det_sieve(), vx_init(), vx_prob_sieve(), and vx_stream().

#### 14.17.2.13 y

mpz_t VX_SEG::y

Segment index y.

Definition at line 187 of file iZ_toolkit.h.

Referenced by vx_collect_p_gaps(), vx_det_sieve(), and vx_set_base_values().

#### 14.17.2.14 yvx

mpz_t VX_SEG::yvx

Cached product y*vx.

Definition at line 188 of file iZ_toolkit.h.

Referenced by vx_prob_sieve(), vx_set_base_values(), and vx_stream().

The documentation for this struct was generated from the following file:

- iZ_toolkit.h

# Chapter 15

# File Documentation

## 15.1 bitmap.h File Reference

Bitmap container and bit operations used by sieve implementations.

#include <utils.h>

Data Structures

- struct BITMAP
  Packed bit-array with optional SHA-256 checksum.

Functions

- int TEST_BITMAP (int verbose)
  Run bitmap module tests.

Lifecycle

- BITMAP * bitmap_init (size_t size, int set_bits)
  Allocate a bitmap with size bits.
- void bitmap_free (BITMAP **bitmap)
  Free a bitmap and set the caller pointer to NULL.
- BITMAP * bitmap_clone (BITMAP *src)
  Deep-copy a bitmap, including data and checksum.

Single-bit Operations

- int bitmap_get_bit (BITMAP *bitmap, size_t idx)
  Read bit value at idx.
- void bitmap_set_bit (BITMAP *bitmap, size_t idx)
  Set bit at idx to 1.
- void bitmap_flip_bit (BITMAP *bitmap, size_t idx)
  Toggle bit at idx.
- void bitmap_clear_bit (BITMAP *bitmap, size_t idx)
  Clear bit at idx (set to 0).

Bulk Operations

- void bitmap_set_all (BITMAP *bitmap)
  Set all bits to 1.
- void bitmap_clear_all (BITMAP *bitmap)
  Clear all bits to 0.
- void bitmap_clear_steps (BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)
  Clear every step-th bit from start_idx up to limit.
- void bitmap_clear_steps_simd (BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)
  SIMD-accelerated variant of bitmap_clear_steps().

Integrity and I/O

- void bitmap_compute_hash (BITMAP *bitmap)
  Compute SHA-256 over bitmap data and store it in BITMAP::sha256.
- int bitmap_validate_hash (BITMAP *bitmap)
  Verify BITMAP::sha256 against current bitmap data.
- int bitmap_fwrite (BITMAP *bitmap, FILE *file)
  Write bitmap payload and checksum to a binary stream.
- BITMAP * bitmap_fread (FILE *file)
  Read bitmap payload and checksum from a binary stream.

### 15.1.1 Detailed Description

Bitmap container and bit operations used by sieve implementations.

The bitmap API is intentionally minimal and fast: a packed bit-array plus helpers for per-bit operations, stepped clearing (for sieving), hashing, and binary serialization.

Definition in file bitmap.h.

## 15.2 bitmap.h

Go to the documentation of this file.

```
00001
00009
00010 #ifndef BITMAP_H
00011 #define BITMAP_H
00012
00013 #include <utils.h>
00014
00018
00026 typedef struct
00027 {
00028     size_t size;
00029     size_t byte_size;
00030     unsigned char *data;
00031     unsigned char sha256[SHA256_DIGEST_LENGTH];
00032 } BITMAP;
00033
00042 BITMAP *bitmap_init(size_t size, int set_bits);
00043
00048 void bitmap_free(BITMAP **bitmap);
00049
00055 BITMAP *bitmap_clone(BITMAP *src);
00057
00066 int bitmap_get_bit(BITMAP *bitmap, size_t idx);
00067
00073 void bitmap_set_bit(BITMAP *bitmap, size_t idx);
00074
00080 void bitmap_flip_bit(BITMAP *bitmap, size_t idx);
00081
```

```
void bitmap_clear_bit(BITMAP *bitmap, size_t idx);

void bitmap_set_all(BITMAP *bitmap);

void bitmap_clear_all(BITMAP *bitmap);

void bitmap_clear_steps(BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit);

void bitmap_clear_steps_simd(BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit);

void bitmap_compute_hash(BITMAP *bitmap);

int bitmap_validate_hash(BITMAP *bitmap);

int bitmap_fwrite(BITMAP *bitmap, FILE *file);

BITMAP *bitmap_fread(FILE *file);

int TEST_BITMAP(int verbose);


#endif // BITMAP_H
```

## 15.3 cli.h File Reference

Public entrypoint for the iZprime CLI application layer.

Functions

- int cli_run (int argc, char **argv)
  Run CLI command dispatch.

### 15.3.1 Detailed Description

Public entrypoint for the iZprime CLI application layer.

Definition in file cli.h.

### 15.3.2 Function Documentation

#### 15.3.2.1 cli_run()

```
int cli_run (
            int argc,
            char ** argv)
```

Run CLI command dispatch.

Parameters

| | |
|---|---|
| argc | Argument count from process entry. |
| argv | Argument vector from process entry. |

Returns

Process exit code.

Definition at line 1143 of file cli_app.c.

References create_dir(), DIR_output, log_set_log_level(), and LOG_WARNING.

## 15.4 cli.h

Go to the documentation of this file.

```
00001
00005
00006 #ifndef IZ_CLI_H
00007 #define IZ_CLI_H
00008
00015 int cli_run(int argc, char **argv);
00016
00017 #endif // IZ_CLI_H
```

## 15.5 int_arrays.h File Reference

Dynamic arrays for uint16_t, uint32_t, and uint64_t values.

#include <utils.h>

Data Structures

- struct UI16_ARRAY
  Dynamic array for uint16_t values.
- struct UI32_ARRAY
  Dynamic array for uint32_t values.
- struct UI64_ARRAY
  Dynamic array for uint64_t values.

Functions

- int TEST_GENERIC_INT_ARRAYS (int verbose)
  Run generic dispatch tests for C11 _Generic helper macros.

UI16 API

- UI16_ARRAY * ui16_init (int capacity)
  Allocate a UI16 array with an initial capacity.
- void ui16_free (UI16_ARRAY **array)
  Free a UI16 array and null the caller pointer.
- void ui16_resize_to (UI16_ARRAY *array, int new_capacity)
  Resize UI16 storage to new_capacity (must be >= count).
- void ui16_resize_to_fit (UI16_ARRAY *array)
  Shrink UI16 storage so capacity equals count.
- void ui16_push (UI16_ARRAY *array, uint16_t element)
  Append a uint16 value, growing storage if needed.
- void ui16_sort (UI16_ARRAY *array)
  Sort values in ascending order.
- void ui16_pop (UI16_ARRAY *array)
  Remove the last element if the array is non-empty.
- void ui16_compute_hash (UI16_ARRAY *array)
  Compute SHA-256 checksum over active payload.
- int ui16_verify_hash (UI16_ARRAY *array)
  Verify the stored checksum against current payload.
- int ui16_fwrite (UI16_ARRAY *array, FILE *file)

Serialize count, payload, and checksum to a binary stream.
- UI16_ARRAY * ui16_fread (FILE *file)
  Deserialize a UI16 array from a binary stream.
- int TEST_UI16_ARRAY (int verbose)
  Execute UI16 test suite.

UI32 API

- UI32_ARRAY * ui32_init (int capacity)
  Allocate a UI32 array with an initial capacity.
- void ui32_free (UI32_ARRAY **array)
  Free a UI32 array and null the caller pointer.
- void ui32_resize_to (UI32_ARRAY *array, int new_capacity)
  Resize UI32 storage to new_capacity (must be >= count).
- void ui32_resize_to_fit (UI32_ARRAY *array)
  Shrink UI32 storage so capacity equals count.
- void ui32_push (UI32_ARRAY *array, uint32_t element)
  Append a uint32 value, growing storage if needed.
- void ui32_sort (UI32_ARRAY *array)
  Sort values in ascending order.
- void ui32_pop (UI32_ARRAY *array)
  Remove the last element if the array is non-empty.
- void ui32_compute_hash (UI32_ARRAY *array)
  Compute SHA-256 checksum over active payload.
- int ui32_verify_hash (UI32_ARRAY *array)
  Verify the stored checksum against current payload.
- int ui32_fwrite (UI32_ARRAY *array, FILE *file)
  Serialize count, payload, and checksum to a binary stream.
- UI32_ARRAY * ui32_fread (FILE *file)
  Deserialize a UI32 array from a binary stream.
- int TEST_UI32_ARRAY (int verbose)
  Execute UI32 test suite.

UI64 API

- UI64_ARRAY * ui64_init (int capacity)
  Allocate a UI64 array with an initial capacity.
- void ui64_free (UI64_ARRAY **array)
  Free a UI64 array and null the caller pointer.
- void ui64_resize_to (UI64_ARRAY *array, int new_capacity)
  Resize UI64 storage to new_capacity (must be >= count).
- void ui64_resize_to_fit (UI64_ARRAY *array)
  Shrink UI64 storage so capacity equals count.
- void ui64_push (UI64_ARRAY *array, uint64_t element)
  Append a uint64 value, growing storage if needed.
- void ui64_sort (UI64_ARRAY *array)
  Sort values in ascending order.
- void ui64_pop (UI64_ARRAY *array)
  Remove the last element if the array is non-empty.
- void ui64_compute_hash (UI64_ARRAY *array)
  Compute SHA-256 checksum over active payload.
- int ui64_verify_hash (UI64_ARRAY *array)
  Verify the stored checksum against current payload.
- int ui64_fwrite (UI64_ARRAY *array, FILE *file)
  Serialize count, payload, and checksum to a binary stream.
- UI64_ARRAY * ui64_fread (FILE *file)
  Deserialize a UI64 array from a binary stream.
- int TEST_UI64_ARRAY (int verbose)
  Execute UI64 test suite.

### 15.5.1 Detailed Description

Dynamic arrays for uint16_t, uint32_t, and uint64_t values.

The module provides uniform typed arrays with append/pop operations, deterministic resizing, optional SHA-256 integrity checks, and binary I/O.

Definition in file int_arrays.h.

## 15.6 int_arrays.h

Go to the documentation of this file.

```
#ifndef INT_ARRAYS_H
#define INT_ARRAYS_H

#include <utils.h>


typedef struct
{
    int capacity;
    int count;
    uint16_t *array;
    int ordered;
    unsigned char sha256[SHA256_DIGEST_LENGTH];
} UI16_ARRAY;

typedef struct
{
    int capacity;
    int count;
    uint32_t *array;
    int ordered;
    unsigned char sha256[SHA256_DIGEST_LENGTH];
} UI32_ARRAY;

typedef struct
{
    int capacity;
    int count;
    uint64_t *array;
    int ordered;
    unsigned char sha256[SHA256_DIGEST_LENGTH];
} UI64_ARRAY;

UI16_ARRAY *ui16_init(int capacity);
void ui16_free(UI16_ARRAY **array);
void ui16_resize_to(UI16_ARRAY *array, int new_capacity);
void ui16_resize_to_fit(UI16_ARRAY *array);
void ui16_push(UI16_ARRAY *array, uint16_t element);
void ui16_sort(UI16_ARRAY *array);
void ui16_pop(UI16_ARRAY *array);
void ui16_compute_hash(UI16_ARRAY *array);
int ui16_verify_hash(UI16_ARRAY *array);
int ui16_fwrite(UI16_ARRAY *array, FILE *file);
UI16_ARRAY *ui16_fread(FILE *file);
int TEST_UI16_ARRAY(int verbose);

UI32_ARRAY *ui32_init(int capacity);
void ui32_free(UI32_ARRAY **array);
void ui32_resize_to(UI32_ARRAY *array, int new_capacity);
void ui32_resize_to_fit(UI32_ARRAY *array);
void ui32_push(UI32_ARRAY *array, uint32_t element);
void ui32_sort(UI32_ARRAY *array);
void ui32_pop(UI32_ARRAY *array);
void ui32_compute_hash(UI32_ARRAY *array);
int ui32_verify_hash(UI32_ARRAY *array);
int ui32_fwrite(UI32_ARRAY *array, FILE *file);
UI32_ARRAY *ui32_fread(FILE *file);
int TEST_UI32_ARRAY(int verbose);

UI64_ARRAY *ui64_init(int capacity);
void ui64_free(UI64_ARRAY **array);
void ui64_resize_to(UI64_ARRAY *array, int new_capacity);
void ui64_resize_to_fit(UI64_ARRAY *array);
```

```
void ui64_push(UI64_ARRAY *array, uint64_t element);
void ui64_sort(UI64_ARRAY *array);
void ui64_pop(UI64_ARRAY *array);
void ui64_compute_hash(UI64_ARRAY *array);
int ui64_verify_hash(UI64_ARRAY *array);
int ui64_fwrite(UI64_ARRAY *array, FILE *file);
UI64_ARRAY *ui64_fread(FILE *file);
int TEST_UI64_ARRAY(int verbose);

int TEST_GENERIC_INT_ARRAYS(int verbose);

#if __STDC_VERSION__ >= 201112L

#define int_array_free(arr) _Generic((arr), \
    UI16_ARRAY * *: ui16_free,              \
    UI32_ARRAY * *: ui32_free,              \
    UI64_ARRAY * *: ui64_free)(arr)

#define int_array_resize_to(arr, cap) _Generic((arr), \
    UI16_ARRAY *: ui16_resize_to,                     \
    UI32_ARRAY *: ui32_resize_to,                     \
    UI64_ARRAY *: ui64_resize_to)(arr, cap)

#define int_array_resize_to_fit(arr) _Generic((arr), \
    UI16_ARRAY *: ui16_resize_to_fit,                \
    UI32_ARRAY *: ui32_resize_to_fit,                \
    UI64_ARRAY *: ui64_resize_to_fit)(arr)

#define int_array_push(arr, val) _Generic((arr), \
    UI16_ARRAY *: ui16_push,                     \
    UI32_ARRAY *: ui32_push,                     \
    UI64_ARRAY *: ui64_push)(arr, val)

#define int_array_sort(arr) _Generic((arr), \
    UI16_ARRAY *: ui16_sort,                \
    UI32_ARRAY *: ui32_sort,                \
    UI64_ARRAY *: ui64_sort)(arr)

#define int_array_pop(arr) _Generic((arr), \
    UI16_ARRAY *: ui16_pop,                \
    UI32_ARRAY *: ui32_pop,                \
    UI64_ARRAY *: ui64_pop)(arr)

#define int_array_compute_hash(arr) _Generic((arr), \
    UI16_ARRAY *: ui16_compute_hash,                \
    UI32_ARRAY *: ui32_compute_hash,                \
    UI64_ARRAY *: ui64_compute_hash)(arr)

#define int_array_verify_hash(arr) _Generic((arr), \
    UI16_ARRAY *: ui16_verify_hash,                \
    UI32_ARRAY *: ui32_verify_hash,                \
    UI64_ARRAY *: ui64_verify_hash)(arr)

#define int_array_fwrite(arr, file) _Generic((arr), \
    UI16_ARRAY *: ui16_fwrite,                      \
    UI32_ARRAY *: ui32_fwrite,                      \
    UI64_ARRAY *: ui64_fwrite)(arr, file)

#endif // __STDC_VERSION__ >= 201112L


#endif // INT_ARRAYS_H
```

## 15.7 iZ_api.h File Reference

Public API for prime sieving, range scans, and prime generation.

#include <utils.h>
#include <iZ_toolkit.h>

Data Structures

- struct INPUT_SIEVE_RANGE

  Input parameters for range sieving/counting.

Functions

Classic Prime Sieve Algorithms

Baseline sieves up to a numeric limit.

All classic sieve functions take a limit n and return an ascending list of primes <= n.

- UI64_ARRAY * SoE (uint64_t n)
  Optimized Sieve of Eratosthenes.
- UI64_ARRAY * SSoE (uint64_t n)
  Segmented Sieve of Eratosthenes.
- UI64_ARRAY * SoS (uint64_t n)
  Sieve of Sundaram.
- UI64_ARRAY * SSoS (uint64_t n)
  Segmented Sieve of Sundaram.
- UI64_ARRAY * SoEu (uint64_t n)
  Euler (linear) sieve.
- UI64_ARRAY * SoA (uint64_t n)
  Sieve of Atkin.

iZ-based Sieve Algorithms

Sieve family operating in the 6x-1 / 6x+1 index space.

- UI64_ARRAY * SiZ (uint64_t n)
  Solid Sieve-iZ (wheel 6, iZ index space).
- UI64_ARRAY * SiZm (uint64_t n)
  Segmented Sieve-iZm (VX segmented, horizontal processing).
- UI64_ARRAY * SiZm_vy (uint64_t n)
  Segmented Sieve-iZm with vertical (y-major) traversal.

Prime Generators

Probabilistic prime searches using iZm/VX machinery.

- int vy_random_prime (mpz_t p, int bit_size, int cores_num)
  Search for a random prime using vertical (vy) sieving.
- int vx_random_prime (mpz_t p, int bit_size, int cores_num)
  Search for a random prime using horizontal (vx) sieving.
- int iZ_next_prime (mpz_t p, mpz_t base, int forward)
  Advance to the next (or previous) prime from a base value.

SiZ Range Variants

Count/stream primes over a numeric interval.

- typedef struct INPUT_SIEVE_RANGE INPUT_SIEVE_RANGE
  Input parameters for range sieving/counting.
- uint64_t SiZ_stream (INPUT_SIEVE_RANGE *range)
  Stream primes (or prime gaps) in a range to filepath (and return the count).
- uint64_t SiZ_count (INPUT_SIEVE_RANGE *input_range, int cores_num)
  Count primes in a range using multiple worker processes.

### 15.7.1 Detailed Description

Public API for prime sieving, range scans, and prime generation.

This header provides the stable, high-level entry points for the iZ library. Most functions return a heap-allocated UI64_ARRAY∗ that must be released with ui64_free().

Definition in file iZ_api.h.

## 15.8 iZ_api.h

Go to the documentation of this file.

```
00001
00009
00010 #ifndef IZ_API_H
00011 #define IZ_API_H
00012
00013 #include <utils.h>
00014 #include <iZ_toolkit.h>
00015
00026
00034
00041 UI64_ARRAY *SoE(uint64_t n);
00042
00049 UI64_ARRAY *SSoE(uint64_t n);
00050
00057 UI64_ARRAY *SoS(uint64_t n);
00058
00065 UI64_ARRAY *SSoS(uint64_t n);
00066
00073 UI64_ARRAY *SoEu(uint64_t n);
00074
00081 UI64_ARRAY *SoA(uint64_t n);
00082
00084
00089
00096 UI64_ARRAY *SiZ(uint64_t n);
00097
00104 UI64_ARRAY *SiZm(uint64_t n);
00105
00117 UI64_ARRAY *SiZm_vy(uint64_t n);
00118
00120
00125
00133 typedef struct INPUT_SIEVE_RANGE
00134 {
00135     char *start;
00136     uint64_t range;
00137     int mr_rounds;
00138     char *filepath;
00139     int stream_gaps;
00140 } INPUT_SIEVE_RANGE;
00141
00147 uint64_t SiZ_stream(INPUT_SIEVE_RANGE *range);
00148
00155 uint64_t SiZ_count(INPUT_SIEVE_RANGE *input_range, int cores_num);
00156
00158
00163
00171 int vy_random_prime(mpz_t p, int bit_size, int cores_num);
00172
00180 int vx_random_prime(mpz_t p, int bit_size, int cores_num);
00181
00189 int iZ_next_prime(mpz_t p, mpz_t base, int forward);
00190
00192
00194
00195 #endif // IZ_API_H
```

## 15.9 iZ_toolkit.h File Reference

Core iZ/iZm primitives used by SiZ-family algorithms.

#include <utils.h>
#include <int_arrays.h>
#include <bitmap.h>

Data Structures

- struct IZM

  Precomputed iZm assets for repeated VX-segment sieving.
- struct VX_SEG

  Runtime state for one VX segment at a specific y.
- struct IZM_RANGE_INFO

  Precomputed iZm coordinates for an inclusive numeric interval.

Macros

- #define MR_ROUNDS 25

Standard VX Sizes (primorial products excluding 2,3)

- #define VX2 (5 ∗ 7ULL)
- #define VX3 (VX2 ∗ 11ULL)
- #define VX4 (VX3 ∗ 13ULL)
- #define VX5 (VX4 ∗ 17ULL)
- #define VX6 (VX5 ∗ 19ULL)
- #define VX7 (VX6 ∗ 23ULL)
- #define VX8 (VX7 ∗ 29ULL)

Functions

- void iZm_construct_vx_base (uint64_t vx, BITMAP ∗base_x5, BITMAP ∗base_x7)

  Build pre-sieved base bitmaps for a VX segment.
- IZM_RANGE_INFO range_info_init (INPUT_SIEVE_RANGE ∗input_range, int vx)

  Map a decimal range input into iZm coordinates and segment bounds.
- void range_info_free (IZM_RANGE_INFO ∗info)

  Clear all GMP fields owned by info.

iZ Mapping Helpers

- uint64_t iZ (uint64_t x, int i)

  Map iZ coordinates to an integer: 6∗x + i.
- void iZ_mpz (mpz_t z, mpz_t x, int i)

  GMP variant of iZ().
- void process_iZ_bitmaps (UI64_ARRAY ∗primes, BITMAP ∗x5, BITMAP ∗x7, uint64_t x↩_limit)

  Traverse iZ bitmaps, emit surviving primes, and mark composites.
- void get_root_primes (UI64_ARRAY ∗primes, uint64_t limit)

  Generate primes up to limit for deterministic sieving.

- int test_primality (mpz_t n, int rounds)
  Check the primality of a number using GMP's probabilistic test.

IZM Lifecycle

- IZM * iZm_init (size_t vx)
  Allocate and initialize an IZM object for a given VX.
- IZM * iZm_clone (IZM *src)
  Deep-copy an IZM object for per-worker ownership.
- void iZm_free (IZM **iZm)
  Release an IZM object and set the caller pointer to NULL.

VX Selection Helpers

- uint64_t compute_vx_k (int k)
  Calculate VX_{ k }.
- uint64_t compute_l2_vx (uint64_t n)
  Choose VX using an L2-cache-aware heuristic.
- void compute_max_vx (mpz_t vx, int bit_size)
  Compute largest VX below 2^bit_size.

Modular Hit Solvers

- uint64_t iZm_solve_for_x0 (int m_id, uint64_t p, uint64_t vx, uint64_t y)
  Solve first x-hit of prime p for line m_id in segment y.
- uint64_t iZm_solve_for_x0_mpz (int m_id, uint64_t p, uint64_t vx, mpz_t y)
  GMP variant of iZm_solve_for_x0() for very large y.
- int64_t iZm_solve_for_y0 (int m_id, uint64_t p, uint64_t vx, uint64_t x)
  Solve first y-hit for fixed x in vertical (vy) scanning.

VX Segment Lifecycle and Execution

- VX_SEG * vx_init (IZM *iZm, int start_x, int end_x, char *y_str, int mr_rounds)
  Initialize and deterministically sieve one VX segment.
- void vx_free (VX_SEG **vx_obj)
  Free a VX segment and all owned resources.
- void vx_collect_p_gaps (VX_SEG *vx_obj)
  Extract prime-gap encoding from a fully sieved segment.
- void vx_full_sieve (VX_SEG *vx_obj, int collect_p_gaps)
  Complete segment processing (probabilistic stage and optional gaps).
- void vx_stream (VX_SEG *vx_obj, FILE *output, int stream_gaps)
  Stream segment primes to an output stream.

Random Prime Search Routines

- int vx_search_prime (mpz_t p, int m_id, int vx, int bit_size)
  Horizontal iZm/VX random-prime search.
- int vy_search_prime (mpz_t p, int m_id, mpz_t vx)
  Vertical iZm/VY random-prime search.

Toolkit Tests

- int TEST_IZM (int verbose)
  Run IZM construction and solver tests.
- int TEST_VX_SEG (int verbose)
  Run VX segment tests.

### 15.9.1 Detailed Description

Core iZ/iZm primitives used by SiZ-family algorithms.

This layer exposes index-space mapping helpers, VX sizing utilities, pre-sieved iZm base construction, per-segment sieve objects, and internal prime-search routines used by high-level APIs in iZ_api.h.

Definition in file iZ_toolkit.h.

## 15.10 iZ_toolkit.h

Go to the documentation of this file.

```
#ifndef IZ_TOOLKIT_H
#define IZ_TOOLKIT_H

#include <utils.h>      // Common utilities, types, and dependencies.
#include <int_arrays.h> // Integer array containers.
#include <bitmap.h>     // Packed bit-array utilities.

typedef struct INPUT_SIEVE_RANGE INPUT_SIEVE_RANGE;


uint64_t iZ(uint64_t x, int i);

void iZ_mpz(mpz_t z, mpz_t x, int i);

void process_iZ_bitmaps(UI64_ARRAY *primes, BITMAP *x5, BITMAP *x7, uint64_t x_limit);

void get_root_primes(UI64_ARRAY *primes, uint64_t limit);

int test_primality(mpz_t n, int rounds);

#define VX2 (5 * 7ULL)
#define VX3 (VX2 * 11ULL)
#define VX4 (VX3 * 13ULL)
#define VX5 (VX4 * 17ULL)
#define VX6 (VX5 * 19ULL)
#define VX7 (VX6 * 23ULL)
#define VX8 (VX7 * 29ULL)

#define MR_ROUNDS 25

typedef struct
{
    int vx;
    int k_vx;
    BITMAP *base_x5;
    BITMAP *base_x7;
    UI64_ARRAY *root_primes;
} IZM;

IZM *iZm_init(size_t vx);

IZM *iZm_clone(IZM *src);

void iZm_free(IZM **iZm);

uint64_t compute_vx_k(int k);

uint64_t compute_l2_vx(uint64_t n);

void compute_max_vx(mpz_t vx, int bit_size);

void iZm_construct_vx_base(uint64_t vx, BITMAP *base_x5, BITMAP *base_x7);

uint64_t iZm_solve_for_x0(int m_id, uint64_t p, uint64_t vx, uint64_t y);

uint64_t iZm_solve_for_x0_mpz(int m_id, uint64_t p, uint64_t vx, mpz_t y);

int64_t iZm_solve_for_y0(int m_id, uint64_t p, uint64_t vx, uint64_t x);

typedef struct
{
    int vx;
    mpz_t y;
```

```
    mpz_t yvx;
    mpz_t root_limit;
    int is_large_limit;
    int mr_rounds;
    int start_x;
    int end_x;
    BITMAP *x5;
    BITMAP *x7;
    int p_count;
    UI16_ARRAY *p_gaps;
    int bit_ops;
    int p_test_ops;
} VX_SEG;

VX_SEG *vx_init(IZM *iZm, int start_x, int end_x, char *y_str, int mr_rounds);

void vx_free(VX_SEG **vx_obj);

void vx_collect_p_gaps(VX_SEG *vx_obj);

void vx_full_sieve(VX_SEG *vx_obj, int collect_p_gaps);

void vx_stream(VX_SEG *vx_obj, FILE *output, int stream_gaps);

typedef struct
{
    int vx;
    mpz_t Zs;
    mpz_t Ze;
    mpz_t Xs;
    mpz_t Xe;
    mpz_t Ys;
    mpz_t Ye;
    int y_range;
} IZM_RANGE_INFO;

IZM_RANGE_INFO range_info_init(INPUT_SIEVE_RANGE *input_range, int vx);
void range_info_free(IZM_RANGE_INFO *info);

int vx_search_prime(mpz_t p, int m_id, int vx, int bit_size);

int vy_search_prime(mpz_t p, int m_id, mpz_t vx);

int TEST_IZM(int verbose);
int TEST_VX_SEG(int verbose);


#endif // IZ_TOOLKIT_H
```

## 15.11 izprime_ffi.h File Reference

Stable C-ABI surface for language bindings.

#include <stddef.h>
#include <stdint.h>

### Data Structures

- struct IZP_U64_BUFFER

  Owned buffer of uint64_t values returned by FFI sieve routines.

### Enumerations

- enum IZP_FFI_STATUS {
  IZP_FFI_OK = 0 , IZP_FFI_ERR_INVALID_ARG = 1 , IZP_FFI_ERR_PARSE = 2 , IZP_FFI_ERR_ALLOC = 3 ,
  IZP_FFI_ERR_NOT_FOUND = 4 , IZP_FFI_ERR_OPERATION = 5 }

  Status/error codes returned by FFI functions.

- enum IZP_SIEVE_KIND {
  IZP_SIEVE_SOE = 0 , IZP_SIEVE_SSOE = 1 , IZP_SIEVE_SOS = 2 , IZP_SIEVE_SSOS = 3 ,
  IZP_SIEVE_SOEU = 4 , IZP_SIEVE_SOA = 5 , IZP_SIEVE_SIZ = 6 , IZP_SIEVE_SIZM = 7 ,
  IZP_SIEVE_SIZM_VY = 8 }

  Sieve model selector used by izp_ffi_sieve_u64.

Functions

- IZP_FFI_API const char * izp_ffi_version (void)

  Return the iZprime semantic version string.
- IZP_FFI_API const char * izp_ffi_status_message (int code)

  Return a human-readable status message for code.
- IZP_FFI_API const char * izp_ffi_last_error (void)

  Return the last FFI error message from the current process.
- IZP_FFI_API void izp_ffi_clear_error (void)

  Clear the stored last-error string.
- IZP_FFI_API int izp_ffi_sieve_u64 (IZP_SIEVE_KIND kind, uint64_t limit, IZP_U64_BUFFER *out)

  Run a selected sieve model up to limit and return all primes.
- IZP_FFI_API int izp_ffi_count_range (const char *start, uint64_t range, int mr_rounds, int cores_num, uint64_t *out_count)

  Count primes in [start, start + range - 1].
- IZP_FFI_API int izp_ffi_stream_range (const char *start, uint64_t range, int mr_rounds, const char *filepath, int stream_gaps, uint64_t *out_count)

  Stream primes (or gaps) in [start, start + range - 1] to a file.
- IZP_FFI_API int izp_ffi_next_prime (const char *base_expr, int forward, char **out_prime_↩base10)

  Find the next/previous prime from base_expr.
- IZP_FFI_API int izp_ffi_random_prime_vx (int bit_size, int cores_num, char **out_prime_↩base10)

  Generate a random prime using vx-based search.
- IZP_FFI_API int izp_ffi_random_prime_vy (int bit_size, int cores_num, char **out_prime_↩base10)

  Generate a random prime using vy-based search.
- IZP_FFI_API void izp_ffi_free_u64_buffer (IZP_U64_BUFFER *buffer)

  Free a buffer returned by izp_ffi_sieve_u64.
- IZP_FFI_API void izp_ffi_free_string (char **str)

  Free a decimal string returned by FFI prime APIs.

### 15.11.1 Detailed Description

Stable C-ABI surface for language bindings.

This header exposes a narrow, FFI-safe layer over iZprime's native C API. It is designed for bindings in Python, Rust, Node.js, Go, and similar ecosystems that can call C functions directly.

Definition in file izprime_ffi.h.

### 15.11.2 Macro Definition Documentation

#### 15.11.2.1 IZP_FFI_API

#define IZP_FFI_API

Definition at line 30 of file izprime_ffi.h.

## 15.12 izprime_ffi.h

Go to the documentation of this file.

```
00001
00009
00010 #ifndef IZPRIME_FFI_H
00011 #define IZPRIME_FFI_H
00012
00013 #include <stddef.h>
00014 #include <stdint.h>
00015
00016 #if defined(_WIN32) || defined(__CYGWIN__)
00017 /*
00018  * Default to static-link semantics on Windows. This avoids requiring
00019  * __declspec(dllimport) for in-repo static builds/tests.
00020  * Define IZP_FFI_USE_DLL in consumers that link against a DLL import lib.
00021  */
00022 #if defined(IZP_FFI_BUILD)
00023 #define IZP_FFI_API __declspec(dllexport)
00024 #elif defined(IZP_FFI_USE_DLL)
00025 #define IZP_FFI_API __declspec(dllimport)
00026 #else
00027 #define IZP_FFI_API
00028 #endif
00029 #else
00030 #define IZP_FFI_API
00031 #endif
00032
00033 #ifdef __cplusplus
00034 extern "C" {
00035 #endif
00036
00040
00042 typedef enum
00043 {
00044     IZP_FFI_OK = 0,
00045     IZP_FFI_ERR_INVALID_ARG = 1,
00046     IZP_FFI_ERR_PARSE = 2,
00047     IZP_FFI_ERR_ALLOC = 3,
00048     IZP_FFI_ERR_NOT_FOUND = 4,
00049     IZP_FFI_ERR_OPERATION = 5
00050 } IZP_FFI_STATUS;
00051
00053 typedef enum
00054 {
00055     IZP_SIEVE_SOE = 0,
00056     IZP_SIEVE_SSOE = 1,
00057     IZP_SIEVE_SOS = 2,
00058     IZP_SIEVE_SSOS = 3,
00059     IZP_SIEVE_SOEU = 4,
00060     IZP_SIEVE_SOA = 5,
00061     IZP_SIEVE_SIZ = 6,
00062     IZP_SIEVE_SIZM = 7,
00063     IZP_SIEVE_SIZM_VY = 8
00064 } IZP_SIEVE_KIND;
00065
00071 typedef struct
00072 {
00073     uint64_t *data;
00074     size_t len;
00075 } IZP_U64_BUFFER;
00076
00078 IZP_FFI_API const char *izp_ffi_version(void);
00079
00084 IZP_FFI_API const char *izp_ffi_status_message(int code);
00085
00092 IZP_FFI_API const char *izp_ffi_last_error(void);
00093
```

```
IZP_FFI_API void izp_ffi_clear_error(void);

IZP_FFI_API int izp_ffi_sieve_u64(IZP_SIEVE_KIND kind, uint64_t limit, IZP_U64_BUFFER *out);

IZP_FFI_API int izp_ffi_count_range(const char *start, uint64_t range, int mr_rounds, int cores_num, uint64_t
    *out_count);

IZP_FFI_API int izp_ffi_stream_range(const char *start, uint64_t range, int mr_rounds, const char *filepath, int
    stream_gaps, uint64_t *out_count);

IZP_FFI_API int izp_ffi_next_prime(const char *base_expr, int forward, char **out_prime_base10);

IZP_FFI_API int izp_ffi_random_prime_vx(int bit_size, int cores_num, char **out_prime_base10);

IZP_FFI_API int izp_ffi_random_prime_vy(int bit_size, int cores_num, char **out_prime_base10);

IZP_FFI_API void izp_ffi_free_u64_buffer(IZP_U64_BUFFER *buffer);

IZP_FFI_API void izp_ffi_free_string(char **str);


#ifdef __cplusplus
}
#endif

#endif // IZPRIME_FFI_H
```

## 15.13 logger.h File Reference

Header file for the logging system.

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

### Macros

- #define LOGGER_FORMAT_PRINTF(fmt_idx, arg_idx)

  Compiler attribute wrapper for printf-style format checking.
- #define LOG_DIR "logs/"

  Directory where logs are stored.
- #define LOG_FILE LOG_DIR "log.txt"

  Default log file.
- #define LOG_MAX_SIZE 1024 ∗ 1024 ∗ 5

  Maximum log file size (5 MB).

### Enumerations

- enum LogLevel {
  LOG_DEBUG , LOG_INFO , LOG_WARNING , LOG_ERROR ,
  LOG_FATAL }

  Enumeration of log levels.

Functions

- const char * log_level_to_string (LogLevel level)

  Returns a string representation of the log level.
- void log_init (const char *log_file)

  Initializes the logging system.
- void log_shutdown (void)

  Shuts down the logging system and cleans up resources.
- void log_set_log_level (LogLevel level)

  Sets the current log level. Messages below this level will not be logged.
- void log_message (LogLevel level, const char *format,...) LOGGER_FORMAT_PRINTF(2

  Logs a formatted message at the given log level.
- void void log_message_extended (LogLevel level, const char *file_name, int line_number, const char *format,...) LOGGER_FORMAT_PRINTF(4

  Logs a formatted message with extended information (file name, line number).
- void void void log_console (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs a timestamped message to the console only, without requiring a log level.
- void void void void log_debug (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs a debug message.
- void void void void void log_info (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs an info message.
- void void void void void void log_warn (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs a warning message.
- void void void void void void void log_error (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs an error message.
- void void void void void void void void log_fatal (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs a fatal message.

### 15.13.1 Detailed Description

Header file for the logging system.

This file contains function declarations and macros for a logging system that provides various logging levels (DEBUG, INFO, WARNING, ERROR, FATAL). The logging system supports thread-safe logging, log rotation, and console output. It also includes convenience functions for logging messages with extended information (file name, line number).

Note

> This module is part of a larger project that includes various components requiring logging functionality. It is designed to be flexible and easy to use, allowing developers to log messages at different levels and with different formats.

API:

- log_init: Initializes the logging system.
- log_shutdown: Shuts down the logging system.
- log_set_log_level: Sets the current log level.
- log_message: Logs a formatted message at the given log level.

- log_message_extended: Logs a formatted message with extended information.
- log_console: Logs a formatted message to the console.
- log_debug: Logs a debug message.
- log_info: Logs an info message.
- log_warn: Logs a warning message.
- log_error: Logs an error message.
- log_fatal: Logs a fatal message.

Definition in file logger.h.

## 15.14 logger.h

Go to the documentation of this file.

```
00031
00032 #ifndef LOGGER_H
00033 #define LOGGER_H
00034
00035 #include <stdio.h>
00036 #include <stdlib.h>
00037 #include <string.h>
00038
00042
00047 #if defined(__GNUC__) || defined(__clang__)
00048 #define LOGGER_FORMAT_PRINTF(fmt_idx, arg_idx) __attribute__((format(printf, fmt_idx, arg_idx)))
00049 #else
00050 #define LOGGER_FORMAT_PRINTF(fmt_idx, arg_idx)
00051 #endif
00052
00053 // Log directory and file configuration
00054 #define LOG_DIR "logs/"
00055 #define LOG_FILE LOG_DIR "log.txt"
00056 #define LOG_MAX_SIZE 1024 * 1024 * 5
00057
00065 typedef enum
00066 {
00067     LOG_DEBUG,
00068     LOG_INFO,
00069     LOG_WARNING,
00070     LOG_ERROR,
00071     LOG_FATAL
00072 } LogLevel;
00073
00074 // Function declarations
00075
00082 const char *log_level_to_string(LogLevel level);
00083
00091 void log_init(const char *log_file);
00092
00096 void log_shutdown(void);
00097
00103 void log_set_log_level(LogLevel level);
00104
00113 void log_message(LogLevel level, const char *format, ...) LOGGER_FORMAT_PRINTF(2, 3);
00114
00125 void log_message_extended(LogLevel level, const char *file_name, int line_number, const char *format, ...)
      LOGGER_FORMAT_PRINTF(4, 5);
00126
00134 void log_console(const char *format, ...) LOGGER_FORMAT_PRINTF(1, 2);
00135
00136 // Convenience logging functions for different log levels
00142 void log_debug(const char *format, ...) LOGGER_FORMAT_PRINTF(1, 2);
00143
00149 void log_info(const char *format, ...) LOGGER_FORMAT_PRINTF(1, 2);
00150
00156 void log_warn(const char *format, ...) LOGGER_FORMAT_PRINTF(1, 2);
00157
00163 void log_error(const char *format, ...) LOGGER_FORMAT_PRINTF(1, 2);
00164
00170 void log_fatal(const char *format, ...) LOGGER_FORMAT_PRINTF(1, 2);
00171
00173
00174 #endif // LOGGER_H
```

## 15.15 platform.h File Reference

Cross-platform system abstraction utilities.

#include <stddef.h>
#include <stdint.h>
#include <time.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

Macros

- #define IZ_PLATFORM_WINDOWS 0
- #define IZ_PLATFORM_POSIX 1
- #define IZ_PLATFORM_HAS_FORK 1

Functions

- int iz_platform_create_dir (const char *dir)

  Create a directory if it does not already exist.
- int iz_platform_fill_random (void *buffer, size_t bytes)

  Fill a buffer with random bytes.
- int iz_platform_cpu_cores_count (void)

  Return number of online logical CPU cores (>= 1).
- int iz_platform_l2_cache_size_bits (void)

  Best-effort L2 cache size query in bits.
- double iz_platform_monotonic_seconds (void)

  Return a monotonic timestamp in seconds.
- int iz_platform_localtime (const time_t *timestamp, struct tm *out)

  Convert calendar time to local broken-down time (thread-safe).

### 15.15.1 Detailed Description

Cross-platform system abstraction utilities.

This header centralizes OS-dependent capabilities used by iZprime. It provides a single layer for process capability flags, entropy, monotonic timing, directory creation, and hardware query helpers.

Definition in file platform.h.

### 15.15.2 Macro Definition Documentation

#### 15.15.2.1 IZ_PLATFORM_HAS_FORK

#define IZ_PLATFORM_HAS_FORK 1

Definition at line 25 of file platform.h.

#### 15.15.2.2 IZ_PLATFORM_POSIX

#define IZ_PLATFORM_POSIX 1

Definition at line 24 of file platform.h.

#### 15.15.2.3 IZ_PLATFORM_WINDOWS

#define IZ_PLATFORM_WINDOWS 0

Definition at line 23 of file platform.h.

## 15.16 platform.h

Go to the documentation of this file.

```
00001
00009
00010 #ifndef IZ_PLATFORM_H
00011 #define IZ_PLATFORM_H
00012
00013 #include <stddef.h>
00014 #include <stdint.h>
00015 #include <time.h>
00016
00017 #if defined(_WIN32) || defined(_WIN64)
00018 #define IZ_PLATFORM_WINDOWS 1
00019 #define IZ_PLATFORM_POSIX 0
00020 #define IZ_PLATFORM_HAS_FORK 0
00021
00022 #else
00023 #define IZ_PLATFORM_WINDOWS 0
00024 #define IZ_PLATFORM_POSIX 1
00025 #define IZ_PLATFORM_HAS_FORK 1
00026
00027 #include <sys/types.h>
00028 #include <sys/stat.h>
00029 #include <fcntl.h>
00030 #include <unistd.h>
00031 #include <signal.h>
00032 #include <sys/wait.h>
00033
00034 #if defined(__APPLE__) || defined(__FreeBSD__) || defined(__OpenBSD__) || defined(__NetBSD__)
00035 #include <sys/sysctl.h>
00036 #endif
00037 #endif
00038
00043
00049 int iz_platform_create_dir(const char *dir);
00050
00057 int iz_platform_fill_random(void *buffer, size_t bytes);
00058
00063 int iz_platform_cpu_cores_count(void);
00064
00069 int iz_platform_l2_cache_size_bits(void);
00070
00075 double iz_platform_monotonic_seconds(void);
00076
00083 int iz_platform_localtime(const time_t *timestamp, struct tm *out);
00084
00086
00087 #endif // IZ_PLATFORM_H
```

## 15.17 test_api.h File Reference

Public test/benchmark entry points for the iZ library.

#include <iZ_api.h>

Data Structures

- struct SIEVE_LIMIT
- struct SIEVE_MODEL

Typedefs

- typedef UI64_ARRAY *(* SIEVE_FN) (uint64_t)

Functions

Sieve model tests/benchmarks

- int TEST_UTILS (int verbose)
  Validate utility helpers (numeric/range parsers and shared utilities).
- int TEST_FFI (int verbose)
  Validate the bindings-oriented C FFI surface (izprime_ffi.h).
- int TEST_SIEVE_MODELS_INTEGRITY (int verbose)
  Run correctness checks across all registered sieve models.
- void BENCHMARK_SIEVE_MODELS (int save_results)
  Benchmark registered sieve models and optionally persist results.

Range sieve tests/benchmarks

- int TEST_SiZ_stream (int verbose)
  Validate SiZ_stream behavior and output formatting.
- int TEST_SiZ_count (int verbose)
  Validate SiZ_count correctness across worker counts.
- void BENCHMARK_SiZ_count (int save_results)
  Benchmark SiZ_count over increasing ranges starting from 10^10, 10^20, ..., 10^100 using max cores.

Prime generation tests/benchmarks

- int TEST_CLI (int verbose)
  Run CLI command-dispatch smoke tests.
- int TEST_iZ_next_prime (int verbose)
  Validate iZ_next_prime for forward/backward traversal cases.
- int TEST_vy_random_prime (int verbose)
  Validate vy_random_prime for primality and bit-length constraints.
- int TEST_vx_random_prime (int verbose)
  Validate vx_random_prime for primality and bit-length constraints.
- int BENCHMARK_P_GEN_ALGORITHMS (int bit_size, int test_rounds, int save_results)
  Benchmark random-prime generation routines over repeated trials.

### 15.17.1 Detailed Description

Public test/benchmark entry points for the iZ library.

Doxygen can include this header when documenting the project to expose the test harness API and benchmark entry points.

Definition in file test_api.h.

## 15.18 test_api.h

Go to the documentation of this file.

```
#ifndef TESTS_H
#define TESTS_H

#include <iZ_api.h>

typedef UI64_ARRAY *(*SIEVE_FN)(uint64_t);

typedef struct
{
    int base;
    int exp;
} SIEVE_LIMIT;

typedef struct
{
    SIEVE_FN function;
    const char name[32];
} SIEVE_MODEL;


int TEST_UTILS(int verbose);
int TEST_FFI(int verbose);
int TEST_SIEVE_MODELS_INTEGRITY(int verbose);
void BENCHMARK_SIEVE_MODELS(int save_results);


int TEST_SiZ_stream(int verbose);
int TEST_SiZ_count(int verbose);
void BENCHMARK_SiZ_count(int save_results);


int TEST_CLI(int verbose);
int TEST_iZ_next_prime(int verbose);
int TEST_vy_random_prime(int verbose);
int TEST_vx_random_prime(int verbose);

int BENCHMARK_P_GEN_ALGORITHMS(int bit_size, int test_rounds, int save_results);


#endif // TESTS_H
```

## 15.19 utils.h File Reference

Shared utilities and common includes for the iZprime library.

#include <stdlib.h>
#include <stdint.h>
#include <inttypes.h>
#include <stddef.h>
#include <string.h>
#include <assert.h>
#include <math.h>
#include <stdio.h>
#include <stdarg.h>
#include <time.h>
#include <platform.h>
#include <logger.h>
#include <gmp.h>
#include <openssl/sha.h>

Data Structures

- struct STOPWATCH

  Stopwatch state for elapsed wall-clock measurements.

Macros

- #define MIN(a, b)

  Return the smaller of two expressions.
- #define MAX(a, b)

  Return the larger of two expressions.
- #define DIR_output "./output"
- #define MAX_CORES get_cpu_cores_count()

Functions

- int create_dir (const char *dir)

  Create a directory if it does not exist.
- int is_numeric_str (const char *str)

  Return non-zero if str contains only ASCII digits.
- int parse_numeric_expr_mpz (mpz_t out, const char *expr)

  Parse an integer expression into an mpz value.
- int parse_numeric_expr_u64 (const char *expr, uint64_t *out)

  Parse an integer expression into uint64_t.
- int parse_inclusive_range_mpz (const char *range_expr, mpz_t lower, mpz_t upper)

  Parse an inclusive range expression into lower/upper bounds.
- uint64_t gcd (uint64_t a, uint64_t b)

  Compute the greatest common divisor of a and b.
- uint64_t lcm (uint64_t a, uint64_t b)

  Compute the least common multiple of a and b.
- void gcd_mpz (mpz_t out, const mpz_t a, const mpz_t b)

  Compute the greatest common divisor of two arbitrary-precision integers.
- void lcm_mpz (mpz_t out, const mpz_t a, const mpz_t b)

  Compute the least common multiple of two arbitrary-precision integers.
- uint64_t modular_inverse (uint64_t a, uint64_t m)

  Compute the modular inverse of a modulo m.
- void gmp_seed_randstate (gmp_randstate_t state)

  Seed a GMP random state.
- int get_cpu_cores_count (void)

  Get the number of online CPU cores (>= 1).
- int get_cpu_L2_cache_size_bits (void)

  Return the CPU L2 cache size in bits (best effort).
- void print_sha256_hash (const unsigned char *hash)

  Print a SHA-256 digest as hex to stdout.
- void print_line (int length, char fill_char)

  Print a repeated-character horizontal line.
- void print_centered_text (const char *text, int line_length, char fill_char)

  Print text centered inside a padded line.
- void print_test_table_header (void)

  Print the generic test runner header.
- void print_test_module_header (char *module_name)

  Print a test-suite header banner for a module.
- void print_test_fn_header (char *fn_name)

  Print a formatted header for a test function.
- void print_test_module_result (int result, int test_id, const char *unit_name, const char *format,...)

Print a single test-row result.

- void print_test_summary (char *module_name, int passed, int failed, int verbose)

  Print module-level test summary.
- void sw_start (STOPWATCH *sw)

  Start or restart a stopwatch.
- void sw_stop (STOPWATCH *sw)

  Stop a running stopwatch, capturing the elapsed time in seconds.
- double sw_elapsed_seconds (const STOPWATCH *sw)

  Return elapsed seconds for the stopwatch.
- double sw_elapsed_now_seconds (void)

  Capture the current monotonic time in seconds.

### 15.19.1 Detailed Description

Shared utilities and common includes for the iZprime library.

This header centralizes cross-cutting helpers (string checks, arithmetic, formatting, GMP helpers, and system queries).

Definition in file utils.h.

## 15.20 utils.h

Go to the documentation of this file.

```
00001
00008
00009 #ifndef IZ_UTILS_H
00010 #define IZ_UTILS_H
00011
00012 // Standard library includes
00013 #include <stdlib.h>   // For malloc, free, etc.
00014 #include <stdint.h>   // For fixed-width integer types like uint64_t
00015 #include <inttypes.h> // For PRIu64 and other format macros
00016 #include <stddef.h>   // For size_t
00017 #include <string.h>   // For string manipulation functions like snprintf
00018 #include <assert.h>   // For assertions
00019 #include <math.h>     // For math functions like sqrt
00020 #include <stdio.h>    // For printf, FILE, fopen, fwrite, fread, etc.
00021 #include <stdarg.h>   // For variadic functions (va_list, va_start, va_end)
00022 #include <time.h>     // For struct timespec and time-based helpers
00023
00024 #include <platform.h> // OS abstraction layer (process/timing/system helpers)
00025
00026 // Internal includes
00027 #include <logger.h> // Logger module for logging timestamped messages.
00028
00029 // Third-party libraries
00030 #include <gmp.h>        // GMP library for arbitrary precision arithmetic
00031 #include <openssl/sha.h> // For SHA-256 hashing
00032
00037
00038 #ifndef MIN
00040 #define MIN(a, b) (((a) < (b)) ? (a) : (b))
00041 #endif
00042 #ifndef MAX
00044 #define MAX(a, b) (((a) > (b)) ? (a) : (b))
00045 #endif
00046
00048 #define DIR_output "./output"
00050 #define MAX_CORES get_cpu_cores_count()
00051
00052 // file utilities
00058 int create_dir(const char *dir);
00059
00060 // string utilities
00066 int is_numeric_str(const char *str);
00067
```

```
int parse_numeric_expr_mpz(mpz_t out, const char *expr);

int parse_numeric_expr_u64(const char *expr, uint64_t *out);

int parse_inclusive_range_mpz(const char *range_expr, mpz_t lower, mpz_t upper);

// math utilities
uint64_t gcd(uint64_t a, uint64_t b);
uint64_t lcm(uint64_t a, uint64_t b);
void gcd_mpz(mpz_t out, const mpz_t a, const mpz_t b);
void lcm_mpz(mpz_t out, const mpz_t a, const mpz_t b);
uint64_t modular_inverse(uint64_t a, uint64_t m);

// gmp utilities
void gmp_seed_randstate(gmp_randstate_t state);

// system utilities
int get_cpu_cores_count(void);
int get_cpu_L2_cache_size_bits(void);


void print_sha256_hash(const unsigned char *hash);

void print_line(int length, char fill_char);

void print_centered_text(const char *text, int line_length, char fill_char);

void print_test_table_header(void);

void print_test_module_header(char *module_name);

void print_test_fn_header(char *fn_name);

void print_test_module_result(int result, int test_id, const char *unit_name, const char *format, ...);

void print_test_summary(char *module_name, int passed, int failed, int verbose);


typedef struct
{
    struct timespec start_time;
    struct timespec end_time;
    int running;
    double elapsed_sec;
} STOPWATCH;

void sw_start(STOPWATCH *sw);

void sw_stop(STOPWATCH *sw);

double sw_elapsed_seconds(const STOPWATCH *sw);

double sw_elapsed_now_seconds(void);


#endif // IZ_UTILS_H
```

## 15.21 cli_app.c File Reference

CLI application dispatcher and command handlers.

#include <cli.h>
#include <iZ_api.h>
#include <inttypes.h>
#include <limits.h>
#include <openssl/crypto.h>
#include <time.h>

Data Structures

- struct CLI_SIEVE_MODEL

- struct CLI_COMMAND
- struct CLI_RANGE
- struct STREAM_CMD_OPTIONS
- struct CLI_BINARY_EXPR_OPTIONS

Macros

- #define IZ_VERSION "dev"
- #define CLI_STRING_CAP 8192U

Typedefs

- typedef UI64_ARRAY *(* SIEVE_FN) (uint64_t)
- typedef int(* CLI_HANDLER) (int argc, char **argv)
- typedef void(* MPZ_BINARY_OP) (mpz_t out, const mpz_t a, const mpz_t b)

Enumerations

- enum STREAM_PARSE_RESULT { STREAM_PARSE_OK = 0 , STREAM_PARSE_HELP = 1 , STREAM_PARSE_ERROR = 2 }

Functions

- static int run_directional_prime_cmd (int argc, char **argv, int forward)
- static int run_binary_mpz_cmd (int argc, char **argv, const char *command_name, MPZ_↩ BINARY_OP op_fn)
- static int parse_expr_u64 (const char *value, uint64_t *out)
- static int parse_expr_int (const char *value, int *out)
- static int parse_cores_value (const char *value, int *out)
- static int read_cli_option_value (int argc, char **argv, int *index, const char **value, const char *option_name)
- static char * dup_cli_string (const char *src)
- static int parse_cli_range (const char *value, CLI_RANGE *range)
- static void print_general_help (const char *prog)
- static void print_stream_help (const char *prog)
- static void print_count_help (const char *prog)
- static void print_next_prime_help (const char *prog)
- static void print_prev_prime_help (const char *prog)
- static void print_is_prime_help (const char *prog)
- static void print_gcd_help (const char *prog)
- static void print_lcm_help (const char *prog)
- static void print_test_help (const char *prog)
- static void print_benchmark_help (const char *prog)
- static int stream_read_option_value (int argc, char **argv, int *index, const char **value, const char *option)
- static STREAM_PARSE_RESULT parse_stream_range_option (int argc, char **argv, int *index, STREAM_CMD_OPTIONS *options)
- static STREAM_PARSE_RESULT parse_stream_output_option (int argc, char **argv, int *index, STREAM_CMD_OPTIONS *options, const char *option)
- static STREAM_PARSE_RESULT parse_stream_rounds_option (int argc, char **argv, int *index, STREAM_CMD_OPTIONS *options)

- static STREAM_PARSE_RESULT parse_stream_primes_option (int argc, char **argv, int *index, STREAM_CMD_OPTIONS *options)
- static STREAM_PARSE_RESULT parse_stream_primes_args (int argc, char **argv, STREAM_CMD_OPTIONS *options)
- static int validate_stream_primes_options (const STREAM_CMD_OPTIONS *options)
- static STREAM_PARSE_RESULT parse_binary_mpz_args (int argc, char **argv, CLI_BINARY_EXPR_OPTI *options, void(*print_help)(const char *prog))
- static const char * resolve_stream_path (const STREAM_CMD_OPTIONS *options, char *default_path, size_t default_path_size)
- static int run_stream_primes_cmd (int argc, char **argv)
- static int run_count_primes_cmd (int argc, char **argv)
- static int run_test_cmd (int argc, char **argv)
- static int run_benchmark_cmd (int argc, char **argv)
- static int run_next_prime_cmd (int argc, char **argv)
- static int run_prev_prime_cmd (int argc, char **argv)
- static int run_is_prime_cmd (int argc, char **argv)
- static int run_gcd_cmd (int argc, char **argv)
- static int run_lcm_cmd (int argc, char **argv)
- static int run_doctor_cmd (int argc, char **argv)
- static const CLI_COMMAND * parse_command (const char *name)
- int cli_run (int argc, char **argv)

  Run CLI command dispatch.

Variables

- static const CLI_SIEVE_MODEL k_sieve_models []
- static const size_t k_sieve_models_count = sizeof(k_sieve_models) / sizeof(k_sieve_models[0])
- static const CLI_COMMAND k_commands []

## 15.21.1 Detailed Description

CLI application dispatcher and command handlers.

Definition in file cli_app.c.

## 15.21.2 Macro Definition Documentation

### 15.21.2.1 CLI_STRING_CAP

#define CLI_STRING_CAP 8192U

Definition at line 17 of file cli_app.c.

### 15.21.2.2 IZ_VERSION

#define IZ_VERSION "dev"

Definition at line 14 of file cli_app.c.

### 15.21.3 Typedef Documentation

#### 15.21.3.1 CLI_HANDLER

typedef int(* CLI_HANDLER) (int argc, char **argv)

Definition at line 20 of file cli_app.c.

#### 15.21.3.2 MPZ_BINARY_OP

typedef void(* MPZ_BINARY_OP) (mpz_t out, const mpz_t a, const mpz_t b)

Definition at line 21 of file cli_app.c.

#### 15.21.3.3 SIEVE_FN

typedef UI64_ARRAY *(* SIEVE_FN) (uint64_t)

Definition at line 19 of file cli_app.c.

### 15.21.4 Enumeration Type Documentation

#### 15.21.4.1 STREAM_PARSE_RESULT

enum STREAM_PARSE_RESULT

Definition at line 291 of file cli_app.c.

### 15.21.5 Function Documentation

#### 15.21.5.1 cli_run()

```
int cli_run (
            int argc,
            char ** argv)
```

Run CLI command dispatch.

Parameters

| | |
|---|---|
| argc | Argument count from process entry. |
| argv | Argument vector from process entry. |

Returns

Process exit code.

Definition at line 1143 of file cli_app.c.

References create_dir(), DIR_output, log_set_log_level(), and LOG_WARNING.

#### 15.21.5.2 dup_cli_string()

```
char * dup_cli_string (
            const char * src)    [static]
```

Definition at line 99 of file cli_app.c.

#### 15.21.5.3 parse_binary_mpz_args()

```
STREAM_PARSE_RESULT parse_binary_mpz_args (
            int argc,
            char ** argv,
            CLI_BINARY_EXPR_OPTIONS * options,
            void(* print_help )(const char *prog))    [static]
```

Definition at line 438 of file cli_app.c.

#### 15.21.5.4 parse_cli_range()

```
int parse_cli_range (
            const char * value,
            CLI_RANGE * range)    [static]
```

Definition at line 124 of file cli_app.c.

#### 15.21.5.5 parse_command()

```
const CLI_COMMAND * parse_command (
            const char * name)    [static]
```

Definition at line 1132 of file cli_app.c.

#### 15.21.5.6 parse_cores_value()

```
int parse_cores_value (
            const char * value,
            int * out)    [static]
```

Definition at line 64 of file cli_app.c.

#### 15.21.5.7 parse_expr_int()

```
int parse_expr_int (
            const char * value,
            int * out)    [static]
```

Definition at line 55 of file cli_app.c.

#### 15.21.5.8 parse_expr_u64()

```
int parse_expr_u64 (
            const char * value,
            uint64_t * out)    [static]
```

Definition at line 50 of file cli_app.c.

#### 15.21.5.9 parse_stream_output_option()

```
STREAM_PARSE_RESULT parse_stream_output_option (
            int argc,
            char ** argv,
            int * index,
            STREAM_CMD_OPTIONS * options,
            const char * option)    [static]
```

Definition at line 334 of file cli_app.c.

#### 15.21.5.10 parse_stream_primes_args()

```
STREAM_PARSE_RESULT parse_stream_primes_args (
            int argc,
            char ** argv,
            STREAM_CMD_OPTIONS * options)    [static]
```

Definition at line 397 of file cli_app.c.

#### 15.21.5.11 parse_stream_primes_option()

```
STREAM_PARSE_RESULT parse_stream_primes_option (
            int argc,
            char ** argv,
            int * index,
            STREAM_CMD_OPTIONS * options)    [static]
```

Definition at line 367 of file cli_app.c.

#### 15.21.5.12 parse_stream_range_option()

```
STREAM_PARSE_RESULT parse_stream_range_option (
            int argc,
            char ** argv,
            int * index,
            STREAM_CMD_OPTIONS * options)    [static]
```

Definition at line 314 of file cli_app.c.

#### 15.21.5.13 parse_stream_rounds_option()

```
STREAM_PARSE_RESULT parse_stream_rounds_option (
            int argc,
            char ** argv,
            int * index,
            STREAM_CMD_OPTIONS * options)    [static]
```

Definition at line 348 of file cli_app.c.

#### 15.21.5.14 print_benchmark_help()

```
void print_benchmark_help (
            const char * prog)    [static]
```

Definition at line 271 of file cli_app.c.

#### 15.21.5.15 print_count_help()

```
void print_count_help (
            const char * prog)    [static]
```

Definition at line 215 of file cli_app.c.

#### 15.21.5.16 print_gcd_help()

```
void print_gcd_help (
            const char * prog)    [static]
```

Definition at line 246 of file cli_app.c.

#### 15.21.5.17 print_general_help()

```
void print_general_help (
            const char * prog)    [static]
```

Definition at line 185 of file cli_app.c.

#### 15.21.5.18 print_is_prime_help()

```
void print_is_prime_help (
            const char * prog)    [static]
```

Definition at line 239 of file cli_app.c.

#### 15.21.5.19 print_lcm_help()

```
void print_lcm_help (
            const char * prog)    [static]
```

Definition at line 255 of file cli_app.c.

#### 15.21.5.20 print_next_prime_help()

```
void print_next_prime_help (
            const char * prog)    [static]
```

Definition at line 225 of file cli_app.c.

#### 15.21.5.21 print_prev_prime_help()

```
void print_prev_prime_help (
            const char * prog)    [static]
```

Definition at line 232 of file cli_app.c.

#### 15.21.5.22 print_stream_help()

```
void print_stream_help (
            const char * prog)    [static]
```

Definition at line 205 of file cli_app.c.

#### 15.21.5.23 print_test_help()

```
void print_test_help (
            const char * prog)    [static]
```

Definition at line 264 of file cli_app.c.

#### 15.21.5.24 read_cli_option_value()

```
int read_cli_option_value (
            int argc,
            char ** argv,
            int * index,
            const char ** value,
            const char * option_name)    [static]
```

Definition at line 81 of file cli_app.c.

#### 15.21.5.25 resolve_stream_path()

```
const char * resolve_stream_path (
            const STREAM_CMD_OPTIONS * options,
            char * default_path,
            size_t default_path_size)    [static]
```

Definition at line 506 of file cli_app.c.

#### 15.21.5.26 run_benchmark_cmd()

```
int run_benchmark_cmd (
            int argc,
            char ** argv)   [static]
```

Definition at line 748 of file cli_app.c.

#### 15.21.5.27 run_binary_mpz_cmd()

```
int run_binary_mpz_cmd (
            int argc,
            char ** argv,
            const char * command_name,
            MPZ_BINARY_OP op_fn)   [static]
```

Definition at line 1038 of file cli_app.c.

#### 15.21.5.28 run_count_primes_cmd()

```
int run_count_primes_cmd (
            int argc,
            char ** argv)   [static]
```

Definition at line 572 of file cli_app.c.

#### 15.21.5.29 run_directional_prime_cmd()

```
int run_directional_prime_cmd (
            int argc,
            char ** argv,
            int forward)   [static]
```

Definition at line 886 of file cli_app.c.

#### 15.21.5.30 run_doctor_cmd()

```
int run_doctor_cmd (
            int argc,
            char ** argv)   [static]
```

Definition at line 1081 of file cli_app.c.

#### 15.21.5.31 run_gcd_cmd()

```
int run_gcd_cmd (
            int argc,
            char ** argv)   [static]
```

Definition at line 1071 of file cli_app.c.

#### 15.21.5.32 run_is_prime_cmd()

```
int run_is_prime_cmd (
            int argc,
            char ** argv)   [static]
```

Definition at line 961 of file cli_app.c.

#### 15.21.5.33 run_lcm_cmd()

```
int run_lcm_cmd (
            int argc,
            char ** argv)   [static]
```

Definition at line 1076 of file cli_app.c.

#### 15.21.5.34 run_next_prime_cmd()

```
int run_next_prime_cmd (
            int argc,
            char ** argv)   [static]
```

Definition at line 881 of file cli_app.c.

#### 15.21.5.35 run_prev_prime_cmd()

```
int run_prev_prime_cmd (
            int argc,
            char ** argv)   [static]
```

Definition at line 956 of file cli_app.c.

#### 15.21.5.36 run_stream_primes_cmd()

```
int run_stream_primes_cmd (
            int argc,
            char ** argv)   [static]
```

Definition at line 525 of file cli_app.c.

#### 15.21.5.37 run_test_cmd()

```
int run_test_cmd (
            int argc,
            char ** argv)   [static]
```

Definition at line 659 of file cli_app.c.

#### 15.21.5.38 stream_read_option_value()

```
int stream_read_option_value (
            int argc,
            char ** argv,
            int * index,
            const char ** value,
            const char * option)    [static]
```

Definition at line 304 of file cli_app.c.

#### 15.21.5.39 validate_stream_primes_options()

```
int validate_stream_primes_options (
            const STREAM_CMD_OPTIONS * options)    [static]
```

Definition at line 417 of file cli_app.c.

### 15.21.6 Variable Documentation

#### 15.21.6.1 k_commands

```
const CLI_COMMAND k_commands[]    [static]
```

Initial value:

```
                    = {
    {"stream_primes", run_stream_primes_cmd},
    {"sieve", run_stream_primes_cmd},
    {"count_primes", run_count_primes_cmd},
    {"count", run_count_primes_cmd},
    {"next_prime", run_next_prime_cmd},
    {"next", run_next_prime_cmd},
    {"prev_prime", run_prev_prime_cmd},
    {"prev", run_prev_prime_cmd},
    {"is_prime", run_is_prime_cmd},
    {"gcd", run_gcd_cmd},
    {"lcm", run_lcm_cmd},
    {"test", run_test_cmd},
    {"benchmark", run_benchmark_cmd},
    {"doctor", run_doctor_cmd},
}
```

Definition at line 1115 of file cli_app.c.

#### 15.21.6.2 k_sieve_models

```
const CLI_SIEVE_MODEL k_sieve_models[]    [static]
```

Initial value:

```
                    = {
    {"SoE", SoE},
    {"SSoE", SSoE},
    {"SoEu", SoEu},
    {"SoS", SoS},
    {"SoA", SoA},
    {"SiZ", SiZ},
    {"SiZm", SiZm},
}
```

Definition at line 29 of file cli_app.c.

#### 15.21.6.3 k_sieve_models_count

const size_t k_sieve_models_count = sizeof(k_sieve_models) / sizeof(k_sieve_models[0]) [static]

Definition at line 39 of file cli_app.c.

## 15.22 cli_app.c

Go to the documentation of this file.

```
00001
00005
00006 #include <cli.h>
00007 #include <iZ_api.h>
00008 #include <inttypes.h>
00009 #include <limits.h>
00010 #include <openssl/crypto.h>
00011 #include <time.h>
00012
00013 #ifndef IZ_VERSION
00014 #define IZ_VERSION "dev"
00015 #endif
00016
00017 #define CLI_STRING_CAP 8192U
00018
00019 typedef UI64_ARRAY *(*SIEVE_FN)(uint64_t);
00020 typedef int (*CLI_HANDLER)(int argc, char **argv);
00021 typedef void (*MPZ_BINARY_OP)(mpz_t out, const mpz_t a, const mpz_t b);
00022
00023 typedef struct
00024 {
00025     const char *name;
00026     SIEVE_FN fn;
00027 } CLI_SIEVE_MODEL;
00028
00029 static const CLI_SIEVE_MODEL k_sieve_models[] = {
00030     {"SoE", SoE},
00031     {"SSoE", SSoE},
00032     {"SoEu", SoEu},
00033     {"SoS", SoS},
00034     {"SoA", SoA},
00035     {"SiZ", SiZ},
00036     {"SiZm", SiZm},
00037 };
00038
00039 static const size_t k_sieve_models_count = sizeof(k_sieve_models) / sizeof(k_sieve_models[0]);
00040
00041 typedef struct
00042 {
00043     const char *name;
00044     CLI_HANDLER handler;
00045 } CLI_COMMAND;
00046
00047 static int run_directional_prime_cmd(int argc, char **argv, int forward);
00048 static int run_binary_mpz_cmd(int argc, char **argv, const char *command_name, MPZ_BINARY_OP op_fn);
00049
00050 static int parse_expr_u64(const char *value, uint64_t *out)
00051 {
00052     return parse_numeric_expr_u64(value, out);
00053 }
00054
00055 static int parse_expr_int(const char *value, int *out)
00056 {
00057     uint64_t parsed = 0;
00058     if (!parse_numeric_expr_u64(value, &parsed) || parsed > INT_MAX)
00059         return 0;
00060     *out = (int)parsed;
00061     return 1;
00062 }
00063
00064 static int parse_cores_value(const char *value, int *out)
00065 {
00066     if (value == NULL || out == NULL)
00067         return 0;
00068
00069     if (strcmp(value, "max") == 0)
00070     {
00071         *out = get_cpu_cores_count();
00072         return 1;
```

```
    }

    if (!parse_expr_int(value, out) || *out < 1)
        return 0;

    return 1;
}

static int read_cli_option_value(
    int argc,
    char **argv,
    int *index,
    const char **value,
    const char *option_name)
{
    if (*index + 1 >= argc)
    {
        fprintf(stderr, "Missing value after %s.\n", option_name);
        return 0;
    }

    *index += 1;
    *value = argv[*index];
    return 1;
}

static char *dup_cli_string(const char *src)
{
    if (src == NULL)
        return NULL;

    size_t len = strnlen(src, CLI_STRING_CAP + 1);
    if (len > CLI_STRING_CAP)
        return NULL;

    len += 1;
    char *dst = malloc(len);
    if (dst == NULL)
        return NULL;

    memcpy(dst, src, len);
    return dst;
}

typedef struct
{
    char lower[1024];
    char upper[1024];
    uint64_t range_size;
} CLI_RANGE;

static int parse_cli_range(const char *value, CLI_RANGE *range)
{
    if (range == NULL)
        return 0;

    mpz_t lower;
    mpz_t upper;
    mpz_t width;
    mpz_inits(lower, upper, width, NULL);

    int ok = parse_inclusive_range_mpz(value, lower, upper);
    if (!ok)
    {
        mpz_clears(lower, upper, width, NULL);
        return 0;
    }

    mpz_sub(width, upper, lower);
    mpz_add_ui(width, width, 1);
    if (mpz_sgn(width) <= 0 || mpz_sizeinbase(width, 2) > 64)
    {
        mpz_clears(lower, upper, width, NULL);
        return 0;
    }

    size_t words = 0;
    uint64_t span = 0;
    mpz_export(&span, &words, 1, sizeof(span), 0, 0, width);
    if (words == 0 || span == 0)
    {
        mpz_clears(lower, upper, width, NULL);
        return 0;
    }

    char *lower_str = mpz_get_str(NULL, 10, lower);
    char *upper_str = mpz_get_str(NULL, 10, upper);
```

```
    if (lower_str == NULL || upper_str == NULL)
    {
        free(lower_str);
        free(upper_str);
        mpz_clears(lower, upper, width, NULL);
        return 0;
    }

    int lower_len = snprintf(range->lower, sizeof(range->lower), "%s", lower_str);
    int upper_len = snprintf(range->upper, sizeof(range->upper), "%s", upper_str);
    if (lower_len < 0 || upper_len < 0 || (size_t)lower_len >= sizeof(range->lower) || (size_t)upper_len >= sizeof(range->upper))
    {
        free(lower_str);
        free(upper_str);
        mpz_clears(lower, upper, width, NULL);
        return 0;
    }
    range->range_size = span;

    free(lower_str);
    free(upper_str);
    mpz_clears(lower, upper, width, NULL);
    return 1;
}

static void print_general_help(const char *prog)
{
    printf("iZprime CLI\n");
    printf("Usage: %s <command> [options]\n\n", prog);
    printf("Commands:\n");
    printf("  stream_primes  Stream primes over a range (uses SiZ_stream)\n");
    printf("  count_primes   Count primes over a range (uses SiZ_count)\n");
    printf("  next_prime     Find the next prime after n (uses iZ_next_prime)\n");
    printf("  prev_prime     Find the previous prime before n (uses iZ_next_prime)\n");
    printf("  is_prime       Check primality for n (uses test_primality)\n");
    printf("  gcd            Compute gcd(a, b) for arbitrary-size integers\n");
    printf("  lcm            Compute lcm(a, b) for arbitrary-size integers\n");
    printf("  test           Run API-level consistency tests\n");
    printf("  benchmark      Benchmark sieve models\n");
    printf("  doctor         Check runtime environment and dependencies\n");
    printf("  help           Show this message\n\n");
    printf("Aliases: sieve -> stream_primes, count -> count_primes, next -> next_prime, prev -> prev_prime\n");
    printf("Use '%s <command> --help' for command-specific options.\n", prog);
}

static void print_stream_help(const char *prog)
{
    printf("Usage: %s stream_primes --range \"[LOWER, UPPER]\" [--print | --stream-to FILE] [--print-gaps] [--mr-rounds N]\n", prog);
    printf("Notes:\n");
    printf("  - Range is inclusive and accepts large-number expressions.\n");
    printf("  - Supported numeric operators: + - * / ^ e and parentheses.\n");
    printf("  - If no output option is set, output defaults to output/stream_<timestamp>.txt\n");
    printf("  - --print-gaps emits prime gaps from segment base (implies --print).\n");
}

static void print_count_help(const char *prog)
{
    printf("Usage: %s count_primes --range \"[LOWER, UPPER]\" [--cores N|max] [--mr-rounds N]\n", prog);
    printf("Notes:\n");
    printf("  - Range is inclusive and accepts large-number expressions.\n");
    printf("  - --cores accepts an integer >= 1 or the literal 'max'.\n");
    printf("  - core count is clamped to available CPU cores.\n");
    printf("  - --cores-number is accepted as a backward-compatible alias.\n");
}

static void print_next_prime_help(const char *prog)
{
    printf("Usage: %s next_prime --n VALUE\n", prog);
    printf("Notes:\n");
    printf("  - VALUE accepts the same numeric expression syntax as range bounds.\n");
}

static void print_prev_prime_help(const char *prog)
{
    printf("Usage: %s prev_prime --n VALUE\n", prog);
    printf("Notes:\n");
    printf("  - VALUE accepts the same numeric expression syntax as range bounds.\n");
}

static void print_is_prime_help(const char *prog)
{
    printf("Usage: %s is_prime --n VALUE [--rounds N]\n", prog);
    printf("Notes:\n");
    printf("  - --rounds defaults to %d.\n", MR_ROUNDS);
}
```

```c

static void print_gcd_help(const char *prog)
{
    printf("Usage: %s gcd --a VALUE --b VALUE\n", prog);
    printf("   or: %s gcd VALUE_A VALUE_B\n", prog);
    printf("Notes:\n");
    printf("  - VALUE inputs accept the same numeric expression syntax as range bounds.\n");
    printf("  - Prints the gcd in base-10 to stdout.\n");
}

static void print_lcm_help(const char *prog)
{
    printf("Usage: %s lcm --a VALUE --b VALUE\n", prog);
    printf("   or: %s lcm VALUE_A VALUE_B\n", prog);
    printf("Notes:\n");
    printf("  - VALUE inputs accept the same numeric expression syntax as range bounds.\n");
    printf("  - Prints the lcm in base-10 to stdout.\n");
}

static void print_test_help(const char *prog)
{
    printf("Usage: %s test [--limit N]\n", prog);
    printf("Notes:\n");
    printf("  - Runs cross-model consistency checks against SoE.\n");
}

static void print_benchmark_help(const char *prog)
{
    printf("Usage: %s benchmark [--limit N] [--repeat N] [--algo NAME|all] [--save-results FILE]\n", prog);
    printf("Supported models:\n");
    for (size_t i = 0; i < k_sieve_models_count; ++i)
    {
        printf("  - %s\n", k_sieve_models[i].name);
    }
}

typedef struct
{
    int has_range;
    CLI_RANGE range;
    int mr_rounds;
    int print_to_console;
    int print_gaps;
    const char *stream_path;
} STREAM_CMD_OPTIONS;

typedef enum
{
    STREAM_PARSE_OK = 0,
    STREAM_PARSE_HELP = 1,
    STREAM_PARSE_ERROR = 2,
} STREAM_PARSE_RESULT;

typedef struct
{
    const char *a_expr;
    const char *b_expr;
} CLI_BINARY_EXPR_OPTIONS;

static int stream_read_option_value(
    int argc,
    char **argv,
    int *index,
    const char **value,
    const char *option)
{
    return read_cli_option_value(argc, argv, index, value, option);
}

static STREAM_PARSE_RESULT parse_stream_range_option(
    int argc,
    char **argv,
    int *index,
    STREAM_CMD_OPTIONS *options)
{
    const char *value = NULL;
    if (!stream_read_option_value(argc, argv, index, &value, "--range"))
        return STREAM_PARSE_ERROR;

    if (!parse_cli_range(value, &options->range))
    {
        fprintf(stderr, "Invalid --range value. Expected [LOWER, UPPER] with LOWER<=UPPER.\n");
        return STREAM_PARSE_ERROR;
    }

    options->has_range = 1;
    return STREAM_PARSE_OK;
```

```
}

static STREAM_PARSE_RESULT parse_stream_output_option(
    int argc,
    char **argv,
    int *index,
    STREAM_CMD_OPTIONS *options,
    const char *option)
{
    const char *value = NULL;
    if (!stream_read_option_value(argc, argv, index, &value, option))
        return STREAM_PARSE_ERROR;
    options->stream_path = value;
    return STREAM_PARSE_OK;
}

static STREAM_PARSE_RESULT parse_stream_rounds_option(
    int argc,
    char **argv,
    int *index,
    STREAM_CMD_OPTIONS *options)
{
    const char *value = NULL;
    if (!stream_read_option_value(argc, argv, index, &value, "--mr-rounds"))
        return STREAM_PARSE_ERROR;

    if (!parse_expr_int(value, &options->mr_rounds))
    {
        fprintf(stderr, "Invalid --mr-rounds value.\n");
        return STREAM_PARSE_ERROR;
    }

    return STREAM_PARSE_OK;
}

static STREAM_PARSE_RESULT parse_stream_primes_option(
    int argc,
    char **argv,
    int *index,
    STREAM_CMD_OPTIONS *options)
{
    const char *arg = argv[*index];

    if (strcmp(arg, "--range") == 0)
        return parse_stream_range_option(argc, argv, index, options);
    if (strcmp(arg, "--print") == 0)
    {
        options->print_to_console = 1;
        return STREAM_PARSE_OK;
    }
    if (strcmp(arg, "--print-gaps") == 0)
    {
        options->print_gaps = 1;
        options->print_to_console = 1;
        return STREAM_PARSE_OK;
    }
    if (strcmp(arg, "--stream_to") == 0 || strcmp(arg, "--stream-to") == 0)
        return parse_stream_output_option(argc, argv, index, options, arg);
    if (strcmp(arg, "--mr-rounds") == 0)
        return parse_stream_rounds_option(argc, argv, index, options);

    fprintf(stderr, "Unknown option: %s\n", arg);
    return STREAM_PARSE_ERROR;
}

static STREAM_PARSE_RESULT parse_stream_primes_args(int argc, char **argv, STREAM_CMD_OPTIONS
    *options)
{
    STREAM_PARSE_RESULT result = STREAM_PARSE_OK;

    for (int i = 2; i < argc; ++i)
    {
        if (strcmp(argv[i], "--help") == 0 || strcmp(argv[i], "-h") == 0)
        {
            print_stream_help(argv[0]);
            result = STREAM_PARSE_HELP;
            break;
        }
        result = parse_stream_primes_option(argc, argv, &i, options);
        if (result != STREAM_PARSE_OK)
            break;
    }

    return result;
}

static int validate_stream_primes_options(const STREAM_CMD_OPTIONS *options)
```

```
{
    if (!options->has_range)
    {
        fprintf(stderr, "Missing required option: --range \"[LOWER, UPPER]\"\n");
        return EXIT_FAILURE;
    }
    if (options->print_gaps && options->stream_path != NULL)
    {
        fprintf(stderr, "--print-gaps cannot be combined with --stream-to.\n");
        return EXIT_FAILURE;
    }
    if (options->print_to_console && options->stream_path != NULL)
    {
        fprintf(stderr, "Use either --print or --stream-to, not both.\n");
        return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}

static STREAM_PARSE_RESULT parse_binary_mpz_args(
    int argc,
    char **argv,
    CLI_BINARY_EXPR_OPTIONS *options,
    void (*print_help)(const char *prog))
{
    const char *positional[2] = {0};
    int positional_count = 0;

    for (int i = 2; i < argc; ++i)
    {
        if (strcmp(argv[i], "--help") == 0 || strcmp(argv[i], "-h") == 0)
        {
            print_help(argv[0]);
            return STREAM_PARSE_HELP;
        }
        if (strcmp(argv[i], "--a") == 0)
        {
            if (positional_count != 0)
            {
                fprintf(stderr, "Do not mix positional operands with --a/--b.\n");
                return STREAM_PARSE_ERROR;
            }
            if (!read_cli_option_value(argc, argv, &i, &options->a_expr, "--a"))
                return STREAM_PARSE_ERROR;
            continue;
        }
        if (strcmp(argv[i], "--b") == 0)
        {
            if (positional_count != 0)
            {
                fprintf(stderr, "Do not mix positional operands with --a/--b.\n");
                return STREAM_PARSE_ERROR;
            }
            if (!read_cli_option_value(argc, argv, &i, &options->b_expr, "--b"))
                return STREAM_PARSE_ERROR;
            continue;
        }
        if (argv[i][0] != '-' && positional_count < 2)
        {
            if (options->a_expr != NULL || options->b_expr != NULL)
            {
                fprintf(stderr, "Do not mix positional operands with --a/--b.\n");
                return STREAM_PARSE_ERROR;
            }
            positional[positional_count++] = argv[i];
            continue;
        }

        fprintf(stderr, "Unknown option: %s\n", argv[i]);
        return STREAM_PARSE_ERROR;
    }

    if (options->a_expr == NULL && options->b_expr == NULL && positional_count == 2)
    {
        options->a_expr = positional[0];
        options->b_expr = positional[1];
    }

    if (options->a_expr == NULL || options->b_expr == NULL)
    {
        fprintf(stderr, "Provide both operands using --a VALUE --b VALUE or two positional expressions.\n");
        return STREAM_PARSE_ERROR;
    }

    return STREAM_PARSE_OK;
}
```

```
static const char *resolve_stream_path(const STREAM_CMD_OPTIONS *options, char *default_path, size_t default_path_size)
{
    if (options->print_to_console)
        return NULL;
    if (options->stream_path != NULL)
        return options->stream_path;

    time_t now = time(NULL);
    struct tm tm_now;
    char stamp[64];

    int tm_ok = iz_platform_localtime(&now, &tm_now);
    if (!tm_ok || strftime(stamp, sizeof(stamp), "%Y%m%d_%H%M%S", &tm_now) == 0)
        snprintf(stamp, sizeof(stamp), "unknown");

    snprintf(default_path, default_path_size, "%s/stream_%s.txt", DIR_output, stamp);
    return default_path;
}

static int run_stream_primes_cmd(int argc, char **argv)
{
    STREAM_CMD_OPTIONS options = {0};
    options.mr_rounds = 25;

    STREAM_PARSE_RESULT parse_result = parse_stream_primes_args(argc, argv, &options);
    if (parse_result == STREAM_PARSE_HELP)
        return EXIT_SUCCESS;
    if (parse_result == STREAM_PARSE_ERROR)
        return EXIT_FAILURE;

    if (validate_stream_primes_options(&options) != EXIT_SUCCESS)
        return EXIT_FAILURE;

    create_dir(DIR_output);
    char default_path[256];
    const char *stream_path = resolve_stream_path(&options, default_path, sizeof(default_path));
    char *stream_path_mut = dup_cli_string(stream_path);
    if (stream_path != NULL && stream_path_mut == NULL)
    {
        fprintf(stderr, "Failed to allocate stream path buffer.\n");
        return EXIT_FAILURE;
    }

    INPUT_SIEVE_RANGE input = {
        .start = options.range.lower,
        .range = options.range.range_size,
        .mr_rounds = options.mr_rounds,
        .stream_gaps = options.print_gaps,
        // NULL filepath tells SiZ_stream() to emit directly to stdout.
        .filepath = stream_path_mut};

    STOPWATCH timer;
    sw_start(&timer);
    uint64_t count = SiZ_stream(&input);
    sw_stop(&timer);
    free(stream_path_mut);
    printf("\n");

    if (!options.print_to_console)
        printf("Streamed primes to: %s\n", stream_path);

    printf("Prime count in [%s, %s] = %" PRIu64 "\n", options.range.lower, options.range.upper, count);
    printf("Elapsed (s): %.6f\n", timer.elapsed_sec);
    return EXIT_SUCCESS;
}

static int run_count_primes_cmd(int argc, char **argv)
{
    int has_range = 0;
    CLI_RANGE range = {0};
    int mr_rounds = 25;
    int cores = get_cpu_cores_count();

    for (int i = 2; i < argc; ++i)
    {
        if (strcmp(argv[i], "--help") == 0 || strcmp(argv[i], "-h") == 0)
        {
            print_count_help(argv[0]);
            return EXIT_SUCCESS;
        }
        if (strcmp(argv[i], "--range") == 0)
        {
            const char *range_value = NULL;
            if (!read_cli_option_value(argc, argv, &i, &range_value, "--range"))
                return EXIT_FAILURE;
```

```c
            if (!parse_cli_range(range_value, &range))
            {
                fprintf(stderr, "Invalid --range value. Expected [LOWER, UPPER] with LOWER<=UPPER.\n");
                return EXIT_FAILURE;
            }
            has_range = 1;
            continue;
        }
        if (strcmp(argv[i], "--cores") == 0 || strcmp(argv[i], "--cores-number") == 0)
        {
            const char *cores_value = NULL;
            if (!read_cli_option_value(argc, argv, &i, &cores_value, argv[i]))
                return EXIT_FAILURE;
            if (!parse_cores_value(cores_value, &cores))
            {
                fprintf(stderr, "Invalid --cores value. Use an integer >= 1 or 'max'.\n");
                return EXIT_FAILURE;
            }
            continue;
        }
        if (strcmp(argv[i], "--mr-rounds") == 0)
        {
            const char *rounds_value = NULL;
            if (!read_cli_option_value(argc, argv, &i, &rounds_value, "--mr-rounds"))
                return EXIT_FAILURE;
            if (!parse_expr_int(rounds_value, &mr_rounds))
            {
                fprintf(stderr, "Invalid --mr-rounds value.\n");
                return EXIT_FAILURE;
            }
            continue;
        }

        fprintf(stderr, "Unknown option: %s\n", argv[i]);
        return EXIT_FAILURE;
    }

    if (!has_range)
    {
        fprintf(stderr, "Missing required option: --range \"[LOWER, UPPER]\"\n");
        return EXIT_FAILURE;
    }

    if (range.range_size <= 100)
    {
        fprintf(stderr, "Range size must be > 100 for SiZ_count preconditions.\n");
        return EXIT_FAILURE;
    }

    INPUT_SIEVE_RANGE input = {
        .start = range.lower,
        .range = range.range_size,
        .mr_rounds = mr_rounds,
        .stream_gaps = 0,
        .filepath = NULL,
    };

    STOPWATCH timer;
    sw_start(&timer);
    uint64_t count = SiZ_count(&input, cores);
    sw_stop(&timer);

    printf("Prime count in [%s, %s] = %" PRIu64 "\n", range.lower, range.upper, count);
    printf("Cores used: %d\n", MIN(cores, get_cpu_cores_count()));
    printf("Elapsed (s): %.6f\n", timer.elapsed_sec);
    return EXIT_SUCCESS;
}

static int run_test_cmd(int argc, char **argv)
{
    uint64_t limit = 1000000ULL;

    for (int i = 2; i < argc; ++i)
    {
        if (strcmp(argv[i], "--help") == 0 || strcmp(argv[i], "-h") == 0)
        {
            print_test_help(argv[0]);
            return EXIT_SUCCESS;
        }
        if (strcmp(argv[i], "--limit") == 0)
        {
            const char *limit_value = NULL;
            if (!read_cli_option_value(argc, argv, &i, &limit_value, "--limit"))
                return EXIT_FAILURE;
            if (!parse_expr_u64(limit_value, &limit))
            {
                fprintf(stderr, "Invalid --limit value.\n");
```

```
            return EXIT_FAILURE;
        }
        continue;
    }

    fprintf(stderr, "Unknown option: %s\n", argv[i]);
    return EXIT_FAILURE;
}

if (limit <= 10)
{
    fprintf(stderr, "--limit must be > 10.\n");
    return EXIT_FAILURE;
}

printf("Running consistency test for n=%" PRIu64 "\n", limit);
UI64_ARRAY *baseline = SoE(limit);
if (!baseline)
{
    fprintf(stderr, "SoE baseline failed.\n");
    return EXIT_FAILURE;
}

int failures = 0;
printf("[PASS] SoE count=%d\n", baseline->count);

for (size_t i = 1; i < k_sieve_models_count; ++i)
{
    UI64_ARRAY *arr = k_sieve_models[i].fn(limit);
    if (!arr)
    {
        printf("[FAIL] %s returned NULL\n", k_sieve_models[i].name);
        failures++;
        continue;
    }

    int ok = (arr->count == baseline->count);
    if (ok)
    {
        for (int j = 0; j < baseline->count; ++j)
        {
            if (arr->array[j] != baseline->array[j])
            {
                ok = 0;
                break;
            }
        }
    }

    if (ok)
        printf("[PASS] %s\n", k_sieve_models[i].name);
    else
    {
        printf("[FAIL] %s mismatch against SoE\n", k_sieve_models[i].name);
        failures++;
    }
    ui64_free(&arr);
}

ui64_free(&baseline);
if (failures != 0)
{
    printf("Test failures: %d\n", failures);
    return EXIT_FAILURE;
}

printf("All model consistency tests passed.\n");
return EXIT_SUCCESS;
}

static int run_benchmark_cmd(int argc, char **argv)
{
    uint64_t limit = 10000000ULL;
    int repeat = 3;
    const char *algo = "all";
    const char *save_path = NULL;

    for (int i = 2; i < argc; ++i)
    {
        if (strcmp(argv[i], "--help") == 0 || strcmp(argv[i], "-h") == 0)
        {
            print_benchmark_help(argv[0]);
            return EXIT_SUCCESS;
        }
        if (strcmp(argv[i], "--limit") == 0)
        {
            const char *limit_value = NULL;
```

```c
            if (!read_cli_option_value(argc, argv, &i, &limit_value, "--limit"))
                return EXIT_FAILURE;
            if (!parse_expr_u64(limit_value, &limit))
            {
                fprintf(stderr, "Invalid --limit value.\n");
                return EXIT_FAILURE;
            }
            continue;
        }
        if (strcmp(argv[i], "--repeat") == 0)
        {
            const char *repeat_value = NULL;
            if (!read_cli_option_value(argc, argv, &i, &repeat_value, "--repeat"))
                return EXIT_FAILURE;
            if (!parse_expr_int(repeat_value, &repeat) || repeat < 1)
            {
                fprintf(stderr, "Invalid --repeat value.\n");
                return EXIT_FAILURE;
            }
            continue;
        }
        if (strcmp(argv[i], "--algo") == 0)
        {
            const char *algo_value = NULL;
            if (!read_cli_option_value(argc, argv, &i, &algo_value, "--algo"))
                return EXIT_FAILURE;
            algo = algo_value;
            continue;
        }
        if (strcmp(argv[i], "--save-results") == 0)
        {
            const char *save_value = NULL;
            if (!read_cli_option_value(argc, argv, &i, &save_value, "--save-results"))
                return EXIT_FAILURE;
            save_path = save_value;
            continue;
        }

        fprintf(stderr, "Unknown option: %s\n", argv[i]);
        return EXIT_FAILURE;
    }

    FILE *out = NULL;
    if (save_path != NULL)
    {
        out = fopen(save_path, "w");
        if (!out)
        {
            fprintf(stderr, "Failed to open %s\n", save_path);
            return EXIT_FAILURE;
        }
        fprintf(out, "algorithm,limit,repeat,avg_seconds,prime_count\n");
    }
    printf("Benchmark: n=%" PRIu64 ", repeat=%d\n", limit, repeat);
    printf("%-10s %-12s %-14s %-12s\n", "Model", "Primes", "Avg(s)", "Status");

    int failures = 0;
    int matched_models = 0;
    for (size_t i = 0; i < k_sieve_models_count; ++i)
    {
        if (strcmp(algo, "all") != 0 && strcmp(algo, k_sieve_models[i].name) != 0)
            continue;
        matched_models++;

        double total_seconds = 0.0;
        int prime_count = -1;
        int ok = 1;

        for (int r = 0; r < repeat; ++r)
        {
            STOPWATCH timer;
            sw_start(&timer);
            UI64_ARRAY *arr = k_sieve_models[i].fn(limit);
            sw_stop(&timer);
            double elapsed = timer.elapsed_sec;
            if (!arr)
            {
                ok = 0;
                break;
            }

            total_seconds += elapsed;
            prime_count = arr->count;
            ui64_free(&arr);
        }

        if (!ok)
```

```
        {
            printf("%-10s %-12s %-14s %-12s\n", k_sieve_models[i].name, "-", "-", "FAIL");
            failures++;
            continue;
        }

        double avg = total_seconds / (double)repeat;
        printf("%-10s %-12d %-14.6f %-12s\n", k_sieve_models[i].name, prime_count, avg, "OK");
        if (out != NULL)
            fprintf(out, "%s,%" PRIu64 ",%d,%.6f,%d\n", k_sieve_models[i].name, limit, repeat, avg, prime_count);
    }

    if (save_path != NULL)
    {
        fclose(out);
        printf("Saved benchmark results: %s\n", save_path);
    }

    if (matched_models == 0)
    {
        fprintf(stderr, "Unknown model '%s'. Use --help to list supported names.\n", algo);
        return EXIT_FAILURE;
    }

    if (failures != 0)
        return EXIT_FAILURE;
    return EXIT_SUCCESS;
}

static int run_next_prime_cmd(int argc, char **argv)
{
    return run_directional_prime_cmd(argc, argv, 1);
}

static int run_directional_prime_cmd(int argc, char **argv, int forward)
{
    const char *value_expr = NULL;

    for (int i = 2; i < argc; ++i)
    {
        if (strcmp(argv[i], "--help") == 0 || strcmp(argv[i], "-h") == 0)
        {
            if (forward)
                print_next_prime_help(argv[0]);
            else
                print_prev_prime_help(argv[0]);
            return EXIT_SUCCESS;
        }
        if (strcmp(argv[i], "--n") == 0)
        {
            const char *n_value = NULL;
            if (!read_cli_option_value(argc, argv, &i, &n_value, "--n"))
                return EXIT_FAILURE;
            value_expr = n_value;
            continue;
        }
        if (argv[i][0] != '-' && value_expr == NULL)
        {
            value_expr = argv[i];
            continue;
        }

        fprintf(stderr, "Unknown option: %s\n", argv[i]);
        return EXIT_FAILURE;
    }

    if (value_expr == NULL)
    {
        fprintf(stderr, "Missing required value. Use --n VALUE.\n");
        return EXIT_FAILURE;
    }

    mpz_t base;
    mpz_t prime;
    mpz_inits(base, prime, NULL);
    if (!parse_numeric_expr_mpz(base, value_expr))
    {
        fprintf(stderr, "Invalid numeric expression for --n.\n");
        mpz_clears(base, prime, NULL);
        return EXIT_FAILURE;
    }

    STOPWATCH timer;
    sw_start(&timer);
    int found = iZ_next_prime(prime, base, forward);
    sw_stop(&timer);

```

```
    if (!found)
    {
        fprintf(stderr, "Failed to find the %s prime.\n", forward ? "next" : "previous");
        mpz_clears(base, prime, NULL);
        return EXIT_FAILURE;
    }

    if (forward)
        gmp_printf("Next prime after %Zd is %Zd\n", base, prime);
    else
        gmp_printf("Previous prime before %Zd is %Zd\n", base, prime);
    printf("Elapsed (s): %.6f s\n", timer.elapsed_sec);

    mpz_clears(base, prime, NULL);
    return EXIT_SUCCESS;
}

static int run_prev_prime_cmd(int argc, char **argv)
{
    return run_directional_prime_cmd(argc, argv, 0);
}

static int run_is_prime_cmd(int argc, char **argv)
{
    const char *value_expr = NULL;
    int rounds = MR_ROUNDS;

    for (int i = 2; i < argc; ++i)
    {
        if (strcmp(argv[i], "--help") == 0 || strcmp(argv[i], "-h") == 0)
        {
            print_is_prime_help(argv[0]);
            return EXIT_SUCCESS;
        }
        if (strcmp(argv[i], "--n") == 0)
        {
            const char *n_value = NULL;
            if (!read_cli_option_value(argc, argv, &i, &n_value, "--n"))
                return EXIT_FAILURE;
            value_expr = n_value;
            continue;
        }
        if (strcmp(argv[i], "--rounds") == 0)
        {
            const char *rounds_value = NULL;
            if (!read_cli_option_value(argc, argv, &i, &rounds_value, "--rounds"))
                return EXIT_FAILURE;
            if (!parse_expr_int(rounds_value, &rounds) || rounds < 1)
            {
                fprintf(stderr, "Invalid --rounds value.\n");
                return EXIT_FAILURE;
            }
            continue;
        }
        if (argv[i][0] != '-' && value_expr == NULL)
        {
            value_expr = argv[i];
            continue;
        }

        fprintf(stderr, "Unknown option: %s\n", argv[i]);
        return EXIT_FAILURE;
    }

    if (value_expr == NULL)
    {
        fprintf(stderr, "Missing required value. Use --n VALUE.\n");
        return EXIT_FAILURE;
    }

    mpz_t n;
    mpz_init(n);
    if (!parse_numeric_expr_mpz(n, value_expr))
    {
        fprintf(stderr, "Invalid numeric expression for --n.\n");
        mpz_clear(n);
        return EXIT_FAILURE;
    }

    STOPWATCH timer;
    sw_start(&timer);
    int result = test_primality(n, rounds);
    sw_stop(&timer);

    if (result > 0)
    {
        const char *certainty = (result == 2) ? "definitely prime" : "probably prime";
```

```
        gmp_printf("%Zd is prime (%s)\n", n, certainty);
    }
    else
    {
        gmp_printf("%Zd is composite\n", n);
    }
    printf("Elapsed (s): %.6f s\n", timer.elapsed_sec);

    mpz_clear(n);
    return EXIT_SUCCESS;
}

static int run_binary_mpz_cmd(int argc, char **argv, const char *command_name, MPZ_BINARY_OP op_fn)
{
    CLI_BINARY_EXPR_OPTIONS options = {0};
    STREAM_PARSE_RESULT parse_result = STREAM_PARSE_OK;

    if (strcmp(command_name, "gcd") == 0)
        parse_result = parse_binary_mpz_args(argc, argv, &options, print_gcd_help);
    else
        parse_result = parse_binary_mpz_args(argc, argv, &options, print_lcm_help);

    if (parse_result == STREAM_PARSE_HELP)
        return EXIT_SUCCESS;
    if (parse_result == STREAM_PARSE_ERROR)
        return EXIT_FAILURE;

    mpz_t a;
    mpz_t b;
    mpz_t out;
    mpz_inits(a, b, out, NULL);

    if (!parse_numeric_expr_mpz(a, options.a_expr) || !parse_numeric_expr_mpz(b, options.b_expr))
    {
        fprintf(stderr, "Invalid numeric expression for %s operands.\n", command_name);
        mpz_clears(a, b, out, NULL);
        return EXIT_FAILURE;
    }

    op_fn(out, a, b);
    gmp_printf("%Zd\n", out);
    mpz_clears(a, b, out, NULL);
    return EXIT_SUCCESS;
}

static int run_gcd_cmd(int argc, char **argv)
{
    return run_binary_mpz_cmd(argc, argv, "gcd", gcd_mpz);
}

static int run_lcm_cmd(int argc, char **argv)
{
    return run_binary_mpz_cmd(argc, argv, "lcm", lcm_mpz);
}

static int run_doctor_cmd(int argc, char **argv)
{
    for (int i = 2; i < argc; ++i)
    {
        if (strcmp(argv[i], "--help") == 0 || strcmp(argv[i], "-h") == 0)
        {
            printf("Usage: %s doctor\n", argv[0]);
            return EXIT_SUCCESS;
        }

        fprintf(stderr, "Unknown option: %s\n", argv[i]);
        return EXIT_FAILURE;
    }

    int status = EXIT_SUCCESS;
    printf("iZprime doctor\n");
    printf("Version: %s\n", IZ_VERSION);
    printf("CPU cores: %d\n", get_cpu_cores_count());
    printf("GMP version: %s\n", gmp_version);
    printf("OpenSSL: %s\n", OpenSSL_version(OPENSSL_VERSION));

    if (create_dir(DIR_output) != 0)
    {
        printf("[FAIL] Cannot create output directory: %s\n", DIR_output);
        status = EXIT_FAILURE;
    }
    else
    {
        printf("[PASS] Output directory is writable: %s\n", DIR_output);
    }

    return status;
```

```
}

static const CLI_COMMAND k_commands[] = {
    {"stream_primes", run_stream_primes_cmd},
    {"sieve", run_stream_primes_cmd},
    {"count_primes", run_count_primes_cmd},
    {"count", run_count_primes_cmd},
    {"next_prime", run_next_prime_cmd},
    {"next", run_next_prime_cmd},
    {"prev_prime", run_prev_prime_cmd},
    {"prev", run_prev_prime_cmd},
    {"is_prime", run_is_prime_cmd},
    {"gcd", run_gcd_cmd},
    {"lcm", run_lcm_cmd},
    {"test", run_test_cmd},
    {"benchmark", run_benchmark_cmd},
    {"doctor", run_doctor_cmd},
};

static const CLI_COMMAND *parse_command(const char *name)
{
    size_t count = sizeof(k_commands) / sizeof(k_commands[0]);
    for (size_t i = 0; i < count; ++i)
    {
        if (strcmp(name, k_commands[i].name) == 0)
            return &k_commands[i];
    }
    return NULL;
}

int cli_run(int argc, char **argv)
{
    log_set_log_level(LOG_WARNING);
    create_dir(DIR_output);

    if (argc < 2 || strcmp(argv[1], "--help") == 0 || strcmp(argv[1], "-h") == 0)
    {
        print_general_help(argv[0]);
        return EXIT_SUCCESS;
    }

    if (strcmp(argv[1], "help") == 0)
    {
        if (argc == 2)
        {
            print_general_help(argv[0]);
            return EXIT_SUCCESS;
        }

        const CLI_COMMAND *help_cmd = parse_command(argv[2]);
        if (help_cmd == NULL)
        {
            fprintf(stderr, "Unknown command: %s\n\n", argv[2]);
            print_general_help(argv[0]);
            return EXIT_FAILURE;
        }

        char help_name[64];
        snprintf(help_name, sizeof(help_name), "%s", help_cmd->name);
        char help_flag[] = "--help";
        char *help_argv[] = {argv[0], help_name, help_flag};
        return help_cmd->handler(3, help_argv);
    }

    const CLI_COMMAND *cmd = parse_command(argv[1]);
    if (cmd != NULL)
        return cmd->handler(argc, argv);

    fprintf(stderr, "Unknown command: %s\n\n", argv[1]);
    print_general_help(argv[0]);
    return EXIT_FAILURE;
}
```

## 15.23 izprime_ffi.c File Reference

FFI-safe wrapper layer over the native iZprime API.

#include <izprime_ffi.h>
#include <iZ_api.h>

Macros

- #define IZP_FFI_BUILD 1
- #define IZP_MAX_SIEVE_LIMIT 1000000000000ULL
- #define IZP_FFI_ERROR_CAP 512

Functions

- static void izp_ffi_set_error (const char *message)
- static int izp_ffi_validate_expr_nonnegative (const char *expr, mpz_t out)
- static char * izp_ffi_strdup_heap (const char *src)
- static int izp_ffi_copy_mpz_to_string (mpz_t value, char **out_string)
- static int izp_ffi_export_primes (UI64_ARRAY *primes, IZP_U64_BUFFER *out)
- static uint64_t izp_ffi_count_small_range (const mpz_t start, uint64_t range, int mr_rounds)
- const char * izp_ffi_version (void)

  Return the iZprime semantic version string.
- const char * izp_ffi_status_message (int code)

  Return a human-readable status message for code.
- const char * izp_ffi_last_error (void)

  Return the last FFI error message from the current process.
- void izp_ffi_clear_error (void)

  Clear the stored last-error string.
- int izp_ffi_sieve_u64 (IZP_SIEVE_KIND kind, uint64_t limit, IZP_U64_BUFFER *out)

  Run a selected sieve model up to limit and return all primes.
- int izp_ffi_count_range (const char *start, uint64_t range, int mr_rounds, int cores_num, uint64↩_t *out_count)

  Count primes in [start, start + range - 1].
- int izp_ffi_stream_range (const char *start, uint64_t range, int mr_rounds, const char *filepath, int stream_gaps, uint64_t *out_count)

  Stream primes (or gaps) in [start, start + range - 1] to a file.
- int izp_ffi_next_prime (const char *base_expr, int forward, char **out_prime_base10)

  Find the next/previous prime from base_expr.
- int izp_ffi_random_prime_vx (int bit_size, int cores_num, char **out_prime_base10)

  Generate a random prime using vx-based search.
- int izp_ffi_random_prime_vy (int bit_size, int cores_num, char **out_prime_base10)

  Generate a random prime using vy-based search.
- void izp_ffi_free_u64_buffer (IZP_U64_BUFFER *buffer)

  Free a buffer returned by izp_ffi_sieve_u64.
- void izp_ffi_free_string (char **str)

  Free a decimal string returned by FFI prime APIs.

Variables

- static char g_izp_ffi_last_error [IZP_FFI_ERROR_CAP] = ""

### 15.23.1 Detailed Description

FFI-safe wrapper layer over the native iZprime API.

Definition in file izprime_ffi.c.

## 15.23.2 Macro Definition Documentation

### 15.23.2.1 IZP_FFI_BUILD

```
#define IZP_FFI_BUILD 1
```

Definition at line 6 of file izprime_ffi.c.

### 15.23.2.2 IZP_FFI_ERROR_CAP

```
#define IZP_FFI_ERROR_CAP 512
```

Definition at line 12 of file izprime_ffi.c.

### 15.23.2.3 IZP_MAX_SIEVE_LIMIT

```
#define IZP_MAX_SIEVE_LIMIT 1000000000000ULL
```

Definition at line 11 of file izprime_ffi.c.

## 15.23.3 Function Documentation

### 15.23.3.1 izp_ffi_copy_mpz_to_string()

```
int izp_ffi_copy_mpz_to_string (
            mpz_t value,
            char ** out_string)    [static]
```

Definition at line 47 of file izprime_ffi.c.

### 15.23.3.2 izp_ffi_count_small_range()

```
uint64_t izp_ffi_count_small_range (
            const mpz_t start,
            uint64_t range,
            int mr_rounds)    [static]
```

Definition at line 112 of file izprime_ffi.c.

### 15.23.3.3 izp_ffi_export_primes()

```
int izp_ffi_export_primes (
            UI64_ARRAY * primes,
            IZP_U64_BUFFER * out)    [static]
```

Definition at line 68 of file izprime_ffi.c.

#### 15.23.3.4 izp_ffi_set_error()

```
void izp_ffi_set_error (
                const char * message)   [static]
```

Definition at line 16 of file izprime_ffi.c.

#### 15.23.3.5 izp_ffi_strdup_heap()

```
char * izp_ffi_strdup_heap (
                const char * src)   [static]
```

Definition at line 30 of file izprime_ffi.c.

#### 15.23.3.6 izp_ffi_validate_expr_nonnegative()

```
int izp_ffi_validate_expr_nonnegative (
                const char * expr,
                mpz_t out)   [static]
```

Definition at line 21 of file izprime_ffi.c.

### 15.23.4 Variable Documentation

#### 15.23.4.1 g_izp_ffi_last_error

```
char g_izp_ffi_last_error[IZP_FFI_ERROR_CAP] = ""   [static]
```

Definition at line 14 of file izprime_ffi.c.

## 15.24 izprime_ffi.c

Go to the documentation of this file.

```
00001
00005
00006 #define IZP_FFI_BUILD 1
00007 #include <izprime_ffi.h>
00008
00009 #include <iZ_api.h>
00010
00011 #define IZP_MAX_SIEVE_LIMIT 1000000000000ULL
00012 #define IZP_FFI_ERROR_CAP 512
00013
00014 static char g_izp_ffi_last_error[IZP_FFI_ERROR_CAP] = "";
00015
00016 static void izp_ffi_set_error(const char *message)
00017 {
00018     snprintf(g_izp_ffi_last_error, sizeof(g_izp_ffi_last_error), "%s", message ? message : "");
00019 }
00020
00021 static int izp_ffi_validate_expr_nonnegative(const char *expr, mpz_t out)
00022 {
00023     if (expr == NULL || expr[0] == '\0')
00024         return 0;
00025     if (!parse_numeric_expr_mpz(out, expr))
00026         return 0;
00027     return mpz_sgn(out) >= 0;
00028 }
```

```c

static char *izp_ffi_strdup_heap(const char *src)
{
    if (src == NULL)
        return NULL;

    size_t len = strnlen(src, 8193);
    if (len > 8192)
        return NULL;

    len += 1;
    char *dst = malloc(len);
    if (dst == NULL)
        return NULL;
    memcpy(dst, src, len);
    return dst;
}

static int izp_ffi_copy_mpz_to_string(mpz_t value, char **out_string)
{
    char *tmp = mpz_get_str(NULL, 10, value);
    if (tmp == NULL)
    {
        izp_ffi_set_error("Failed to serialize mpz value to decimal string.");
        return IZP_FFI_ERR_ALLOC;
    }

    *out_string = izp_ffi_strdup_heap(tmp);
    free(tmp);

    if (*out_string == NULL)
    {
        izp_ffi_set_error("Failed to allocate output string.");
        return IZP_FFI_ERR_ALLOC;
    }

    return IZP_FFI_OK;
}

static int izp_ffi_export_primes(UI64_ARRAY *primes, IZP_U64_BUFFER *out)
{
    if (primes == NULL)
    {
        izp_ffi_set_error("Sieve operation failed (NULL prime list).");
        return IZP_FFI_ERR_OPERATION;
    }

    out->data = NULL;
    out->len = 0;

    if (primes->count < 0)
    {
        ui64_free(&primes);
        izp_ffi_set_error("Invalid prime list length returned by core API.");
        return IZP_FFI_ERR_OPERATION;
    }

    size_t len = (size_t)primes->count;
    if (len > 0)
    {
        if (len > SIZE_MAX / sizeof(uint64_t))
        {
            ui64_free(&primes);
            izp_ffi_set_error("Prime list length exceeds addressable memory.");
            return IZP_FFI_ERR_ALLOC;
        }

        out->data = malloc(len * sizeof(uint64_t));
        if (out->data == NULL)
        {
            ui64_free(&primes);
            izp_ffi_set_error("Failed to allocate output prime buffer.");
            return IZP_FFI_ERR_ALLOC;
        }

        memcpy(out->data, primes->array, len * sizeof(uint64_t));
    }

    out->len = len;
    ui64_free(&primes);
    return IZP_FFI_OK;
}

static uint64_t izp_ffi_count_small_range(const mpz_t start, uint64_t range, int mr_rounds)
{
    int rounds = MIN(MAX(mr_rounds, 5), 50);
    uint64_t total = 0;
```

```
    mpz_t z;
    mpz_init_set(z, start);

    for (uint64_t i = 0; i < range; i++)
    {
        if (mpz_cmp_ui(z, 2) >= 0 && mpz_probab_prime_p(z, rounds) > 0)
            total++;
        mpz_add_ui(z, z, 1);
    }

    mpz_clear(z);
    return total;
}

const char *izp_ffi_version(void)
{
#ifdef IZ_VERSION
    return IZ_VERSION;
#else
    return "dev";
#endif
}

const char *izp_ffi_status_message(int code)
{
    switch (code)
    {
    case IZP_FFI_OK:
        return "OK";
    case IZP_FFI_ERR_INVALID_ARG:
        return "Invalid argument";
    case IZP_FFI_ERR_PARSE:
        return "Parse error";
    case IZP_FFI_ERR_ALLOC:
        return "Allocation error";
    case IZP_FFI_ERR_NOT_FOUND:
        return "Not found";
    case IZP_FFI_ERR_OPERATION:
        return "Operation failed";
    default:
        return "Unknown status code";
    }
}

const char *izp_ffi_last_error(void)
{
    return g_izp_ffi_last_error;
}

void izp_ffi_clear_error(void)
{
    g_izp_ffi_last_error[0] = '\0';
}

int izp_ffi_sieve_u64(IZP_SIEVE_KIND kind, uint64_t limit, IZP_U64_BUFFER *out)
{
    izp_ffi_clear_error();

    if (out == NULL)
    {
        izp_ffi_set_error("out buffer pointer is NULL.");
        return IZP_FFI_ERR_INVALID_ARG;
    }

    out->data = NULL;
    out->len = 0;

    if (limit > IZP_MAX_SIEVE_LIMIT)
    {
        izp_ffi_set_error("Sieve limit must be <= 1000000000000.");
        return IZP_FFI_ERR_INVALID_ARG;
    }

    UI64_ARRAY *primes = NULL;

    switch (kind)
    {
    case IZP_SIEVE_SOE:
        primes = SoE(limit);
        break;
    case IZP_SIEVE_SSOE:
        primes = SSoE(limit);
        break;
    case IZP_SIEVE_SOS:
        primes = SoS(limit);
        break;
```

```
    case IZP_SIEVE_SSOS:
        primes = SSoS(limit);
        break;
    case IZP_SIEVE_SOEU:
        primes = SoEu(limit);
        break;
    case IZP_SIEVE_SOA:
        primes = SoA(limit);
        break;
    case IZP_SIEVE_SIZ:
        primes = SiZ(limit);
        break;
    case IZP_SIEVE_SIZM:
        primes = SiZm(limit);
        break;
    case IZP_SIEVE_SIZM_VY:
        primes = SiZm_vy(limit);
        break;
    default:
        izp_ffi_set_error("Unknown sieve kind.");
        return IZP_FFI_ERR_INVALID_ARG;
    }

    return izp_ffi_export_primes(primes, out);
}

int izp_ffi_count_range(const char *start, uint64_t range, int mr_rounds, int cores_num, uint64_t *out_count)
{
    izp_ffi_clear_error();

    if (out_count == NULL)
    {
        izp_ffi_set_error("out_count pointer is NULL.");
        return IZP_FFI_ERR_INVALID_ARG;
    }

    *out_count = 0;

    if (range == 0)
    {
        izp_ffi_set_error("range must be >= 1.");
        return IZP_FFI_ERR_INVALID_ARG;
    }

    mpz_t start_value;
    mpz_init(start_value);
    if (!izp_ffi_validate_expr_nonnegative(start, start_value))
    {
        mpz_clear(start_value);
        izp_ffi_set_error("Failed to parse non-negative start expression.");
        return IZP_FFI_ERR_PARSE;
    }

    // Core SiZ_count currently asserts range > 100. Handle tiny ranges directly.
    if (range <= 100)
    {
        *out_count = izp_ffi_count_small_range(start_value, range, mr_rounds);
        mpz_clear(start_value);
        return IZP_FFI_OK;
    }

    char *start_base10 = mpz_get_str(NULL, 10, start_value);
    if (start_base10 == NULL)
    {
        mpz_clear(start_value);
        izp_ffi_set_error("Failed to normalize start expression.");
        return IZP_FFI_ERR_ALLOC;
    }

    INPUT_SIEVE_RANGE input_range = {
        .start = start_base10,
        .range = range,
        .mr_rounds = mr_rounds,
        .filepath = NULL,
        .stream_gaps = 0,
    };

    int worker_count = MAX(cores_num, 1);
    *out_count = SiZ_count(&input_range, worker_count);

    free(start_base10);
    mpz_clear(start_value);
    return IZP_FFI_OK;
}

int izp_ffi_stream_range(const char *start, uint64_t range, int mr_rounds, const char *filepath, int stream_gaps,
    uint64_t *out_count)
```

```
{
    izp_ffi_clear_error();

    if (out_count == NULL)
    {
        izp_ffi_set_error("out_count pointer is NULL.");
        return IZP_FFI_ERR_INVALID_ARG;
    }

    *out_count = 0;

    if (filepath == NULL || filepath[0] == '\0')
    {
        izp_ffi_set_error("filepath must be a non-empty string.");
        return IZP_FFI_ERR_INVALID_ARG;
    }

    if (range == 0)
    {
        izp_ffi_set_error("range must be >= 1.");
        return IZP_FFI_ERR_INVALID_ARG;
    }

    mpz_t start_value;
    mpz_init(start_value);
    if (!izp_ffi_validate_expr_nonnegative(start, start_value))
    {
        mpz_clear(start_value);
        izp_ffi_set_error("Failed to parse non-negative start expression.");
        return IZP_FFI_ERR_PARSE;
    }

    // Probe output path first so path errors map to a deterministic FFI status.
    FILE *probe = fopen(filepath, "w");
    if (probe == NULL)
    {
        mpz_clear(start_value);
        izp_ffi_set_error("Failed to open output path.");
        return IZP_FFI_ERR_OPERATION;
    }
    fclose(probe);

    char *start_base10 = mpz_get_str(NULL, 10, start_value);
    if (start_base10 == NULL)
    {
        mpz_clear(start_value);
        izp_ffi_set_error("Failed to normalize start expression.");
        return IZP_FFI_ERR_ALLOC;
    }

    char *filepath_copy = izp_ffi_strdup_heap(filepath);
    if (filepath_copy == NULL)
    {
        free(start_base10);
        mpz_clear(start_value);
        izp_ffi_set_error("Failed to allocate output path buffer.");
        return IZP_FFI_ERR_ALLOC;
    }

    INPUT_SIEVE_RANGE input_range = {
        .start = start_base10,
        .range = range,
        .mr_rounds = mr_rounds,
        .filepath = filepath_copy,
        .stream_gaps = stream_gaps ? 1 : 0,
    };

    *out_count = SiZ_stream(&input_range);
    free(filepath_copy);
    free(start_base10);
    mpz_clear(start_value);

    return IZP_FFI_OK;
}

int izp_ffi_next_prime(const char *base_expr, int forward, char **out_prime_base10)
{
    izp_ffi_clear_error();

    if (out_prime_base10 == NULL)
    {
        izp_ffi_set_error("out_prime_base10 pointer is NULL.");
        return IZP_FFI_ERR_INVALID_ARG;
    }

    *out_prime_base10 = NULL;

```

```
    mpz_t base;
    mpz_init(base);
    if (!izp_ffi_validate_expr_nonnegative(base_expr, base))
    {
        mpz_clear(base);
        izp_ffi_set_error("Failed to parse non-negative base expression.");
        return IZP_FFI_ERR_PARSE;
    }

    mpz_t next;
    mpz_init(next);

    int found = iZ_next_prime(next, base, forward ? 1 : 0);
    if (!found)
    {
        mpz_clears(base, next, NULL);
        izp_ffi_set_error("No prime found for the requested direction.");
        return IZP_FFI_ERR_NOT_FOUND;
    }

    int status = izp_ffi_copy_mpz_to_string(next, out_prime_base10);
    mpz_clears(base, next, NULL);
    return status;
}

int izp_ffi_random_prime_vx(int bit_size, int cores_num, char **out_prime_base10)
{
    izp_ffi_clear_error();

    if (out_prime_base10 == NULL)
    {
        izp_ffi_set_error("out_prime_base10 pointer is NULL.");
        return IZP_FFI_ERR_INVALID_ARG;
    }

    *out_prime_base10 = NULL;

    if (bit_size < 2)
    {
        izp_ffi_set_error("bit_size must be >= 2.");
        return IZP_FFI_ERR_INVALID_ARG;
    }

    mpz_t prime;
    mpz_init(prime);
    int found = vx_random_prime(prime, bit_size, MAX(cores_num, 1));
    if (!found)
    {
        mpz_clear(prime);
        izp_ffi_set_error("vx_random_prime failed.");
        return IZP_FFI_ERR_OPERATION;
    }

    int status = izp_ffi_copy_mpz_to_string(prime, out_prime_base10);
    mpz_clear(prime);
    return status;
}

int izp_ffi_random_prime_vy(int bit_size, int cores_num, char **out_prime_base10)
{
    izp_ffi_clear_error();

    if (out_prime_base10 == NULL)
    {
        izp_ffi_set_error("out_prime_base10 pointer is NULL.");
        return IZP_FFI_ERR_INVALID_ARG;
    }

    *out_prime_base10 = NULL;

    if (bit_size < 2)
    {
        izp_ffi_set_error("bit_size must be >= 2.");
        return IZP_FFI_ERR_INVALID_ARG;
    }

    mpz_t prime;
    mpz_init(prime);
    int found = vy_random_prime(prime, bit_size, MAX(cores_num, 1));
    if (!found)
    {
        mpz_clear(prime);
        izp_ffi_set_error("vy_random_prime failed.");
        return IZP_FFI_ERR_OPERATION;
    }

    int status = izp_ffi_copy_mpz_to_string(prime, out_prime_base10);
```

```
    mpz_clear(prime);
    return status;
}

void izp_ffi_free_u64_buffer(IZP_U64_BUFFER *buffer)
{
    if (buffer == NULL)
        return;

    free(buffer->data);
    buffer->data = NULL;
    buffer->len = 0;
}

void izp_ffi_free_string(char **str)
{
    if (str == NULL || *str == NULL)
        return;

    free(*str);
    *str = NULL;
}
```

## 15.25 iZ_apps.c File Reference

High-level application routines built on top of the iZ_toolkit.

#include <iZ_api.h>

### Macros

- #define PRIME_STR_CAP_PADDING 64U

### Functions

- ___attribute___ ((constructor))

#### SiZ Range Variants

Count/stream primes over a numeric interval.

- uint64_t SiZ_stream (INPUT_SIEVE_RANGE *range)
  Stream primes (or prime gaps) in a range to filepath (and return the count).
- uint64_t SiZ_count (INPUT_SIEVE_RANGE *input_range, int cores_num)
  Count primes in a range using multiple worker processes.

#### Prime Generators

Probabilistic prime searches using iZm/VX machinery.

- int vy_random_prime (mpz_t p, int bit_size, int cores_num)
  Search for a random prime using vertical (vy) sieving.
- int vx_random_prime (mpz_t p, int bit_size, int cores_num)
  Search for a random prime using horizontal (vx) sieving.
- int iZ_next_prime (mpz_t p, mpz_t base, int forward)
  Advance to the next (or previous) prime from a base value.

### Variables

- static IZM * iZmX = NULL

### 15.25.1 Detailed Description

High-level application routines built on top of the iZ_toolkit.

This module contains range-oriented entry points (stream/count) and probabilistic prime generation wrappers.

Definition in file iZ_apps.c.

### 15.25.2 Macro Definition Documentation

#### 15.25.2.1 PRIME_STR_CAP_PADDING

#define PRIME_STR_CAP_PADDING 64U

Definition at line 12 of file iZ_apps.c.

### 15.25.3 Function Documentation

#### 15.25.3.1 __attribute__()

__attribute__ (
(constructor) )

Definition at line 33 of file iZ_apps.c.

### 15.25.4 Variable Documentation

#### 15.25.4.1 iZmX

IZM* iZmX = NULL [static]

Definition at line 30 of file iZ_apps.c.

## 15.26 iZ_apps.c

Go to the documentation of this file.

```
#include <iZ_api.h>

#define PRIME_STR_CAP_PADDING 64U

#if IZ_PLATFORM_HAS_FORK
static int write_all_bytes(int fd, const char *buf, size_t len)
{
    size_t total = 0;
    while (total < len)
    {
        ssize_t written = write(fd, buf + total, len - total);
        if (written <= 0)
            return 0;
        total += (size_t)written;
    }
    return 1;
}
#endif

// Global iZm instance for VX6 segments, initialized once and reused by range APIs.
static IZM *iZmX = NULL;

// Constructor to initialize iZmX at program startup
__attribute__((constructor)) static void init_iZmX(void)
{
    int vx = VX6; // 1616615, suitable for up to 10^12 on typical hardware,
    // for larger limits on capable hardware consider VX7 or VX8
    iZmX = iZm_init(vx);
    if (!iZmX)
    {
        log_error("Failed to initialize iZmX in constructor.");
        exit(EXIT_FAILURE);
    }
}

// ============================================================
// * SiZ Range Variants
// ============================================================

uint64_t SiZ_stream(INPUT_SIEVE_RANGE *input_range)
{
    assert(input_range && input_range->start && "Invalid INPUT_SIEVE_RANGE passed to SiZ_stream.");

    int has_output_file = (input_range->filepath && input_range->filepath[0] != '\0');
    FILE *output = stdout; // default to stdout if no valid filepath is provided

    if (has_output_file)
    {
        output = fopen(input_range->filepath, "w");
        if (output == NULL)
        {
            log_error("Failed to open output file: %s", input_range->filepath);
            return 0;
        }
    }

    uint64_t total = 0; // output: total prime count

    int vx = iZmX->vx;
    // Miller-Rabin rounds, bounded [5, 50]
    int mr_rounds = MIN(MAX(input_range->mr_rounds, 5), 50);

    IZM_RANGE_INFO info = range_info_init(input_range, vx);
    if (info.y_range < 0)
    {
        if (has_output_file)
            fclose(output);
        range_info_free(&info);
        return 0;
    }

    mpz_t current_y;
    mpz_init(current_y);
    mpz_set(current_y, info.Ys);
    int start_x = mpz_fdiv_ui(info.Xs, info.vx);
    int end_x = mpz_fdiv_ui(info.Xe, info.vx);

    // If Ys = 0, use SiZm on the first segment (covers small fixed primes too).
    if (mpz_cmp_ui(current_y, 0) == 0)
    {
```

```
        uint64_t limit = mpz_cmp_ui(info.Ye, 0) > 0 ? info.vx : end_x;
        UI64_ARRAY *primes = SiZm(limit * 6 + 1);
        if (!primes)
        {
            total = 0;
            goto stream_cleanup;
        }

        uint64_t s = mpz_get_ui(info.Zs);
        uint64_t e = mpz_get_ui(info.Ze);
        uint64_t last_gap_base = 1;

        for (int i = 0; i < primes->count; i++)
        {
            // only primes in [Zs, Ze]
            if (primes->array[i] > s && primes->array[i] <= e)
            {
                total++;
                if (input_range->stream_gaps)
                {
                    uint64_t gap = primes->array[i] - last_gap_base;
                    fprintf(output, "%" PRIu64 " ", gap);
                    last_gap_base = primes->array[i];
                }
                else
                {
                    fprintf(output, "%" PRIu64 " ", primes->array[i]);
                }
            }
        }

        start_x = 1;                        // next segment starts at x=1
        mpz_add_ui(current_y, current_y, 1); // increment Ys for the next segment

        ui64_free(&primes);
    }

    // No remaining vx segments after the initial one.
    if (mpz_cmp(current_y, info.Ye) > 0)
    {
        goto stream_cleanup;
    }

    if (!iZmX)
    {
        log_error("SiZ_stream: global iZmX is not initialized.");
        total = 0;
        goto stream_cleanup;
    }

    // Process remaining segments for y in [current_y:Ye]
    int first_segment = 1;
    while (mpz_cmp(current_y, info.Ye) <= 0)
    {
        int seg_start_x = first_segment ? start_x : 1;
        int seg_end_x = (mpz_cmp(current_y, info.Ye) == 0) ? end_x : vx;
        char *y_str = mpz_get_str(NULL, 10, current_y);
        if (!y_str)
        {
            total = 0;
            goto stream_cleanup;
        }

        VX_SEG *vx_obj = vx_init(iZmX, seg_start_x, seg_end_x, y_str, mr_rounds);
        free(y_str);
        if (!vx_obj)
        {
            // check logs for errors
            total = 0;
            goto stream_cleanup;
        }

        vx_stream(vx_obj, output, input_range->stream_gaps);
        total += vx_obj->p_count; // accumulate prime count

        vx_free(&vx_obj);
        first_segment = 0;
        mpz_add_ui(current_y, current_y, 1); // increment Ys for the next segment
    }

stream_cleanup:
    range_info_free(&info);
    mpz_clear(current_y);
    if (has_output_file)
        fclose(output);
    else
        fflush(stdout);
```

```
    return total;
}

uint64_t SiZ_count(INPUT_SIEVE_RANGE *input_range, int cores_num)
{
    assert(input_range && input_range->start && input_range->range > 100 &&
           "Invalid INPUT_SIEVE_RANGE passed to SiZ_count.");

    uint64_t total = 0;
    int vx = iZmX->vx;
    // Miller-Rabin rounds, bounded [5, 50]
    int mr_rounds = MIN(MAX(input_range->mr_rounds, 5), 50);
    cores_num = MAX(1, MIN(cores_num, get_cpu_cores_count()));
#if !IZ_PLATFORM_HAS_FORK
    if (cores_num > 1)
    {
        log_info("SiZ_count: multi-process mode is unavailable on this platform; using single-process mode.");
        cores_num = 1;
    }
#endif
#if IZ_PLATFORM_HAS_FORK
    int (*pipe_fds)[2] = NULL;
    pid_t *pids = NULL;
#endif

    IZM_RANGE_INFO info = range_info_init(input_range, vx);
    if (info.y_range < 0)
    {
        range_info_free(&info);
        return 0;
    }

    mpz_t current_y;
    mpz_init(current_y);
    mpz_set(current_y, info.Ys);
    int start_x = mpz_fdiv_ui(info.Xs, vx);
    int end_x = mpz_fdiv_ui(info.Xe, vx);

    // if current_y = 0, use sieve_iZm for the first segment
    if (mpz_cmp_ui(current_y, 0) == 0)
    {
        uint64_t limit = mpz_cmp_ui(info.Ye, 0) > 0 ? vx : end_x;
        UI64_ARRAY *primes = SiZm(limit * 6 + 1);
        if (!primes)
        {
            total = 0;
            goto count_cleanup;
        }

        total += primes->count;
        uint64_t s = mpz_get_ui(info.Zs);
        uint64_t e = mpz_get_ui(info.Ze);

        for (int i = 0; i < primes->count; i++)
        {
            // Exclude values outside [Zs, Ze] from the first solid segment.
            if (primes->array[i] < s || primes->array[i] > e)
            {
                total--;
            }
        }

        ui64_free(&primes);
        start_x = 1;
        mpz_add_ui(current_y, current_y, 1); // increment Ys for the next segment
    }

    if (mpz_cmp(current_y, info.Ye) > 0)
    {
        goto count_cleanup;
    }

    if (!iZmX)
    {
        log_error("SiZ_count: global iZmX is not initialized.");
        total = 0;
        goto count_cleanup;
    }

    // Handle edge cases:
    mpz_t prime_z;
    mpz_init(prime_z);
    // if Ys > 0 and Zs % 6 <= 1
    if (mpz_cmp_ui(current_y, 0) > 0 && mpz_fdiv_ui(info.Zs, 6) <= 1)
    {
        // if iZ(Xs, -1) < Zs and prime, decrement total
```

```
        iZ_mpz(prime_z, info.Xs, -1);
        if (mpz_cmp(prime_z, info.Zs) < 0)
        {
            if (mpz_probab_prime_p(prime_z, 25))
            {
                total--;
            }
        }
    }
    // if Ye > 0 and Ze % 6 <= 1
    if (mpz_cmp_ui(info.Ye, 0) > 0 && mpz_fdiv_ui(info.Ze, 6) <= 1)
    {
        // if iZ(Xe, 1) > Ze and prime, decrement total
        iZ_mpz(prime_z, info.Xe, 1);
        if (mpz_cmp(prime_z, info.Ze) > 0)
        {
            if (mpz_probab_prime_p(prime_z, 25))
            {
                total--;
            }
        }
    }
    mpz_clear(prime_z);

    int total_segments = (int)(mpz_get_ui(info.Ye) - mpz_get_ui(current_y) + 1);
    if (total_segments <= 0)
    {
        goto count_cleanup;
    }

    // Single-process processing of all segments
    if (cores_num == 1)
    {
        int first_segment = 1;
        for (int i = 0; i < total_segments; i++)
        {
            int seg_start_x = first_segment ? start_x : 1;
            int seg_end_x = (i == total_segments - 1) ? end_x : vx;
            char *y_str = mpz_get_str(NULL, 10, current_y);
            if (!y_str)
            {
                total = 0;
                goto count_cleanup;
            }

            VX_SEG *vx_obj = vx_init(iZmX, seg_start_x, seg_end_x, y_str, mr_rounds);
            free(y_str);
            if (!vx_obj)
            {
                total = 0;
                goto count_cleanup;
            }

            vx_full_sieve(vx_obj, 0);
            total += vx_obj->p_count;

            vx_free(&vx_obj);
            first_segment = 0;
            mpz_add_ui(current_y, current_y, 1); // increment Ys for the next segment
        }
        goto count_cleanup;
    }

#if IZ_PLATFORM_HAS_FORK
    // Multi-process processing of remaining segments
    // if total_segments < cores_num, adjust cores_num
    if (total_segments < cores_num)
    {
        cores_num = total_segments;
    }

    int segments_per_core = total_segments / cores_num;
    int remainder_segments = total_segments % cores_num;

    pipe_fds = malloc((size_t)cores_num * sizeof(*pipe_fds));
    pids = malloc((size_t)cores_num * sizeof(*pids));
    if (!pipe_fds || !pids)
    {
        log_error("SiZ_count: Failed to allocate process bookkeeping arrays.");
        total = 0;
        goto count_cleanup;
    }

    for (int i = 0; i < cores_num; i++)
    {
        pids[i] = -1;
        pipe_fds[i][0] = -1;
```

```
        pipe_fds[i][1] = -1;
    }

    for (int core = 0; core < cores_num; core++)
    {
        if (pipe(pipe_fds[core]) == -1)
        {
            log_error("SiZ_count: Failed to create pipe for core %d. Aborting.", core);
            // cleanup and abort on any pipe error
            for (int j = 0; j <= core; j++)
            {
                if (pipe_fds[j][0] != -1)
                    close(pipe_fds[j][0]);
                if (pipe_fds[j][1] != -1)
                    close(pipe_fds[j][1]);
            }
            total = 0;
            goto count_cleanup;
        }

        pids[core] = fork();
        if (pids[core] < 0)
        {
            log_error("SiZ_count: Failed to fork process for core %d. Aborting.", core);
            // close this core's pipe and any previous ones, then exit
            for (int j = 0; j <= core; j++)
            {
                if (pipe_fds[j][0] != -1)
                    close(pipe_fds[j][0]);
                if (pipe_fds[j][1] != -1)
                    close(pipe_fds[j][1]);
            }
            total = 0;
            goto count_cleanup;
        }

        if (pids[core] == 0)
        {
            // Child process
            // Close unrelated pipes
            for (int j = 0; j < cores_num; j++)
            {
                if (j != core && pipe_fds[j][0] != -1)
                    close(pipe_fds[j][0]);
                if (j != core && pipe_fds[j][1] != -1)
                    close(pipe_fds[j][1]);
            }
            close(pipe_fds[core][0]); // close read end

            // Compute this child's starting Y and number of segments
            int offset = core * segments_per_core + (core < remainder_segments ? core : remainder_segments);
            int local_segments = segments_per_core + (core < remainder_segments ? 1 : 0);

            uint64_t child_total = 0;

            if (local_segments > 0)
            {
                mpz_t local_Ys;
                mpz_init(local_Ys);
                mpz_set(local_Ys, current_y);
                mpz_add_ui(local_Ys, local_Ys, offset);

                // Each child has its own IZM to avoid data races
                IZM *iZm_local = iZm_clone(iZmX);
                if (!iZm_local)
                {
                    mpz_clear(local_Ys);
                    close(pipe_fds[core][1]);
                    exit(1);
                }

                for (int i = 0; i < local_segments; i++)
                {
                    int global_segment = offset + i;
                    int seg_start_x = (global_segment == 0) ? start_x : 1;
                    int seg_end_x = (global_segment == total_segments - 1) ? end_x : vx;
                    char *y_str = mpz_get_str(NULL, 10, local_Ys);
                    if (!y_str)
                    {
                        iZm_free(&iZm_local);
                        mpz_clear(local_Ys);
                        close(pipe_fds[core][1]);
                        exit(1);
                    }

                    VX_SEG *vx_obj = vx_init(iZm_local, seg_start_x, seg_end_x, y_str, mr_rounds);
                    free(y_str);
```

```
                if (!vx_obj)
                {
                    iZm_free(&iZm_local);
                    mpz_clear(local_Ys);
                    close(pipe_fds[core][1]);
                    exit(1);
                }

                vx_full_sieve(vx_obj, 0);
                child_total += vx_obj->p_count;

                vx_free(&vx_obj);
                mpz_add_ui(local_Ys, local_Ys, 1);
            }

            iZm_free(&iZm_local);
            mpz_clear(local_Ys);
        }

        // Send result to parent
        ssize_t written = write(pipe_fds[core][1], &child_total, sizeof(child_total));
        if (written != sizeof(child_total))
        {
            log_error("SiZ_count: Child %d failed to write result.", core);
        }

        close(pipe_fds[core][1]);
        exit(0);
    }
    else
    {
        // Parent process
        close(pipe_fds[core][1]); // close write end
        pipe_fds[core][1] = -1;
    }
}

// Collect results
for (int core = 0; core < cores_num; core++)
{
    if (pids[core] > 0 && pipe_fds[core][0] != -1)
    {
        uint64_t child_total = 0;
        ssize_t bytes_read = read(pipe_fds[core][0], &child_total, sizeof(child_total));
        if (bytes_read == sizeof(child_total))
        {
            total += child_total;
        }
        else if (bytes_read == 0)
        {
            log_error("SiZ_count: Child %d closed pipe without sending result.", core);
        }
        else if (bytes_read < 0)
        {
            log_error("SiZ_count: Failed to read result from child %d.", core);
        }
        else
        {
            log_error("SiZ_count: Partial read from child %d (got %zd bytes).", core, bytes_read);
        }

        close(pipe_fds[core][0]);
        pipe_fds[core][0] = -1;

        int status;
        waitpid(pids[core], &status, 0);
        if (WIFEXITED(status) && WEXITSTATUS(status) != 0)
        {
            log_error("SiZ_count: Child %d exited with status %d.", core, WEXITSTATUS(status));
            // propagate child failure as an overall error
            total = 0;
            goto count_cleanup;
        }
        else if (WIFSIGNALED(status))
        {
            log_error("SiZ_count: Child %d terminated by signal %d.", core, WTERMSIG(status));
            total = 0;
            goto count_cleanup;
        }
    }
    else if (pipe_fds[core][0] != -1)
    {
        close(pipe_fds[core][0]);
        pipe_fds[core][0] = -1;
    }
}
#endif
```

```

count_cleanup:
#if IZ_PLATFORM_HAS_FORK
    free(pids);
    free(pipe_fds);
#endif
    range_info_free(&info);
    mpz_clear(current_y);

    return total;
}

// ==========================================================
// * Random Prime Generation
// ==========================================================

int vy_random_prime(mpz_t p, int bit_size, int cores_num)
{
    int found = 0;              // flag to indicate if a prime was found
    bit_size = MAX(bit_size, 10); // Set minimum bit size

    // 1. Compute max vx for the given bit-size
    mpz_t vx;
    mpz_init(vx);
    compute_max_vx(vx, bit_size);

    // 2. If < 2 cores, run the search in-process
    if (cores_num < 2)
    {
        found = vy_search_prime(p, 0, vx);
        mpz_clear(vx);
        return found;
    }

#if !IZ_PLATFORM_HAS_FORK
    log_info("vy_random_prime: multi-process mode is unavailable on this platform; using single-process mode.");
    found = vy_search_prime(p, 0, vx);
    mpz_clear(vx);
    return found;
#else
    // 3. Else, fork multiple processes to search for a prime
    // Create a pipe for inter-process communication.
    int fd[2];
    if (pipe(fd) == -1)
    {
        log_error("Failed to create pipe in vy_random_prime. Falling back to in-process search.");
        found = vy_search_prime(p, 0, vx);
        mpz_clear(vx);
        return found;
    }

    pid_t pids[cores_num];
    int forks_created = 0;

    // Fork child processes.
    for (int i = 0; i < cores_num; i++)
    {
        pid_t pid = fork();
        if (pid < 0)
        {
            log_error("Failed to fork process %d in vy_random_prime", i);
            continue; // Skip this fork failure and continue creating other forks
        }
        else if (pid == 0)
        {
            // Child process: close the read-end.
            close(fd[0]);

            mpz_t local_p; // local prime candidate
            mpz_init(local_p);

            // Search for a candidate prime
            found = vy_search_prime(local_p, 0, vx);

            // If a candidate is found, send it via the pipe.
            if (found)
            {
                char *p_str = mpz_get_str(NULL, 10, local_p);
                if (p_str)
                {
                    size_t p_cap = (size_t)bit_size + PRIME_STR_CAP_PADDING;
                    size_t p_len = strnlen(p_str, p_cap + 1);
                    if (p_len > p_cap || !write_all_bytes(fd[1], p_str, p_len + 1))
                        log_warn("Failed to write prime candidate from child process in vy_random_prime");
                    free(p_str);
                }
            }
```

```
            mpz_clear(local_p);

            close(fd[1]); // Close the write-end.
            exit(0);
        }
        else
        {
            // Parent process saves child's PID.
            pids[i] = pid;
            forks_created++;
        }
    }

    // 4. Parent reads first result from the pipe.
    close(fd[1]); // Close the write-end.

    if (forks_created == 0)
    {
        log_error("No child processes were created in vy_random_prime, falling back to in-process search");
        close(fd[0]);
        found = vy_search_prime(p, 0, vx);
        mpz_clear(vx);
        return found;
    }

    // Allocate a sufficiently large buffer to hold the prime string + null terminator
    // bit_size * 0.302 is approx digits, add extra padding for safety
    size_t buf_size = (size_t)(bit_size * 0.4);
    char *buf = malloc(buf_size);
    if (!buf)
    {
        log_error("Failed to allocate buffer in vy_random_prime");
        found = 0;
    }
    else
    {
        ssize_t n = read(fd[0], buf, buf_size - 1);
        if (n == -1)
        {
            log_error("Failed to read from pipe in vx_random_prime");
            found = 0;
        }
        else if (n > 0)
        {
            buf[n] = '\0'; // Ensure null-termination
            if (mpz_set_str(p, buf, 10) == 0)
            {
                found = 1;
            }
            else
            {
                log_error("Failed to set prime from buffer in vx_random_prime");
                found = 0;
            }
        }
        free(buf);
    }
    close(fd[0]); // Close the read-end of the pipe.

    // 5. Terminate all child processes
    for (int i = 0; i < forks_created; i++)
    {
        kill(pids[i], SIGTERM);    // Terminate child process
        waitpid(pids[i], NULL, 0); // Wait for child process to terminate
    }

    mpz_clear(vx);

    return found;
#endif
}

int vx_random_prime(mpz_t p, int bit_size, int cores_num)
{
    int found = 0;
    bit_size = MAX(bit_size, 10);
    int vx = bit_size <= 2048 ? VX5 : VX6;

    // 2. If < 2 cores, run the search in-process
    if (cores_num < 2)
    {
        found = vx_search_prime(p, 0, vx, bit_size);
        return found;
    }

#if !IZ_PLATFORM_HAS_FORK
    log_info("vx_random_prime: multi-process mode is unavailable on this platform; using single-process mode.");
```

```
    found = vx_search_prime(p, 0, vx, bit_size);
    return found;
#else
    // 3. Multi-core: fork processes
    int fd[2];
    if (pipe(fd) == -1)
    {
        log_error("Failed to create pipe in vx_random_prime. Falling back to in-process search.");
        found = vx_search_prime(p, 0, vx, bit_size);
        return found;
    }

    pid_t pids[cores_num];
    int forks_created = 0;

    for (int i = 0; i < cores_num; i++)
    {
        pid_t pid = fork();
        if (pid < 0)
        {
            log_error("Failed to fork process %d in vx_random_prime", i);
            continue;
        }
        else if (pid == 0)
        {
            // CHILD PROCESS: Initialize everything fresh
            close(fd[0]);

            // Search for prime with child's own objects
            mpz_t local_p;
            mpz_init(local_p);
            int child_found = vx_search_prime(local_p, 0, vx, bit_size);

            if (child_found)
            {
                char *p_str = mpz_get_str(NULL, 10, local_p);
                if (p_str != NULL)
                {
                    size_t p_cap = (size_t)bit_size + PRIME_STR_CAP_PADDING;
                    size_t p_len = strnlen(p_str, p_cap + 1);
                    if (p_len > p_cap || !write_all_bytes(fd[1], p_str, p_len + 1))
                        log_warn("Failed to write prime candidate from child process in vx_random_prime");
                    free(p_str);
                }
            }

            // Cleanup child resources
            mpz_clear(local_p);
            close(fd[1]);
            exit(0);
        }
        else
        {
            pids[i] = pid;
            forks_created++;
        }
    }

    // 4. Parent process reads result
    close(fd[1]);

    if (forks_created == 0)
    {
        log_error("No child processes were created in vx_random_prime, falling back to in-process search");
        close(fd[0]);
        // fallback to in-process search
        found = vx_search_prime(p, 0, vx, bit_size);
        return found;
    }

    // Allocate a sufficiently large buffer to hold the prime string + null terminator
    size_t buf_size = (size_t)(bit_size * 0.4);
    char *buf = malloc(buf_size);
    if (!buf)
    {
        log_error("Failed to allocate buffer in vx_random_prime");
        found = 0;
    }
    else
    {
        ssize_t n = read(fd[0], buf, buf_size - 1);
        if (n > 0)
        {
            buf[n] = '\0'; // Ensure null-termination
            if (mpz_set_str(p, buf, 10) == 0)
            {
                found = 1;
```

```
            }
            else
            {
                log_error("Failed to set prime from buffer in vx_random_prime");
                found = 0;
            }
        }
        free(buf);
    }

    close(fd[0]); // Close the read-end of the pipe.

    // Terminate children
    for (int i = 0; i < forks_created; i++)
    {
        kill(pids[i], SIGTERM);
        waitpid(pids[i], NULL, 0);
    }

    return found;
#endif
}

int iZ_next_prime(mpz_t p, mpz_t base, int forward)
{
    // 1. Initialization
    int found = 0; // flag to indicate if a prime was found
    // tmp variable to hold the value until a prime is found
    mpz_t z;
    mpz_init_set(z, base); // set tmp = base

    int vx = VX6;

    // Edge cases 1:
    // Handle small cases where base < vx
    if (mpz_cmp_ui(base, vx) < 0)
    {
        // find the next/previous prime after/before base in the iZmX->root_primes array
        uint64_t base_ui = mpz_get_ui(base); // get base ui value
        for (int i = 0; i < iZmX->root_primes->count; i++)
        {
            uint64_t prime = iZmX->root_primes->array[i];
            if (forward)
            {
                if (prime > base_ui)
                {
                    mpz_set_ui(p, prime);
                    found = 1;
                    break;
                }
            }
            else
            {
                if (prime < base_ui)
                {
                    mpz_set_ui(p, prime);
                    found = 1;
                }
                else
                {
                    break; // since root_primes are sorted, we can stop once we pass the candidate
                }
            }
        }

        return found;
    }

    // Edge cases 2:
    // if forward and base is iZ-, check next iZ+
    if (mpz_fdiv_ui(z, 6) == 5 && forward)
    {
        mpz_add_ui(z, z, 2); // increment tmp by 2
        if (mpz_probab_prime_p(z, MR_ROUNDS))
        {
            mpz_set(p, z); // set p = tmp + 2
            mpz_clear(z);
            return 1;
        }
    }

    // Edge cases 3:
    // if backward and base is iZ+, check previous iZ-
    if (mpz_fdiv_ui(z, 6) == 1 && !forward)
    {
        mpz_sub_ui(z, z, 2); // decrement tmp by 2
        if (mpz_probab_prime_p(z, MR_ROUNDS))
```

```
        {
            mpz_set(p, z); // set p = tmp - 2
            mpz_clear(z);
            return 1;
        }
    }

    // Initialize and set y and yvx
    mpz_t y, yvx, x_p;
    mpz_init(y);
    mpz_init(yvx);
    mpz_init(x_p);

    mpz_div_ui(y, base, 6 * vx); // compute y = base / 6 * vx
    mpz_mul_ui(yvx, y, vx);      // compute yvx = y * vx
    mpz_div_ui(x_p, z, 6);       // compute x_p = z / 6

    // 2. Iterate over the x5 and x7 bitmaps to find a prime
    // set start_x = x_p % vx +/- 1
    int step = forward ? 1 : -1;
    int start_x = mpz_fdiv_ui(x_p, vx) + step;
    int end_x = forward ? vx : 1;

    int i = 0; // segment counter

    while (!found)
    {
        if (forward)
        {
            if (i > 0)
                start_x = 1; // start from the beginning of the bitmap

            for (int x = start_x; x <= end_x; x++)
            {
                // check if x5[x] is set
                if (bitmap_get_bit(iZmX->base_x5, x))
                {
                    mpz_add_ui(x_p, yvx, x); // set x_p = yvx + x
                    iZ_mpz(z, x_p, -1);      // compute p = iZ(x_p, -1)
                    // check if tmp is prime
                    found = mpz_probab_prime_p(z, MR_ROUNDS);

                    if (found)
                        break;
                }

                // check if x7[x] is set
                if (bitmap_get_bit(iZmX->base_x7, x))
                {
                    mpz_add_ui(x_p, yvx, x); // set x_p = yvx + x
                    iZ_mpz(z, x_p, 1);       // compute tmp = iZ(x_p, 1)
                    // check if tmp is prime
                    found = mpz_probab_prime_p(z, MR_ROUNDS);

                    if (found)
                        break;
                }
            }

            mpz_add_ui(yvx, yvx, vx); // increment yvx by vx for next segment
        }
        else // backward search
        {
            if (i > 0)
                start_x = vx; // start from the end of the bitmaps

            // check iZ+ first if backward
            for (int x = start_x; x >= end_x; x--)
            {
                // check if x7[x] is set
                if (bitmap_get_bit(iZmX->base_x7, x))
                {
                    mpz_add_ui(x_p, yvx, x); // set x_p = yvx + x
                    iZ_mpz(z, x_p, 1);       // compute tmp = iZ(x_p, 1)
                    // check if tmp is prime
                    found = mpz_probab_prime_p(z, MR_ROUNDS);

                    if (found)
                        break;
                }

                // check iZ-
                if (bitmap_get_bit(iZmX->base_x5, x))
                {
                    mpz_add_ui(x_p, yvx, x); // set x_p = yvx + x
                    iZ_mpz(z, x_p, -1);      // compute p = iZ(x_p, -1)
                    // check if tmp is prime
```

```
                found = mpz_probab_prime_p(z, MR_ROUNDS);

                if (found)
                    break;
            }
        }

        mpz_sub_ui(yvx, yvx, vx); // decrement yvx by vx for next segment
    }

    i++; // increment segment counter
}

// 3. Set the found prime
if (found)
    mpz_set(p, z); // set p = tmp
else
    log_debug("No prime found :/");

// cleanup
mpz_clears(y, yvx, x_p, z, NULL);

return found;
}
```

## 15.27 playground.c File Reference

Scratch space for testing new ideas.

#include <iZ_api.h>

### 15.27.1 Detailed Description

Scratch space for testing new ideas.

Definition in file playground.c.

## 15.28 playground.c

Go to the documentation of this file.

```
#include <iZ_api.h>
```

## 15.29 prime_sieve.c File Reference

Implementations of classical and SiZ-family sieve algorithms.

#include <iZ_api.h>

Macros

- #define N_LIMIT (1000000000000ULL)
- #define ASSERT_LIMIT(n)

  Assert that input n is within the valid range for sieve functions.
- #define Pi(n)

  Pi(n) is an approximation of the number of primes up to n, used for initial array sizing.

Functions

- static void push_small_primes (UI64_ARRAY *primes, uint64_t n)
- static void process_N_bitmap (UI64_ARRAY *primes, BITMAP *sieve_bitmap, uint64_t n)

  A helper function to process the bitmap for the Sieve of Eratosthenes and collect primes.

Classic Prime Sieve Algorithms

Baseline sieves up to a numeric limit.

All classic sieve functions take a limit n and return an ascending list of primes <= n.

- UI64_ARRAY * SoE (uint64_t n)

  Optimized Sieve of Eratosthenes.
- UI64_ARRAY * SSoE (uint64_t n)

  Segmented Sieve of Eratosthenes.
- UI64_ARRAY * SoS (uint64_t n)

  Sieve of Sundaram.
- UI64_ARRAY * SSoS (uint64_t n)

  Segmented Sieve of Sundaram.
- UI64_ARRAY * SoEu (uint64_t n)

  Euler (linear) sieve.
- UI64_ARRAY * SoA (uint64_t n)

  Sieve of Atkin.

iZ-based Sieve Algorithms

Sieve family operating in the 6x-1 / 6x+1 index space.

- UI64_ARRAY * SiZ (uint64_t n)

  Solid Sieve-iZ (wheel 6, iZ index space).
- UI64_ARRAY * SiZm (uint64_t n)

  Segmented Sieve-iZm (VX segmented, horizontal processing).
- UI64_ARRAY * SiZm_vy (uint64_t n)

  Segmented Sieve-iZm with vertical (y-major) traversal.

Variables

- static const uint64_t base_primes [] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97}
- static const int base_primes_count = sizeof(base_primes) / sizeof(base_primes[0])

### 15.29.1 Detailed Description

Implementations of classical and SiZ-family sieve algorithms.

This file contains the implementations of various prime sieving algorithms, including classical algorithms (SoE/SSoE/SoEu/SoS/SoA) as well as SiZ-family algorithms (SiZ/SiZm/SiZm_vy). All functions are single-threaded, take an upper limit n and return a pointer to a UI64_ARRAY containing the prime numbers up to n.

Definition in file prime_sieve.c.

## 15.29.2 Macro Definition Documentation

### 15.29.2.1 ASSERT_LIMIT

```
#define ASSERT_LIMIT(
            n)
```

Value:

```
assert((n) <= N_LIMIT && "Input must be in the range <= 10^12.")
```

Assert that input n is within the valid range for sieve functions.

Definition at line 23 of file prime_sieve.c.

Referenced by SiZ(), SiZm(), SiZm_vy(), SoA(), SoE(), SoEu(), SoS(), SSoE(), and SSoS().

### 15.29.2.2 N_LIMIT

```
#define N_LIMIT (1000000000000ULL)
```

Maximum supported sieve limit for standard entry points (10^12).

Definition at line 20 of file prime_sieve.c.

### 15.29.2.3 Pi

```
#define Pi(
            n)
```

Value:

```
((n / log(n)))
```

Pi(n) is an approximation of the number of primes up to n, used for initial array sizing.

Definition at line 26 of file prime_sieve.c.

Referenced by SiZ(), SiZm(), SiZm_vy(), SoA(), SoE(), SoEu(), SoS(), SSoE(), and SSoS().

## 15.29.3 Function Documentation

### 15.29.3.1 process_N_bitmap()

```
void process_N_bitmap (
            UI64_ARRAY * primes,
            BITMAP * sieve_bitmap,
            uint64_t n)   [static]
```

A helper function to process the bitmap for the Sieve of Eratosthenes and collect primes.

Parameters

| | |
|---|---|
| primes | Output prime array. |

| | |
|---|---|
| sieve_bitmap | Candidate bitmap. |
| n | Inclusive numeric upper bound represented in sieve_bitmap. |

Definition at line 48 of file prime_sieve.c.

References bitmap_clear_steps_simd(), bitmap_get_bit(), and ui64_push().

Referenced by SoE(), and SSoE().

#### 15.29.3.2 push_small_primes()

```
void push_small_primes (
            UI64_ARRAY * primes,
            uint64_t n)   [static]
```

Definition at line 32 of file prime_sieve.c.

### 15.29.4 Variable Documentation

#### 15.29.4.1 base_primes

const uint64_t base_primes[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97} [static]

Definition at line 29 of file prime_sieve.c.

#### 15.29.4.2 base_primes_count

const int base_primes_count = sizeof(base_primes) / sizeof(base_primes[0]) [static]

Definition at line 30 of file prime_sieve.c.

## 15.30 prime_sieve.c

Go to the documentation of this file.

```
00001
00012
00013 #include <iZ_api.h>
00014
00015 // ==================================================================
00016 // * Internal helper macros:
00017 // ==================================================================
00018
00020 #define N_LIMIT (1000000000000ULL)
00021
00023 #define ASSERT_LIMIT(n) assert((n) <= N_LIMIT && "Input must be in the range <= 10^12.")
00024
00026 #define Pi(n) ((n / log(n)))
00027
00028 // a small array of the first few primes (<= 100) to use as base for sieving, and to avoid small edge cases in the algorithms.
00029 static const uint64_t base_primes[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89,
      97};
00030 static const int base_primes_count = sizeof(base_primes) / sizeof(base_primes[0]);
00031
00032 static void push_small_primes(UI64_ARRAY *primes, uint64_t n)
```

```
{
    for (int i = 0; i < base_primes_count && base_primes[i] <= n; i++)
        ui64_push(primes, base_primes[i]);
}

// ============================================================
// * Classic Sieve Algorithms
// ============================================================

static void process_N_bitmap(UI64_ARRAY *primes, BITMAP *sieve_bitmap, uint64_t n)
{
    ui64_push(primes, 2); // Add 2 to primes to focus on odd candidates
    uint64_t n_sqrt = sqrt(n);

    // Sieve logic:
    // Iterate through odd numbers i starting from 3,
    // check if i is marked as prime in the bitmap,
    // if so, collect i as a prime and mark its odd multiples if i <= sqrt(n).
    for (uint64_t i = 3; i <= n; i += 2)
    {
        if (bitmap_get_bit(sieve_bitmap, i))
        {
            ui64_push(primes, i);
            if (i <= n_sqrt)
                bitmap_clear_steps_simd(sieve_bitmap, 2 * i, i * i, n + 1);
        }
    }
}

UI64_ARRAY *SoE(uint64_t n)
{
    ASSERT_LIMIT(n); // Validate input limit

    // Initialize the primes object with an estimated capacity
    UI64_ARRAY *primes = ui64_init(Pi(n) * 1.4); // 40% over-estimation to avoid reallocs
    assert(primes && "Memory allocation failed for primes array.");

    if (n <= 100)
    {
        push_small_primes(primes, n);
        ui64_resize_to_fit(primes); // Trim excess memory in primes array
        return primes;
    }

    // Create a bitmap to mark prime numbers
    BITMAP *sieve = bitmap_init(n + 1, 1);
    if (!sieve)
    {
        ui64_free(&primes);
        return NULL;
    }

    // Sieve logic
    process_N_bitmap(primes, sieve, n);

    // cleanup and finalize
    bitmap_free(&sieve);        // free bitmap
    ui64_resize_to_fit(primes); // Trim excess memory in primes array

    return primes;
}

UI64_ARRAY *SSoE(uint64_t n)
{
    ASSERT_LIMIT(n); // Validate input limit

    if (n <= 10000)
    {
        return SoE(n); // For small n, the overhead of segmentation is not worth it
    }

    // Initialize UI64_ARRAY with an estimated capacity
    UI64_ARRAY *primes = ui64_init(Pi(n) * 1.4); // 40% over-estimation to avoid reallocs
    assert(primes && "Memory allocation failed for primes array.");

    // Define the segment size; can be tuned based on memory constraints
    uint64_t segment_size = (uint64_t)sqrt(n);

    // * Step 1: Sieve small primes up to sqrt(n) using the traditional sieve
    BITMAP *sieve = bitmap_init(segment_size + 8, 1);
    if (!sieve)
    {
        ui64_free(&primes);
        return NULL;
    }

    // process first segment to collect root primes
```

```
    process_N_bitmap(primes, sieve, segment_size);

    // * Step 2: Segmented sieve
    uint64_t low = segment_size + 1;
    uint64_t high = low + segment_size - 1;

    // Iterate over segments
    while (low <= n)
    {
        bitmap_set_all(sieve); // Reset segment bitmap
        uint64_t root_limit = sqrt(high);

        // Sieve the current segment using primes <= sqrt(high)
        for (int i = 1; i < primes->count; i++) // skip 2
        {
            uint64_t p = primes->array[i];
            if (p > root_limit)
                break;

            // Find the minimum number in [low, high] that is a multiple of p
            uint64_t start = (low / p) * p;
            start += (start < low) ? p : 0;    // ensure start >= low
            start += (start % 2 == 0) ? p : 0; // ensure start is odd
            start = MAX(p * p, start);         // start from p^2 or higher

            // Mark multiples of p within the segment
            bitmap_clear_steps_simd(sieve, 2 * p, start - low, high - low + 1);
        }

        // Collect primes from the current segment
        uint64_t i = (low % 2 == 0) ? low + 1 : low;
        for (; i <= high; i += 2) // skip even numbers
        {
            if (bitmap_get_bit(sieve, i - low))
                ui64_push(primes, i);
        }

        // Move to the next segment
        low += segment_size;
        high += segment_size;
        if (high > n)
            high = n;
    }

    // * Step 3: Cleanup and finalize
    bitmap_free(&sieve);        // free bitmap
    ui64_resize_to_fit(primes); // Trim excess memory in primes array

    return primes;
}

UI64_ARRAY *SoS(uint64_t n)
{
    ASSERT_LIMIT(n); // Validate input limit

    // Calculate k as the odd numbers up to n.
    uint64_t k = n / 2 + 1;

    // Initialize the primes object with an estimated capacity
    UI64_ARRAY *primes = ui64_init(Pi(n) * 1.4); // 40% over-estimation to avoid reallocs
    assert(primes && "Memory allocation failed for primes array.");

    if (n <= 100)
    {
        push_small_primes(primes, n);
        ui64_resize_to_fit(primes); // Trim excess memory in primes array
        return primes;
    }

    ui64_push(primes, 2);

    // Create a bitmap with size k+1.
    BITMAP *sieve = bitmap_init(k + 8, 1);
    if (sieve == NULL)
    {
        ui64_free(&primes);
        return NULL;
    }

    uint64_t n_sqrt = sqrt(n) + 1;

    // iterate through odd numbers
    for (uint64_t i = 1; i < k; ++i)
    {
        if (bitmap_get_bit(sieve, i))
        {
            uint64_t p = 2 * i + 1;
```

```
            ui64_push(primes, p);
            if (p < n_sqrt)
            {
                // First composite mark xp in the bitmap is given by:
                // xp = 2 * i^2 + 2 * i = p * i + i, corresponding to p^2 in the odd set
                uint64_t xp = p * i + i;
                // Mark composites of p additively in the bitmap using step size p
                bitmap_clear_steps_simd(sieve, p, xp, k);
            }
        }
    }

    // Cleanup
    bitmap_free(&sieve);

    // Guard against overshoot from index-space upper bounds.
    while (primes->count > 0 && primes->array[primes->count - 1] > n)
        ui64_pop(primes);

    // Resize primes array to fit the exact number of primes found
    ui64_resize_to_fit(primes);

    return primes;
}

UI64_ARRAY *SSoS(uint64_t n)
{
    ASSERT_LIMIT(n); // Validate input limit

    // if n < 10000, return SoS(n), doesn't worth segmenting
    if (n < 10000)
        return SoS(n);

    // Initialize the primes object with an estimated capacity
    UI64_ARRAY *primes = ui64_init(Pi(n) * 1.4); // 40% over-estimation to avoid reallocs
    assert(primes && "Memory allocation failed for primes array.");

    // Calculate k = n/2 + 1 as the odd numbers up to n.
    uint64_t x_n = n / 2 + 1;
    // Compute vx wheel size that fits in L2 cache
    int vx = 3 * compute_l2_vx(n);
    BITMAP *base = bitmap_init(vx + 8, 1);
    BITMAP *sieve = bitmap_init(vx + 8, 1);
    if (sieve == NULL || base == NULL)
    {
        ui64_free(&primes);
        return NULL;
    }
    get_root_primes(primes, 2 * vx);

    // construct base bitmap for vx
    int k = 1; // flag to track the pre-sieved primes
    while (vx % base_primes[k] == 0)
    {
        int p = base_primes[k++];
        int xp = p / 2;             // index of p in the odd set
        bitmap_clear_bit(base, xp); // mark p as prime in base bitmap
        bitmap_clear_steps_simd(base, p, p * xp + xp, vx + 1);
    }

    int y_limit = x_n / vx; // number of full segments to process
    uint64_t yvx = vx;

    for (int y = 1; y <= y_limit; y++)
    {
        memcpy(sieve->data, base->data, sieve->byte_size); // reset sieve bitmap to base for the new segment

        int x_limit = (y < y_limit) ? vx : (int)(x_n % (uint64_t)vx); // local x limit adjusted for last segment
        uint64_t root_limit = sqrt(2 * (yvx + x_limit)) + 1;          // local root limit for current segment

        // Mark composites of root primes in the current segment using root primes,
        // starting from the first non-pre-sieved prime
        for (int i = k; i < primes->count; i++)
        {
            uint64_t p = primes->array[i];
            if (p > root_limit)
                break;

            uint64_t xp = p / 2;
            int x0 = p - (vx * y - xp) % p;
            bitmap_clear_steps_simd(sieve, p, x0, x_limit + 1);
        }

        for (int x = k / 2; x <= x_limit; x++)
        {
            if (bitmap_get_bit(sieve, x))
            {
```

```
            uint64_t p = 2 * (yvx + x) + 1;
            ui64_push(primes, p);
        }
    }
    yvx += vx;
}

// Cleanup
bitmap_free(&base);
bitmap_free(&sieve);

// Guard against overshoot from segment tail generation.
while (primes->count > 0 && primes->array[primes->count - 1] > n)
    ui64_pop(primes);

// Resize primes array to fit the exact number of primes found
ui64_resize_to_fit(primes);

return primes;
}

UI64_ARRAY *SoEu(uint64_t n)
{
    ASSERT_LIMIT(n); // Validate input limit

    // initialization
    UI64_ARRAY *primes = ui64_init(Pi(n) * 1.4); // 40% over-estimation to avoid reallocs
    assert(primes && "Memory allocation failed for primes array.");

    if (n <= 100)
    {
        push_small_primes(primes, n);
        return primes;
    }

    BITMAP *sieve = bitmap_init(n + 1, 1);
    if (sieve == NULL)
    {
        ui64_free(&primes);
        return NULL;
    }

    // starting the prime list with 2 to skip reading even numbers
    ui64_push(primes, 2);

    // sieve logic: iterate through odd numbers, marking composites by clearing multiples of primes
    for (uint64_t i = 3; i <= n; i += 2)
    {
        if (bitmap_get_bit(sieve, i))
            ui64_push(primes, i);

        // Mark multiples of the current prime
        for (int j = 1; j < primes->count; ++j)
        {
            uint64_t p = primes->array[j];

            if (p * i > n)
                break;

            bitmap_clear_bit(sieve, p * i);

            if (i % p == 0)
                break;
        }
    }

    // cleanup
    bitmap_free(&sieve);

    // Resize primes array to fit the exact number of primes found
    ui64_resize_to_fit(primes);

    return primes;
}

UI64_ARRAY *SoA(uint64_t n)
{
    ASSERT_LIMIT(n); // Validate input limit

    UI64_ARRAY *primes = ui64_init(Pi(n) * 1.4); // 40% over-estimation to avoid reallocs
    assert(primes && "Memory allocation failed for primes array.");

    if (n <= 100)
    {
        push_small_primes(primes, n);
        return primes;
    }
```

```c

    // Create a bitmap to mark potential primes
    BITMAP *sieve = bitmap_init(n + 1, 0);
    if (sieve == NULL)
    {
        ui64_free(&primes);
        return NULL;
    }

    // Add 2 and 3 to primes
    ui64_push(primes, 2);
    ui64_push(primes, 3);

    // 1. Mark potential primes in the bitmap using the Atkin conditions
    // Condition 1: for all 4x^2 + y^2 <= n
    for (uint64_t x = 1; 4 * x * x < n; x++)
    {
        uint64_t a = 4 * x * x;
        for (uint64_t y = 1; a + y * y <= n; ++y)
        {
            uint64_t b = a + y * y;
            // if 4x^2 + y^2   1 or 5 (mod 12), flip the bit
            if (b % 12 == 1 || b % 12 == 5)
                bitmap_flip_bit(sieve, b);
        }
    }

    // Condition 2: for all 3x^2 + y^2 <= n
    for (uint64_t x = 1; 3 * x * x < n; x++)
    {
        uint64_t a = 3 * x * x;
        for (uint64_t y = 1; a + y * y <= n; ++y)
        {
            uint64_t b = a + y * y;
            // if 3x^2 + y^2   7 (mod 12), flip the bit
            if (b % 12 == 7)
                bitmap_flip_bit(sieve, b);
        }
    }

    // Condition 3: for all 3x^2 - y^2 <= n and x > y
    for (uint64_t x = 1; 2 * x * x < n; x++) // Approximation for loop bound
    {
        uint64_t a = 3 * x * x;
        for (uint64_t y = x - 1; y > 0; y--)
        {
            uint64_t b = a - y * y;
            if (b > n)
                break; // b increases as y decreases

            // if 3x^2 - y^2   11 (mod 12), flip the bit
            if (b % 12 == 11)
                bitmap_flip_bit(sieve, b);
        }
    }

    // 2. Remove remaining composites by sieving out multiples of squares of root primes
    uint64_t n_sqrt = sqrt(n);
    for (uint64_t p = 5; p <= n_sqrt; p += 2)
    {
        if (bitmap_get_bit(sieve, p))
        {
            // Mark odd multiples of p^2 as non-prime
            // starting at p^2 with step size 2p^2
            bitmap_clear_steps_simd(sieve, 2 * p * p, p * p, n + 1);
        }
    }

    // 3. Collect primes from the bitmap
    for (uint64_t p = 5; p <= n; p += 2)
    {
        if (bitmap_get_bit(sieve, p))
            ui64_push(primes, p);
    }

    // cleanup
    bitmap_free(&sieve);

    // Resize primes array to fit the exact number of primes found
    ui64_resize_to_fit(primes);

    return primes;
}

// ============================================================
// * Sieve-iZ Algorithms
// ============================================================
```

```

UI64_ARRAY *SiZ(uint64_t n)
{
    ASSERT_LIMIT(n); // Validate input limit

    // Initialize primes object with enough initial estimation
    UI64_ARRAY *primes = ui64_init(Pi(n) * 1.4); // 40% over-estimation to avoid reallocs
    assert(primes && "Memory allocation failed for primes array.");

    if (n <= 100)
    {
        push_small_primes(primes, n);
        ui64_resize_to_fit(primes); // Trim excess memory in primes array
        return primes;
    }

    // Add 2, 3 to primes
    ui64_push(primes, 2);
    ui64_push(primes, 3);

    // Calculate x_n, max x value in iZ space for given n
    uint64_t x_n = n / 6 + 1;

    // Create bitmap X-Arrays x5, x7, each of size x_n + 1 bits
    BITMAP *x5 = bitmap_init(x_n + 1, 1);
    BITMAP *x7 = bitmap_init(x_n + 1, 1);

    // Memory allocation failed, check logs
    if (!x5 || !x7)
    {
        ui64_free(&primes);
        return NULL;
    }

    // Sieve logic
    process_iZ_bitmaps(primes, x5, x7, x_n);

    // Cleanup: free memory of x5, x7
    bitmap_free(&x5);
    bitmap_free(&x7);

    // Guard against overshoot near the final iZ lane.
    while (primes->count > 0 && primes->array[primes->count - 1] > n)
        ui64_pop(primes);

    // Trim unused memory in primes object
    ui64_resize_to_fit(primes);

    return primes;
}

UI64_ARRAY *SiZm(uint64_t n)
{
    ASSERT_LIMIT(n); // Validate input limit

    // if n < 10000, return SiZ(n), doesn't worth segmenting
    if (n < 10000)
        return SiZ(n);

    // * 1. Initialization:
    // Initialize primes array with enough capacity to avoid reallocs
    UI64_ARRAY *primes = ui64_init(Pi(n) * 1.4); // 40% over-estimation to avoid reallocs
    assert(primes && "Memory allocation failed for primes array in SiZm.");

    // Compute vx wheel size that fits in L2 cache
    int vx = compute_l2_vx(n);

    // Initialize and construct base bitmaps for iZm/vx
    BITMAP *base_x5 = bitmap_init(vx + 8, 1);
    BITMAP *base_x7 = bitmap_init(vx + 8, 1);
    if (!base_x5 || !base_x7)
    {
        ui64_free(&primes);
        return NULL;
    }
    iZm_construct_vx_base(vx, base_x5, base_x7);

    // Add the pre-sieved k primes to primes array
    int k = 0;
    while ((6 * vx) % base_primes[k] == 0)
        ui64_push(primes, base_primes[k++]);

    // * 2. Process first segment (y = 0) to collect root primes:
    uint64_t x_n = n / 6 + 1; // max x value up to n
    // uint64_t root_limit = sqrt(6 * x_limit) + 1;
    // Initialize active sieve bitmaps from base
    BITMAP *x5 = bitmap_clone(base_x5);
```

```
    BITMAP *x7 = bitmap_clone(base_x7);
    if (!x5 || !x7)
    {
        ui64_free(&primes);
        bitmap_free(&base_x5);
        bitmap_free(&base_x7);
        return NULL;
    }
    process_iZ_bitmaps(primes, x5, x7, vx + 1);

    // * 3. Process remaining segments (y >= 1) to collect primes:
    int y_limit = x_n / vx; // number of full segments to process
    uint64_t yvx = vx;      // current base value (y * vx)
    for (int y = 1; y <= y_limit; y++)
    {
        // * a. Reset active bitmaps to base state
        memcpy(x5->data, base_x5->data, x5->byte_size);
        memcpy(x7->data, base_x7->data, x7->byte_size);

        int x_limit = (y < y_limit) ? vx : (int)(x_n % (uint64_t)vx); // local x limit adjusted for last segment
        uint64_t root_limit = sqrt(6 * (yvx + x_limit)) + 1;          // local root limit for current segment

        // * b. Mark composites of root primes in current segment
        for (int i = k; i < primes->count; i++)
        {
            uint64_t p = primes->array[i];
            if (p > root_limit)
                break;

            // mark composites of p in current segment
            bitmap_clear_steps_simd(x5, p, iZm_solve_for_x0(-1, p, vx, y), x_limit);
            bitmap_clear_steps_simd(x7, p, iZm_solve_for_x0(1, p, vx, y), x_limit);
        }

        // * c. Collect unmarked indices as primes in current segment
        for (int x = 2; x <= x_limit; x++)
        {
            if (bitmap_get_bit(x5, x)) // i.e. iZ- prime
                ui64_push(primes, iZ(yvx + x, -1));

            if (bitmap_get_bit(x7, x)) // i.e. iZ+ prime
                ui64_push(primes, iZ(yvx + x, 1));
        }

        yvx += vx; // advance yvx for next segment
    }

    // * 4. Clean up and finalize
    bitmap_free(&x5);
    bitmap_free(&x7);
    bitmap_free(&base_x5);
    bitmap_free(&base_x7);

    // Guard against overshoot near the final iZ lane.
    while (primes->count > 0 && primes->array[primes->count - 1] > n)
        ui64_pop(primes);

    ui64_resize_to_fit(primes); // Trim excess memory in primes array
    return primes;
}

UI64_ARRAY *SiZm_vy(uint64_t n)
{
    ASSERT_LIMIT(n); // Validate input limit

    // if n is less than 10000, return SiZ(n)
    if (n < 10000)
        return SiZ(n);

    // * 1. Initialization:
    // Initialize primes array with enough capacity to avoid reallocs
    UI64_ARRAY *primes = ui64_init(Pi(n) * 1.4);
    assert(primes && "Memory allocation failed for primes array in SiZm.");

    uint64_t x_n = n / 6 + 1; // iZ limit for n
    uint64_t root_limit = sqrt(n) + 1;

    get_root_primes(primes, root_limit);
    int root_count = primes->count;

    int k = 4; // pointing at 11 in root_primes
    int vx = 35;
    if (n >= pow(10, 9))
    {
        vx *= 11;
        k++;
    }
```

```
    if (n >= pow(10, 11))
    {
        vx *= 13;
        k++;
    }

    int vy = x_n / vx;

    BITMAP *sieve = bitmap_init(vy + 8, 1);

    // * 2. Sieve logic: Process iZm's columns as segments
    for (int x = 2; x <= vx; x++)
    {
        // Handle iZ- case:
        // crucial, ensure iZ(x, -1) is coprime to vx before sieving
        if (gcd(iZ(x, -1), vx) == 1)
        {
            // * a. reset sieve bitmap for new segment
            bitmap_set_all(sieve); // set all bits

            // * b. mark composites of root primes in sieve
            for (int i = k; i < root_count; i++)
            {
                uint64_t p = primes->array[i];
                int64_t y_0 = iZm_solve_for_y0(-1, p, vx, x);
                bitmap_clear_steps_simd(sieve, p, y_0, vy);
            }

            // * c. collect primes from sieve up to y = vy-1
            for (int y = 0; y < vy; y++)
            {
                if (bitmap_get_bit(sieve, y))
                    ui64_push(primes, iZ(y * vx + x, -1));
            }
            // handle partial last row where y = vy (check if p < n before pushing)
            if (bitmap_get_bit(sieve, vy))
            {
                uint64_t p = iZ(vy * vx + x, -1);
                if (p < n)
                    ui64_push(primes, p);
            }
        }

        // Handle iZ+ case similarly:
        if (gcd(iZ(x, 1), vx) == 1)
        {
            bitmap_set_all(sieve); // reset sieve bitmap for new segment
            for (int i = k; i < root_count; i++)
            {
                uint64_t p = primes->array[i];
                int64_t y_0 = iZm_solve_for_y0(1, p, vx, x);
                bitmap_clear_steps_simd(sieve, p, y_0, vy);
            }

            for (int y = 0; y < vy; y++)
            {
                if (bitmap_get_bit(sieve, y))
                    ui64_push(primes, iZ(y * vx + x, 1));
            }
            if (bitmap_get_bit(sieve, vy))
            {
                uint64_t p = iZ(vy * vx + x, 1);
                if (p < n)
                    ui64_push(primes, p);
            }
        }
    }

    // * 3. Clean up and finalize
    bitmap_free(&sieve);
    ui64_resize_to_fit(primes); // Trim excess memory in primes array

    primes->ordered = 0; // Mark the array as unordered
    return primes;
}
```

## 15.31 bitmap.c File Reference

Implementation of the bitmap module for efficient bit array operations.

#include <bitmap.h>

Functions

- BITMAP * bitmap_init (size_t size, int set_bits)

  Creates and initializes a new bitmap with the specified number of bits.
- void bitmap_free (BITMAP **bitmap)

  Frees all memory associated with a bitmap and nullifies the pointer.
- void bitmap_set_bit (BITMAP *bitmap, size_t idx)

  Sets a specific bit to 1.
- int bitmap_get_bit (BITMAP *bitmap, size_t idx)

  Gets the value of a specific bit.
- void bitmap_flip_bit (BITMAP *bitmap, size_t idx)

  Flips the value of a specific bit.
- void bitmap_clear_bit (BITMAP *bitmap, size_t idx)

  Clears (sets to 0) a specific bit.
- void bitmap_set_all (BITMAP *bitmap)

  Sets all bits in the array to 1.
- void bitmap_clear_all (BITMAP *bitmap)

  Clears all bits in the bitmap (sets them to 0).
- void bitmap_clear_steps (BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)

  Clears bits at regular intervals, typically used in sieve algorithms.
- void bitmap_clear_steps_simd (BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)

  SIMD-optimized version of bitmap_clear_steps.
- BITMAP * bitmap_clone (BITMAP *src)

  Creates a deep copy (clone) of an existing bitmap.
- void bitmap_compute_hash (BITMAP *bitmap)

  Generates a SHA-256 hash for the bitmap->data.
- int bitmap_validate_hash (BITMAP *bitmap)

  Validates the SHA-256 hash stored in bitmap->sha256.
- int bitmap_fwrite (BITMAP *bitmap, FILE *file)

  Writes the bitmap to a binary file.
- BITMAP * bitmap_fread (FILE *file)

  Reads a bitmap from a binary file.

## 15.31.1 Detailed Description

Implementation of the bitmap module for efficient bit array operations.

This file provides the complete implementation of bitmap functions including memory management, bit manipulation, data integrity verification via SHA-256 hashing, and binary file I/O with automatic hash validation.

### 15.31.1.1 Implementation Notes

- All functions include NULL pointer and bounds checking
- File I/O operations verify all read/write operations succeed
- Overflow protection is implemented in loop-based operations
- Memory is managed with proper cleanup on errors
- SHA-256 hashing uses OpenSSL library

#### 15.31.1.2 Performance Characteristics

- bitmap_init(): O(n/8) where n is number of bits
- Individual bit ops: O(1)
- Bulk operations: O(n/8)
- Hash computation: O(n/8)
- File I/O: O(n/8)

Author

iZprime.com

Date

October 2025

Version

1.0

See also

bitmap.h for API documentation

test/test_bitmap.c for test suite

Definition in file bitmap.c.

## 15.32 bitmap.c

Go to the documentation of this file.

```
00001
00030
00031 #include <bitmap.h>
00032
00033 #ifdef __aarch64__
00034 #include <arm_neon.h>
00035 #elif defined(__x86_64__) || defined(_M_X64)
00036 #include <immintrin.h>
00037 #endif
00038
00057 BITMAP *bitmap_init(size_t size, int set_bits)
00058 {
00059     assert(size > 0 && "Bitmap size must be positive");
00060
00061     BITMAP *bitmap = (BITMAP *)malloc(sizeof(BITMAP));
00062     if (bitmap == NULL)
00063     {
00064         log_error("Memory allocation failed for BITMAP struct");
00065         return NULL;
00066     }
00067
00068     bitmap->size = size;
00069     bitmap->byte_size = (size + 7) / 8;
00070     bitmap->data = (unsigned char *)malloc(bitmap->byte_size);
00071     if (bitmap->data == NULL)
00072     {
00073         free(bitmap);
00074         log_error("bitmap_create: Memory allocation failed for BITMAP");
```

```
        return NULL;
    }

    if (set_bits)
        memset(bitmap->data, 0xFF, bitmap->byte_size);
    else
        memset(bitmap->data, 0x00, bitmap->byte_size);

    memset(bitmap->sha256, 0, SHA256_DIGEST_LENGTH); // Initialize SHA-256 to zero

    return bitmap;
}

void bitmap_free(BITMAP **bitmap)
{
    // Safe guard against double free and NULL pointer
    if (bitmap && *bitmap)
    {
        if ((*bitmap)->data)
        {
            free((*bitmap)->data);
            (*bitmap)->data = NULL;
        }

        free(*bitmap);
        *bitmap = NULL;
    }
}

void bitmap_set_bit(BITMAP *bitmap, size_t idx)
{
    bitmap->data[idx / 8] |= (1 « (idx % 8));
}

int bitmap_get_bit(BITMAP *bitmap, size_t idx)
{
    return (bitmap->data[idx / 8] & (1 « (idx % 8))) != 0;
}

void bitmap_flip_bit(BITMAP *bitmap, size_t idx)
{
    bitmap->data[idx / 8] ^= (1 « (idx % 8));
}

void bitmap_clear_bit(BITMAP *bitmap, size_t idx)
{
    // Clear the bit by ANDing with the negation of the bit mask
    bitmap->data[idx / 8] &= ~(1 « (idx % 8));
}

void bitmap_set_all(BITMAP *bitmap)
{
    memset(bitmap->data, 0xFF, (bitmap->size + 7) / 8);
}

void bitmap_clear_all(BITMAP *bitmap)
{
    memset(bitmap->data, 0x00, (bitmap->size + 7) / 8);
}

void bitmap_clear_steps(BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)
{
    assert(step > 0 && "Step must be positive in bitmap_clear_steps.");
    limit = MIN(limit, bitmap->size - 1);
    if (start_idx > limit) // safe guard against invalid range
        return;

    for (uint64_t idx = start_idx; idx <= limit; idx += step)
    {
        bitmap->data[idx / 8] &= ~(1 « (idx % 8));
    }
}

void bitmap_clear_steps_simd(BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)
{
    assert(step > 0 && "Step must be positive in bitmap_clear_steps_simd.");
    limit = MIN(limit, bitmap->size - 1);
    if (start_idx > limit)
        return;

#ifdef __aarch64__
    // NEON implementation for ARM64
    uint64_t idx = start_idx;

    // Process 4 steps at a time
    // Condition: idx + 3*step <= limit

```

```
    if (limit >= 3 * step && idx <= limit - 3 * step)
    {
        uint64x2_t v_step2 = vdupq_n_u64(2 * step);
        uint64x2_t v_step4 = vdupq_n_u64(4 * step);

        // Initialize indices: [idx, idx+step] and [idx+2step, idx+3step]
        uint64x2_t v_idx_01 = vdupq_n_u64(idx);
        v_idx_01 = vsetq_lane_u64(idx + step, v_idx_01, 1);

        uint64x2_t v_idx_23 = vaddq_u64(v_idx_01, v_step2);

        while (idx <= limit - 3 * step)
        {
            // Extract indices and perform operations
            // Note: NEON doesn't have efficient scatter for bytes, so we extract and do scalar stores

            uint64_t i0 = vgetq_lane_u64(v_idx_01, 0);
            uint64_t i1 = vgetq_lane_u64(v_idx_01, 1);
            uint64_t i2 = vgetq_lane_u64(v_idx_23, 0);
            uint64_t i3 = vgetq_lane_u64(v_idx_23, 1);

            bitmap->data[i0 / 8] &= ~(1 « (i0 % 8));
            bitmap->data[i1 / 8] &= ~(1 « (i1 % 8));
            bitmap->data[i2 / 8] &= ~(1 « (i2 % 8));
            bitmap->data[i3 / 8] &= ~(1 « (i3 % 8));

            // Advance indices
            v_idx_01 = vaddq_u64(v_idx_01, v_step4);
            v_idx_23 = vaddq_u64(v_idx_23, v_step4);

            idx += 4 * step;
        }
    }

    // Handle remaining steps
    for (; idx <= limit; idx += step)
    {
        bitmap->data[idx / 8] &= ~(1 « (idx % 8));
    }

#elif defined(__AVX2__)
    // AVX2 implementation for x86_64
    uint64_t idx = start_idx;

    // Process 4 steps at a time
    if (limit >= 3 * step && idx <= limit - 3 * step)
    {
        __m256i v_step4 = _mm256_set1_epi64x(4 * step);

        // Initialize indices: [idx, idx+step, idx+2step, idx+3step]
        // Note: _mm256_set_epi64x takes arguments in reverse order (e3, e2, e1, e0)
        __m256i v_idx = _mm256_set_epi64x(idx + 3 * step, idx + 2 * step, idx + step, idx);

        while (idx <= limit - 3 * step)
        {
            // Extract indices
            uint64_t i0 = _mm256_extract_epi64(v_idx, 0);
            uint64_t i1 = _mm256_extract_epi64(v_idx, 1);
            uint64_t i2 = _mm256_extract_epi64(v_idx, 2);
            uint64_t i3 = _mm256_extract_epi64(v_idx, 3);

            bitmap->data[i0 / 8] &= ~(1 « (i0 % 8));
            bitmap->data[i1 / 8] &= ~(1 « (i1 % 8));
            bitmap->data[i2 / 8] &= ~(1 « (i2 % 8));
            bitmap->data[i3 / 8] &= ~(1 « (i3 % 8));

            // Advance indices
            v_idx = _mm256_add_epi64(v_idx, v_step4);
            idx += 4 * step;
        }
    }

    // Handle remaining steps
    for (; idx <= limit; idx += step)
    {
        bitmap->data[idx / 8] &= ~(1 « (idx % 8));
    }

#elif defined(__SSE2__)
    // SSE2 implementation for x86_64
    uint64_t idx = start_idx;

    // Process 2 steps at a time
    if (limit >= step && idx <= limit - step)
    {
        __m128i v_step2 = _mm_set1_epi64x(2 * step);
        // Note: _mm_set_epi64x takes arguments in reverse order (e1, e0)
```

```
        __m128i v_idx = _mm_set_epi64x(idx + step, idx);

        while (idx <= limit - step)
        {
            // Extract indices
            // _mm_cvtsi128_si64 extracts the lower 64 bits (element 0)
            uint64_t i0 = _mm_cvtsi128_si64(v_idx);
            // Unpack high to low to extract element 1
            uint64_t i1 = _mm_cvtsi128_si64(_mm_unpackhi_epi64(v_idx, v_idx));

            bitmap->data[i0 / 8] &= ~(1 « (i0 % 8));
            bitmap->data[i1 / 8] &= ~(1 « (i1 % 8));

            // Advance indices
            v_idx = _mm_add_epi64(v_idx, v_step2);
            idx += 2 * step;
        }
    }

    // Handle remaining steps
    for (; idx <= limit; idx += step)
    {
        bitmap->data[idx / 8] &= ~(1 « (idx % 8));
    }

#else
    // Fallback to scalar implementation
    for (uint64_t idx = start_idx; idx <= limit; idx += step)
    {
        bitmap->data[idx / 8] &= ~(1 « (idx % 8));
    }
#endif
}

BITMAP *bitmap_clone(BITMAP *src)
{
    assert(src && src->data && "Invalid source bitmap passed to bitmap_clone.");

    BITMAP *dest = bitmap_init(src->size, 0);
    memcpy(dest->data, src->data, src->byte_size);
    memcpy(dest->sha256, src->sha256, SHA256_DIGEST_LENGTH);

    return dest;
}

void bitmap_compute_hash(BITMAP *bitmap)
{
    assert(bitmap && bitmap->data && "Invalid bitmap passed to bitmap_compute_hash.");

    // Generate SHA-256 hash and store it in the struct
    SHA256((unsigned char *)bitmap->data, bitmap->byte_size, bitmap->sha256);
}

int bitmap_validate_hash(BITMAP *bitmap)
{
    assert(bitmap && bitmap->data && "Invalid bitmap");

    unsigned char correct_hash[SHA256_DIGEST_LENGTH]; // Buffer to hold the computed hash

    // Generate SHA-256 hash and store it in correct_hash
    SHA256((unsigned char *)bitmap->data, bitmap->byte_size, correct_hash);

    // Compare actual_hash with the stored hash in bitmap->sha256
    if (memcmp(correct_hash, bitmap->sha256, SHA256_DIGEST_LENGTH) == 0)
    {
        return 1; // SHA-256 match
    }
    else
    {
        log_error("SHA-256 checksum validation failed.");
        return 0; // SHA-256 mismatch
    }
}

int bitmap_fwrite(BITMAP *bitmap, FILE *file)
{
    assert(bitmap && bitmap->data && "Invalid bitmap passed to bitmap_fwrite.");
    assert(file && "File pointer is NULL in bitmap_fwrite.");

    // Write size
    if (fwrite(&bitmap->size, sizeof(size_t), 1, file) != 1)
    {
        log_error("Failed to write bitmap size to file.");
        return 0;
    }

    // Write data
```

```
    if (fwrite(bitmap->data, sizeof(unsigned char), bitmap->byte_size, file) != bitmap->byte_size)
    {
        log_error("Failed to write bitmap data to file.");
        return 0;
    }

    // Compute SHA-256 hash if not already computed
    // Check if hash is all zeros by examining all bytes
    int hash_is_zero = 1;
    for (size_t i = 0; i < SHA256_DIGEST_LENGTH; i++)
    {
        if (bitmap->sha256[i] != 0)
        {
            hash_is_zero = 0;
            break;
        }
    }

    if (hash_is_zero)
    {
        bitmap_compute_hash(bitmap);
    }

    // Write SHA-256 hash
    if (fwrite(bitmap->sha256, 1, SHA256_DIGEST_LENGTH, file) != SHA256_DIGEST_LENGTH)
    {
        log_error("Failed to write SHA-256 hash to file.");
        return 0;
    }

    return 1; // Success
}

BITMAP *bitmap_fread(FILE *file)
{
    assert(file && "File pointer is NULL in bitmap_fread.");

    // Read size
    size_t size;
    if (fread(&size, sizeof(size_t), 1, file) != 1)
    {
        log_error("Failed to read bitmap size from file.");
        return NULL;
    }

    BITMAP *bitmap = bitmap_init(size, 0);
    if (bitmap == NULL)
    {
        log_error("Failed to initialize bitmap.");
        return NULL;
    }

    // Read data directly into the already-allocated buffer
    if (fread(bitmap->data, sizeof(unsigned char), bitmap->byte_size, file) != bitmap->byte_size)
    {
        log_error("Failed to read complete bitmap data from file.");
        bitmap_free(&bitmap);
        return NULL;
    }

    // Read SHA-256 hash
    if (fread(bitmap->sha256, 1, SHA256_DIGEST_LENGTH, file) != SHA256_DIGEST_LENGTH)
    {
        log_error("Failed to read SHA-256 hash from file.");
        bitmap_free(&bitmap);
        return NULL;
    }

    if (!bitmap_validate_hash(bitmap))
    {
        log_error("SHA-256 hash validation failed.");
        bitmap_free(&bitmap);
        return NULL;
    }

    return bitmap;
}
```

## 15.33 int_arrays.c File Reference

Implementation of dynamic integer array module for 16-bit, 32-bit, and 64-bit unsigned integers.

#include <int_arrays.h>
#include "templates/int_array_impl.inc"

### 15.33.1 Detailed Description

Implementation of dynamic integer array module for 16-bit, 32-bit, and 64-bit unsigned integers.

This file provides the complete implementation of UI16_ARRAY, UI32_ARRAY, and UI64_ARRAY functions including memory management, automatic capacity growth, data integrity verification via SHA-256 hashing, and binary file I/O with automatic hash validation.

#### 15.33.1.1 Implementation Notes

- All functions include NULL pointer checking
- Automatic capacity doubling (2x growth) ensures amortized O(1) append operations
- File I/O operations automatically compute and validate SHA-256 hashes
- Memory is managed with proper cleanup on errors
- SHA-256 hashing uses OpenSSL library

#### 15.33.1.2 Performance Characteristics

- init(): O(1)
- push() (amortized): O(1)
- push() (worst case): O(n) during resize
- Hash operations: O(n)
- File I/O: O(n)

Author

iZprime.com

Date

October 2025

Version

1.0

See also

int_arrays.h for API documentation

Definition in file int_arrays.c.

## 15.34 int_arrays.c

Go to the documentation of this file.

```
00029
00030 #include <int_arrays.h>
00031
00032 //
      ==========================================================================
00033 // UI16_ARRAY IMPLEMENTATION
00034 //
      ==========================================================================
00036 #define TEMPLATE_TYPE uint16_t
00037 #define TEMPLATE_STRUCT UI16_ARRAY
00038 #define TEMPLATE_FUNC(name) ui16_##name
00039 #define TEMPLATE_NAME_STR "UI16_ARRAY"
00041 #include "templates/int_array_impl.inc"
00043 #undef TEMPLATE_TYPE
00044 #undef TEMPLATE_STRUCT
00045 #undef TEMPLATE_FUNC
00046 #undef TEMPLATE_NAME_STR
00048
00049 //
      ==========================================================================
00050 // UI32_ARRAY IMPLEMENTATION
00051 //
      ==========================================================================
00053 #define TEMPLATE_TYPE uint32_t
00054 #define TEMPLATE_STRUCT UI32_ARRAY
00055 #define TEMPLATE_FUNC(name) ui32_##name
00056 #define TEMPLATE_NAME_STR "UI32_ARRAY"
00058 #include "templates/int_array_impl.inc"
00060 #undef TEMPLATE_TYPE
00061 #undef TEMPLATE_STRUCT
00062 #undef TEMPLATE_FUNC
00063 #undef TEMPLATE_NAME_STR
00065
00066 //
      ==========================================================================
00067 // UI64_ARRAY IMPLEMENTATION
00068 //
      ==========================================================================
00070 #define TEMPLATE_TYPE uint64_t
00071 #define TEMPLATE_STRUCT UI64_ARRAY
00072 #define TEMPLATE_FUNC(name) ui64_##name
00073 #define TEMPLATE_NAME_STR "UI64_ARRAY"
00075 #include "templates/int_array_impl.inc"
00077 #undef TEMPLATE_TYPE
00078 #undef TEMPLATE_STRUCT
00079 #undef TEMPLATE_FUNC
00080 #undef TEMPLATE_NAME_STR
```

## 15.35 iZ_toolkit.c File Reference

Implementation of iZ index space helpers and iZm/VX segment machinery.

#include <iZ_api.h>

Functions

- static int gmp_random_below (gmp_randstate_t state, int upper_bound)
- uint64_t iZ (uint64_t x, int i)

  Computes 6x + i for a given x and i.
- void iZ_mpz (mpz_t z, mpz_t x, int i)

  Computes 6x + i for arbitrary precision values using GMP.
- static int compute_k_vx (IZM *iZm)

  Count how many small primes (>3) divide vx.
- void iZm_construct_vx_base (uint64_t vx, BITMAP *base_x5, BITMAP *base_x7)

Build pre-sieved base bitmaps for a VX segment.

- static int vx_set_base_values (VX_SEG *vx_obj, char *y_str)

  Initialize mpz-dependent base fields for a VX segment object.
- static void vx_det_sieve (IZM *iZm, VX_SEG *vx_obj)

  Deterministic phase: mark composites using root primes.
- static void vx_prob_sieve (VX_SEG *vx_obj)

  Perform probabilistic sieve cleanup for large numeric ranges.
- IZM_RANGE_INFO range_info_init (INPUT_SIEVE_RANGE *input_range, int vx)

  Map a decimal range input into iZm coordinates and segment bounds.
- void range_info_free (IZM_RANGE_INFO *info)

  Clear all GMP fields owned by info.

iZ Mapping Helpers

- void process_iZ_bitmaps (UI64_ARRAY *primes, BITMAP *x5, BITMAP *x7, uint64_t x↩ _limit)

  Traverse iZ bitmaps, emit surviving primes, and mark composites.
- void get_root_primes (UI64_ARRAY *primes, uint64_t limit)

  Generate primes up to limit for deterministic sieving.
- int test_primality (mpz_t n, int rounds)

  Check the primality of a number using GMP's probabilistic test.

IZM Lifecycle

- IZM * iZm_init (size_t vx)

  Allocate and initialize an IZM object for a given VX.
- IZM * iZm_clone (IZM *src)

  Deep-copy an IZM object for per-worker ownership.
- void iZm_free (IZM **iZm)

  Release an IZM object and set the caller pointer to NULL.

VX Selection Helpers

- uint64_t compute_vx_k (int k)

  Calculate VX_{ k }.
- uint64_t compute_l2_vx (uint64_t n)

  Choose VX using an L2-cache-aware heuristic.
- void compute_max_vx (mpz_t vx, int bit_size)

  Compute largest VX below 2^bit_size.

Modular Hit Solvers

- uint64_t iZm_solve_for_x0 (int m_id, uint64_t p, uint64_t vx, uint64_t y)

  Solve first x-hit of prime p for line m_id in segment y.
- uint64_t iZm_solve_for_x0_mpz (int m_id, uint64_t p, uint64_t vx, mpz_t y)

  GMP variant of iZm_solve_for_x0() for very large y.
- int64_t iZm_solve_for_y0 (int m_id, uint64_t p, uint64_t vx, uint64_t x)

  Solve first y-hit for fixed x in vertical (vy) scanning.

VX Segment Lifecycle and Execution

- VX_SEG * vx_init (IZM *iZm, int start_x, int end_x, char *y_str, int mr_rounds)

  Initialize and deterministically sieve one VX segment.
- void vx_free (VX_SEG **vx_obj)

Free a VX segment and all owned resources.
- void vx_collect_p_gaps (VX_SEG *vx_obj)
  Extract prime-gap encoding from a fully sieved segment.
- void vx_full_sieve (VX_SEG *vx_obj, int collect_p_gaps)
  Complete segment processing (probabilistic stage and optional gaps).
- void vx_stream (VX_SEG *vx_obj, FILE *output, int stream_gaps)
  Stream segment primes to an output stream.

Random Prime Search Routines

- int vx_search_prime (mpz_t p, int m_id, int vx, int bit_size)
  Horizontal iZm/VX random-prime search.
- int vy_search_prime (mpz_t p, int m_id, mpz_t vx)
  Vertical iZm/VY random-prime search.

Variables

- static const int s_primes []
- static int s_primes_count = sizeof(s_primes) / sizeof(int)

## 15.35.1 Detailed Description

Implementation of iZ index space helpers and iZm/VX segment machinery.

Definition in file iZ_toolkit.c.

## 15.35.2 Function Documentation

### 15.35.2.1 compute_k_vx()

```
int compute_k_vx (
            IZM * iZm)    [static]
```

Count how many small primes (>3) divide vx.

Parameters

| | |
|---|---|
| iZm | Initialized toolkit context. |

Returns

Number of pre-sieved small primes encoded in vx.

Definition at line 68 of file iZ_toolkit.c.

References s_primes, and IZM::vx.

Referenced by iZm_init().

#### 15.35.2.2 gmp_random_below()

```
int gmp_random_below (
            gmp_randstate_t state,
            int upper_bound)   [static]
```

Definition at line 15 of file iZ_toolkit.c.

#### 15.35.2.3 vx_det_sieve()

```
void vx_det_sieve (
            IZM * iZm,
            VX_SEG * vx_obj)   [static]
```

Deterministic phase: mark composites using root primes.

Parameters

| | |
|---|---|
| iZm | Toolkit context containing root-prime table and base bitmaps. |
| vx_obj | Segment object to update. |

Definition at line 580 of file iZ_toolkit.c.

References UI64_ARRAY::array, VX_SEG::bit_ops, bitmap_clear_steps_simd(), bitmap_get_bit(), UI64_ARRAY::count, VX_SEG::end_x, VX_SEG::is_large_limit, iZm_solve_for_x0(), iZm_solve_for_x0_mpz(), IZM::k_vx, VX_SEG::p_count, VX_SEG::root_limit, IZM::root_primes, VX_SEG::start_x, VX_SEG::vx, VX_SEG::x5, VX_SEG::x7, and VX_SEG::y.

Referenced by vx_init().

#### 15.35.2.4 vx_prob_sieve()

```
void vx_prob_sieve (
            VX_SEG * vx_obj)   [static]
```

Perform probabilistic sieve cleanup for large numeric ranges.

Parameters

| | |
|---|---|
| vx_obj | Segment object containing deterministic survivors. |

Definition at line 647 of file iZ_toolkit.c.

References bitmap_clear_bit(), bitmap_get_bit(), VX_SEG::end_x, VX_SEG::is_large_limit, iZ_mpz(), log_info(), VX_SEG::mr_rounds, VX_SEG::p_count, VX_SEG::p_test_ops, VX_SEG::start_x, test_primality(), VX_SEG::x5, VX_SEG::x7, and VX_SEG::yvx.

Referenced by vx_full_sieve().

#### 15.35.2.5 vx_set_base_values()

```
int vx_set_base_values (
            VX_SEG * vx_obj,
            char * y_str)   [static]
```

Initialize mpz-dependent base fields for a VX segment object.

Parameters

| | |
|---|---|
| vx_obj | Segment object to populate. |

| | |
|---|---|
| y_str | Decimal y-coordinate for the segment. |

Returns

1 on success, 0 on parse/initialization failure.

Definition at line 543 of file iZ_toolkit.c.

References VX_SEG::is_large_limit, iZ_mpz(), log_error(), VX_SEG::root_limit, VX_SEG::vx, VX_SEG::y, and VX_SEG::yvx.

Referenced by vx_init().

### 15.35.3 Variable Documentation

#### 15.35.3.1 s_primes

const int s_primes[] [static]

Initial value:

```
= {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,
   43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97}
```

Small primes used to compute and construct wheel structures.

Definition at line 10 of file iZ_toolkit.c.

Referenced by compute_k_vx(), compute_l2_vx(), compute_vx_k(), and iZm_construct_vx_base().

#### 15.35.3.2 s_primes_count

int s_primes_count = sizeof(s_primes) / sizeof(int) [static]

Number of entries available in s_primes.

Definition at line 13 of file iZ_toolkit.c.

Referenced by iZm_construct_vx_base().

## 15.36 iZ_toolkit.c

Go to the documentation of this file.

```c

#include <iZ_api.h>

static const int s_primes[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,
                               43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97};
static int s_primes_count = sizeof(s_primes) / sizeof(int);

static int gmp_random_below(gmp_randstate_t state, int upper_bound)
{
    if (upper_bound <= 1)
        return 0;

    mpz_t range;
    mpz_t sample;
    mpz_init_set_ui(range, (unsigned long)upper_bound);
    mpz_init(sample);
    mpz_urandomm(sample, state, range);
    int value = (int)mpz_get_ui(sample);
    mpz_clear(range);
    mpz_clear(sample);
    return value;
}

inline uint64_t iZ(uint64_t x, int i)
{
    return 6 * x + i;
}

void iZ_mpz(mpz_t z, mpz_t x, int i)
{
    mpz_mul_ui(z, x, 6); // z = 6 * x

    if (i > 0)
        mpz_add_ui(z, z, i); // z = z + i
    else
        mpz_sub_ui(z, z, -i); // z = z - i
}

static int compute_k_vx(IZM *iZm)
{
    int k = 0;
    while (iZm->vx % s_primes[k + 2] == 0)
    {
        k++;
    }
    return k;
}

void process_iZ_bitmaps(UI64_ARRAY *primes, BITMAP *x5, BITMAP *x7, uint64_t x_limit)
{
    uint64_t root_limit = sqrt(6 * x_limit) + 1;

    // Iterate through x values in range 0 < x < x_n
    for (uint64_t x = 1; x < x_limit; x++)
    {
        // if x5[x], implying it's iZ- prime
        if (bitmap_get_bit(x5, x))
        {
            uint64_t p = iZ(x, -1); // compute p = iZ(x, -1)
            ui64_push(primes, p);   // add p to primes

            // if p is root prime, mark its multiples in x5, x7
            if (p < root_limit)
            {
                bitmap_clear_steps_simd(x5, p, p * x + x, x_limit);
                bitmap_clear_steps_simd(x7, p, p * x - x, x_limit);
            }
        }

        // Do the same if x7[x], inverting the xp signs
        if (bitmap_get_bit(x7, x))
        {
            uint64_t p = iZ(x, 1);
            ui64_push(primes, p);

            if (p < root_limit)
            {
                bitmap_clear_steps_simd(x5, p, p * x - x, x_limit);
                bitmap_clear_steps_simd(x7, p, p * x + x, x_limit);
            }
        }
```

```
    }
}

void get_root_primes(UI64_ARRAY *primes, uint64_t limit)
{
    // Add 2, 3 to primes
    ui64_push(primes, 2);
    ui64_push(primes, 3);

    // Calculate x_n, max x value in iZ space for given n
    uint64_t x_n = limit / 6 + 1;

    // Create bitmap X-Arrays x5, x7, each of size x_n + 1 bits
    BITMAP *x5 = bitmap_init(x_n + 1, 1);
    BITMAP *x7 = bitmap_init(x_n + 1, 1);

    // Memory allocation failed, check logs
    if (!x5 || !x7)
    {
        ui64_free(&primes);
        return;
    }

    // Sieve logic: mark composites and collect primes up to limit
    process_iZ_bitmaps(primes, x5, x7, x_n);

    // Cleanup: free memory of x5, x7
    bitmap_free(&x5);
    bitmap_free(&x7);

    return;
}

int test_primality(mpz_t n, int rounds)
{
    // GMP's mpz_probab_prime_p returns:
    // 0 if n is composite,
    // 1 if n is probably prime (without being certain),
    // 2 if n is definitely prime (only for small n).
    int is_prime = mpz_probab_prime_p(n, rounds);
    // ToDo: implement Baillie–PSW primality test for a deterministic alternative to Miller-Rabin, which would return a boolean result instead of a probability.

    return is_prime;
}

// ============================================================
// * iZm structure:
// ============================================================

IZM *iZm_init(size_t vx)
{
    // validate vx
    assert(vx >= 35 && "vx must be at least 35.");

    IZM *iZm = malloc(sizeof(IZM));
    if (!iZm)
    {
        log_error("Memory allocation failed for iZm.");
        return NULL;
    }

    iZm->vx = vx;
    // get root primes for deterministic sieving
    iZm->root_primes = SiZ(vx);
    // iZm->root_primes = root_limit < pow(10, 7) ? SiZ(root_limit) : SiZm(root_limit);
    if (!iZm->root_primes)
    {
        log_error("Root primes generation failed for iZm.");
        free(iZm);
        return NULL;
    }

    iZm->k_vx = compute_k_vx(iZm);

    // initialize base bitmaps
    iZm->base_x5 = bitmap_init(vx + 10, 1);
    iZm->base_x7 = bitmap_init(vx + 10, 1);
    if (!iZm->base_x5 || !iZm->base_x7)
    {
        log_error("Bitmap initialization failed for iZm base segment.");
        iZm_free(&iZm);
        ui64_free(&iZm->root_primes);
        return NULL;
    }

    // construct pre-sieved base_x5, base_x7 bitmaps
```

```
    iZm_construct_vx_base(vx, iZm->base_x5, iZm->base_x7);

    return iZm;
}

IZM *iZm_clone(IZM *src)
{
    assert(src && "Invalid source IZM structure for cloning.");

    IZM *clone = malloc(sizeof(IZM));
    assert(clone && "Memory allocation failed for iZm clone.");

    clone->vx = src->vx;
    clone->k_vx = src->k_vx;
    clone->root_primes = ui64_init(src->root_primes->capacity);
    if (clone->root_primes == NULL)
    {
        log_error("Memory allocation failed for root primes in iZm clone.");
        iZm_free(&clone);
        return NULL;
    }
    // copy root primes
    memcpy(clone->root_primes->array, src->root_primes->array, src->root_primes->count * sizeof(uint64_t));
    clone->root_primes->count = src->root_primes->count;

    // clone base bitmaps
    clone->base_x5 = bitmap_clone(src->base_x5);
    clone->base_x7 = bitmap_clone(src->base_x7);

    return clone;
}

void iZm_free(IZM **iZm)
{
    if (iZm == NULL || *iZm == NULL)
        return;

    // Free root primes object allocated in iZm_init
    ui64_free(&(*iZm)->root_primes);
    bitmap_free(&(*iZm)->base_x5);
    bitmap_free(&(*iZm)->base_x7);
    free(*iZm);
    *iZm = NULL;
}

uint64_t compute_vx_k(int k)
{
    uint64_t vx = 1;
    k += 2;    // adjust k to account for skipping 2 and 3
    int i = 2; // start from prime 5

    while (i < k && s_primes[i] * vx < UINT64_MAX)
    {
        vx *= s_primes[i++];
    }

    return vx;
}

uint64_t compute_l2_vx(uint64_t n)
{
    uint64_t l2 = get_cpu_L2_cache_size_bits();
    uint64_t x_n = n / 6;
    uint64_t vx = 35; // minimum useful vx size
    int k = 4;        // pointing to prime 11 in s_primes

    // while vx * k-th prime doesn't exceed x_n and l2 cache size
    while (vx * s_primes[k] < MIN(l2, x_n))
    {
        vx *= s_primes[k];
        k++;
    }

    return vx;
}

void compute_max_vx(mpz_t vx, int bit_size)
{
    // get some primes to compute the primorial vx
    UI64_ARRAY *primes = SiZ(10000);
    assert(primes && "Failed to generate primes");

    int i = 2;                        // to skip 2, 3
    mpz_set_ui(vx, primes->array[i]); // set vx = 5

    // while vx * primes->array[i] < 2^bit_size
    while (mpz_sizeinbase(vx, 2) < (size_t)bit_size)
```

```
    {
        i++;
        mpz_mul_ui(vx, vx, primes->array[i]);
    }

    mpz_div_ui(vx, vx, primes->array[i]); // divide by the last prime
}

void iZm_construct_vx_base(uint64_t vx, BITMAP *base_x5, BITMAP *base_x7)
{
    assert(base_x5 && base_x7 && "Invalid bitmaps passed to iZm_construct_vx_base.");

    bitmap_set_all(base_x5);
    bitmap_set_all(base_x7);
    bitmap_clear_bit(base_x5, 0); // clear the 0th bit
    bitmap_clear_bit(base_x7, 0); // clear the 0th bit

    // iterate through small primes that divide vx
    for (int i = 2; i < s_primes_count; i++)
    {
        int p = s_primes[i];
        if (vx % p != 0)
            continue;

        int ip = (p % 6 == 1) ? 1 : -1;
        int xp = (p + 1) / 6;
        // clear composites in iZ- and iZ+
        if (ip == -1)
        {
            bitmap_clear_bit(base_x5, xp);                    // clear the prime itself
            bitmap_clear_steps_simd(base_x5, p, p * xp + xp, vx); // mark composites in iZ-
            bitmap_clear_steps_simd(base_x7, p, p * xp - xp, vx); // mark composites in iZ+
        }
        else
        {
            bitmap_clear_bit(base_x7, xp);                    // clear the prime itself
            bitmap_clear_steps_simd(base_x5, p, p * xp - xp, vx); // mark composites in iZ-
            bitmap_clear_steps_simd(base_x7, p, p * xp + xp, vx); // mark composites in iZ+
        }
    }
}

uint64_t iZm_solve_for_x0(int m_id, uint64_t p, uint64_t vx, uint64_t y)
{
    uint64_t xp = (p + 1) / 6;
    int ip = (p % 6 == 1) ? 1 : -1;

    // Optimized start for first segment (y = 0)
    if (y == 0)
    {
        return p * xp + m_id * ip * xp;
    }

    // Normalize x_p to x_p if p_id = m_id, else to p - x_p
    xp = (m_id == ip) ? xp : p - xp;
    // Compute the first composite hit of p in the yth vx segment in iZm index space
    uint64_t x0 = p - (vx * y - xp) % p;
    // return x0 if it's within the segment, else return 0 for p > vx that doesn't have a hit in the segment
    return x0 > vx ? 0 : x0;
}

uint64_t iZm_solve_for_x0_mpz(int m_id, uint64_t p, uint64_t vx, mpz_t y)
{
    mpz_t tmp;
    mpz_init(tmp);

    // 1. Normalize x_p to x_p if p_id = m_id, else to p - x_p
    uint64_t xp = (p + 1) / 6;
    int ip = (p % 6 == 1) ? 1 : -1;
    xp = (m_id == ip) ? xp : p - xp;

    mpz_mul_ui(tmp, y, vx);
    mpz_sub_ui(tmp, tmp, xp);
    mpz_mod_ui(tmp, tmp, p);

    uint64_t x = p < vx ? p - mpz_get_ui(tmp) : mpz_get_ui(tmp);

    mpz_clear(tmp);
    return x;
}

int64_t iZm_solve_for_y0(int m_id, uint64_t p, uint64_t vx, uint64_t x)
{
    // No solution check, if vx and p are not coprime
    if (gcd(vx, p) != 1)
    {
        printf("There's no solution for the given parameters\n");
```

```
        return -1; // No solution can be found
    }

    uint64_t xp = (p + 1) / 6;
    int ip = (p % 6 == 1) ? 1 : -1;
    xp = (m_id == ip) ? xp : p - xp;

    // Immediate solution check
    if (x % p == xp)
        return 0;

    // Compute delta
    uint64_t x_mod_p = x % p;
    uint64_t delta = (xp + p - x_mod_p) % p;

    // Find modular inverse
    uint64_t vx_inv = modular_inverse(vx % p, p);

    // Compute and return y
    uint64_t y = (delta * vx_inv) % p;
    return y;
}

// ==================================================
// * VX_SEG structure:
// ==================================================

static int vx_set_base_values(VX_SEG *vx_obj, char *y_str)
{

    // assert y_str is numeric
    mpz_t y_tmp;
    mpz_init(y_tmp);
    if (mpz_set_str(y_tmp, y_str, 10) != 0)
    {
        log_error("Invalid numeric string for y in vx_set_base_values\n");
        mpz_clear(y_tmp);
        return 0;
    }

    mpz_init_set(vx_obj->y, y_tmp);
    mpz_clear(y_tmp);

    // Compute yvx = y * vx
    mpz_init(vx_obj->yvx);
    mpz_mul_ui(vx_obj->yvx, vx_obj->y, vx_obj->vx);

    // Compute root_limit = sqrt(iZ(vx * (y+1), 1))
    mpz_init(vx_obj->root_limit);
    mpz_add_ui(vx_obj->root_limit, vx_obj->yvx, vx_obj->vx);
    iZ_mpz(vx_obj->root_limit, vx_obj->root_limit, 1);
    mpz_sqrt(vx_obj->root_limit, vx_obj->root_limit);

    // Set is_large_limit to determine if probabilistic primality test is needed
    // if root_limit > vx
    vx_obj->is_large_limit = mpz_cmp_ui(vx_obj->root_limit, vx_obj->vx) > 0;
    return 1;
}

static void vx_det_sieve(IZM *iZm, VX_SEG *vx_obj)
{
    assert(iZm && "iZm is NULL in vx_det_sieve");
    assert(vx_obj && "vx_obj is NULL in vx_det_sieve");

    int vx = vx_obj->vx; // segment size

    // * Deterministic Sieve: Mark composites of primes < vx in x5, x7
    int start_x = vx_obj->start_x;
    int end_x = vx_obj->end_x;
    int k = 2 + iZm->k_vx; // skip 2, 3 and pre-sieved k_vx primes
    UI64_ARRAY *root_primes = iZm->root_primes;

    // if y < 2^64, use iZm_solve_for_x0 version for efficiency
    if (mpz_sizeinbase(vx_obj->y, 2) <= 64)
    {
        uint64_t y = mpz_get_ui(vx_obj->y);
        uint64_t root_limit = mpz_get_ui(vx_obj->root_limit);

        // Iterate through root primes, skipping the first k pre-sieved primes
        for (int i = k; i < root_primes->count; i++)
        {
            uint64_t p = root_primes->array[i];

            if (p > root_limit)
                break;

            // Mark composites of p in x5 and x7
```

```
            bitmap_clear_steps_simd(vx_obj->x5, p, iZm_solve_for_x0(-1, p, vx, y), end_x);
            bitmap_clear_steps_simd(vx_obj->x7, p, iZm_solve_for_x0(1, p, vx, y), end_x);

            vx_obj->bit_ops += (2 * end_x) / p; // approximate number of bit operations
        }
    }
    else
    {
        // the same as above but using iZm_solve_for_x0_mpz version
        for (int i = k; i < root_primes->count; i++)
        {
            int p = root_primes->array[i];

            bitmap_clear_steps_simd(vx_obj->x5, p, iZm_solve_for_x0_mpz(-1, p, vx, vx_obj->y), end_x);
            bitmap_clear_steps_simd(vx_obj->x7, p, iZm_solve_for_x0_mpz(1, p, vx, vx_obj->y), end_x);

            vx_obj->bit_ops += (2 * end_x) / p;
        }
    }

    // if no further probabilistic testing is needed,
    // count unmarked bits in x5 and x7 as p_count
    if (!vx_obj->is_large_limit)
    {
        vx_obj->p_count = 0;
        for (int x = start_x; x <= end_x; x++)
        {
            if (bitmap_get_bit(vx_obj->x5, x))
                vx_obj->p_count++;
            if (bitmap_get_bit(vx_obj->x7, x))
                vx_obj->p_count++;
        }
    }
}

static void vx_prob_sieve(VX_SEG *vx_obj)
{
    // If is_large_limit is false, no need for probabilistic testing
    if (!vx_obj->is_large_limit)
    {
        log_info("vx_obj->is_large_limit is false, no need for probabilistic testing");
        return;
    }

    // Initialize GMP reusable variables p, x_p
    mpz_t p, x_p;
    mpz_init(p);
    mpz_init(x_p);

    int r = vx_obj->mr_rounds;
    int s = vx_obj->start_x <= 1 ? 1 : vx_obj->start_x;

    // Iterate through x values in the range start_x <= x <= end_x
    for (int x = s; x <= vx_obj->end_x; x++)
    {
        // Check if iZ(vx * y + x, -1) is prime, if not, clear x in x5
        if (bitmap_get_bit(vx_obj->x5, x))
        {
            // Compute x_p = yvx + x
            mpz_add_ui(x_p, vx_obj->yvx, x);
            iZ_mpz(p, x_p, -1); // Compute p = iZ(x_p, -1)
            int is_prime = test_primality(p, r);
            vx_obj->p_test_ops++;

            // if is_prime, increment count, else clear composite from x5
            if (is_prime)
            {
                vx_obj->p_count++;
            }
            else
            {
                bitmap_clear_bit(vx_obj->x5, x); // Clear composite from x5
            }
        }

        // same for the 6x+1 candidate
        if (bitmap_get_bit(vx_obj->x7, x))
        {
            mpz_add_ui(x_p, vx_obj->yvx, x);
            iZ_mpz(p, x_p, 1); // Compute p = iZ(x_p, 1)
            int is_prime = test_primality(p, r);
            vx_obj->p_test_ops++;

            if (is_prime)
            {
                vx_obj->p_count++;
            }
```

```
            else
            {
                bitmap_clear_bit(vx_obj->x7, x); // Clear composite from x7
            }
        }
    }

    vx_obj->is_large_limit = 0; // all composites cleared

    // Cleanup
    mpz_clears(p, x_p, NULL);
}

VX_SEG *vx_init(IZM *iZm, int start_x, int end_x, char *y_str, int mr_rounds)
{
    // assert vx > 5 and not a multiple of 2 or 3
    assert(iZm && "iZm is NULL in vx_init");
    assert(y_str && "y_str is NULL in vx_init");

    VX_SEG *vx_obj = malloc(sizeof(VX_SEG));
    if (vx_obj == NULL)
    {
        log_error("Memory allocation failed in vx_init\n");
        return NULL;
    }

    // Initialize struct members
    vx_obj->vx = iZm->vx;

    // Set base values
    if (!vx_set_base_values(vx_obj, y_str))
    {
        // check logs
        free(vx_obj);
        return NULL;
    }

    vx_obj->mr_rounds = (mr_rounds == 0) ? MR_ROUNDS : mr_rounds; // default 25 rounds

    vx_obj->start_x = MAX(start_x, 1);
    vx_obj->end_x = MIN(end_x, vx_obj->vx);
    vx_obj->x5 = bitmap_clone(iZm->base_x5);
    vx_obj->x7 = bitmap_clone(iZm->base_x7);
    vx_obj->p_count = 0;
    vx_obj->p_gaps = NULL;
    vx_obj->bit_ops = 0;
    vx_obj->p_test_ops = 0;

    // perform deterministic sieving to prepare for probabilistic sieving or streaming
    vx_det_sieve(iZm, vx_obj);

    return vx_obj;
}

void vx_free(VX_SEG **vx_obj)
{
    if (vx_obj == NULL || *vx_obj == NULL)
        return;

    // clear bitmaps
    bitmap_free(&(*vx_obj)->x5);
    bitmap_free(&(*vx_obj)->x7);

    // clear p_gaps array
    ui16_free(&(*vx_obj)->p_gaps);

    // clear mpz_t variables
    mpz_clears((*vx_obj)->y, (*vx_obj)->yvx, (*vx_obj)->root_limit, NULL);

    free(*vx_obj);
    *vx_obj = NULL;
}

void vx_collect_p_gaps(VX_SEG *vx_obj)
{
    assert(vx_obj && vx_obj->p_count > 0 && "Invalid vx_obj in vx_collect_p_gaps");
    assert(mpz_cmp_ui(vx_obj->y, 0) > 0 && "First segment requires special handling in vx_collect_p_gaps");
    if (vx_obj->is_large_limit)
    {
        vx_full_sieve(vx_obj, 0);
    }

    vx_obj->p_gaps = ui16_init(vx_obj->p_count + 2);
    assert(vx_obj->p_gaps && "Memory allocation failed for vx_obj->p_gaps in vx_collect_p_gaps");

    // Initialize gap counter
    int gap = 0;
```

```
    // Iterate through x values in the range start_x <= x <= end_x
    for (int x = vx_obj->start_x; x <= vx_obj->end_x; x++)
    {
        // Increment gap by 4 since: iZ(x, -1) - iZ(x-1, 1) = 4
        gap += 4;

        // Check if iZ(vx * y + x, -1) is prime
        if (bitmap_get_bit(vx_obj->x5, x))
        {
            // Append gap to p_gaps
            ui16_push(vx_obj->p_gaps, gap);
            gap = 0; // Reset gap
        }

        // Increment gap by 2
        gap += 2;

        // Check if iZ(vx * y + x, 1) is prime
        if (bitmap_get_bit(vx_obj->x7, x))
        {
            // Append gap to p_gaps
            ui16_push(vx_obj->p_gaps, gap);
            gap = 0; // Reset gap
        }
    }

    // append final gap for backward calculations
    ui16_push(vx_obj->p_gaps, gap);
}

void vx_full_sieve(VX_SEG *vx_obj, int collect_p_gaps)
{
    assert(vx_obj && "vx_obj is NULL in vx_full_sieve");

    // If is_large_limit is true, perform probabilistic primality tests
    if (vx_obj->is_large_limit)
        vx_prob_sieve(vx_obj);

    // If collect_p_gaps is true, populate the p_gaps array
    if (collect_p_gaps)
        vx_collect_p_gaps(vx_obj);
}

void vx_stream(VX_SEG *vx_obj, FILE *output, int stream_gaps)
{
    assert(vx_obj && "vx_obj is NULL in vx_stream");
    assert(output && "output stream is NULL in vx_stream");

    // Initialize GMP reusable variables.
    mpz_t last_p, gap, p, x_p;
    mpz_init(last_p);
    mpz_init(gap);
    mpz_init(p);
    mpz_init(x_p);

    // Prime gaps are reported from this segment base.
    iZ_mpz(last_p, vx_obj->yvx, 1);
    if (stream_gaps)
    {
        gmp_fprintf(output, "First prime gap computed from: %Zd\n", last_p);
    }

    int r = vx_obj->mr_rounds;

    // Iterate through x values in the range start_x <= x <= end_x
    for (int x = vx_obj->start_x; x <= vx_obj->end_x; x++)
    {
        // Check if iZ(vx * y + x, -1) is prime, if not, clear x in x5
        if (bitmap_get_bit(vx_obj->x5, x))
        {
            // Compute x_p = yvx + x
            mpz_add_ui(x_p, vx_obj->yvx, x);
            iZ_mpz(p, x_p, -1); // Compute p = iZ(x_p, -1)
            int is_prime = 1;

            if (vx_obj->is_large_limit)
            {
                vx_obj->p_test_ops++;
                is_prime = test_primality(p, r);
            }

            if (is_prime)
            {
                if (vx_obj->is_large_limit)
                {
                    vx_obj->p_count++; // otherwise already counted in det_sieve
```

```
            }
            if (stream_gaps)
            {
                mpz_sub(gap, p, last_p);
                gmp_fprintf(output, "%Zd ", gap);
                mpz_set(last_p, p);
            }
            else
            {
                gmp_fprintf(output, "%Zd ", p);
            }
        }
        else
        {
            bitmap_clear_bit(vx_obj->x5, x); // Clear composite from x5
        }
    }

    // test iZ(vx * y + x, 1) for primality
    if (bitmap_get_bit(vx_obj->x7, x))
    {
        mpz_add_ui(x_p, vx_obj->yvx, x);
        iZ_mpz(p, x_p, 1); // Compute p = iZ(x_p, 1)
        int is_prime = 1;

        if (vx_obj->is_large_limit)
        {
            vx_obj->p_test_ops++;
            is_prime = test_primality(p, r);
        }

        if (is_prime)
        {
            if (vx_obj->is_large_limit)
            {
                vx_obj->p_count++;
            }
            if (stream_gaps)
            {
                mpz_sub(gap, p, last_p);
                gmp_fprintf(output, "%Zd ", gap);
                mpz_set(last_p, p);
            }
            else
            {
                gmp_fprintf(output, "%Zd ", p);
            }
        }
        else
        {
            bitmap_clear_bit(vx_obj->x7, x); // Clear composite from x7
        }
    }
}

mpz_clears(last_p, gap, p, x_p, NULL);
}

// ====================================================
// * IZM_RANGE_INFO structure:
// ====================================================

IZM_RANGE_INFO range_info_init(INPUT_SIEVE_RANGE *input_range, int vx)
{
    IZM_RANGE_INFO info = {0};
    info.vx = vx;
    info.y_range = -1;

    mpz_inits(info.Zs, info.Ze, info.Xs, info.Xe, info.Ys, info.Ye, NULL);

    if (!input_range || !input_range->start || vx < 35 || mpz_set_str(info.Zs, input_range->start, 10) != 0)
    {
        log_error("range_info_init: invalid input.");
        return info;
    }

    if (input_range->range == 0)
    {
        mpz_set(info.Ze, info.Zs);
    }
    else
    {
        // inclusive upper bound for range size semantics: [start, start + range - 1]
        mpz_add_ui(info.Ze, info.Zs, input_range->range - 1);
    }

    mpz_fdiv_q_ui(info.Xs, info.Zs, 6);
```

```
    mpz_fdiv_q_ui(info.Xe, info.Ze, 6);
    mpz_fdiv_q_ui(info.Ys, info.Xs, vx);
    mpz_fdiv_q_ui(info.Ye, info.Xe, vx);

    mpz_t y_delta;
    mpz_init(y_delta);
    mpz_sub(y_delta, info.Ye, info.Ys);

    if (mpz_sgn(y_delta) < 0 || !mpz_fits_sint_p(y_delta))
    {
        log_error("range_info_init: computed y-range is out of supported bounds.");
        mpz_clear(y_delta);
        return info;
    }

    info.y_range = (int)mpz_get_si(y_delta);
    mpz_clear(y_delta);

    return info;
}

void range_info_free(IZM_RANGE_INFO *info)
{
    if (info == NULL)
        return;

    mpz_clears(info->Zs, info->Ze, info->Xs, info->Xe, info->Ys, info->Ye, NULL);
}

// ==================================================
// * iZm Search primes routines:
// ==================================================

int vx_search_prime(mpz_t p, int m_id, int vx, int bit_size)
{
    // assert p is not NULL
    assert(p && "p cannot be NULL in vx_search_prime.");

    bit_size = MAX(bit_size, 10);

    gmp_randstate_t state;
    gmp_randinit_default(state);
    gmp_seed_randstate(state);

    // set m_id randomly if not provided
    if (m_id != -1 && m_id != 1)
        m_id = gmp_random_below(state, 2) ? 1 : -1;

    UI64_ARRAY *root_primes = SiZm(vx);
    if (!root_primes)
    {
        log_error("Failed to initialize root primes in vx_search_prime");
        gmp_randclear(state);
        return 0;
    }

    mpz_t z, x_z, y, yvx;
    mpz_init(z);
    mpz_init(x_z);
    mpz_init(y);
    mpz_init(yvx);

    mpz_urandomb(y, state, bit_size);
    mpz_fdiv_q_ui(y, y, 6 * vx);
    mpz_mul_ui(yvx, y, vx); // compute yvx = vx * y

    int found = 0;
    while (!found)
    {
        BITMAP *bitmap = bitmap_init(vx + 10, 1);

        // sieve the bitmap with root primes skipping 2 and 3
        for (int i = 2; i < root_primes->count; i++)
        {
            uint64_t q = root_primes->array[i];
            // mark composites of q in the bitmap
            bitmap_clear_steps(bitmap, q, iZm_solve_for_x0_mpz(m_id, q, vx, y), vx);
        }

        int random_x = gmp_random_below(state, MAX(vx / 2, 1)); // random x < vx/2
        for (int x = random_x; x < vx; x++)
        {
            // check if z is prime candidate in bitmap
            if (bitmap_get_bit(bitmap, x))
            {
                // compute x_z = vx * y + x
                mpz_add_ui(x_z, yvx, x);
```

```
            iZ_mpz(z, x_z, m_id);
            // check if z is prime
            found = test_primality(z, MR_ROUNDS);

            // if z is prime, set p = z
            if (found)
            {
                mpz_set(p, z);
                break;
            }
        }
    }

    // cleanup
    bitmap_free(&bitmap);

    // increment y by 1 for next vx segment
    mpz_add_ui(y, y, 1);
    mpz_add_ui(yvx, yvx, vx);
}

ui64_free(&root_primes);
mpz_clears(z, x_z, y, yvx, NULL);
gmp_randclear(state);

return found;
}

int vy_search_prime(mpz_t p, int m_id, mpz_t vx)
{
    assert(p && vx && "Input parameters cannot be NULL");

    int found = 0; // flag to indicate if a prime was found

    gmp_randstate_t state;
    gmp_randinit_default(state);
    gmp_seed_randstate(state); // seed random state

    // set m_id randomly if not provided
    if (m_id != -1 && m_id != 1)
        m_id = gmp_random_below(state, 2) ? 1 : -1;

    mpz_t z, g;
    mpz_init(z);
    mpz_init(g);

    // set random x value in the range of vx
    mpz_urandomm(z, state, vx);
    // compute z = 6 * z + i
    iZ_mpz(z, z, m_id);

    // search for x value such that gcd(iZ(x, m_id), z) = 1
    for (;;)
    {
        // increment z by 6 to advance x by 1
        mpz_add_ui(z, z, 6);

        // Compute g = gcd(vx, z)
        mpz_gcd(g, vx, z);

        // break if g = 1,
        // implying current x value can yield primes of the form iZ(x + vx * y, p_id)
        if (mpz_cmp_ui(g, 1) == 0)
            break;
    }

    // set g = 6 * vx to use as a step
    mpz_mul_ui(g, vx, 6);
    // add z by rand * g
    int rand_steps = gmp_random_below(state, 100);
    mpz_addmul_ui(z, g, rand_steps);

    while (!found)
    {
        // increment z by 6 * vx to advance y by 1
        mpz_add(z, z, g);

        // check if z is prime
        found = test_primality(z, MR_ROUNDS);

        // if z is prime, set p = z
        if (found)
            mpz_set(p, z);
    }

    // cleanup
    gmp_randclear(state);
```

```
    mpz_clears(z, g, NULL);

    return found;
}
```

## 15.37 logger.c File Reference

Logging module.

### 15.37.1 Detailed Description

Logging module.

This module provides functions for logging messages at different levels.

Definition in file logger.c.

## 15.38 logger.c

Go to the documentation of this file.

```

#ifdef ENABLE_LOGGING

#include "logger.h"
#include <platform.h>
#include <stdio.h>
#include <time.h>
#include <stdarg.h>
#include <string.h>
#include <sys/stat.h>
#include <pthread.h>
#include <stdlib.h>
#include <errno.h>

#if defined(__clang__)
#define IZ_DIAG_PUSH _Pragma("clang diagnostic push")
#define IZ_DIAG_POP _Pragma("clang diagnostic pop")
#define IZ_DIAG_IGNORE_FORMAT_NONLITERAL _Pragma("clang diagnostic ignored \"-Wformat-nonliteral\"")
#elif defined(__GNUC__)
#define IZ_DIAG_PUSH _Pragma("GCC diagnostic push")
#define IZ_DIAG_POP _Pragma("GCC diagnostic pop")
#define IZ_DIAG_IGNORE_FORMAT_NONLITERAL _Pragma("GCC diagnostic ignored \"-Wformat-nonliteral\"")
#else
#define IZ_DIAG_PUSH
#define IZ_DIAG_POP
#define IZ_DIAG_IGNORE_FORMAT_NONLITERAL
#endif

// Initialize the mutex for thread-safe logging
static pthread_mutex_t log_mutex = PTHREAD_MUTEX_INITIALIZER;
static LogLevel current_log_level = LOG_DEBUG; // Default log level

const char *log_level_to_string(LogLevel level)
{
    switch (level)
    {
    case LOG_DEBUG:
        return "DEBUG";
    case LOG_INFO:
        return "INFO";
    case LOG_WARNING:
        return "WARNING";
    case LOG_ERROR:
        return "ERROR";
    case LOG_FATAL:
        return "FATAL";
    default:
        return "UNKNOWN";
    }
```

```c
}

void get_current_timestamp(char *buffer, size_t size)
{
    time_t now = time(NULL);
    struct tm tm_now;
    if (!iz_platform_localtime(&now, &tm_now) || strftime(buffer, size, "%Y-%m-%d %H:%M:%S", &tm_now) == 0)
        snprintf(buffer, size, "1970-01-01 00:00:00");
}

void log_rotate(const char *log_file, size_t max_size)
{
    struct stat st;
    if (stat(log_file, &st) == 0 && st.st_size >= (off_t)max_size) // Cast max_size to off_t to match types
    {
        char old_log_file[256];
        for (int i = 5; i > 0; --i)
        {
            snprintf(old_log_file, sizeof(old_log_file), "%s.%d", log_file, i);
            if (i == 5)
            {
                remove(old_log_file); // Remove the oldest log file
            }
            else
            {
                char prev_log_file[256];
                snprintf(prev_log_file, sizeof(prev_log_file), "%s.%d", log_file, i + 1);
                rename(old_log_file, prev_log_file);
            }
        }

        char new_log_file[256];
        snprintf(new_log_file, sizeof(new_log_file), "%s.1", log_file);
        rename(log_file, new_log_file);
    }
}

void log_init(const char *log_file)
{
    if (iz_platform_create_dir(LOG_DIR) != 0)
    {
        perror("Failed to create log directory");
    }

    log_rotate(log_file, LOG_MAX_SIZE); // Rotate logs if needed
}

void log_shutdown(void)
{
    pthread_mutex_destroy(&log_mutex);
}

void log_set_log_level(LogLevel level)
{
    current_log_level = level;
}

static void log_message_va(LogLevel level, const char *format, va_list args)
{
    if (level < current_log_level)
        return; // Don't log messages below the current log level

    char message[1024];
    // Format string comes from a variadic function parameter - suppress warning
    IZ_DIAG_PUSH;
    IZ_DIAG_IGNORE_FORMAT_NONLITERAL;
    vsnprintf(message, sizeof(message), format, args);
    IZ_DIAG_POP;

    char timestamp[20];
    get_current_timestamp(timestamp, sizeof(timestamp));

    pthread_mutex_lock(&log_mutex);

    FILE *file = fopen(LOG_FILE, "a");
    if (!file)
    {
        perror("Failed to open log file");
        pthread_mutex_unlock(&log_mutex); // Ensure mutex is unlocked
        return;
    }

    fprintf(file, "[%s] [%s] %s\n", timestamp, log_level_to_string(level), message);
    fclose(file);

    pthread_mutex_unlock(&log_mutex);
}
```

```
00190
00200 void log_message(LogLevel level, const char *format, ...)
00201 {
00202     va_list args;
00203     va_start(args, format);
00204     log_message_va(level, format, args);
00205     va_end(args);
00206 }
00207
00216 void log_message_extended(LogLevel level, const char *file_name, int line_number, const char *format, ...)
00217 {
00218     if (level < current_log_level)
00219         return; // Don't log messages below the current log level
00220
00221     va_list args;
00222     va_start(args, format);
00223     char message[1024];
00224     // Format string comes from a variadic function parameter - suppress warning
00225     IZ_DIAG_PUSH;
00226     IZ_DIAG_IGNORE_FORMAT_NONLITERAL;
00227     vsnprintf(message, sizeof(message), format, args);
00228     IZ_DIAG_POP;
00229     va_end(args);
00230
00231     char timestamp[20];
00232     get_current_timestamp(timestamp, sizeof(timestamp));
00233
00234     pthread_mutex_lock(&log_mutex);
00235
00236     FILE *file = fopen(LOG_FILE, "a");
00237     if (!file)
00238     {
00239         perror("Failed to open log file");
00240         pthread_mutex_unlock(&log_mutex);
00241         return;
00242     }
00243
00244     fprintf(file, "[%s] [%s] %s (File: %s, Line: %d)\n",
00245             timestamp, log_level_to_string(level), message, file_name, line_number);
00246     fclose(file);
00247
00248     pthread_mutex_unlock(&log_mutex);
00249 }
00250
00258 void log_console(const char *format, ...)
00259 {
00260     va_list args;
00261     va_start(args, format);
00262     char message[1024];
00263     // Format string comes from a variadic function parameter - suppress warning
00264     IZ_DIAG_PUSH;
00265     IZ_DIAG_IGNORE_FORMAT_NONLITERAL;
00266     vsnprintf(message, sizeof(message), format, args);
00267     IZ_DIAG_POP;
00268     va_end(args);
00269
00270     char timestamp[20];
00271     get_current_timestamp(timestamp, sizeof(timestamp));
00272
00273     // Print the message with a timestamp to the console
00274     printf("[%s] %s\n", timestamp, message);
00275 }
00276
00282 void log_debug(const char *format, ...)
00283 {
00284     va_list args;
00285     va_start(args, format);
00286     log_message_va(LOG_DEBUG, format, args);
00287     va_end(args);
00288 }
00289
00295 void log_info(const char *format, ...)
00296 {
00297     va_list args;
00298     va_start(args, format);
00299     log_message_va(LOG_INFO, format, args);
00300     va_end(args);
00301 }
00302
00308 void log_warn(const char *format, ...)
00309 {
00310     va_list args;
00311     va_start(args, format);
00312     log_message_va(LOG_WARNING, format, args);
00313     va_end(args);
00314 }
00315
```

```
void log_error(const char *format, ...)
{
    va_list args;
    va_start(args, format);
    log_message_va(LOG_ERROR, format, args);
    va_end(args);
}

void log_fatal(const char *format, ...)
{
    va_list args;
    va_start(args, format);
    log_message_va(LOG_FATAL, format, args);
    va_end(args);
}

#endif // ENABLE_LOGGING
```

## 15.39 platform.c File Reference

Platform abstraction implementation.

#include <platform.h>
#include <errno.h>
#include <limits.h>
#include <stdio.h>
#include <string.h>
#include <openssl/rand.h>

Functions

- int iz_platform_create_dir (const char *dir)

  Create a directory if it does not already exist.
- int iz_platform_fill_random (void *buffer, size_t bytes)

  Fill a buffer with random bytes.
- int iz_platform_cpu_cores_count (void)

  Return number of online logical CPU cores (>= 1).
- int iz_platform_l2_cache_size_bits (void)

  Best-effort L2 cache size query in bits.
- double iz_platform_monotonic_seconds (void)

  Return a monotonic timestamp in seconds.
- int iz_platform_localtime (const time_t *timestamp, struct tm *out)

  Convert calendar time to local broken-down time (thread-safe).

### 15.39.1 Detailed Description

Platform abstraction implementation.

Definition in file platform.c.

## 15.40 platform.c

Go to the documentation of this file.

```
#include <platform.h>

#include <errno.h>
#include <limits.h>
#include <stdio.h>
#include <string.h>

#include <openssl/rand.h>

#if IZ_PLATFORM_WINDOWS
#ifndef WIN32_LEAN_AND_MEAN
#define WIN32_LEAN_AND_MEAN
#endif
#ifndef NOMINMAX
#define NOMINMAX
#endif
#include <windows.h>
#include <direct.h>
#endif

int iz_platform_create_dir(const char *dir)
{
    if (dir == NULL || dir[0] == '\0')
        return -1;

#if IZ_PLATFORM_WINDOWS
    if (_mkdir(dir) == 0 || errno == EEXIST)
        return 0;
#else
    if (mkdir(dir, 0700) == 0 || errno == EEXIST)
        return 0;
#endif
    return -1;
}

int iz_platform_fill_random(void *buffer, size_t bytes)
{
    if (buffer == NULL)
        return 0;

    if (bytes == 0)
        return 1;

    unsigned char *out = (unsigned char *)buffer;
    size_t offset = 0;

    while (offset < bytes)
    {
        size_t chunk = bytes - offset;
        if (chunk > (size_t)INT_MAX)
            chunk = (size_t)INT_MAX;

        if (RAND_bytes(out + offset, (int)chunk) != 1)
            return 0;

        offset += chunk;
    }

    return 1;
}

int iz_platform_cpu_cores_count(void)
{
#if IZ_PLATFORM_WINDOWS
    SYSTEM_INFO info;
    GetSystemInfo(&info);
    int cores = (int)info.dwNumberOfProcessors;
    return (cores > 0) ? cores : 1;
#else
    long cores = sysconf(_SC_NPROCESSORS_ONLN);
    return (cores > 0) ? (int)cores : 1;
#endif
}

int iz_platform_l2_cache_size_bits(void)
{
#if defined(__linux__)
    FILE *fp = fopen("/sys/devices/system/cpu/cpu0/cache/index2/size", "r");
    if (fp != NULL)
    {
```

```
        char buffer[32];
        if (fgets(buffer, sizeof(buffer), fp) != NULL)
        {
            int size_kb = 0;
            if (sscanf(buffer, "%dK", &size_kb) == 1 && size_kb > 0)
            {
                fclose(fp);
                return size_kb * 1024 * 8;
            }
        }
        fclose(fp);
    }
#endif

#if defined(__APPLE__) || defined(__FreeBSD__) || defined(__OpenBSD__) || defined(__NetBSD__)
    size_t size_bytes = 0;
    size_t len = sizeof(size_bytes);
    if (sysctlbyname("hw.l2cachesize", &size_bytes, &len, NULL, 0) == 0 && size_bytes > 0)
        return (int)(size_bytes * 8);

    len = sizeof(size_bytes);
    if (sysctlbyname("machdep.cpu.cache.L2_cache_size", &size_bytes, &len, NULL, 0) == 0 && size_bytes > 0)
        return (int)(size_bytes * 8);
#endif

    return 256 * 1024 * 8;
}

double iz_platform_monotonic_seconds(void)
{
#if IZ_PLATFORM_WINDOWS
    LARGE_INTEGER counter;
    LARGE_INTEGER frequency;
    if (QueryPerformanceFrequency(&frequency) && QueryPerformanceCounter(&counter) && frequency.QuadPart > 0)
    {
        return (double)counter.QuadPart / (double)frequency.QuadPart;
    }
    return (double)GetTickCount64() / 1000.0;
#else
    struct timespec ts;
    if (clock_gettime(CLOCK_MONOTONIC, &ts) == 0)
        return (double)ts.tv_sec + (double)ts.tv_nsec / 1000000000.0;

    timespec_get(&ts, TIME_UTC);
    return (double)ts.tv_sec + (double)ts.tv_nsec / 1000000000.0;
#endif
}

int iz_platform_localtime(const time_t *timestamp, struct tm *out)
{
    if (timestamp == NULL || out == NULL)
        return 0;

#if IZ_PLATFORM_WINDOWS
    return localtime_s(out, timestamp) == 0;
#else
    return localtime_r(timestamp, out) != NULL;
#endif
}
```

## 15.41 print_utils.c File Reference

Print formatting helpers for tests, benchmarks, and CLI output.

#include <utils.h>

### Functions

- void print_line (int length, char fill_char)

  Print a repeated-character horizontal line.
- void print_centered_text (const char *text, int line_length, char fill_char)

  Print text centered inside a padded line.

- void print_sha256_hash (const unsigned char *hash)

  Print a SHA-256 digest as hex to stdout.
- void print_test_module_header (char *module_name)

  Print a test-suite header banner for a module.
- void print_test_table_header (void)

  Print the generic test runner header.
- void print_test_module_result (int result, int test_id, const char *unit_name, const char *format,...)

  Print a single test-row result.
- void print_test_summary (char *module_name, int passed, int failed, int verbose)

  Print module-level test summary.
- void print_test_fn_header (char *fn_name)

  Print a formatted header for a test function.

### 15.41.1 Detailed Description

Print formatting helpers for tests, benchmarks, and CLI output.

Definition in file print_utils.c.

### 15.41.2 Macro Definition Documentation

#### 15.41.2.1 IZ_DIAG_IGNORE_FORMAT_NONLITERAL

#define IZ_DIAG_IGNORE_FORMAT_NONLITERAL

Definition at line 20 of file print_utils.c.

#### 15.41.2.2 IZ_DIAG_POP

#define IZ_DIAG_POP

Definition at line 19 of file print_utils.c.

#### 15.41.2.3 IZ_DIAG_PUSH

#define IZ_DIAG_PUSH

Definition at line 18 of file print_utils.c.

## 15.42 print_utils.c

Go to the documentation of this file.

```
#include <utils.h>

#if defined(__clang__)
#define IZ_DIAG_PUSH _Pragma("clang diagnostic push")
#define IZ_DIAG_POP _Pragma("clang diagnostic pop")
#define IZ_DIAG_IGNORE_FORMAT_NONLITERAL _Pragma("clang diagnostic ignored \"-Wformat-nonliteral\"")
#elif defined(__GNUC__)
#define IZ_DIAG_PUSH _Pragma("GCC diagnostic push")
#define IZ_DIAG_POP _Pragma("GCC diagnostic pop")
#define IZ_DIAG_IGNORE_FORMAT_NONLITERAL _Pragma("GCC diagnostic ignored \"-Wformat-nonliteral\"")
#else
#define IZ_DIAG_PUSH
#define IZ_DIAG_POP
#define IZ_DIAG_IGNORE_FORMAT_NONLITERAL
#endif

void print_line(int length, char fill_char)
{
    if (fill_char == '\0')
        fill_char = '-';

    for (int i = 0; i < length; ++i)
        printf("%c", fill_char);
    printf("\n");
}

void print_centered_text(const char *text, int line_length, char fill_char)
{
    if (text == NULL)
        text = "";

    size_t scan_cap = (line_length > 0) ? (size_t)line_length + 1 : 1;
    int text_length = (int)strnlen(text, scan_cap);
    if (text_length >= line_length)
    {
        printf("%s\n", text);
        return;
    }

    int left_padding = (line_length - text_length) / 2;
    int right_padding = line_length - text_length - left_padding;

    for (int i = 0; i < left_padding; ++i)
        printf("%c", fill_char);
    printf("%s", text);
    for (int i = 0; i < right_padding; ++i)
        printf("%c", fill_char);
    printf("\n");
}

void print_sha256_hash(const unsigned char *hash)
{
    for (int i = 0; i < SHA256_DIGEST_LENGTH; i++)
        printf("%02x", hash[i]);

    printf("\n");
}

void print_test_module_header(char *module_name)
{
    print_line(60, '*');
    printf("* %s MODULE TEST SUITE\n", module_name);
    print_line(60, '*');
}

void print_test_table_header(void)
{
    print_line(92, '-');
    printf("[%s] %-30s %s %-66s\n", "ID", "Unit Name", "Result", "Details");
    print_line(92, '-');
}

void print_test_module_result(int result, int test_id, const char *unit_name, const char *format, ...)
{
    va_list args;
    va_start(args, format);

    char message[1024];
    IZ_DIAG_PUSH;
    IZ_DIAG_IGNORE_FORMAT_NONLITERAL;
```

```
    vsnprintf(message, sizeof(message), format, args);
    IZ_DIAG_POP;
    va_end(args);

    if (result)
    {
        printf("[%02d] %-30s [PASS] %s\n", test_id, unit_name, message);
    }
    else
    {
        printf("[%02d] %-30s [FAIL] %s\n", test_id, unit_name, message);
    }
}

void print_test_summary(char *module_name, int passed, int failed, int verbose)
{
    (void)verbose;

    print_line(60, '*');
    printf("Results Summary for %s\n", module_name);
    print_line(60, '-');
    printf("%-32s: %d\n", "Total Tests", passed + failed);
    printf("%-32s: %d\n", "Passed", passed);
    printf("%-32s: %d\n", "Failed", failed);
    print_line(60, '-');
    if (failed == 0)
    {
        printf("[SUCCESS] ALL %s TESTS PASSED!\n", module_name);
    }
    else
    {
        printf("[FAILURE] SOME %s TESTS FAILED :\\\n", module_name);
    }
    print_line(60, '*');
}

void print_test_fn_header(char *fn_name)
{
    char header[64];
    snprintf(header, sizeof(header), " Testing %s ", fn_name);

    print_line(60, '*');
    print_centered_text(header, 60, '=');
    print_line(60, '*');
    printf("\n");
}
```

## 15.43 stopwatch.c File Reference

Stopwatch helpers based on monotonic wall-clock time.

#include <utils.h>

Functions

- void sw_start (STOPWATCH *sw)

  Start or restart a stopwatch.
- void sw_stop (STOPWATCH *sw)

  Stop a running stopwatch, capturing the elapsed time in seconds.
- double sw_elapsed_seconds (const STOPWATCH *sw)

  Return elapsed seconds for the stopwatch.
- double sw_elapsed_now_seconds (void)

  Capture the current monotonic time in seconds.

### 15.43.1 Detailed Description

Stopwatch helpers based on monotonic wall-clock time.

Definition in file stopwatch.c.

## 15.44 stopwatch.c

Go to the documentation of this file.

```
#include <utils.h>

static struct timespec sw_now_timespec(void)
{
    double now = iz_platform_monotonic_seconds();
    if (now < 0.0)
        now = 0.0;

    struct timespec ts;
    ts.tv_sec = (time_t)now;
    ts.tv_nsec = (long)((now - (double)ts.tv_sec) * 1000000000.0);
    if (ts.tv_nsec < 0)
        ts.tv_nsec = 0;
    if (ts.tv_nsec >= 1000000000L)
        ts.tv_nsec = 999999999L;
    return ts;
}

static double sw_elapsed_between(struct timespec start, struct timespec end)
{
    time_t sec = end.tv_sec - start.tv_sec;
    long nsec = end.tv_nsec - start.tv_nsec;
    if (nsec < 0)
    {
        sec -= 1;
        nsec += 1000000000L;
    }
    return (double)sec + (double)nsec / 1000000000.0;
}

void sw_start(STOPWATCH *sw)
{
    assert(sw && "Invalid stopwatch passed to sw_start.");
    sw->start_time = sw_now_timespec();
    sw->end_time = sw->start_time;
    sw->running = 1;
}

void sw_stop(STOPWATCH *sw)
{
    assert(sw && "Invalid stopwatch passed to sw_stop.");
    if (!sw->running)
        return;
    sw->end_time = sw_now_timespec();
    sw->running = 0;
    sw->elapsed_sec = sw_elapsed_between(sw->start_time, sw->end_time);
}

double sw_elapsed_seconds(const STOPWATCH *sw)
{
    assert(sw && "Invalid stopwatch passed to sw_elapsed_seconds.");
    if (sw->running)
    {
        struct timespec now = sw_now_timespec();
        return sw_elapsed_between(sw->start_time, now);
    }
    return sw->elapsed_sec;
}

double sw_elapsed_now_seconds(void)
{
    return iz_platform_monotonic_seconds();
}
```

## 15.45 utils.c File Reference

Implementations for the shared utility layer.

#include <utils.h>
#include <ctype.h>

Functions

- int is_numeric_str (const char *str)

  Check if a string is numeric.
- int parse_numeric_expr_mpz (mpz_t out, const char *expr)

  Parse an integer expression into an mpz value.
- int parse_numeric_expr_u64 (const char *expr, uint64_t *out)

  Parse an integer expression into uint64_t.
- int parse_inclusive_range_mpz (const char *range_expr, mpz_t lower, mpz_t upper)

  Parse an inclusive range expression into lower/upper bounds.
- int create_dir (const char *dir)

  Create a directory if it does not exist.
- uint64_t gcd (uint64_t a, uint64_t b)

  Compute the greatest common divisor (GCD) of two integers.
- uint64_t lcm (uint64_t a, uint64_t b)

  Compute the least common multiple (LCM) of two integers.
- void gcd_mpz (mpz_t out, const mpz_t a, const mpz_t b)

  Compute the greatest common divisor for arbitrary-precision integers.
- void lcm_mpz (mpz_t out, const mpz_t a, const mpz_t b)

  Compute the least common multiple for arbitrary-precision integers.
- uint64_t modular_inverse (uint64_t a, uint64_t m)

  Compute the modular inverse of a modulo m.
- void gmp_seed_randstate (gmp_randstate_t state)

  Seed the GMP random state.
- int get_cpu_cores_count (void)

  Compute the number of CPU cores available.
- int get_cpu_L2_cache_size_bits (void)

  Get the CPU L2 cache size in bits.

### 15.45.1 Detailed Description

Implementations for the shared utility layer.

Definition in file utils.c.

## 15.46 utils.c

Go to the documentation of this file.

```
00001
00006
00007 #include <utils.h>
00008 #include <ctype.h>
00009
00011 #define IZ_MAX_TEXT_INPUT_BYTES (1U « 20)
00012
00013 static int bounded_cstr_len(const char *s, size_t cap, size_t *out_len)
00014 {
00015     if (s == NULL || out_len == NULL)
00016         return 0;
00017
00018     size_t len = strnlen(s, cap + 1);
00019     if (len > cap)
00020         return 0;
00021
00022     *out_len = len;
00023     return 1;
```

```
}

static const char *skip_spaces(const char *s)
{
    while (*s && isspace((unsigned char)*s))
        s++;
    return s;
}

static char *dup_trimmed(const char *src)
{
    if (src == NULL)
        return NULL;

    const char *start = skip_spaces(src);
    size_t start_len = 0;
    if (!bounded_cstr_len(start, IZ_MAX_TEXT_INPUT_BYTES, &start_len))
        return NULL;

    const char *end = start + start_len;
    while (end > start && isspace((unsigned char)end[-1]))
        --end;

    size_t len = (size_t)(end - start);
    char *out = malloc(len + 1);
    if (!out)
        return NULL;
    memcpy(out, start, len);
    out[len] = '\0';
    return out;
}

static char *normalize_decimal_token(const char *token)
{
    char *trimmed = dup_trimmed(token);
    if (trimmed == NULL)
        return NULL;

    const char *s = trimmed;
    if (*s == '+')
        ++s;

    if (*s == '\0')
    {
        free(trimmed);
        return NULL;
    }

    size_t s_len = 0;
    if (!bounded_cstr_len(s, IZ_MAX_TEXT_INPUT_BYTES, &s_len))
    {
        free(trimmed);
        return NULL;
    }

    size_t cap = s_len + 1;
    char *normalized = malloc(cap);
    if (!normalized)
    {
        free(trimmed);
        return NULL;
    }

    size_t n = 0;
    int has_comma = strchr(s, ',') != NULL;

    if (!has_comma)
    {
        for (const char *p = s; *p; ++p)
        {
            if (*p == '_')
                continue;
            if (!isdigit((unsigned char)*p))
            {
                free(normalized);
                free(trimmed);
                return NULL;
            }
            normalized[n++] = *p;
        }
    }
    else
    {
        const char *seg = s;
        const char *s_end = s + s_len;
        int group_idx = 0;
        while (1)
```

```
        {
            const char *comma = strchr(seg, ',');
            size_t seg_len = comma ? (size_t)(comma - seg) : (size_t)(s_end - seg);
            if (seg_len == 0)
            {
                free(normalized);
                free(trimmed);
                return NULL;
            }

            if (group_idx == 0)
            {
                if (seg_len < 1 || seg_len > 3)
                {
                    free(normalized);
                    free(trimmed);
                    return NULL;
                }
            }
            else if (seg_len != 3)
            {
                free(normalized);
                free(trimmed);
                return NULL;
            }

            for (size_t i = 0; i < seg_len; ++i)
            {
                if (!isdigit((unsigned char)seg[i]))
                {
                    free(normalized);
                    free(trimmed);
                    return NULL;
                }
                normalized[n++] = seg[i];
            }

            group_idx++;
            if (comma == NULL)
                break;
            seg = comma + 1;
        }
    }

    normalized[n] = '\0';
    free(trimmed);
    return normalized;
}

static int parse_integer_token_mpz(mpz_t out, const char *token)
{
    char *normalized = normalize_decimal_token(token);
    if (normalized == NULL)
        return 0;

    int ok = (mpz_set_str(out, normalized, 10) == 0);
    free(normalized);
    return ok;
}

typedef struct
{
    const char *cursor;
} MPZ_EXPR_PARSER;

static void parser_skip_spaces(MPZ_EXPR_PARSER *parser)
{
    parser->cursor = skip_spaces(parser->cursor);
}

static int parse_expr_mpz_impl(MPZ_EXPR_PARSER *parser, mpz_t out);

static int parse_primary_mpz(MPZ_EXPR_PARSER *parser, mpz_t out)
{
    parser_skip_spaces(parser);
    if (*parser->cursor == '(')
    {
        parser->cursor++;
        if (!parse_expr_mpz_impl(parser, out))
            return 0;
        parser_skip_spaces(parser);
        if (*parser->cursor != ')')
            return 0;
        parser->cursor++;
        return 1;
    }

```

```
    const char *start = parser->cursor;
    while (*parser->cursor)
    {
        char c = *parser->cursor;
        if (!(isdigit((unsigned char)c) || c == ',' || c == '_'))
            break;
        parser->cursor++;
    }
    if (parser->cursor == start)
        return 0;

    size_t len = (size_t)(parser->cursor - start);
    char *token = malloc(len + 1);
    if (!token)
        return 0;
    memcpy(token, start, len);
    token[len] = '\0';

    int ok = parse_integer_token_mpz(out, token);
    free(token);
    return ok;
}

static int parse_unary_mpz(MPZ_EXPR_PARSER *parser, mpz_t out)
{
    parser_skip_spaces(parser);
    if (*parser->cursor == '+')
    {
        parser->cursor++;
        return parse_unary_mpz(parser, out);
    }
    if (*parser->cursor == '-')
    {
        parser->cursor++;
        if (!parse_unary_mpz(parser, out))
            return 0;
        mpz_neg(out, out);
        return 1;
    }
    return parse_primary_mpz(parser, out);
}

static int parse_power_mpz(MPZ_EXPR_PARSER *parser, mpz_t out)
{
    mpz_t base, exp_value, pow10;
    mpz_inits(base, exp_value, pow10, NULL);

    if (!parse_unary_mpz(parser, base))
    {
        mpz_clears(base, exp_value, pow10, NULL);
        return 0;
    }

    parser_skip_spaces(parser);
    char op = *parser->cursor;
    if (op == '^' || op == 'e' || op == 'E')
    {
        parser->cursor++;
        if (!parse_power_mpz(parser, exp_value))
        {
            mpz_clears(base, exp_value, pow10, NULL);
            return 0;
        }

        if (mpz_sgn(exp_value) < 0 || !mpz_fits_ulong_p(exp_value))
        {
            mpz_clears(base, exp_value, pow10, NULL);
            return 0;
        }

        unsigned long exp = mpz_get_ui(exp_value);
        if (op == '^')
        {
            mpz_pow_ui(base, base, exp);
        }
        else
        {
            mpz_ui_pow_ui(pow10, 10, exp);
            mpz_mul(base, base, pow10);
        }
    }

    mpz_set(out, base);
    mpz_clears(base, exp_value, pow10, NULL);
    return 1;
}

```

```
static int parse_term_mpz(MPZ_EXPR_PARSER *parser, mpz_t out)
{
    mpz_t rhs;
    mpz_init(rhs);

    if (!parse_power_mpz(parser, out))
    {
        mpz_clear(rhs);
        return 0;
    }

    while (1)
    {
        parser_skip_spaces(parser);
        char op = *parser->cursor;
        if (op != '*' && op != '/')
            break;

        parser->cursor++;
        if (!parse_power_mpz(parser, rhs))
        {
            mpz_clear(rhs);
            return 0;
        }

        if (op == '*')
        {
            mpz_mul(out, out, rhs);
        }
        else
        {
            if (mpz_sgn(rhs) == 0)
            {
                mpz_clear(rhs);
                return 0;
            }
            mpz_tdiv_q(out, out, rhs);
        }
    }

    mpz_clear(rhs);
    return 1;
}

static int parse_expr_mpz_impl(MPZ_EXPR_PARSER *parser, mpz_t out)
{
    mpz_t rhs;
    mpz_init(rhs);

    if (!parse_term_mpz(parser, out))
    {
        mpz_clear(rhs);
        return 0;
    }

    while (1)
    {
        parser_skip_spaces(parser);
        char op = *parser->cursor;
        if (op != '+' && op != '-')
            break;

        parser->cursor++;
        if (!parse_term_mpz(parser, rhs))
        {
            mpz_clear(rhs);
            return 0;
        }

        if (op == '+')
            mpz_add(out, out, rhs);
        else
            mpz_sub(out, out, rhs);
    }

    mpz_clear(rhs);
    return 1;
}

static int parse_range_parts(const char *left_expr, const char *right_expr, mpz_t lower, mpz_t upper)
{
    mpz_t left, right;
    mpz_inits(left, right, NULL);

    int ok = parse_numeric_expr_mpz(left, left_expr) && parse_numeric_expr_mpz(right, right_expr) &&
mpz_cmp(right, left) >= 0;
    if (ok)
```

```
    {
        mpz_set(lower, left);
        mpz_set(upper, right);
    }

    mpz_clears(left, right, NULL);
    return ok;
}

int is_numeric_str(const char *str)
{
    if (str == NULL || *str == '\0')
        return 0;

    while (*str)
    {
        if (*str < '0' || *str > '9')
            return 0;
        ++str;
    }
    return 1;
}

int parse_numeric_expr_mpz(mpz_t out, const char *expr)
{
    if (expr == NULL)
        return 0;

    MPZ_EXPR_PARSER parser = {.cursor = expr};
    if (!parse_expr_mpz_impl(&parser, out))
        return 0;

    parser_skip_spaces(&parser);
    return *parser.cursor == '\0';
}

int parse_numeric_expr_u64(const char *expr, uint64_t *out)
{
    if (out == NULL)
        return 0;

    mpz_t value;
    mpz_init(value);
    if (!parse_numeric_expr_mpz(value, expr) || mpz_sgn(value) < 0 || mpz_sizeinbase(value, 2) > 64)
    {
        mpz_clear(value);
        return 0;
    }

    uint64_t parsed = 0;
    size_t words = 0;
    mpz_export(&parsed, &words, 1, sizeof(parsed), 0, 0, value);
    *out = (words == 0) ? 0 : parsed;
    mpz_clear(value);
    return 1;
}

int parse_inclusive_range_mpz(const char *range_expr, mpz_t lower, mpz_t upper)
{
    if (range_expr == NULL)
        return 0;

    char *range = dup_trimmed(range_expr);
    if (range == NULL || range[0] == '\0')
    {
        free(range);
        return 0;
    }

    size_t len = 0;
    if (!bounded_cstr_len(range, IZ_MAX_TEXT_INPUT_BYTES, &len))
    {
        free(range);
        return 0;
    }

    if (len > 7 && strncmp(range, "range[", 6) == 0 && range[len - 1] == ']')
    {
        memmove(range, range + 6, len - 7);
        range[len - 7] = '\0';
        len -= 7;
    }

    if (len > 2 && range[0] == '[' && range[len - 1] == ']')
    {
        memmove(range, range + 1, len - 2);
        range[len - 2] = '\0';
```

```
        len -= 2;
    }

    char *sep = strstr(range, "..");
    if (sep != NULL)
    {
        *sep = '\0';
        int ok = parse_range_parts(range, sep + 2, lower, upper);
        free(range);
        return ok;
    }

    sep = strchr(range, ':');
    if (sep != NULL)
    {
        *sep = '\0';
        int ok = parse_range_parts(range, sep + 1, lower, upper);
        free(range);
        return ok;
    }

    for (char *comma = range; (comma = strchr(comma, ',')) != NULL; ++comma)
    {
        char saved = *comma;
        *comma = '\0';
        int ok = parse_range_parts(range, comma + 1, lower, upper);
        *comma = saved;
        if (ok)
        {
            free(range);
            return 1;
        }
    }

    free(range);
    return 0;
}

int create_dir(const char *dir)
{
    if (iz_platform_create_dir(dir) != 0)
    {
        log_error("Failed to create directory: %s", dir ? dir : "(null)");
        return -1;
    }
    return 0;
}

uint64_t gcd(uint64_t a, uint64_t b)
{
    while (b != 0)
    {
        uint64_t temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

uint64_t lcm(uint64_t a, uint64_t b)
{
    if (a == 0 || b == 0)
        return 0;

    return (a / gcd(a, b)) * b;
}

void gcd_mpz(mpz_t out, const mpz_t a, const mpz_t b)
{
    mpz_gcd(out, a, b);
}

void lcm_mpz(mpz_t out, const mpz_t a, const mpz_t b)
{
    mpz_lcm(out, a, b);
}

uint64_t modular_inverse(uint64_t a, uint64_t m)
{
    if (m == 1)
        return 0;

    uint64_t m0 = m;
    int64_t x0 = 0, x1 = 1;

    while (a > 1)
    {
```

```
        uint64_t q = a / m;
        uint64_t t = m;

        m = a % m;
        a = t;
        t = x0;

        x0 = x1 - (int64_t)q * x0;
        x1 = t;
    }

    if (x1 < 0)
        x1 += (int64_t)m0;
    return (uint64_t)x1;
}

void gmp_seed_randstate(gmp_randstate_t state)
{
    unsigned long seed;
    if (!iz_platform_fill_random(&seed, sizeof(seed)))
        seed = (unsigned long)time(NULL);

    gmp_randseed_ui(state, seed);
}

int get_cpu_cores_count(void)
{
    return iz_platform_cpu_cores_count();
}

int get_cpu_L2_cache_size_bits(void)
{
    return iz_platform_l2_cache_size_bits();
}
```

# Index


__attribute__
    iZ_apps.c, 201

a_expr
    CLI_BINARY_EXPR_OPTIONS, 123
array
    UI16_ARRAY, 136
    UI32_ARRAY, 137
    UI64_ARRAY, 138
ASSERT_LIMIT
    prime_sieve.c, 215

b_expr
    CLI_BINARY_EXPR_OPTIONS, 123
base
    SIEVE_LIMIT, 132
base_primes
    prime_sieve.c, 216
base_primes_count
    prime_sieve.c, 216
base_x5
    IZM, 127
base_x7
    IZM, 127
Benchmarks, 3
bit_ops
    VX_SEG, 140
BITMAP, 121
    byte_size, 121
    data, 121
    sha256, 122
    size, 122
Bitmap Module, 35
    bitmap_clear_all, 37
    bitmap_clear_bit, 37
    bitmap_clear_steps, 37
    bitmap_clear_steps_simd, 39
    bitmap_clone, 40
    bitmap_compute_hash, 41
    bitmap_flip_bit, 41
    bitmap_fread, 42
    bitmap_free, 42
    bitmap_fwrite, 43
    bitmap_get_bit, 44
    bitmap_init, 44
    bitmap_set_all, 45
    bitmap_set_bit, 46
    bitmap_validate_hash, 46
    TEST_BITMAP, 47
bitmap.c, 224, 226
bitmap.h, 143, 144
bitmap_clear_all
    Bitmap Module, 37
bitmap_clear_bit
    Bitmap Module, 37
bitmap_clear_steps
    Bitmap Module, 37
bitmap_clear_steps_simd
    Bitmap Module, 39
bitmap_clone
    Bitmap Module, 40
bitmap_compute_hash
    Bitmap Module, 41
bitmap_flip_bit
    Bitmap Module, 41
bitmap_fread
    Bitmap Module, 42
bitmap_free
    Bitmap Module, 42
bitmap_fwrite
    Bitmap Module, 43
bitmap_get_bit
    Bitmap Module, 44
bitmap_init
    Bitmap Module, 44
bitmap_set_all
    Bitmap Module, 45
bitmap_set_bit
    Bitmap Module, 46
bitmap_validate_hash
    Bitmap Module, 46
byte_size
    BITMAP, 121

capacity
    UI16_ARRAY, 136
    UI32_ARRAY, 137
    UI64_ARRAY, 138
cli.h, 145, 146
    cli_run, 145
cli_app.c, 167, 178
    CLI_HANDLER, 170
    cli_run, 170
    CLI_STRING_CAP, 169
    dup_cli_string, 170
    IZ_VERSION, 169
    k_commands, 177
    k_sieve_models, 177
    k_sieve_models_count, 177
    MPZ_BINARY_OP, 170

parse_binary_mpz_args, 171
parse_cli_range, 171
parse_command, 171
parse_cores_value, 171
parse_expr_int, 171
parse_expr_u64, 171
parse_stream_output_option, 172
parse_stream_primes_args, 172
parse_stream_primes_option, 172
parse_stream_range_option, 172
parse_stream_rounds_option, 172
print_benchmark_help, 173
print_count_help, 173
print_gcd_help, 173
print_general_help, 173
print_is_prime_help, 173
print_lcm_help, 173
print_next_prime_help, 173
print_prev_prime_help, 174
print_stream_help, 174
print_test_help, 174
read_cli_option_value, 174
resolve_stream_path, 174
run_benchmark_cmd, 174
run_binary_mpz_cmd, 175
run_count_primes_cmd, 175
run_directional_prime_cmd, 175
run_doctor_cmd, 175
run_gcd_cmd, 175
run_is_prime_cmd, 175
run_lcm_cmd, 176
run_next_prime_cmd, 176
run_prev_prime_cmd, 176
run_stream_primes_cmd, 176
run_test_cmd, 176
SIEVE_FN, 170
STREAM_PARSE_RESULT, 170
stream_read_option_value, 176
validate_stream_primes_options, 177
CLI_BINARY_EXPR_OPTIONS, 122
a_expr, 123
b_expr, 123
CLI_COMMAND, 123
handler, 123
name, 123
CLI_HANDLER
cli_app.c, 170
CLI_RANGE, 124
lower, 124
range_size, 124
upper, 124
cli_run
cli.h, 145
cli_app.c, 170
CLI_SIEVE_MODEL, 124
fn, 125
name, 125
CLI_STRING_CAP
cli_app.c, 169
compute_k_vx
iZ_toolkit.c, 234
compute_l2_vx
Toolkit (iZ/iZm), 69
compute_max_vx
Toolkit (iZ/iZm), 69
compute_vx_k
Toolkit (iZ/iZm), 70
Contributing to iZprime, 13
count
UI16_ARRAY, 136
UI32_ARRAY, 137
UI64_ARRAY, 139
create_dir
Utilities, 106

data
BITMAP, 121
IZP_U64_BUFFER, 131
DIR_output
Utilities, 104
dup_cli_string
cli_app.c, 170

elapsed_sec
STOPWATCH, 133
end_time
STOPWATCH, 133
end_x
VX_SEG, 140
exp
SIEVE_LIMIT, 132

FFI API, 85
izp_ffi_clear_error, 86
izp_ffi_count_range, 86
IZP_FFI_ERR_ALLOC, 86
IZP_FFI_ERR_INVALID_ARG, 86
IZP_FFI_ERR_NOT_FOUND, 86
IZP_FFI_ERR_OPERATION, 86
IZP_FFI_ERR_PARSE, 86
izp_ffi_free_string, 88
izp_ffi_free_u64_buffer, 88
izp_ffi_last_error, 88
izp_ffi_next_prime, 88
IZP_FFI_OK, 86
izp_ffi_random_prime_vx, 90
izp_ffi_random_prime_vy, 90
izp_ffi_sieve_u64, 90
IZP_FFI_STATUS, 86
izp_ffi_status_message, 92
izp_ffi_stream_range, 92
izp_ffi_version, 93
IZP_SIEVE_KIND, 86
filepath
INPUT_SIEVE_RANGE, 126
fn
CLI_SIEVE_MODEL, 125

function
SIEVE_MODEL, 132

g_izp_ffi_last_error
izprime_ffi.c, 194
gcd
Utilities, 106
gcd_mpz
Utilities, 106
get_cpu_cores_count
Utilities, 107
get_cpu_L2_cache_size_bits
Utilities, 107
get_root_primes
Toolkit (iZ/iZm), 70
gmp_random_below
iZ_toolkit.c, 234
gmp_seed_randstate
Utilities, 108

handler
CLI_COMMAND, 123
has_range
STREAM_CMD_OPTIONS, 134

INPUT_SIEVE_RANGE, 125
filepath, 126
iZ Public API, 52
mr_rounds, 126
range, 126
start, 126
stream_gaps, 126
int_arrays.c, 230, 232
int_arrays.h, 146, 148
Integer Arrays, 47
TEST_GENERIC_INT_ARRAYS, 50
is_large_limit
VX_SEG, 140
is_numeric_str
Utilities, 108
iZ
Toolkit (iZ/iZm), 71
iZ Public API, 50
INPUT_SIEVE_RANGE, 52
iZ_next_prime, 52
SiZ, 53
SiZ_count, 54
SiZ_stream, 55
SiZm, 56
SiZm_vy, 57
SoA, 57
SoE, 58
SoEu, 59
SoS, 60
SSoE, 61
SSoS, 62
vx_random_prime, 63
vy_random_prime, 64
iZ_api.h, 149, 151
iZ_apps.c, 200, 202
__attribute__, 201
iZmX, 201
PRIME_STR_CAP_PADDING, 201
IZ_DIAG_IGNORE_FORMAT_NONLITERAL
print_utils.c, 254
IZ_DIAG_POP
print_utils.c, 254
IZ_DIAG_PUSH
print_utils.c, 254
iZ_mpz
Toolkit (iZ/iZm), 71
iZ_next_prime
iZ Public API, 52
iz_platform_cpu_cores_count
Platform Abstraction, 100
iz_platform_create_dir
Platform Abstraction, 100
iz_platform_fill_random
Platform Abstraction, 100
IZ_PLATFORM_HAS_FORK
platform.h, 161
iz_platform_l2_cache_size_bits
Platform Abstraction, 101
iz_platform_localtime
Platform Abstraction, 101
iz_platform_monotonic_seconds
Platform Abstraction, 101
IZ_PLATFORM_POSIX
platform.h, 161
IZ_PLATFORM_WINDOWS
platform.h, 162
iZ_toolkit.c, 232, 237
compute_k_vx, 234
gmp_random_below, 234
s_primes, 236
s_primes_count, 236
vx_det_sieve, 235
vx_prob_sieve, 235
vx_set_base_values, 235
iZ_toolkit.h, 152, 154
IZ_VERSION
cli_app.c, 169
IZM, 127
base_x5, 127
base_x7, 127
k_vx, 127
root_primes, 127
vx, 128
iZm_clone
Toolkit (iZ/iZm), 72
iZm_construct_vx_base
Toolkit (iZ/iZm), 72
iZm_free
Toolkit (iZ/iZm), 73
iZm_init
Toolkit (iZ/iZm), 73
IZM_RANGE_INFO, 128

vx, 129
Xe, 129
Xs, 129
y_range, 129
Ye, 129
Ys, 129
Ze, 130
Zs, 130
iZm_solve_for_x0
Toolkit (iZ/iZm), 74
iZm_solve_for_x0_mpz
Toolkit (iZ/iZm), 75
iZm_solve_for_y0
Toolkit (iZ/iZm), 77
iZmX
iZ_apps.c, 201
IZP_FFI_API
izprime_ffi.h, 157
IZP_FFI_BUILD
izprime_ffi.c, 193
izp_ffi_clear_error
FFI API, 86
izp_ffi_copy_mpz_to_string
izprime_ffi.c, 193
izp_ffi_count_range
FFI API, 86
izp_ffi_count_small_range
izprime_ffi.c, 193
IZP_FFI_ERR_ALLOC
FFI API, 86
IZP_FFI_ERR_INVALID_ARG
FFI API, 86
IZP_FFI_ERR_NOT_FOUND
FFI API, 86
IZP_FFI_ERR_OPERATION
FFI API, 86
IZP_FFI_ERR_PARSE
FFI API, 86
IZP_FFI_ERROR_CAP
izprime_ffi.c, 193
izp_ffi_export_primes
izprime_ffi.c, 193
izp_ffi_free_string
FFI API, 88
izp_ffi_free_u64_buffer
FFI API, 88
izp_ffi_last_error
FFI API, 88
izp_ffi_next_prime
FFI API, 88
IZP_FFI_OK
FFI API, 86
izp_ffi_random_prime_vx
FFI API, 90
izp_ffi_random_prime_vy
FFI API, 90
izp_ffi_set_error
izprime_ffi.c, 193
izp_ffi_sieve_u64
FFI API, 90
IZP_FFI_STATUS
FFI API, 86
izp_ffi_status_message
FFI API, 92
izp_ffi_strdup_heap
izprime_ffi.c, 194
izp_ffi_stream_range
FFI API, 92
izp_ffi_validate_expr_nonnegative
izprime_ffi.c, 194
izp_ffi_version
FFI API, 93
IZP_MAX_SIEVE_LIMIT
izprime_ffi.c, 193
IZP_SIEVE_KIND
FFI API, 86
IZP_U64_BUFFER, 130
data, 131
len, 131
izprime CLI, 9
iZprime User Manual, 1
izprime_ffi.c, 191, 194
g_izp_ffi_last_error, 194
IZP_FFI_BUILD, 193
izp_ffi_copy_mpz_to_string, 193
izp_ffi_count_small_range, 193
IZP_FFI_ERROR_CAP, 193
izp_ffi_export_primes, 193
izp_ffi_set_error, 193
izp_ffi_strdup_heap, 194
izp_ffi_validate_expr_nonnegative, 194
IZP_MAX_SIEVE_LIMIT, 193
izprime_ffi.h, 155, 157
IZP_FFI_API, 157

k_commands
cli_app.c, 177
k_sieve_models
cli_app.c, 177
k_sieve_models_count
cli_app.c, 177
k_vx
IZM, 127

Language Bindings Strategy (FFI), 7
lcm
Utilities, 109
lcm_mpz
Utilities, 109
len
IZP_U64_BUFFER, 131
log_console
Logging, 96
LOG_DEBUG
Logging, 95
log_debug
Logging, 96

LOG_DIR
Logging, 94
LOG_ERROR
Logging, 95
log_error
Logging, 96
LOG_FATAL
Logging, 95
log_fatal
Logging, 96
LOG_FILE
Logging, 94
LOG_INFO
Logging, 95
log_info
Logging, 97
log_init
Logging, 97
log_level_to_string
Logging, 97
LOG_MAX_SIZE
Logging, 94
log_message
Logging, 98
log_message_extended
Logging, 98
log_set_log_level
Logging, 99
log_warn
Logging, 99
LOG_WARNING
Logging, 95
logger.c, 248
logger.h, 158, 160
LOGGER_FORMAT_PRINTF
Logging, 95
Logging, 93
log_console, 96
LOG_DEBUG, 95
log_debug, 96
LOG_DIR, 94
LOG_ERROR, 95
log_error, 96
LOG_FATAL, 95
log_fatal, 96
LOG_FILE, 94
LOG_INFO, 95
log_info, 97
log_init, 97
log_level_to_string, 97
LOG_MAX_SIZE, 94
log_message, 98
log_message_extended, 98
log_set_log_level, 99
log_warn, 99
LOG_WARNING, 95
LOGGER_FORMAT_PRINTF, 95
LogLevel, 95
LogLevel
Logging, 95
lower
CLI_RANGE, 124

Makefile documentation (iZprime), 17
MAX
Utilities, 104
MAX_CORES
Utilities, 105
MIN
Utilities, 105
modular_inverse
Utilities, 110
MPZ_BINARY_OP
cli_app.c, 170
MR_ROUNDS
Toolkit (iZ/iZm), 67
mr_rounds
INPUT_SIEVE_RANGE, 126
STREAM_CMD_OPTIONS, 134
VX_SEG, 140

N_LIMIT
prime_sieve.c, 215
name
CLI_COMMAND, 123
CLI_SIEVE_MODEL, 125
SIEVE_MODEL, 132
Native Package Plan, 23

ordered
UI16_ARRAY, 136
UI32_ARRAY, 137
UI64_ARRAY, 139

p_count
VX_SEG, 141
p_gaps
VX_SEG, 141
p_test_ops
VX_SEG, 141
parse_binary_mpz_args
cli_app.c, 171
parse_cli_range
cli_app.c, 171
parse_command
cli_app.c, 171
parse_cores_value
cli_app.c, 171
parse_expr_int
cli_app.c, 171
parse_expr_u64
cli_app.c, 171
parse_inclusive_range_mpz
Utilities, 111
parse_numeric_expr_mpz
Utilities, 111
parse_numeric_expr_u64

Utilities, 113
parse_stream_output_option
cli_app.c, 172
parse_stream_primes_args
cli_app.c, 172
parse_stream_primes_option
cli_app.c, 172
parse_stream_range_option
cli_app.c, 172
parse_stream_rounds_option
cli_app.c, 172
Pi
prime_sieve.c, 215
Platform Abstraction, 99
iz_platform_cpu_cores_count, 100
iz_platform_create_dir, 100
iz_platform_fill_random, 100
iz_platform_l2_cache_size_bits, 101
iz_platform_localtime, 101
iz_platform_monotonic_seconds, 101
platform.c, 251, 252
platform.h, 161, 162
IZ_PLATFORM_HAS_FORK, 161
IZ_PLATFORM_POSIX, 161
IZ_PLATFORM_WINDOWS, 162
playground.c, 213
prime_sieve.c, 213, 216
ASSERT_LIMIT, 215
base_primes, 216
base_primes_count, 216
N_LIMIT, 215
Pi, 215
process_N_bitmap, 215
push_small_primes, 216
PRIME_STR_CAP_PADDING
iZ_apps.c, 201
print_benchmark_help
cli_app.c, 173
print_centered_text
Printer, implemented in toolkit/print_utils.c, 114
print_count_help
cli_app.c, 173
print_gaps
STREAM_CMD_OPTIONS, 135
print_gcd_help
cli_app.c, 173
print_general_help
cli_app.c, 173
print_is_prime_help
cli_app.c, 173
print_lcm_help
cli_app.c, 173
print_line
Printer, implemented in toolkit/print_utils.c, 114
print_next_prime_help
cli_app.c, 173
print_prev_prime_help
cli_app.c, 174
print_sha256_hash
Printer, implemented in toolkit/print_utils.c, 115
print_stream_help
cli_app.c, 174
print_test_fn_header
Printer, implemented in toolkit/print_utils.c, 115
print_test_help
cli_app.c, 174
print_test_module_header
Printer, implemented in toolkit/print_utils.c, 115
print_test_module_result
Printer, implemented in toolkit/print_utils.c, 116
print_test_summary
Printer, implemented in toolkit/print_utils.c, 116
print_test_table_header
Printer, implemented in toolkit/print_utils.c, 118
print_to_console
STREAM_CMD_OPTIONS, 135
print_utils.c, 253, 255
IZ_DIAG_IGNORE_FORMAT_NONLITERAL, 254
IZ_DIAG_POP, 254
IZ_DIAG_PUSH, 254
Printer, implemented in toolkit/print_utils.c, 113
print_centered_text, 114
print_line, 114
print_sha256_hash, 115
print_test_fn_header, 115
print_test_module_header, 115
print_test_module_result, 116
print_test_summary, 116
print_test_table_header, 118
process_iZ_bitmaps
Toolkit (iZ/iZm), 78
process_N_bitmap
prime_sieve.c, 215
push_small_primes
prime_sieve.c, 216

range
INPUT_SIEVE_RANGE, 126
STREAM_CMD_OPTIONS, 135
range_info_free
Toolkit (iZ/iZm), 78
range_info_init
Toolkit (iZ/iZm), 78
range_size
CLI_RANGE, 124
read_cli_option_value
cli_app.c, 174
Releasing iZprime, 25

resolve_stream_path
cli_app.c, 174
root_limit
VX_SEG, 141
root_primes
IZM, 127
run_benchmark_cmd
cli_app.c, 174
run_binary_mpz_cmd
cli_app.c, 175
run_count_primes_cmd
cli_app.c, 175
run_directional_prime_cmd
cli_app.c, 175
run_doctor_cmd
cli_app.c, 175
run_gcd_cmd
cli_app.c, 175
run_is_prime_cmd
cli_app.c, 175
run_lcm_cmd
cli_app.c, 176
run_next_prime_cmd
cli_app.c, 176
run_prev_prime_cmd
cli_app.c, 176
run_stream_primes_cmd
cli_app.c, 176
run_test_cmd
cli_app.c, 176
running
STOPWATCH, 133

s_primes
iZ_toolkit.c, 236
s_primes_count
iZ_toolkit.c, 236
sha256
BITMAP, 122
UI16_ARRAY, 136
UI32_ARRAY, 138
UI64_ARRAY, 139
SIEVE_FN
cli_app.c, 170
Tests and Benchmarks, 103
SIEVE_LIMIT, 131
base, 132
exp, 132
SIEVE_MODEL, 132
function, 132
name, 132
SiZ
iZ Public API, 53
SiZ_count
iZ Public API, 54
SiZ_stream
iZ Public API, 55
size
BITMAP, 122
SiZm
iZ Public API, 56
SiZm_vy
iZ Public API, 57
SoA
iZ Public API, 57
SoE
iZ Public API, 58
SoEu
iZ Public API, 59
SoS
iZ Public API, 60
SSoE
iZ Public API, 61
SSoS
iZ Public API, 62
start
INPUT_SIEVE_RANGE, 126
start_time
STOPWATCH, 134
start_x
VX_SEG, 141
STOPWATCH, 133
elapsed_sec, 133
end_time, 133
running, 133
start_time, 134
Stopwatch, 118
sw_elapsed_now_seconds, 119
sw_elapsed_seconds, 119
sw_start, 119
sw_stop, 119
stopwatch.c, 256, 257
STREAM_CMD_OPTIONS, 134
has_range, 134
mr_rounds, 134
print_gaps, 135
print_to_console, 135
range, 135
stream_path, 135
stream_gaps
INPUT_SIEVE_RANGE, 126
STREAM_PARSE_RESULT
cli_app.c, 170
stream_path
STREAM_CMD_OPTIONS, 135
stream_read_option_value
cli_app.c, 176
sw_elapsed_now_seconds
Stopwatch, 119
sw_elapsed_seconds
Stopwatch, 119
sw_start
Stopwatch, 119
sw_stop
Stopwatch, 119

test_api.h, 162, 164
TEST_BITMAP

Bitmap Module, 47
TEST_GENERIC_INT_ARRAYS
Integer Arrays, 50
test_primality
Toolkit (iZ/iZm), 79
Tests, 27
Tests and Benchmarks, 102
SIEVE_FN, 103
Toolkit (iZ/iZm), 65
compute_l2_vx, 69
compute_max_vx, 69
compute_vx_k, 70
get_root_primes, 70
iZ, 71
iZ_mpz, 71
iZm_clone, 72
iZm_construct_vx_base, 72
iZm_free, 73
iZm_init, 73
iZm_solve_for_x0, 74
iZm_solve_for_x0_mpz, 75
iZm_solve_for_y0, 77
MR_ROUNDS, 67
process_iZ_bitmaps, 78
range_info_free, 78
range_info_init, 78
test_primality, 79
VX2, 67
VX3, 67
VX4, 67
VX5, 68
VX6, 68
VX7, 68
VX8, 68
vx_collect_p_gaps, 80
vx_free, 80
vx_full_sieve, 81
vx_init, 81
vx_search_prime, 82
vx_stream, 83
vy_search_prime, 84

UI16_ARRAY, 135
array, 136
capacity, 136
count, 136
ordered, 136
sha256, 136
UI32_ARRAY, 137
array, 137
capacity, 137
count, 137
ordered, 137
sha256, 138
UI64_ARRAY, 138
array, 138
capacity, 138
count, 139
ordered, 139
sha256, 139
upper
CLI_RANGE, 124
Utilities, 103
create_dir, 106
DIR_output, 104
gcd, 106
gcd_mpz, 106
get_cpu_cores_count, 107
get_cpu_L2_cache_size_bits, 107
gmp_seed_randstate, 108
is_numeric_str, 108
lcm, 109
lcm_mpz, 109
MAX, 104
MAX_CORES, 105
MIN, 105
modular_inverse, 110
parse_inclusive_range_mpz, 111
parse_numeric_expr_mpz, 111
parse_numeric_expr_u64, 113
utils.c, 257, 258
utils.h, 164, 166

validate_stream_primes_options
cli_app.c, 177
vx
IZM, 128
IZM_RANGE_INFO, 129
VX_SEG, 142
VX2
Toolkit (iZ/iZm), 67
VX3
Toolkit (iZ/iZm), 67
VX4
Toolkit (iZ/iZm), 67
VX5
Toolkit (iZ/iZm), 68
VX6
Toolkit (iZ/iZm), 68
VX7
Toolkit (iZ/iZm), 68
VX8
Toolkit (iZ/iZm), 68
vx_collect_p_gaps
Toolkit (iZ/iZm), 80
vx_det_sieve
iZ_toolkit.c, 235
vx_free
Toolkit (iZ/iZm), 80
vx_full_sieve
Toolkit (iZ/iZm), 81
vx_init
Toolkit (iZ/iZm), 81
vx_prob_sieve
iZ_toolkit.c, 235
vx_random_prime
iZ Public API, 63
vx_search_prime

Toolkit (iZ/iZm), 82
VX_SEG, 139
bit_ops, 140
end_x, 140
is_large_limit, 140
mr_rounds, 140
p_count, 141
p_gaps, 141
p_test_ops, 141
root_limit, 141
start_x, 141
vx, 142
x5, 142
x7, 142
y, 142
yvx, 142
vx_set_base_values
iZ_toolkit.c, 235
vx_stream
Toolkit (iZ/iZm), 83
vy_random_prime
iZ Public API, 64
vy_search_prime
Toolkit (iZ/iZm), 84

x5
VX_SEG, 142
x7
VX_SEG, 142
Xe
IZM_RANGE_INFO, 129
Xs
IZM_RANGE_INFO, 129

y
VX_SEG, 142
y_range
IZM_RANGE_INFO, 129
Ye
IZM_RANGE_INFO, 129
Ys
IZM_RANGE_INFO, 129
yvx
VX_SEG, 142

Ze
IZM_RANGE_INFO, 130
Zs
IZM_RANGE_INFO, 130